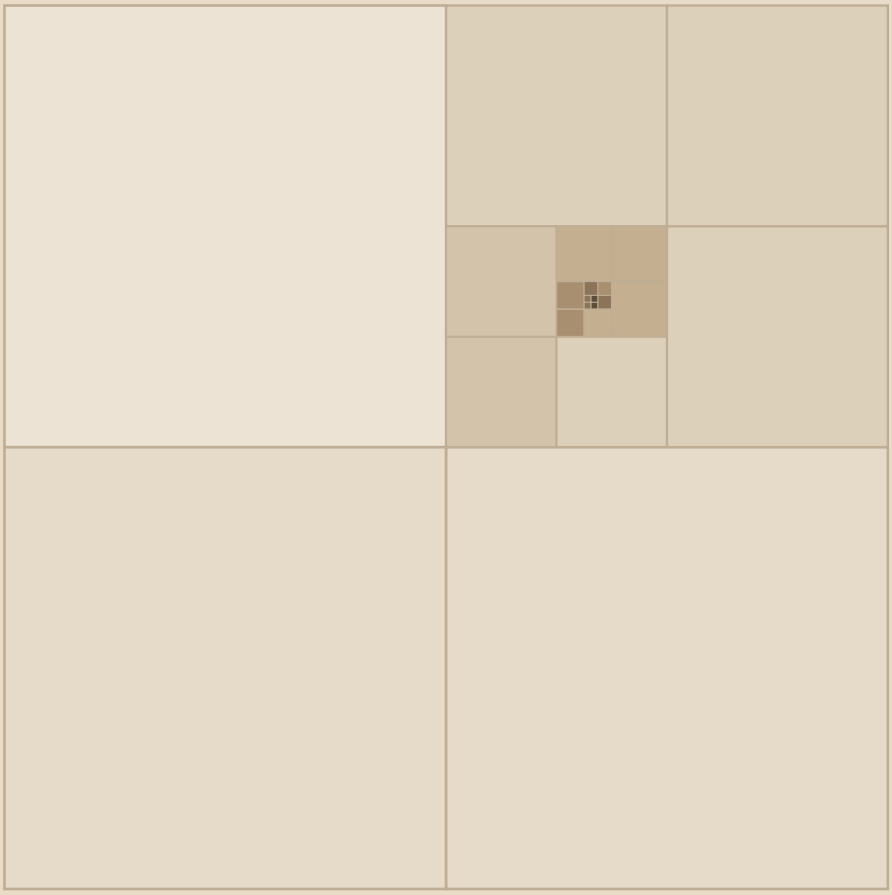
Tessera



Ten Days *in Ink*

Every story of Tessera, one working day at a time, in C# 14 on .NET 10

‘Work in pencil can be rubbed out. Work in ink stays in the record.’

Phase Oct Edition · October 2026

A companion to The World in Squares

How to read this book

> IN SHORT This book takes you through sixty working days on Tessera, from an empty folder on Monday 19 October 2026 to the last of its thirty stories on Monday 18 January 2027. Each chapter is one day, and each day ends with work you can show: passing tests and commits on GitHub.

Read the answer at the top of each chapter first, then the steps, then do them. Every chapter gives its answer, then the reasons, then the detail, so you can stop at any level and still know the main point. The order follows Barbara Minto's *Pyramid Principle*: one idea first, then the few ideas that hold it up, then the facts under each.

The book runs in the order you use it:

- **Chapter 1** gives the whole plan: the sixty days, the four milestones, the three rules of 'ink', your two folders and the five steps of every day.
- **Part one** gives the first ten days in full depth: the machine, then US-00, US-01, US-03 and US-24, the first tests of US-07, and the review that reaches milestone M1.
- **Parts two to four** carry the delivery plan on to the release of Monday 7 December: the safe tiles and the map in the office, the map that reads well, then clusters, names and the release.
- **Parts five and six** follow the second plan, from Wednesday 9 December to Monday 18 January, until all thirty stories stand in your own copy.
- **Appendices A and B** list the planned differences between your work and the worked answer, and the words this book uses.

The parts of each day

Each day has the same five parts, so you always know where to look, and Exhibit 0.1 gives the question each one answers.

EXHIBIT 0.1 Every day chapter answers the same five questions in the same order

PART	THE QUESTION IT ANSWERS
In short	What will you have at the end of the day, and which story, worth how many points, does it serve?
The day on one page	Which story and criteria, which files of the worked answer, what to look for, what to leave for later, and which tests end the day?
The steps	What do you do, and in what order? Each step goes red, then green, then tidy.
In ink	Which commits do you make, with which messages, and when do you push?
The proof	Which tests pass tonight, and what do you write in your notes for tomorrow?

Source: this book

The panels

Labelled panels sit beside the main text, each kind with one job, as Exhibit 0.2 lists.

EXHIBIT 0.2 Eight kinds of panel, each with one job

PANEL	ITS JOB
In short	The answer of the chapter, in one to three sentences
Key idea	The point most worth remembering from a section
From school	How the work links to something you learnt in computer science lessons
Watch out	A mistake that is easy to make and hard to find, or a known fault in the worked answer
Try it	A small task at the keyboard, with what you should see
Note	A test the worked answer lacks, which you write all the same, marked 'beyond the worked answer'
In ink	The day's commits, with their messages, and the push
The proof	What passes tonight, and tomorrow's first test

Source: this book

The Note panel matters more in this edition than in the first. The repository's requirements files, in `docs/requirements/`, list 151 requirements and say for each which test proves it: 99 in full, 33 in part, and 19 marked 'No test yet'. Where a day meets a criterion with no test, it says so and has you write one; Appendix A lists each test your copy holds and the worked answer lacks.

The Watch out panel carries a second load. The worked answer has four known faults, and the day that meets each one names it, in the words of *The World in Squares*: a disguised path reaches a route (S-06, day 14); a misspelt key stops the start alone (N-21, day 41); no log on a 500 (N-22, day 44); and N-26's known limit names the wrong signal (day 49). Read them as findings, not as rules to copy.

Two folders and five tools

You keep two folders, and you type only in the second. The **worked answer**, `~/src/tessera`, is the repository this book describes, `nathanwh71/Tessera` on GitHub: read it and run it, but never change it. **Your copy**, `~/src/tessera-rebuild`, starts empty on day 2 and holds every line you write. Chapter 1 sets the two side by side.

Five tools do all the work, on Ubuntu 24.04 inside WSL2 on Windows 11, the same set-up as the office machine. Day 1 installs them and proves them by running the worked answer's `npm test`; the worked answer's files name each version, as Exhibit 0.3 shows, so you copy the versions rather than guess them.

EXHIBIT 0.3 Five tools, the version each needs, and the command that checks it

TOOL	VERSION, AND WHERE IT IS NAMED	WHAT IT DOES FOR YOU	CHECK
.NET 10 SDK	10.0.100 or a later feature band, never .NET 11: `global.json`	Builds, tests and runs the C# service; `dotnet new` starts your projects	`dotnet --version`
VS Code, with the WSL extension and the C# Dev Kit	Not pinned	Edits the code inside Ubuntu and marks mistakes as you type	'WSL: Ubuntu-24.04' in the status bar

Docker Engine	Not pinned; each image is named in its Dockerfile	Runs the database, Nginx, and the databases that Testcontainers starts for the integration tests	`docker version`
Node.js	26 or later: `package.json`	Runs the zx scripts, such as `npm test`, and builds the map in `web/`	`node --version`
uv	Not pinned; it installs Python 3.13, named in `etl/.python-version`	Runs the Python pipeline's checks and tests in `etl/`	`uv --version`

Source: `global.json`; `package.json`; `etl/.python-version`; `.github/workflows/build.yml`

Git carries your work to GitHub, and some of the pipeline's tests call GDAL's `ogr2ogr`, so install Ubuntu's `gdal-bin` as well. The C# Dev Kit runs the same compiler and analysers as `dotnet build`; when the two seem to disagree, the terminal has the last word.

> WATCH OUT An answer from the web may fit an older .NET, an older xUnit or an older TypeScript, and fail to compile here. Check it against the worked answer's pins in `global.json`, `Directory.Packages.props`, `package.json` and `web/package.json` before you trust it.

The words in bold italics, and the words in grey

Two kinds of word stand out in the main text and the tables. An important idea appears in ***bold italics*** where the book explains it, with its meaning in plain words beside it; Appendix B lists them all. Words from the Oxford list, the common English words every reader knows, appear in grey. Headings, panel labels, code, commands and test names stay plain.

What you need to know already

You need only what a school course in computer science teaches:

- a function or subroutine, with inputs and a return value;
- a record and a list;
- a database table, its records and fields, and an SQL `SELECT`;
- the parts of a network: an IP address, a port, a protocol such as HTTP;
- testing with normal, boundary and erroneous data;
- binary, and how a character set stores text as bytes.

The book builds every new idea on one of these. The service's language is C# 14, and the From school panels show the same ideas in the pseudocode or Python you know.

> KEY IDEA Read the answer at the top of each chapter first. If it makes sense, read the steps. If the steps make sense, do them, in your own folder, and only then open the worked answer's version.

Contents

How to read this book 2

PART ONE *The first ten days: the skeleton and the places*

1 **Sixty days, thirty stories, four milestones** 8

2 **Day 1 · Monday 19 October: the machine** 16

3 **Day 2 · Tuesday 20 October: US-00, the walking skeleton** 19

4 **Day 3 · Wednesday 21 October: US-01, one database for the service and its tests** 24

5 **Day 4 · Thursday 22 October: US-03, one way to quote a name** 28

6 **Day 5 · Friday 23 October: US-03, rows become layers** 32

7 **Day 6 · Monday 26 October: US-03, the catalogue as the real role** 36
US-03 ends: the database, not the service, decides what the service may see

8 **Day 7 · Tuesday 27 October: US-24, each seed script once** 41
The program learns a second command: run each new script of places once

9 **Day 8 · Wednesday 28 October: US-24, publish by grant** 45
The seed loads the places; only a grant puts them on the map

10 **Day 9 · Thursday 29 October: US-07 begun, the tile's address** 49
US-07 begins: a tile's address is checked before any query runs

11 **Day 10 · Friday 30 October: the review, and M1** 52
Prove two weeks from a clean start, show M1 twice, and plan the next two weeks

PART TWO *Iteration 2: safe tiles, and the map in the office*

12 **Day 11 · Tuesday 3 November: US-07, a tile through one query** 55
One query builds the tile, and every failure has one shape

13 **Day 12 · Wednesday 4 November: S-01, values beside the query** 61
Values travel beside the query, and one text serves every tile

14 **Day 13 · Thursday 5 November: S-01, names from the catalogue alone** 65
A name enters SQL only through one quoting function

15 **Day 14 · Friday 6 November: S-06, refuse malformed addresses** 67
Nginx stands at the door, and refuses what it cannot read

16 **Day 15 · Monday 9 November: US-35, the office reaches port 80** 71
Port 80 on the office machine, and Nginx sees who asks

17 **Day 16 · Tuesday 10 November: US-23, the area in the page and the nine layers** 75

18 **Day 17 · Wednesday 11 November: US-23, one model and one pure update** 81

19 **Day 18 · Thursday 12 November: US-23, the page in a browser** 85

20 **Day 19 · Friday 13 November: the review, US-35 on the office machine, and M2** 89

PART THREE *Iteration 3: the map reads well*

21 **Day 20 · Monday 16 November: UI-09, levels that double** 91
Two counts of zoom, kept apart by the compiler

22 **Day 21 · Tuesday 17 November: UI-09, buttons that say the level** 94
Buttons that speak, and a world with an edge

23 **Day 22 · Wednesday 18 November: UI-15, one theme for the page** 97
Every colour in one place, every icon from one font

24 **Day 23 · Thursday 19 November: UI-15, movement that stays calm** 100
Nothing moves for long, and nothing moves at all on request

25 **Day 24 · Friday 20 November: UI-12, shapes as well as colours** 103
A shape for each group, and a check of the week

26 **Day 25 · Monday 23 November: UI-19, the scale bar** 105
A round length that fits in 140 pixels

27 **Day 26 · Tuesday 24 November: UI-19, the compass and the coordinates** 108
North, and the point under the pointer

28 **Day 27 · Wednesday 25 November: UI-10 begins, where pins merge** 110
Where pins merge, and why the browser decides

29 **Day 28 · Thursday 26 November: UI-10, places fed and ringed** 113
The loaded places reach the clusters, ringed by group

30 **Day 29 · Friday 27 November: the review, and M3** 117
A clean run, the milestone, and the plan for the release

PART FOUR *Iteration 4: clusters, names and the release*

31 **Day 30 · Monday 30 November: UI-10, opening a cluster** 119
A click opens a cluster, and a screen reader hears what it holds

32 **Day 31 · Tuesday 1 December: UI-11, names from level 16** 121
From level 16, every pin carries its name

33 **Day 32 · Wednesday 2 December: UI-17, names that give way** 124
Four sides to try, and pins that always win

34 **Day 33 · Thursday 3 December: UI-17, a fixed order and three approved images** 127
The same name wins every time, and three images prove it

35 **Day 34 · Friday 4 December: S-07, office machines alone** 130
The day in hand closes the door to strangers

36 **Day 35 · Monday 7 December: the review and the release, M4** 134
Milestone M4: Parts A and B released to the office

PART FIVE *The second plan: Parts C and D*

37 **Day 36 · Wednesday 9 December: the second plan, and N-18, a tile the browser keeps** 137

38 **Day 37 · Thursday 10 December: N-19, a tag made from the tile's bytes** 140

39 **Day 38 · Friday 11 December: N-19, 304 and Nginx's store** 142

40 **Day 39 · Monday 14 December: N-20, tiles and files travel packed** 145

41 **Day 40 · Tuesday 15 December: N-21, every setting in one file** 147

42 **Day 41 · Wednesday 16 December: N-21, the environment, the limits, and no password** 151

43 **Day 42 · Thursday 17 December: N-22, four kinds of database failure** 155

44 **Day 43 · Friday 18 December: the second plan's first review** 158

45 **Day 44 · Monday 21 December: N-22, ask the browser to wait** 160

46 **Day 45 · Wednesday 23 December: N-23, a list of layers that lives** 163

47 **Day 46 · Monday 28 December: N-23, reread the list on a schedule** 166

48 **Day 47 · Tuesday 29 December: N-24, one query for a crowd** 169

49 **Day 48 · Wednesday 30 December: N-25, a health check that says UP only when it means it** 172

PART SIX *The second plan: Parts D, E and F*

50 **Day 49 · Thursday 31 December: N-26, first come, first served** 176

51 **Day 50 · Monday 4 January: N-28 begins, kept tiles with the service down** 180

52 **Day 51 · Tuesday 5 January: N-28 ends, an old tile and a hanging database** 183

53 **Day 52 · Wednesday 6 January: S-30 begins, a steady rate for each visitor** 186

54 **Day 53 · Thursday 7 January: S-30 ends, thirty-two at once** 189

55 **Day 54 · Friday 8 January: the second plan's second review** 191

56 **Day 55 · Monday 11 January: N-29 begins, the restart loop with a fake clock** 194

57 **Day 56 · Tuesday 12 January: N-29 ends, two copies behind Nginx** 197

58 **Day 57 · Wednesday 13 January: US-36 begins, a chain of filters** 201

59 **Day 58 · Thursday 14 January: US-36 continues, clean text and the stop rules** 205

60 **Day 59 · Friday 15 January: US-36 ends, the log and a whole run** 209

61 **Day 60 · Monday 18 January: the third review, every story in ink** 212

A **Planned differences** 216

B **The words this book uses** 221

1
Sixty days, thirty stories, four milestones

> IN SHORT In sixty working days you rebuild all thirty of Tessera's stories in an empty folder of your own: twenty-nine planned points, and S-07 on the day in hand, by the release of 7 December (milestone M4), then twenty-two more points in the second plan by 18 January. You work test-first and finish every day in ink.

Four ideas make the plan work. 'Ink' means three rules that turn each day into proof; the sixty days follow the delivery plan's fixed order of stories; two folders stay apart, the worked answer you read and the copy you write; and every day has the same five steps.

1.1 What 'in ink' means

Every day ends with its work committed and pushed. A change left on your machine overnight is pencil: a broken disk, a careless command or a tired morning can rub it out. Work in ***ink*** stays in the record. A ***commit*** (a saved snapshot of your files, with a message) on GitHub has a date, a message and a list of changes that anyone can read, including you next week.

Ink comes with three rules:

1. **Commit only when every test passes.** Each commit holds one change, and its message names the story, such as `US-03: quote every name one way`.
2. **Push before you stop.** The day ends only when `git push` has copied your commits to GitHub and the build there has started. ***Continuous integration***, CI for short, is a machine on GitHub that builds your project and runs every test after each push.
3. **Never commit a failing test to remind you of tomorrow's work.** Write tomorrow's first test in a note outside the repository.

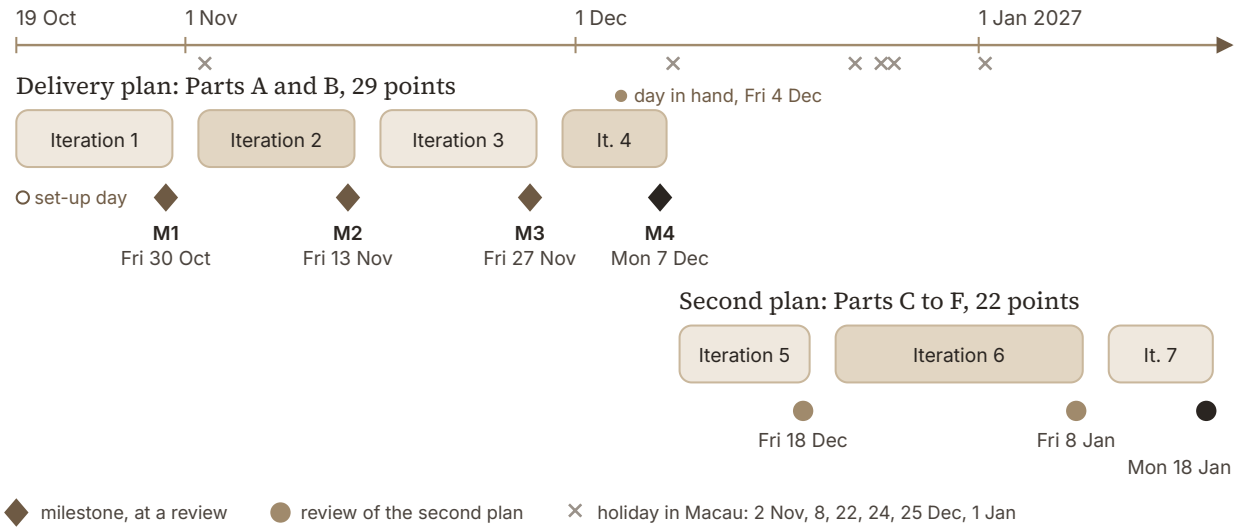
> FROM SCHOOL At school you met iterative testing: testing each part as you build it, not only at the end. Ink is iterative testing with a written record. Each commit says 'these tests passed at this moment', and GitHub keeps that record for good.

Sixty days of ink give sixty days of proof: at each review you can show, commit by commit, how each criterion came to pass.

1.2 The sixty days at a glance

One day sets up the machine, fifty-two days build the thirty stories, and seven reviews close the seven iterations, the last on day 60. The delivery plan, `docs/guides/delivery-plan.md`, covers the first thirty-five days in four iterations; the second plan, which it sketches for the days after 8 December, covers the rest in three. One ***point*** is about one working day, tests included, so a story worth three points takes three days.

EXHIBIT 1.1 Seven iterations from 19 October to 18 January: the four milestones of the delivery plan, then the three reviews of the second plan



Source: `docs/guides/delivery-plan.md`, its iterations, its capacity sum and 'After 8 December'; the second plan's iterations follow days 36 to 60 of this book

As Exhibit 1.1 shows, each holiday costs a day. Monday 19 October, after the Chung Yeung Festival, is a rest day only for government staff, so the plan counts it as a working day. Each ***milestone*** is a state of the product that someone could see and test, named in a sentence:

- **M1, Friday 30 October:** 'The places are published by grant'.
- **M2, Friday 13 November:** 'The map opens in the office, on safe tiles'.
- **M3, Friday 27 November:** 'The map zooms, keeps calm and tells places apart'.
- **M4, Monday 7 December:** 'Parts A and B released to the office'.

The delivery plan has the second plan written at the review of 7 December; this book writes it on Wednesday 9 December, beside N-18, and splits it into iterations 5, 6 and 7, each closed by a review. The delivery plan names only four milestones, so these reviews check goals, and their tags name the parts they close: `review-c`, `review-d` and `review-ef`. Twenty-five working days, less three reviews, leave twenty-two days for twenty-two points: the second plan holds no day in hand. Three tables give each day, its story and the test that ends it: Exhibit 1.2 for iterations 1 and 2, Exhibit 1.3 for iterations 3 and 4, and Exhibit 1.4 for the second plan.

EXHIBIT 1.2 Days 1 to 19: the skeleton, the places, safe tiles and the map in the office

DAY	DATE	STORY	DONE WHEN
Iteration 1 · 19–30 October · 8 points · M1			
1	Mon 19 Oct	The machine	The worked answer's `npm test` ends with no error
2	Tue 20 Oct	US-00 (1 of 1)	`Names_a_missing_or_bad_database_url` and `Refuses_to_serve_and_names_each_missing_variable` pass, and CI passes
3	Wed 21 Oct	US-01 (1 of 1)	`Signs_in_as_the_shipped_reader_role` and `Keeps_the_reader_read_only` pass against the image of `db/Dockerfile`

4	Thu 22 Oct	US-03 (1 of 3)	`Quotes_a_name_and_doubles_each_inner_quote` and the property test `Quotes_every_name_safely` pass
5	Fri 23 Oct	US-03 (2 of 3)	`Turns_a_row_into_a_layer_whose_id_is_schema_dot_name`, `Publishes_neither_of_two_tables_whose_ids_clash` and `Finds_a_layer_exactly_and_case_sensitively` pass; burn-up 2 of 2
6	Mon 26 Oct	US-03 (3 of 3)	`Publishes_what_is_granted_and_nothing_else` passes, signed in as the reader
7	Tue 27 Oct	US-24 (1 of 2)	`Loads_the_shipped_seed_once` passes
8	Wed 28 Oct	US-24 (2 of 2)	`Publishes_nothing_until_the_grant` and `Undoes_a_script_that_fails` pass
9	Thu 29 Oct	US-07 (1 of 2)	`Accepts_a_tile_that_exists` and `Refuses_a_tile_outside_the_grid` pass
10	Fri 30 Oct	Review	M1: `npm test` passes from a clean start; burn-up 7 of 7; tag `m1`
Iteration 2 · 3–13 November · 8 points · M2 (Monday 2 November is All Souls' Day)			
11	Tue 3 Nov	US-07 (2 of 2)	`Serves_a_tile_as_a_mapbox_vector_tile` and `Answers_an_unknown_layer_with_a_problem_document` pass
12	Wed 4 Nov	S-01 (1 of 2)	`Matches_the_approved_tile_query` and `Keeps_every_tile_address_out_of_the_text` pass
13	Thu 5 Nov	S-01 (2 of 2)	`Answers_an_attack_with_404_and_keeps_every_row` passes: `food.cafes` keeps its 100 rows
14	Fri 6 Nov	S-06 (1 of 1)	`npm run e2e` passes the checks of Nginx; burn-up 12
15	Mon 9 Nov	US-35 (1 of 1)	The three tests of `office.test.ts` pass; a tile reaches another office machine
16	Tue 10 Nov	US-23 (1 of 3)	`Writes_the_area_block_before_the_head_closes` and `Answers_404_when_there_is_no_page_to_show` pass
17	Wed 11 Nov	US-23 (2 of 3)	'starts from one model and changes it only through the update's handlers' passes
18	Thu 12 Nov	US-23 (3 of 3)	`Serves_the_map_page_with_the_area_written_in` and `npm run e2e -- --grep "US-23"` pass; burn-up 16
19	Fri 13 Nov	Review	M2: a tile from another office machine; burn-up 16 of 16; tag `m2`

Source: `docs/guides/delivery-plan.md`; `docs/user-stories/user-stories.md`; the tests named, in `tests/`, `scripts/lib` and `web/`; the days' own chapters

EXHIBIT 1.3 Days 20 to 35: the map reads well, then clusters, names and the release

DAY	DATE	STORY	DONE WHEN
Iteration 3 · 16–27 November · 9 points · M3			
20	Mon 16 Nov	UI-09 (1 of 2)	'doubles and halves the scale one level at a time' and 'keeps the level within 0 to 22' pass
21	Tue 17 Nov	UI-09 (2 of 2)	'states the current level in each label, and nowhere on screen' passes
22	Wed 18 Nov	UI-15 (1 of 2)	'keeps the theme in styles.css' and 'draws every icon from Material Symbols' pass

23	Thu 19 Nov	UI-15 (2 of 2)	‘makes every transition and animation last 300 ms or less’ and ‘lets nothing loop’ pass
24	Fri 20 Nov	UI-12 (1 of 1)	‘gives each group its own shape’ and ‘meets 4.5:1 for text in each group colour on white’ pass; burn-up 21
25	Mon 23 Nov	UI-19 (1 of 2)	‘chooses 500 m at level 15 and 50 m at level 18’ passes
26	Tue 24 Nov	UI-19 (2 of 2)	‘names north for screen readers, as a sign rather than a button’ passes
27	Wed 25 Nov	UI-10 (1 of 3)	‘merges pins that sit closer than the distance for the level’ passes
28	Thu 26 Nov	UI-10 (2 of 3)	‘drops features that are not points or carry no id’ passes
29	Fri 27 Nov	Review	M3 shown in Chrome; burn-up 23 of 29; tag `m3`
Iteration 4 · 30 November–7 December · 4 points and a day in hand · M4			
30	Mon 30 Nov	UI-10 (3 of 3)	The ‘UI-10 opening a cluster’ tests pass; burn-up 26
31	Tue 1 Dec	UI-11 (1 of 1)	‘shows names from level 16, and none inside a cluster’ passes; burn-up 27
32	Wed 2 Dec	UI-17 (1 of 2)	‘gives the names four anchors, and lets a name give way while its pin stays’ passes
33	Thu 3 Dec	UI-17 (2 of 2)	The three ‘matches the approved image at level …’ tests pass; burn-up 29
34	Fri 4 Dec	S-07 (1 of 1), the day in hand	‘accepts an office network, and refuses one too wide or misspelt’ passes; burn-up 30
35	Mon 7 Dec	Review and release	M4: `npm test` in a clean clone; the map opens from another office machine; tag `m4`

Source: `docs/guides/delivery-plan.md`*; the tests named, in* `web/src`, `web/e2e` *and* `scripts/lib`

EXHIBIT 1.4 Days 36 to 60: the second plan, Parts C to F, and its three reviews

DAY	DATE	STORY	DONE WHEN
Iteration 5 · 9–18 December · 6 points, and N-22 begun			
36	Wed 9 Dec	The second plan; N-18 (1 of 1)	`docs/second-plan.md` is committed, and `Tells_the_browser_and_nginx_how_long_to_keep_a_tile` passes; burn-up 31
37	Thu 10 Dec	N-19 (1 of 2)	`Tags_a_tile_by_its_bytes` passes, in the unit and the integration tests
38	Fri 11 Dec	N-19 (2 of 2)	`Answers_a_matching_tag_with_304` passes, and `npm run e2e` reads `X-Cache-Status: HIT`; burn-up 33
39	Mon 14 Dec	N-20 (1 of 1)	`npm run e2e` unpacks a packed tile; burn-up 34
40	Tue 15 Dec	N-21 (1 of 2)	`Reads_the_shipped_file` and `Refuses_a_key_it_does_not_know` pass
41	Wed 16 Dec	N-21 (2 of 2)	`Lets_a_variable_override_a_key` passes, and your copy moves to port 7800 behind Nginx; burn-up 36

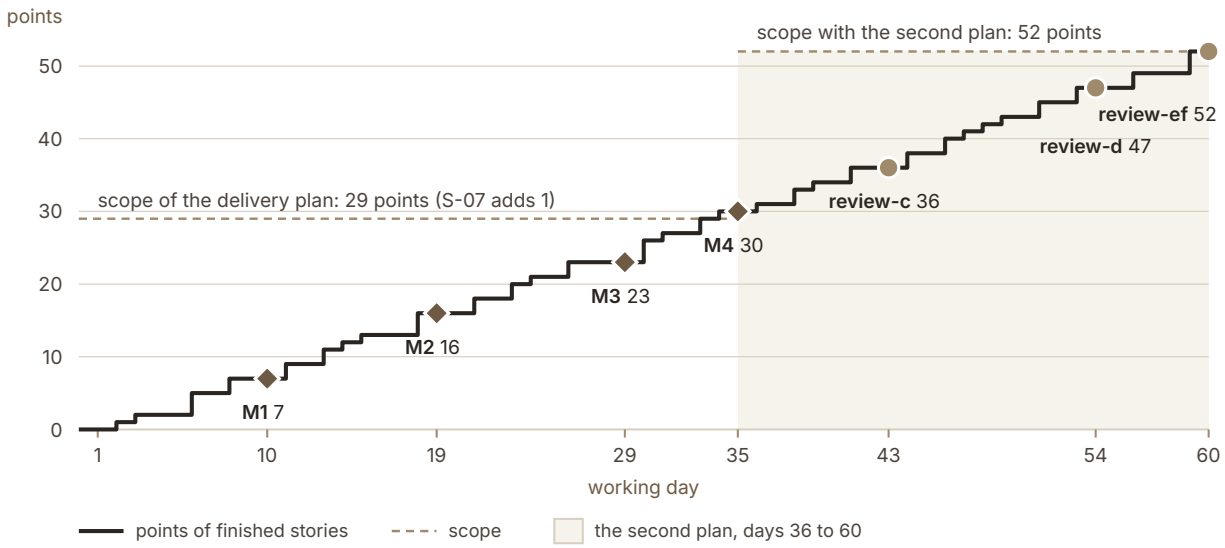
42	Thu 17 Dec	N-22 (1 of 2)	`Asks_the_browser_to_wait_for_a_failure_that_may_pass` and `Sorts_any_other_failure_into_its_kind` pass
43	Fri 18 Dec	First review	Burn-up 36 of 52; tag `review-c`
Iteration 6 · 21 December–8 January · 11 points (22, 24 and 25 December and 1 January are off)			
44	Mon 21 Dec	N-22 (2 of 2)	`Answers_503_when_the_database_cannot_be_reached` and `Answers_503_when_a_query_runs_past_its_time` pass; burn-up 38
45	Wed 23 Dec	N-23 (1 of 2)	`Holds_no_list_until_the_first_load_succeeds` and `Keeps_the_last_good_list_through_a_failure` pass
46	Mon 28 Dec	N-23 (2 of 2)	`Sees_a_grant_and_a_revoke_within_one_refresh` passes, with the tile half of `Starts_with_the_database_down_and_answers_503`; burn-up 40
47	Tue 29 Dec	N-24 (1 of 1)	`npm run e2e` passes its crowd check; burn-up 41
48	Wed 30 Dec	N-25 (1 of 1)	`Says_up_when_the_service_can_serve_tiles` and `Reports_degraded_until_the_first_list_loads` pass, with `Starts_with_the_database_down_and_answers_503` whole; burn-up 42
49	Thu 31 Dec	N-26 (1 of 1)	The three tests of `PoolTests` pass; burn-up 43
50	Mon 4 Jan	N-28 (1 of 2)	A kept tile arrives with the service stopped
51	Tue 5 Jan	N-28 (2 of 2)	`Starts_with_the_database_down_and_answers_503` passes with `/health` held to one second; burn-up 45
52	Wed 6 Jan	S-30 (1 of 2)	A flood of 600 meets 429, and another address still gets 200
53	Thu 7 Jan	S-30 (2 of 2)	Forty requests at once leave at least eight answered 429; burn-up 47
54	Fri 8 Jan	Second review	Burn-up 47 of 52; tag `review-d`
Iteration 7 · 11–18 January · 5 points			
55	Mon 11 Jan	N-29 (1 of 2)	The three tests of `supervise.test.ts` pass
56	Tue 12 Jan	N-29 (2 of 2)	With one copy stopped, six fresh tiles answer 200 and `/health` says `UP`; burn-up 49
57	Wed 13 Jan	US-36 (1 of 3)	The shape filters' tests in `test_filters.py` pass
58	Thu 14 Jan	US-36 (2 of 3)	`test_stops_a_table_that_drops_more_than_ten_per_cent_by_default` passes
59	Fri 15 Jan	US-36 (3 of 3)	`test_cleans_a_geodatabase_into_one_script_and_logs_each_row` passes; burn-up 52
60	Mon 18 Jan	Third review	All thirty stories, 52 points, in your copy; tag `review-ef`

Source: `docs/guides/delivery-plan.md`, 'After 8 December'; the tests named, in `tests/`, `scripts/lib` and `etl/tests`; the days' own chapters

The last day of each iteration is a ***review***, and it takes the whole day: run every test from a clean start, check the milestone, read the week's notes and plan the next iteration. Iteration 1 has eight building days for eight points, so a day that runs late takes time from the next, and the review on day 10 sorts out the delay. Only iteration 4 holds a spare day, Friday 4 December, which builds S-07 if nothing has slipped.

A ***burn-up*** chart sets the points of finished stories against the scope. Only a finished story counts: a story at 90 per cent counts as nothing, because half a story cannot ship. Exhibit 1.5 draws the count this book plans. Keep the same chart in your notes and add your own line every Friday; a gap of one day uses the day in hand, and a gap of two at a review calls for the delivery plan's first cut, S-07.

EXHIBIT 1.5 The planned burn-up: 30 points at the release of 7 December, 52 at the third review



Source: `docs/guides/delivery-plan.md`, *'Tracking progress with a burn-up' (weeks 1 to 8); days 36 to 60 follow the tables above*

KEY IDEA The stories follow the build order of `docs/user-stories/user-stories.md`: each builds only on stories above it. So you can always build the next story with what you have, and a story cut at a review leaves no gap behind it.

WATCH OUT Four days meet a known fault in the worked answer, named as *The World in Squares* names it, and each has you write the missing test, beyond the worked answer. Day 14: **a disguised path reaches a route.** Kestrel decodes `%2e%2e` and removes the dot segment before routing, so `/x/%2e%2e/food.cafes/0/0/0.pbf` reaches the tile route, and S-06 c3 holds only for the target `checkEdge` sends. Day 41: **a misspelt key stops the start alone.** The binder names it and nothing else, so N-21 c4's 'every such problem at once' holds for values, not for a misspelt key. Day 44: **no log on a 500.** `TileServer.Answer` answers `DatabaseError.Other` with a 500 and logs nothing, though `Problems.Fault` 's comment says 'the log holds the cause'. Day 49: **N-26's known limit names the wrong signal.** It says a tile query runs on the service's stop signal; the code hands it the request's own `CancellationToken` .

1.3 Two folders, never mixed

Keep the worked answer and your copy in two folders, and type only in the second. The worked answer, `~/src/tessera`, holds the whole of Phase Oct, with about 500 tests: 106 for the service and 253 for the pipeline. Read it and run it, but never change it. Your copy, `~/src/tessera-rebuild`, starts empty on day 2 and takes every line you write.

EXHIBIT 1.6 The two folders differ in purpose, place, database and ports

	THE WORKED ANSWER	YOUR COPY
Folder	`~/src/tessera`	`~/src/tessera-rebuild`
GitHub	`nathanwh71/Tessera`, read only	A private repository of your own, `tessera-rebuild`
Database you start by hand	`tessera-db`, through `npm start`, on 127.0.0.1, port 5432	`rebuild-db`, through `docker run`, on 127.0.0.1, port 5434
Database the tests start	The image `tessera-db-test`, built from `db/Dockerfile` by Testcontainers, on a free port	The same, from your own `db/Dockerfile`
The service	Two copies on ports 7800 and 7801, behind Nginx on port 80	Port 8080 until day 41 (N-21); then 7800, with a second copy on 7801 from N-29, behind Nginx
The rule	Read it; never change it	Every line you type goes here

Source: `scripts/start.ts`; `scripts/lib/plan.ts`; `.env.example`; `tests/Tessera.IntegrationTests/DatabaseFixture.cs`

As Exhibit 1.6 shows, the two hand-started databases have different names and ports, so both can run at once. The tests start a fresh database each time and throw it away after, so neither can hide a mistake in your scripts.

> WATCH OUT Typing in the wrong folder is the easiest mistake of the sixty days. Before you change a file, check the path at the top of your editor: it must contain `tessera-rebuild`.

1.4 The shape of every day

Every day follows the same five steps, with building taking most of the day, as Exhibit 1.7 shows; the times are a guide, not a limit.

EXHIBIT 1.7 Five steps, with building in the middle

STEP	TIME	WHAT YOU DO
1 Start	15 minutes	Read yesterday's changes as a reviewer would; check that nothing is left to commit
2 Read	No more than 45 minutes	Read the story and the files of the worked answer that the day names, until you can name your first failing test
3 Build	Most of the day	Red, green, tidy; commit each time every test passes
4 Compare	30 minutes	Your files beside the worked answer's; a short note of what is different
5 Close	15 minutes	Every test in the terminal; push; write tomorrow's first test in your notes

Source: `docs/guides/delivery-plan.md`, *'The weekly and daily rhythm'; the times are this book's*

1.4.1 Start

Open an Ubuntu terminal in `~/src/tessera-rebuild` and read yesterday's work as a reviewer would: `git status -sb` should show nothing to commit, in step with `origin/main`, and `git log -p -3` shows the last three commits with their changes. If they show a mistake, fix it first, in a commit of its own.

1.4.2 Read

Each day's table names the story, the files to read and what to look for. Read the story's page in *The World in Squares* first, then the worked answer's tests before its code, because the tests say what the code must do. Stop reading when you can name the first test you will write.

1.4.3 Build

For each criterion, follow the ***test-first loop***:

1. Write the test and run it. Check that it fails for the reason you expect (**red**).
2. Write the least code that makes it pass (**green**).
3. Tidy the code while the tests still pass, then commit.

The .NET SDK runs the service's tests one project, one class or one test at a time:

```bash
dotnet test --project tests/Tessera.UnitTests          # one test project
dotnet test --project tests/Tessera.UnitTests --filter-class Tessera.UnitTests.SqlTests
dotnet test --project tests/Tessera.UnitTests \
  --filter-method Tessera.UnitTests.SqlTests.Quotes_a_name_and_doubles_each_inner_quote
```

`--project` names the test project, because `global.json` chooses Microsoft.Testing.Platform as the test runner; `--filter-class` and `--filter-method` take the full name of a class or a test. The worked answer keeps the service's tests in three .NET test projects: `Tessera.ArchitectureTests`, six rules that guard the borders between the service's layers; `Tessera.UnitTests`, 60 tests of pure functions; and `Tessera.IntegrationTests`, 40 tests that start the packaged database in Docker. Your copy makes the second and third on day 2, and the first on day 12. The worked answer measures no coverage, so no coverage floor stops a run.

> KEY IDEA Red comes first for a reason. A test you have never seen fail might pass for the wrong reason, and then it proves nothing.

1.4.4 Compare

When your version passes, set it beside the worked answer's:

```bash
git diff --no-index --stat ~/src/tessera/src ~/src/tessera-rebuild/src
git diff --no-index ~/src/tessera/src/Tessera/Modules/Tessera/Domain/SqlName.cs \
  ~/src/tessera-rebuild/src/Tessera/Modules/Tessera/Domain/SqlName.cs
```

The first line lists which files differ, and by how much; the second compares one pair, line by line. Expect many differences in the first weeks: the worked answer holds all thirty stories, your copy only a few. Read the differences in today's files, then write a note of three lines: what matched, what differed, and why. Appendix A lists the planned differences you will meet.

1.4.5 Close

Run `npm test`, every check your copy has so far, then `git push`, and write tomorrow's first test in your notes, outside the repository.

2

Day 1 · Monday 19 October: the machine

> IN SHORT Today you set up a machine on which every later command works, and prove it by running every test of the worked answer. You write no code; the day carries no story and no points.

The day has three parts, in time order. You install the tools in a fixed order, check that each answers with the right version, then copy the worked answer from GitHub and run all its tests. Exhibit 2.1 sets out the day.

EXHIBIT 2.1 Day 1 ends when the worked answer's tests all pass on your machine

Goal	A machine that runs all of the worked answer's tests
Story	None: the set-up day, which the plan counts apart from the stories
In the worked answer	`README.md`, 'What you need' and 'The commands'; `global.json`; `scripts/test.ts`; `.github/workflows/build.yml`
Look for	The versions the build needs; the order in which `npm test` runs its checks; the steps CI takes on a new machine
Leave for later	`.env` and `npm start`: the worked answer's on day 10, your copy's on day 34; Nginx and port 80 (days 14 and 15)
Done when	`npm test` ends with no error, and `echo $?` prints `0`

Source: `README.md`; `docs/guides/delivery-plan.md`, *'Monday 19: the machine'*

2.1 Install the tools in order

Each tool needs the one before it, so the order matters. Appendix F of *The World in Squares*, session 1, gives every command; tick each line here when its check passes.

1. **WSL2 and Ubuntu 24.04.** In PowerShell, as administrator, install WSL with Ubuntu 24.04; restart Windows, open Ubuntu and choose a user name and password. Set `networkingMode=mirrored` in `%UserProfile%\.wslconfig` and `systemd=true` in Ubuntu's `/etc/wsl.conf` now: the office needs the first (US-35), and the second starts Docker each time Ubuntu starts.

```text
wsl --install -d Ubuntu-24.04
```

2. **Docker Engine inside Ubuntu.** Install it from Docker's own package repository, not as Docker Desktop. Add yourself to the `docker` group, so you and the tests can use Docker without `sudo`, and let systemd start it:

```bash
sudo usermod -aG docker "$USER"
sudo systemctl enable --now docker
```

3. **The .NET 10 SDK**, the Software Development Kit: the compiler, the `dotnet` command and its project templates. Microsoft's script `dotnet-install.sh --channel 10.0` puts it in `~/.dotnet` with no administrator's rights; add that folder to `PATH` and set `DOTNET_ROOT` in `~/.bashrc`.
4. **Node.js 26.** Both `package.json` files ask for `"node": ">=26"`. Install it through nvm with `nvm install 26`.
5. **uv**, Python's project tool, which installs the pipeline's Python and its packages, and **GDAL's tools** from Ubuntu, as CI does: `sudo apt-get install gdal-bin`.
6. **Git**, with your name and email address, and `main` as the first branch of each new repository: `git config --global init.defaultBranch main`.
7. **Your editor:** Visual Studio Code on Windows, with its WSL extension, opened on folders inside Ubuntu, and the C# Dev Kit installed inside Ubuntu from the Extensions view.

> FROM SCHOOL An **operating system** manages the hardware and runs programs. WSL2 runs a second operating system, Linux, inside Windows, in a light ***virtual machine***: a computer made of software. Docker goes one step further. It runs each program in a ***container***, a closed box with its own files, so the database on your machine matches the database in the office.

2.2 Check every version

Each tool must answer with the right version before you go on.

BASH

```
dotnet --version          # 10.0.…
node --version            # v26.…
uv --version
ogr2ogr --version         # GDAL 3.…
docker --version
docker run --rm hello-world
```

`dotnet --version` answers for the folder you stand in. A `global.json` file in that folder, or any folder above it, names the SDK the build needs; the worked answer's asks for 10.0.100, and its `"rollForward": "latestFeature"` lets a newer release of .NET 10 stand in, never .NET 11. The same file names Microsoft.Testing.Platform as the test runner, the one xUnit v3 runs on.

> WATCH OUT A machine can hold several SDKs at once. If the worked answer's folder finds only an older one, every `dotnet` command there stops at once with `A compatible .NET SDK was not found.` and `Requested SDK version: 10.0.100`. Run `dotnet --list-sdks`; if no line starts with 10.0, install the SDK again and open a new terminal.

2.3 Run the worked answer

Copy the worked answer from GitHub, install its tools, and run every test:

BASH

```
mkdir -p ~/src && cd ~/src
git clone https://github.com/nathanwh71/Tessera.git tessera
cd tessera
npm install                                          # zx, TypeScript and Node's types
(cd web && npm ci && npx playwright install --with-deps chromium)
npm test
```

Keep both folders in Ubuntu's own files, never under `/mnt/c`, where every build would run slowly across Windows' file sharing. The tests need no settings from you: the integration tests build the database image from `db/Dockerfile`, start a fresh copy for each test class, with passwords written in the tests, and remove it when they finish. `.env.example` and the `.env` copied from it serve `npm start`, which runs the service for the office; you first use them in the worked answer on day 10, and your copy makes its own `.env` on day 34.

> FROM SCHOOL `127.0.0.1` is the ***loopback address***: every computer uses it to mean 'this computer'. A ***port*** is the number that picks one program among the many that listen on a machine. The worked answer's `tessera.json` sets `"Urls": "http://127.0.0.1:7800"`, so the service answers only this machine, on port 7800, and Nginx later carries the office's requests to it.

> TRY IT Once `npm test` has built the program, start it with no settings:
>
> ```bash
> dotnet src/Tessera/bin/Release/net10.0/Tessera.dll; echo "exit code $?"
> ```
>
> It prints one line, `DataAnnotation validation failed for 'DatabaseConnection' members: '' with the error: 'DATABASE_URL: missing or empty'.`, and the shell prints `exit code 1`. The service refuses to start without its database's address, and names the variable that should hold it. Tomorrow you build that refusal yourself.

2.4 The proof

`npm test` runs `scripts/test.ts`, which does all of this, in order:

1. Checks the scripts' types with `tsc`, and runs the scripts' own tests with `node --test`.
2. Runs the pipeline's four checks in `etl/` through uv: pyright, ruff, lint-imports and pytest.
3. Runs the map's type check, lint and unit tests in `web/`.
4. Builds the map, then the solution in Release, with the map copied inside the service.
5. Runs the three .NET test projects with `dotnet run`: the architecture tests, the unit tests and the integration tests, 6, 60 and 40 tests.
6. Runs the browser tests and the checks of Nginx in `scripts/e2e.ts`.

The first run is slow, because it downloads every package and builds every image. The script stops at the first check that fails, with that check's error and a non-zero exit code. zx keeps each command's output to itself unless the command fails, so a run that reaches the end prints no closing line. Ask the shell:

```bash
npm test; echo "exit code $?"     # exit code 0
```

Nothing is committed today; your copy starts tomorrow. Write tomorrow's first test in your notes: `Answers_a_request_on_its_port`.

3

Day 2 · Tuesday 20 October: US-00, the walking skeleton

> IN SHORT Today you turn an empty folder into a solution that builds with one command, tests with another, answers a request on its port, and refuses to start without its database's address. This is US-00, worth 1 point.

US-00 is the *walking skeleton*: the thinnest program that works from end to end. You build it in five parts: the solution and its shared rules; a service that answers a request; a start that names the missing database; one command to build and one to test; and the first push, where CI checks your work. Exhibit 3.1 sets out the day.

EXHIBIT 3.1 Day 2 ends with seven passing tests, a built program and a first green run on GitHub

Goal	A service that answers on its port; a start that names a missing or malformed database address; one command to build and one to test
Story	US-00 c1 to c4, in `docs/user-stories/user-stories.md`
In the worked answer	`global.json`; `Directory.Build.props`; `Directory.Packages.props`; the three `.editorconfig` files; `src/Tessera/Tessera.csproj`, `Program.cs`, `TesseraHost.cs` and `Modules/Tessera/Infrastructure/Sql/Connection.cs`; `tests/Tessera.IntegrationTests/StartTests.cs`; `tests/Tessera.UnitTests/SettingsTests.cs`; `scripts/build.ts` and `scripts/test.ts`; `.github/workflows/build.yml`
Look for	Why every warning stops the build; why `StartTests` runs the built program in a process of its own; why `ConnectionString` has a private constructor
Leave for later	The database (day 3); the `seed` command (day 7); the run-command test of a tile (day 11); the architecture tests (S-01, day 12); the map (day 16); `tessera.json` (N-21, day 40)
Done when	`Names_a_missing_or_bad_database_url` and `Refuses_to_serve_and_names_each_missing_variable` pass, with `Answers_a_request_on_its_port`; `npm run build` makes `Tessera.dll`; the first CI run passes

Source: `docs/user-stories/user-stories.md`, *US-00;* `docs/requirements/functional-requirements.md`, *FR-01 to FR-03;* `docs/requirements/security-requirements.md`, *SR-25*

3.1 Make the solution from templates

A new project needs a folder under version control, a list of files to ignore, and a ***solution***: the file that lists every project the build compiles. The SDK's templates make the start:

```bash
mkdir -p ~/src/tessera-rebuild && cd ~/src/tessera-rebuild && git init -b main
cp ~/src/tessera/.gitignore .
dotnet new globaljson --sdk-version 10.0.100 --roll-forward latestFeature \
  --test-runner Microsoft.Testing.Platform
dotnet new sln -n Tessera                              # writes Tessera.slnx
dotnet new web -o src/Tessera                          # ASP.NET Core Empty: the service
dotnet new xunit -o tests/Tessera.UnitTests            # xUnit Test Project
dotnet new xunit -o tests/Tessera.IntegrationTests
dotnet sln add src/Tessera tests/Tessera.UnitTests tests/Tessera.IntegrationTests
```

Copy `.gitignore` before your first commit: it names the build's output, npm's packages and the file of secrets, `.env`, and once a password is in the record, ink keeps it there. `globaljson` writes the worked answer's `global.json`, key for key. Then delete what Tessera does not use: `src/Tessera/Properties/`, whose `launchSettings.json` would pick a port of its own; the two `appsettings*.json` files, since your copy reads the environment alone until N-21 brings `tessera.json`; and each `UnitTest1.cs`.

3.1.1 Every warning stops the build

The build is the first test: the worked answer turns every warning of the compiler and its analysers into an error, so a fault the compiler can see never reaches a test. Copy `Directory.Build.props` and the three `.editorconfig` files (the root's, `tests/`'s and `src/Tessera/`'s) from the worked answer to the same places. MSBuild reads a `Directory.Build.props` into every project below its folder, so `net10.0`, nullable checking, `TreatWarningsAsErrors`, `AnalysisMode All` and central package versions reach all three projects; each `.editorconfig` switches off, with its reason, the few rules that clash, such as the one against underscores in a test's name. Delete the `TargetFramework`, `Nullable` and `ImplicitUsings` lines from each project file. *Sixteen Tiles*, chapter 10, explains the build's rules in full.

Write `Directory.Packages.props` with today's four packages, at the worked answer's versions: `Npgsql` 10.0.3, `CSharpFunctionalExtensions` 3.7.0, `Meziantou.Analyzer` 2.0.295 and `xunit.v3` 4.0.2. `src/Tessera/Tessera.csproj` takes the worked answer's `AssemblyName`, `Tessera`; an empty `RootNamespace`, so each folder's path becomes its ***namespace***; `PackageReference` lines for `Npgsql` and `CSharpFunctionalExtensions`, with no version; and two `InternalsVisibleTo` lines for the test projects. In both test projects, replace the template's four package lines (xUnit 2.9.3 and Microsoft's older runner) with the worked answer's: `<OutputType>Exe</OutputType>`, since xUnit v3 builds each test project as a program; `xunit.v3`; a `ProjectReference` to `../../src/Tessera/Tessera.csproj`; and `<Using Include="CSharpFunctionalExtensions" />`.

3.2 The skeleton answers

The first criterion worth a test is the second: started by the run command, the service answers an HTTP request on its port. In `tests/Tessera.IntegrationTests/StartTests.cs`, copy the worked answer's two helpers: `Start` describes a process that runs the built `Tessera.dll` with an empty environment and only the variables a test gives it, and `Run` starts it and waits for it to end. Add a helper that finds a free port, as the worked answer's `Http.FreePort` does, and the test:

CSHARP

```csharp
// StartTests.cs, in your copy: US-00 c2, beyond the worked answer
[Fact]
public async Task Answers_a_request_on_its_port()
{
    var port = FreePort();
    using var program = Process.Start(Start(new Dictionary<string, string>
    {
        ["Urls"] = $"http://127.0.0.1:{port}",
        ["DATABASE_URL"] = "postgresql://tessera_reader:x@127.0.0.1:1/tessera",
    }))!;
    // … in a try: ask for "/" until the program listens, then
    Assert.Equal(HttpStatusCode.NotFound, answer.StatusCode);
    // … and finally: program.Kill(entireProcessTree: true)
}
```

`Urls` is the variable ASP.NET Core reads for the address and port to listen on. The database address leads to port 1, where nothing listens; the skeleton never asks the database. Run `dotnet test --project tests/Tessera.IntegrationTests`: it fails with `Expected: NotFound`, `Actual: OK`, since the template's `Program.cs` answers `/` with `Hello World!`.

> NOTE **Beyond the worked answer.** The worked answer proves US-00 c2 with `Starts_from_the_run_command_and_answers`, in `ServiceTests`, which asks the running program for a tile and expects 200. That needs a database, a seed and a tile route, so it waits for day 11; today you write a smaller test of your own. A 404 still proves the point: a program listens on the port and answers in HTTP.

Green splits the template's four lines into the worked answer's two files. `TesseraHost.Build` makes the web service, and lets a test lay settings of its own over the environment; `Program.cs` chooses what to do from its arguments, with a `switch` over ***list patterns***, where `[]` matches no arguments and `_` anything else:

CSHARP

```csharp
// src/Tessera/TesseraHost.cs, in your copy (first form)
public static WebApplication Build(IReadOnlyDictionary<string, string?>? overrides = null)
{
    var builder = WebApplication.CreateBuilder();
    if (overrides is not null)
        builder.Configuration.AddInMemoryCollection(overrides);
    return builder.Build();
}

// src/Tessera/Program.cs, in your copy (first form)
return args switch
{
    [] => await Serve().ConfigureAwait(false),   // TesseraHost.Build(), then RunAsync
    _ => Usage(),                                  // prints the one command; returns 2
};
```

The test passes with a 404, since the skeleton has no routes. The worked answer's `switch` has a third line, `["seed"] => Seed()`, which day 7 adds.

3.3 Machines come from the environment

The service must read its database's address from the environment and stop at once without it, naming what is missing. Criterion 4 names one ***environment variable***, `DATABASE_URL`, a value the operating system

hands the program as it starts, such as `postgresql://tessera_reader:secret@127.0.0.1:5432/tessera`. Copy the worked answer's `Refuses_to_serve_and_names_each_missing_variable` into `StartTests`: it calls `Run()` with no variables and expects exit code 1 and `DATABASE_URL` in the error output. Run it, and the run never ends: the program listens on ASP.NET Core's own default port, 5000, while the test waits for it to stop. Press Ctrl+C. A test that hangs is red too.

Copy `src/Tessera/Modules/Tessera/Infrastructure/Sql/Connection.cs` from the worked answer, whole. Its first type, `ConnectionString`, turns `postgresql://user:password@host:port/database` into the text Npgsql, the database's client, reads:

CSHARP

```csharp
// src/Tessera/Modules/Tessera/Infrastructure/Sql/Connection.cs (shortened)
public sealed record ConnectionString
{
    private ConnectionString(string text) => Text = text;
    public string Text { get; }
    public static Result<ConnectionString, string> FromUrl(string? url)
    {
        if (string.IsNullOrEmpty(url))
            return Result.Failure<ConnectionString, string>("missing or empty");
        if (!Uri.TryCreate(url, UriKind.Absolute, out var uri)
            || uri.Scheme is not ("postgresql" or "postgres")
            || uri.UserInfo.Split(':', 2) is not [{ Length: > 0 } user, var password]
            || uri.AbsolutePath.Length < 2)
            return Result.Failure<ConnectionString, string>(
                "expected postgresql://user:password@host:port/database");
        // … an NpgsqlConnectionStringBuilder filled from the URL's five parts
    }
}
```

`FromUrl` returns a `Result`, from the CSharpFunctionalExtensions library: either the connection string, or the reason it could not make one. This is the first `Result` in your copy, and day 4 shows how results join into a chain. The constructor is private, so holding a `ConnectionString` proves the URL was well formed: the worked answer calls this rule 'parse, don't validate'. The other two types hand the parsed URL to ASP.NET Core's settings: `DatabaseConnection` refuses a failed result with a message that names its variable, and `DatabaseConnections.Register` reads the variable and asks for the check to run at start-up. In `TesseraHost.Build`, call `DatabaseConnections.Register(builder.Services, builder.Configuration, DatabaseConnections.ReaderDatabase)`, and after `builder.Build()` run every start-up check with `app.Services.GetRequiredService<IStartupValidator>().Validate()`. In `Serve`, catch `OptionsValidationException` around `Build`, write each line of its `Failures` to the error output, and return 1. Both tests pass, and the built program, given `DATABASE_URL=http://db/tessera`, prints:

TEXT

```
DataAnnotation validation failed for 'DatabaseConnection' members: '' with the error:
'DATABASE_URL: expected postgresql://user:password@host:port/database'.
```

FROM SCHOOL Every program ends with an ***exit code***, a whole number it hands back to the operating system. 0 means success; any other number names a kind of failure. Tessera exits with 1 when a setting is missing or wrong, and with 2 when it meets a command it does not know, so a script that starts it can tell the two apart.

3.3.1 The same rule, without a process

A test that starts a program takes seconds; a unit test of the same rule takes a moment. In `tests/Tessera.UnitTests/SettingsTests.cs`, write the worked answer's two tests of the URL. `Names_a_missing_or_bad_database_url` is a theory, run once for each of its three `InlineData` rows: `null` and `""` must name 'missing or empty', and `http://db/tessera` 'expected postgresql://', each beside `DATABASE_URL`. Your helper `Problems(url)` registers `DATABASE_URL` alone and returns the joined `Failures`; day 36 swaps in the worked answer's helpers. `Reads_the_database_url_into_a_connection_string` gives `FromUrl` a password holding `%40`, an encoded `@`, and the port 5433, and expects each part read, not assumed.

> NOTE **Beyond the worked answer.** `docs/requirements/security-requirements.md` marks SR-25 'in part': no test checks that the code names no machine. Write `Names_no_machine_in_the_code` in `SettingsTests`. It walks up from the test's folder to the one that holds `Tessera.slnx`, reads every `.cs` file under `src/`, and asserts that none holds `localhost` or `127.0.0.1`. Plant `// http://127.0.0.1:8080` in `TesseraHost.cs` and watch it fail; take the line out.

3.4 One command to build, one to test

Criteria 1 and 3 ask for one build command and one test command. The worked answer writes both as short TypeScript scripts that zx runs, named in `package.json`. Run `npm init -y`, give `package.json` the worked answer's `"type"` and `"engines"`, install its exact versions with `npm install --save-dev --save-exact zx@8.8.5 typescript@6.0.3 @types/node@26.6.5`, and copy its `tsconfig.json`. Your `scripts/build.ts` holds the worked answer's last line alone, `await $\`dotnet build --configuration Release\`;`, since you have no map yet. Your `scripts/test.ts` runs the quick check first and the slow ones after: `npx tsc --noEmit`, which checks the scripts' types, since Node.js 26 runs a `.ts` file by stripping its types and checks nothing; then `node scripts/build.ts`; then, for each test project, `dotnet run --no-build --configuration Release --project tests/Tessera.${suite}`. Each test project is a program, so `dotnet run` starts it and it runs its own tests; `--no-build` reuses the Release build that `build.ts` made a moment before. Add `"build"`, `"test"` and `"typecheck"` to the scripts in `package.json`, then run `npm run build` and `npm test`. Until N-21 moves your copy to the worked answer's port 7800, the variable `Urls` sets its port, 8080: started with `Urls=http://127.0.0.1:8080` and any well-formed `DATABASE_URL`, the built `Tessera.dll` prints `Now listening on: http://127.0.0.1:8080`.

> IN INK Commit each time the tests pass, with messages such as `US-00: refuse to start without DATABASE_URL`. Create an empty private repository, `tessera-rebuild`, on GitHub; copy the worked answer's `.github/workflows/build.yml`, keeping the steps your copy can run today (`setup-dotnet` with `global.json`, Node.js 26, the Release build, the two test projects, `npm ci` and `npm run typecheck`); then add the remote and `git push -u origin main`.

> THE PROOF Seven tests pass: five unit tests (one URL read, three refused, no machine named) and two integration tests. `npm run build` writes `Tessera.dll`, `npm test` ends with exit code 0, and the first CI run passes. Tomorrow's first test: `Signs_in_as_the_shipped_reader_role`.

4

Day 3 · Wednesday 21 October: US-01, one database for the service and its tests

> IN SHORT Today you package one spatial database with the service's role, and make the tests start that same image: a test against a role made by hand proves nothing about the shipped one. This is US-01, worth 1 point.

The day has four parts, in time order. You build the database image and watch it lack a role; write the roles script; make the tests build and start the image, and prove a test can fail; and carry the repository's seed, with a test of its fingerprint. Exhibit 4.1 sets out the day.

EXHIBIT 4.1 Day 3 ends when the tests prove the shipped role exists and may only read

Goal	An image that makes the two roles, started by the tests themselves
Story	US-01 c1, c2 and c3
In the worked answer	`db/Dockerfile`; `db/initdb/20-roles.sql`; `tests/Tessera.IntegrationTests/DatabaseFixture.cs` and `DatabaseTests.cs`; `db/seed/0001-macau-places.sql`; `data/macau-pois.geojson`
Look for	Why the roles script reads its passwords with psql's `\getenv`; why `tessera_reader` sets `default_transaction_read_only`; why the fixture builds the image rather than pulling PostGIS
Leave for later	The catalogue (days 4 to 6); the seed run and the grants (US-24, days 7 and 8); the browser tests, which read the shipped seed (US-23); `npm start` (day 34)
Done when	`Signs_in_as_the_shipped_reader_role` and `Keeps_the_reader_read_only` pass against the image of `db/Dockerfile`, with `Ships_the_seed_made_from_the_places`, here and in CI

Source: `docs/user-stories/user-stories.md`, *US-01;* `docs/requirements/functional-requirements.md`, *FR-04 and FR-05;* `docs/requirements/security-requirements.md`, *SR-01*

> FROM SCHOOL A network can give each user an access level, and PostgreSQL does the same with roles. A ***role*** is a name and a password that can sign in, with a list of what it may do. The service signs in as `tessera_reader`, which may only read; the seed as `tessera_publisher`, which owns the places' tables.

4.1 Build the image, and watch the role fail

An ***image*** is a packed set of files from which Docker starts a container. Yours starts from PostGIS, which is PostgreSQL with maps added, and runs your scripts on its first start.

Copy the worked answer's `db/Dockerfile`, with an empty `db/initdb/` beside it. It starts `FROM postgis/postgis:18-3.6`, so every machine and every test meets PostgreSQL 18 with PostGIS 3.6; copies `initdb/` to `/docker-entrypoint-initdb.d/`, whose scripts the image runs once, on a new, empty data

folder; and starts `postgres` with settings tuned for the office machine. Build the image, start it under its own name and port, apart from the worked answer's, and try to sign in as the reader:

```bash
docker build --tag tessera-rebuild-db db
docker run --rm --detach --name rebuild-db --publish 127.0.0.1:5434:5432 \
  --env POSTGRES_DB=tessera --env POSTGRES_PASSWORD=owner-local \
  --env TESSERA_READER_PASSWORD=reader-local --env TESSERA_PUBLISHER_PASSWORD=publisher-local \
  tessera-rebuild-db
docker logs --follow rebuild-db   # wait for the second 'ready to accept connections'; Ctrl+C
docker exec rebuild-db psql -U tessera_reader -d tessera -c 'CREATE TABLE public.mine (id int)'
```

```text
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed:
FATAL:  role "tessera_reader" does not exist
```

That is today's first red, by hand. `--publish 127.0.0.1:5434:5432` carries this machine's port 5434 to the container's 5432, and to no other machine. `POSTGRES_PASSWORD` belongs to the owner, `postgres`; the other two wait for the script you write next. Your copy keeps `rebuild-db` on port 5434 to the end, so it never meets the worked answer's `tessera-db` on 5432.

4.2 Write the roles script

One script makes both roles, reads their passwords from the environment, and locks the reader to reading. Copy it from the worked answer, reading each line as you type:

```sql
-- db/initdb/20-roles.sql (shortened)
\getenv reader_password TESSERA_READER_PASSWORD
\getenv publisher_password TESSERA_PUBLISHER_PASSWORD

\if :{?reader_password}
\else
  DO $$ BEGIN RAISE EXCEPTION
    'Set TESSERA_READER_PASSWORD before the database first starts.'; END $$;
\endif
-- … the same check for the publisher's password

CREATE ROLE tessera_reader LOGIN PASSWORD :'reader_password';
-- A second lock beside the grants: even a table granted for writing by mistake stays
-- read-only.
ALTER ROLE tessera_reader SET default_transaction_read_only = on;

CREATE ROLE tessera_publisher LOGIN PASSWORD :'publisher_password';
-- … CONNECT on the database for both roles, and CREATE for the publisher
```

`\getenv` copies an environment variable into a psql variable, so no password ever sits in the file, and `\if :{?reader_password}` asks whether it was set: a missing password stops the first start with a sentence that names it. The image runs its scripts only on a first start, on an empty data folder, so stop the container, which `--rm` then removes, build and run again, and repeat the last command. It fails again, now for the right reason:

```text
ERROR:  cannot execute CREATE TABLE in a read-only transaction
```

4.3 Let the tests start the image

The tests must use the image you ship, built fresh, so no change made by hand can hide a fault. Add `Testcontainers.PostgreSql` 4.16.0 to `Directory.Packages.props`, and reference it and `Npgsql` in the integration tests' project file. Then write `tests/Tessera.IntegrationTests/DatabaseFixture.cs`, whose head copies the worked answer's:

CSHARP

```
// tests/Tessera.IntegrationTests/DatabaseFixture.cs (shortened)
private static readonly Lazy<Task<IFutureDockerImage>> Image = new(async () =>
{
    var image = new ImageFromDockerfileBuilder()
        .WithDockerfileDirectory(CommonDirectoryPath.GetGitDirectory(), "db")
        .WithName("tessera-rebuild-db-test:latest")
        .WithCleanUp(false)
        .Build();
    await image.CreateAsync().ConfigureAwait(false);
    return image;
});
// … InitializeAsync: a PostgreSqlBuilder on that image, with the owner's password and
// TESSERA_READER_PASSWORD and TESSERA_PUBLISHER_PASSWORD set, then StartAsync
```

`ImageFromDockerfileBuilder` builds the image from the folder `db` beside your `.git` folder; the worked answer names its image `tessera-db-test`, and yours `tessera-rebuild-db-test`. `Lazy` builds it once for every class. The passwords, `reader-test`, `publisher-test` and `owner-test`, sit in plain sight, since each container lives only as long as its tests. Copy the rest of the fixture: `DisposeAsync`, which removes the container; the connection strings `Reader`, `ReaderUrl` and `Publisher`; and `Owner` and `OwnerValue`, which run SQL as the owner to set a test's stage. Leave out `GrantPlaces` until day 8. Testcontainers asks `pg_isready` until the server answers, and on a first start the image runs its scripts on a private server first, so the answer comes only once both roles exist.

A test class asks xUnit for the fixture through `IClassFixture<DatabaseFixture>`, so each class gets a database of its own. Write `DatabaseTests.cs` with the worked answer's three tests. The first signs in as the reader:

CSHARP

```
// tests/Tessera.IntegrationTests/DatabaseTests.cs, in your copy
public sealed class DatabaseTests(DatabaseFixture database) : IClassFixture<DatabaseFixture>
{
    // US-01 test 1: the roles come from the shipped script.
    [Fact]
    public async Task Signs_in_as_the_shipped_reader_role()
    {
        await using var reader = new NpgsqlConnection(database.Reader);
        await reader.OpenAsync(TestContext.Current.CancellationToken);
        await using var command = new NpgsqlCommand("SELECT current_user", reader);

        Assert.Equal("tessera_reader",
            await command.ExecuteScalarAsync(TestContext.Current.CancellationToken));
    }
    // …
}
```

The worked answer's first test signs in by loading the catalogue, which your copy builds on day 6; until then, `SELECT current_user` proves the same sign-in. The second, `Keeps_the_reader_read_only`, reads the

role's own settings as the owner, from PostgreSQL's table `pg_db_role_setting`, and expects `default_transaction_read_only=on` among them.

Both pass at once, because you wrote the script first, and a test never seen to fail proves little. So put `--` before the line `ALTER ROLE tessera_reader SET default_transaction_read_only = on`, and run that test alone:

```bash
dotnet test --project tests/Tessera.IntegrationTests \
  --filter-method "*Keeps_the_reader_read_only"
```

```text
failed Tessera.IntegrationTests.DatabaseTests.Keeps_the_reader_read_only (317ms)
  Xunit.MicrosoftTestingPlatform.XunitException: System.ArgumentNullException : Value cannot be
  null. (Parameter 'collection')
```

Remove the `--`, and the test passes again: only now do you know it checks the line its name promises. GitHub's Ubuntu machines have Docker, so Testcontainers builds the same image in CI.

> WATCH OUT The test fails, but not with an assertion's message. Without the line the role has no settings, so the query finds no row, `OwnerValue` hands back `null`, and `Assert.Contains` refuses a list that is not there. Note it for your comparison: a test that first asserts the row exists would name the cause plainly.

4.4 The seed, as the repository holds it

The package also carries the area's places, ready for US-24's seed: `db/seed/0001-macau-places.sql`, made from `data/macau-pois.geojson`. Copy both files from the worked answer; you rebuild the pipeline that makes the script on days 7 and 8.

The story's second test asks that the GeoJSON file match its recorded ***SHA-256 fingerprint***, a 64-digit number computed from every byte of a file, and that the script's first line name the file. `sha256sum data/macau-pois.geojson` prints the value by hand. The worked answer's third test, `Ships_the_seed_made_from_the_places`, finds the repository's root with `CommonDirectoryPath.GetGitDirectory()`, compares `SHA256.HashData` of the GeoJSON file, written as lower-case hexadecimal, with the recorded `d9319335…e2f3`, and checks that the script starts `-- Made from data/macau-pois.geojson.`; copy it.

The worked answer reads the script from `SeedScripts.Folder`, the copy beside the program, which your copy gains with the seed on day 7. The script's second line reads 'Never change its SQL after it has run'; day 7 explains why.

> IN INK Commit with messages such as `US-01: package the database with the service's two roles` and `US-01: start the packaged database in the tests`, push, and stop `rebuild-db` with `docker stop rebuild-db`.

> THE PROOF The three tests of `DatabaseTests` pass in the terminal and in CI, beside yesterday's two in `StartTests`, and `npm test` passes. Criterion 3's browser tests, which read the shipped seed, arrive with the map. Tomorrow's first test: `Quotes_a_name_and_doubles_each_inner_quote`.

5

Day 4 · Thursday 22 October: US-03, one way to quote a name

> IN SHORT Today you start US-03 with its two smallest parts, on which every later query rests: a result that can fail without throwing an error, and the one safe way to put a table's name into SQL. This is the first of US-03's 3 points.

US-03, worth 3 points, makes each table the service may read into a *layer* of the map. A table becomes a layer when the data publisher ***grants*** the service's role the right to read it. The database keeps the guest list; the service only reads it. Today you build two things that rest on nothing else: a firm grasp of `Result`, and `SqlName`, which quotes a name, as Exhibit 5.1 sets out.

EXHIBIT 5.1 Day 4 ends when every name, however odd, comes out safely quoted

Goal	`SqlName`, with its two tests; a test that fixes how a `Result` carries a failure
Story	US-03 test 1, 'One way to quote a name'; the rule 'Only the catalogue mints names'
In the worked answer	`src/Tessera/Modules/Tessera/Domain/SqlName.cs`; `tests/Tessera.UnitTests/SqlTests.cs`; `docs/adr/0004-functional-style.md`; `Program.cs`, `Seed`
Look for	Why `Map` and `Bind` skip a failure; why the code quotes an awkward name rather than refusing it; why the property test checks a rule rather than an expected string; why `SqlName`'s constructor is private
Leave for later	Layers and the catalogue (day 5); `Queries` and `Database` (day 6); the approved query text and the architecture tests (S-01, days 12 and 13)
Done when	`Quotes_a_name_and_doubles_each_inner_quote` and the property test `Quotes_every_name_safely` pass, with `A_failure_skips_every_later_step`

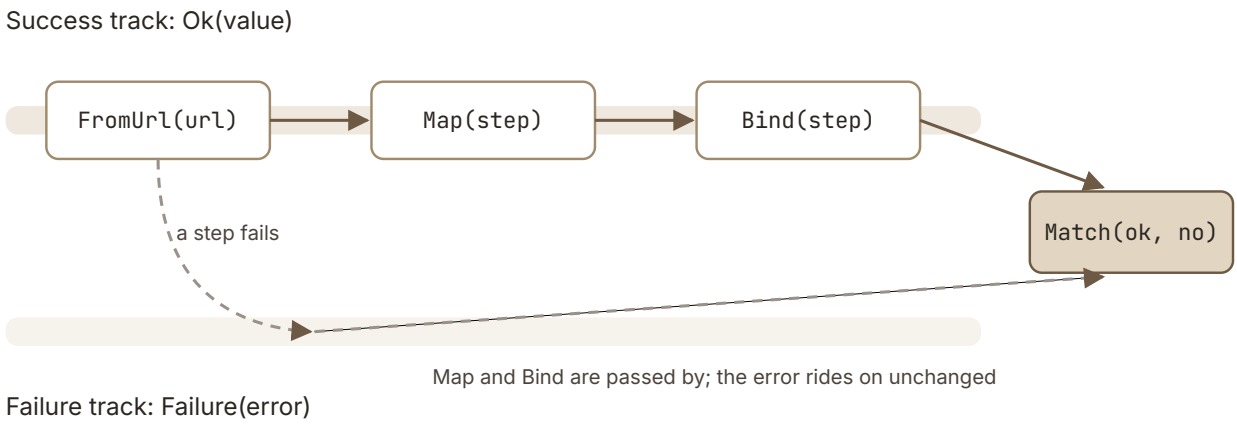
Source: `docs/user-stories/user-stories.md`, *US-03;* `docs/adr/0004-functional-style.md`

Make the worked answer's folder for records that use no web, no SQL and no files: `src/Tessera/Modules/Tessera/Domain/`. Its files declare the namespace `Modules.Tessera.Domain`, the folder's own path, since your project's namespace root is empty, and `src/Tessera/.editorconfig` stops the build when a file's namespace and folder disagree. Tessera has one ***module***, `Modules/Tessera`, in three layers: `Domain`, `Infrastructure`, where yesterday's `Connection.cs` lives, and `Features`, which day 11 starts.

5.1 A result is either success or a refusal

Tessera's code never throws an error to say 'no': it returns 'here is the answer' or 'here is why not'. ADR 0004 sets the rule, and the CSharpFunctionalExtensions library gives the type, so Tessera neither writes nor tests it. You met `Result<ConnectionString, string>` on day 2; today you fix how a chain of results behaves, before any code leans on it.

EXHIBIT 5.2 A `Result` runs on two tracks: once a step fails, every later step is passed by



Source: `docs/adr/0004-functional-style.md`; `src/Tessera/Program.cs`, `Seed`

5.1.1 A learning test: a failure skips the rest

As Exhibit 5.2 shows, picture a railway with two tracks. `Map` runs a step that cannot fail on the value; `Bind` runs a step that returns a `Result` of its own; both run only on the success track. Once a step fails, the train switches to the failure track and passes every later step by, until `Match` turns either track into one answer. In `tests/Tessera.UnitTests/ResultTests.cs`, write a test that chains two steps after a failure:

CSHARP

```csharp
// tests/Tessera.UnitTests/ResultTests.cs, in your copy
[Fact]
public void A_failure_skips_every_later_step()
{
    var failed = Result.Failure<int, string>("no");
    var after = failed.Map(n => n + 1).Bind(n => Result.Success<int, string>(n * 2));

    Assert.Equal(Result.Failure<int, string>("no"), after);
    Assert.Equal(Result.Success<int, string>(2),
        Result.Success<int, string>(1).Map(n => n + 1));
}
```

It passes at its first run, since the library already keeps the rule. A test of a library you did not write is a ***learning test***: it records what you rely on, and fails on the day an upgrade changes it.

NOTE **Beyond the worked answer.** The worked answer tests `Result` only through the code that uses it, such as `Keeps_the_last_good_list_through_a_failure` in the unit tests' `CatalogueTests`. Appendix A lists this test of your own.

WATCH OUT The library has two kinds of result: `Result<T>`, whose error is always a string, and `Result<T, E>`. Given a string error, `MapError` can pick the overload that returns the first kind, and the next `Bind` then finds no step to run. The worked answer's `Seed` names all three types, `MapError<ConnectionString, string, string>`, for that reason. When a chain will not compile, name the types.

5.2 Quote every name one way

A table's name can hold spaces, quotes or capital letters, so the code must quote it before it goes into SQL. One function does it, and only that one.

5.2.1 Red: two tests and no type

Copy the worked answer's `tests/Tessera.UnitTests/SqlTests.cs`. Add `FsCheck.Xunit.v3` 3.4.0 to `Directory.Packages.props` and reference it in the unit tests' project file first.

CSHARP

```csharp
// tests/Tessera.UnitTests/SqlTests.cs
[Fact]
public void Quotes_a_name_and_doubles_each_inner_quote() =>
    Assert.Equal("\"odd \"\"name\"\"\"", SqlName.Quote("odd \"name\"").Sql);

// Every name, however odd, opens with a quote, closes with one, and holds no lone quote
// between.
[Property]
public bool Quotes_every_name_safely(NonNull<string> name)
{
    var quoted = SqlName.Quote(name.Get).Sql;
    return quoted.StartsWith('"') && quoted.EndsWith('"')
        && !quoted[1..^1].Replace("\"\"", "", StringComparison.Ordinal).Contains('"');
}
```

The first is an example: `odd "name"` must become `"odd ""name"""`. The second is a ***property test***. FsCheck, a testing library, makes up a hundred strings, from the empty one to strings of control characters and letters from every alphabet, and the test checks one rule for each: the quoted form opens and closes with a quote, and with every doubled quote taken out, no quote is left inside. Run `dotnet test --project tests/Tessera.UnitTests`. The compiler refuses: `error CS0234: The type or namespace name 'Domain' does not exist in the namespace 'Modules.Tessera'`.

5.2.2 Red again: a quote that is not doubled

Write the smallest `SqlName` that compiles, one that wraps the name in quotes and nothing more, `new($"\"{name}\"")`, and run again. Both tests fail. The example shows where the two strings part, at position 6; the property test shows more:

TEXT

```
failed Tessera.UnitTests.SqlTests.Quotes_every_name_safely (8ms)
  FsCheck.Xunit.PropertyFailedException :
  Falsifiable, after 39 tests (33 shrinks) …
  Original:
  NonNull "4&Km>G[,"`e\0259_~iSrM+J,\015Hw\026e\020\030n\001\001C+" (…)
  Shrunk:
  NonNull """
```

FsCheck found a string that broke the rule on its 39th try, then ***shrank*** it: it tried smaller and smaller strings that still fail, 33 times, until one was left that cannot be made smaller. The answer is a single `"`, written inside FsCheck's own quotes. One quote mark is the whole fault, and a person reads it at a glance.

5.2.3 Green: `SqlName.Quote`

```csharp
// src/Tessera/Modules/Tessera/Domain/SqlName.cs (shortened)
public sealed record SqlName
{
    public string Sql { get; }

    private SqlName(string sql) => Sql = sql;

    public static SqlName Quote(string name) =>
        new($"\"{name.Replace("\"", "\"\"", StringComparison.Ordinal)}\"");

    public static SqlName Qualified(string schema, string table) =>
        new($"{Quote(schema).Sql}.{Quote(table).Sql}");

    public override string ToString() => Sql;
}
```

Copy it from the worked answer, comments and all, and both tests pass. `Qualified` joins a ***schema***, a named folder of tables inside a database, to a table, each quoted: `"food"."cafes"`. Day 5's first test checks it.

5.3 A wrong value cannot be built

`SqlName` is a ***smart constructor***: a private constructor, a property with `get` alone, and static functions that keep the rule. A method that builds SQL can ask for a `SqlName` and refuse plain text, and every `SqlName` in the program is quoted, because there is no other way to make one. Try it: in any test file, write `new Modules.Tessera.Domain.SqlName("x")`. The build stops with `error CS1729: 'SqlName' does not contain a constructor that takes 1 arguments`, since a private constructor is invisible outside its type. A `with` expression cannot copy past the rule either, because `Sql` has no setter. Delete the line.

Day 2's `ConnectionString` is built the same way, and ADR 0004 names both, with `TileAddress` of day 9, as the three smart constructors of the service.

> WATCH OUT Never build SQL by adding a name to a string yourself, even once. A name such as `x"; DROP TABLE cafes; --` would end your quotes early and run its own command. That attack is ***SQL injection***, and quoting in one place, through a type no other code can make, is the guard against it.

> IN INK Commit `US-03: fix how a failure travels` and `US-03: quote every name one way`, and push.

> THE PROOF Eight unit tests pass: day 2's five, the learning test and the two of `SqlTests`, one of which tried a hundred names. `npm test` passes, and your `SqlName.cs` matches the worked answer's line for line. Tomorrow's first test: `Turns_a_row_into_a_layer_whose_id_is_schema_dot_name`.

6

Day 5 · Friday 23 October: US-03, rows become layers

IN SHORT Today you build the catalogue from rows you write by hand, with no database. A row read from PostgreSQL on Monday gives the same layer, so today's tests stay true when the real database arrives. This is the second of US-03's 3 points.

The day builds four parts, from the smallest up: the `Layer` record and its ***catalogue***, the row that describes a table, the function that turns rows into layers, and the exact lookup of a layer by its id. It ends with the weekly check; Exhibit 6.1 sets out the day.

EXHIBIT 6.1 Day 5 ends when rows become layers and a lookup matches exactly

Goal	`Layer`, `LayerCatalogue`, `CatalogueRow`, `CatalogueReader.ToLayer` and `CatalogueReader.FromRows`, with their tests
Story	US-03 test 2, 'Rows become layers, purely'
In the worked answer	`Domain/Layers.cs`; `Infrastructure/Catalogue.cs`, under `src/Tessera/Modules/Tessera/`; in `tests/Tessera.UnitTests/CatalogueTests.cs`, the row `Cafes` and the first three tests
Look for	Why a layer's id stays unquoted while its table is quoted; why `FromRows` drops both of two clashing ids; why `Find` returns `Layer?`
Leave for later	`Queries.Layers`, `Database` and `CatalogueReader.Load`, with story tests 3 and 4 (day 6); `LiveCatalogue`, which rereads the list (N-23)
Done when	`Turns_a_row_into_a_layer_whose_id_is_schema_dot_name`, `Publishes_neither_of_two_tables_whose_ids_clash` and `Finds_a_layer_exactly_and_case_sensitively` pass; the burn-up reads 2 of 2

Source: `docs/user-stories/user-stories.md`, *US-03*

6.1 A layer is a record that cannot change

A layer holds all a tile query needs to know about one table, and nothing can alter it once made. Copy `src/Tessera/Modules/Tessera/Domain/Layers.cs` from the worked answer; its two records live in one file, because they make one idea.

```csharp
// src/Tessera/Modules/Tessera/Domain/Layers.cs (shortened)
public sealed record Layer(
    string Id, SqlName Table, SqlName Geometry, int Srid, string GeometryType,
    ImmutableArray<SqlName> Properties, ImmutableArray<SqlName> Order);

public sealed record LayerCatalogue(ImmutableDictionary<string, Layer> Layers)
{
    public Layer? Find(string id) => Layers.GetValueOrDefault(id);
}
```

The id is the name an address uses, a schema and a table joined by a dot, such as `food.cafes`. Every other name is a `SqlName`, already quoted, and `Order` holds the table's key columns, which fix the order of its features; a view has none. The ***SRID*** is the number that names a coordinate system: 4326 for longitude and latitude.

> FROM SCHOOL In pseudocode you may have written `RECORD Student … ENDRECORD`, with a field for each fact. A C# ***record*** is the same idea, written in one line, with its fields in brackets. These records never change once made, and their lists are `ImmutableArray` and `ImmutableDictionary`, which cannot change either. No part of the program can alter a layer behind another part's back, so a request can read the catalogue while a later story builds the next one.

6.2 Rows become layers

A row describes one table as the database lists it; the catalogue reader turns each row into a layer, quoting every name on the way in.

6.2.1 Red: a row written by hand

In `tests/Tessera.UnitTests/CatalogueTests.cs`, copy the worked answer's row and its first test. Add `<Using Include="System.Collections.Immutable" />` to the unit tests' project file first, as the worked answer's has.

```csharp
// tests/Tessera.UnitTests/CatalogueTests.cs (shortened)
private static readonly CatalogueRow Cafes =
    new("food", "cafes", "geom", 4326, "POINT", """["layer","name"]""");

[Fact]
public void Turns_a_row_into_a_layer_whose_id_is_schema_dot_name()
{
    var layer = CatalogueReader.ToLayer(Cafes);

    Assert.Equal("food.cafes", layer.Id);
    Assert.Equal("\"food\".\"cafes\"", layer.Table.Sql);
    Assert.Equal(["\"layer\"", "\"name\""], layer.Properties.Select(name => name.Sql));
    Assert.Equal(4326, layer.Srid);
}
```

Run `dotnet test --project tests/Tessera.UnitTests`. The compiler refuses: `CatalogueRow` and `CatalogueReader` do not exist.

6.2.2 Green: `CatalogueRow` and `ToLayer`

Copy `src/Tessera/Modules/Tessera/Infrastructure/Catalogue.cs` from the worked answer, without `Load` and its `using` lines, which need Monday's `Database`:

CSHARP

```
// src/Tessera/Modules/Tessera/Infrastructure/Catalogue.cs (shortened)
public sealed record CatalogueRow(
    string Schema, string Name, string GeometryColumn, int Srid, string GeometryType,
    string Columns, string KeyColumns = "[]");

public static class CatalogueReader
{
    public static Layer ToLayer(CatalogueRow row) => new(
        $"{row.Schema}.{row.Name}",
        SqlName.Qualified(row.Schema, row.Name),
        SqlName.Quote(row.GeometryColumn),
        row.Srid,
        row.GeometryType,
        Names(row.Columns),
        Names(row.KeyColumns));

    private static ImmutableArray<SqlName> Names(string json) =>
        JsonSerializer.Deserialize<string[]>(json) is { } names
            ? [.. names.Select(SqlName.Quote)] : [];
    // …
}
```

The row's seven fields match the seven columns the catalogue query returns on Monday, by name, since Dapper fills a record by matching names. The query writes each list of columns as JSON text, such as `["layer","name"]`, and `Names` reads it back and quotes each name. `ToLayer` is a ***pure function***: it reads only its input and changes nothing outside itself, so the test can pass it a row written by hand in place of a real database.

> WATCH OUT The build stops at the comment above `CatalogueRow`, `error CS1574`: it points to `<see cref="Queries.Layers"/>`, and `Queries` arrives on Monday. The compiler checks every `///` comment, and every warning is an error, so write the name as plain text until Monday.

6.3 Two ids that clash

Two tables can earn the same id, and the catalogue then publishes neither. Schema `a.b` with table `c`, and schema `a` with table `b.c`, both give the id `a.b.c`; an address cannot say which it means. Write the worked answer's `Publishes_neither_of_two_tables_whose_ids_clash`: three rows go in, the two that clash and `food.cafes`, and the test expects the keys to be `["food.cafes"]` alone. Then the rest of the reader:

CSHARP

```
// src/Tessera/Modules/Tessera/Infrastructure/Catalogue.cs (shortened)
public static LayerCatalogue FromRows(IEnumerable<CatalogueRow> rows) =>
    new(rows.Select(ToLayer)
        .GroupBy(layer => layer.Id, StringComparer.Ordinal)
        .Where(sharing => sharing.Count() == 1)
        .ToImmutableDictionary(sharing => sharing.Key, sharing => sharing.Single(),
            StringComparer.Ordinal));
```

`GroupBy` gathers the layers that share an id into one group; the filter keeps each group of exactly one. Take the `Where` line out and run the test, to see why it matters: `Single` meets a group of two and throws `Sequence contains more than one element`. Serving the wrong table is worse than serving none, so both clashing tables stay out, and every other table is still published.

6.4 A lookup matches case exactly

The catalogue finds a layer by its id, letter for letter, case included. Write the worked answer's third test:

CSHARP

```
// tests/Tessera.UnitTests/CatalogueTests.cs
[Fact]
public void Finds_a_layer_exactly_and_case_sensitively()
{
    var catalogue = CatalogueReader.FromRows([Cafes]);

    Assert.NotNull(catalogue.Find("food.cafes"));
    Assert.Null(catalogue.Find("Food.Cafes"));
    Assert.Null(catalogue.Find("food"));
}
```

It passes at once, because `FromRows` already builds the dictionary with `StringComparer.Ordinal`, which compares character by character. `Find` answers a `Layer?`: either a layer, or `null`. Nullable checking is on and every warning is an error, so the compiler refuses to read `.Table` from the answer until the code has checked it, as day 11's route does with `is not { } found`. ADR 0004 chose this over a wrapper type: a missing value says so in its type.

> FROM SCHOOL In ASCII, capital `C` is 67 and small `c` is 99, so `Cafes` and `cafes` are different words to a computer. PostgreSQL keeps them apart in quoted names, and so does Tessera.

6.5 The weekly check

On Friday, end the day with the burn-up chart of chapter 1, kept in your notes. Its first row reads: week 1, 19 to 23 October, US-00 and US-01 finished, 2 points planned and 2 done; week 2 plans 7, milestone M1. US-03 has two days of work behind it and one left, and counts nothing until it is finished on Monday.

> IN INK Commit with messages such as `US-03: turn a row into a layer, every name quoted`, `US-03: publish neither of two clashing ids` and `US-03: find a layer by its exact id`. Push before the weekend.

> THE PROOF The three tests named today pass, with eight from the days before: eleven unit tests in all. The five integration tests pass, `npm test` passes, and the chart reads 2 of 2. Monday's first test: `Publishes_what_is_granted_and_nothing_else`, the catalogue read from the real database as `tessera_reader`.

7

Day 6 · Monday 26 October: US-03, the catalogue as the real role

IN SHORT Today you join the pure catalogue of days 4 and 5 to the real database, and a test signed in as `tessera_reader` finds the granted tables and no others. US-03 ends today: 3 points.

The service keeps no list of what it may show: it asks the database, as its own role, and the database answers from its own grants. Today you build the other half of `CatalogueReader`, from the inside out: the record that carries a query, the catalogue query, `Database`, the one class that talks to PostgreSQL, and a test signed in as the real role.

EXHIBIT 7.1 Day 6 ends when the catalogue, loaded as tessera_reader, lists only the granted tables

Goal	`Query`, `Queries.Layers`, `Database` and `CatalogueReader.Load`; the catalogue loaded as `tessera_reader`
Story	US-03 c1, c2, c3 and c5, and the 'present' half of c4; story tests 3 and 4
In the worked answer	Under `src/Tessera/Modules/Tessera/`: `Infrastructure/Sql/Database.cs`, `Infrastructure/Sql/Queries.cs`, `Infrastructure/Catalogue.cs`. Under `tests/`: `Tessera.UnitTests/CatalogueTests.cs`, `Tessera.IntegrationTests/CatalogueTests.cs`
Look for	Why the query checks rights with `has_table_privilege`, not with a list of names; how `Query` keeps values out of the text; why `Database` returns a `Result` and never lets an exception escape
Leave for later	US-03 c4's tile and c6's withdrawn grant, which need the tile route (day 11); the rule that one class alone sends SQL (S-01, day 12); `LiveCatalogue`, which rereads the list (N-23); the pool's limits (N-21, day 41); the four kinds of `DatabaseError` (N-22, day 42)
Done when	`Publishes_what_is_granted_and_nothing_else` passes, signed in as the reader

Source: `docs/user-stories/user-stories.md`, *US-03;* `docs/requirements/functional-requirements.md`, *FR-06 to FR-09;* `docs/requirements/security-requirements.md`, *SR-02 and SR-03*

7.1 One record carries a query and its values

A query travels as two things side by side: its fixed text, and an object that holds the values it needs. Write `Query` at the top of `Infrastructure/Sql/Database.cs`, as the worked answer does:

```csharp
public sealed record Query(string Text, object? Parameters = null);
```

A value from a request, such as a zoom level, travels as a ***parameter***: a value sent beside the query, never inside its text, so the database can never mistake it for a command. The text names each parameter, such as `@z`, and Dapper, the library that sends your queries, fills it from the property of the same name. Today's queries need none; day 11's tile query is the first to fill `Parameters`, and day 12 proves that no part of a request enters the text.

Write `Queries` beside it, in `Infrastructure/Sql/Queries.cs`: one static class that holds every SQL text the service sends, so a reviewer reads them all in one place. Today it holds two: `Ping`, which is `SELECT 1`, and the catalogue query, `Layers`.

7.2 The catalogue query asks the database who may read what

The catalogue query lists every table and view that has a typed geometry column and that the signed-in role may read. Write it as a raw string literal, the C# text between triple quotes that keeps its lines as typed. Its heart is the `FROM` and `WHERE`:

```sql
-- The text of Queries.Layers (trimmed)
SELECT DISTINCT ON (n.nspname, c.relname)
       n.nspname::text AS schema,
       c.relname::text AS name,
       -- … geometrycolumn, srid, geometrytype, and two lists: columns, keycolumns
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
JOIN pg_attribute a ON a.attrelid = c.oid AND a.attnum > 0 AND NOT a.attisdropped
JOIN pg_type t ON t.oid = a.atttypid AND t.typname = 'geometry'
WHERE c.relkind IN ('r', 'p', 'v', 'm')
  AND n.nspname NOT IN ('pg_catalog', 'information_schema')
  AND has_schema_privilege(n.oid, 'USAGE')
  AND has_table_privilege(c.oid, 'SELECT')
  AND postgis_typmod_srid(a.atttypmod) > 0
ORDER BY n.nspname, c.relname, a.attnum
```

`pg_class`, `pg_namespace` and `pg_attribute` are PostgreSQL's own tables of tables, so the query is a `SELECT` with `JOIN`s, as in your lessons. Each line of its `WHERE` clause answers one criterion of the story:

- `relkind IN ('r', 'p', 'v', 'm')` keeps tables, split tables, views and views that store their rows (c1, c2). `pg_class` never lists a function, so a granted function can never appear (c5).
- `has_schema_privilege` and `has_table_privilege` ask PostgreSQL whether the role running the query may use the schema and read the table (c3).
- `postgis_typmod_srid(...) > 0` keeps only a typed geometry column: a column with SRID 0 names no coordinate system, so the map could not place its shapes.

The two lists at the end, `columns` and `keycolumns`, name the columns a tile carries and the table's primary key. Each travels as JSON text, such as `["fid"]`, because Dapper cannot fill a list from a database array; `CatalogueReader`, from day 5, reads the text back into names.

7.2.1 Red, then green: story test 3

Dapper fills a `CatalogueRow` by matching each column's name to a field, so a column named `srid` fills `Srid`, and a misspelt name would leave a field empty without a word. Story test 3 guards against that. Add it to your unit tests' `CatalogueTests`, as the worked answer has it, with its neighbour for c5:

```csharp
// tests/Tessera.UnitTests/CatalogueTests.cs (rewrapped)
// US-03 test 3: each column the query returns maps to one field of the row, by name.
[Fact]
public void Names_each_column_after_a_field_of_the_row()
{
    foreach (var field in typeof(CatalogueRow).GetConstructors().Single().GetParameters()
                 .Select(parameter => parameter.Name!.ToLowerInvariant()))
        Assert.Contains($" AS {field}", Queries.Layers.Text, StringComparison.Ordinal);
}

// US-03 c5: a function never becomes a layer; the query reads pg_class, where functions
// never appear.
[Fact]
public void Lists_tables_and_views_alone() =>
    Assert.Contains("c.relkind IN ('r', 'p', 'v', 'm')", Queries.Layers.Text,
        StringComparison.Ordinal);
```

Both fail to compile until `Queries.Layers` exists. Write the query in full from the worked answer, run the class with `--filter-class Tessera.UnitTests.CatalogueTests`, and both pass. Misspell one alias, such as `AS srd`, watch the first fail, and put it back.

7.3 One class talks to the database

`Database` is the only class that sends SQL, a rule day 12 enforces, and it never lets the database's exception escape: each query returns the `Result` you met on days 2 and 4, a success with the rows or a failure with the reason. Add `Dapper` 2.1.89 to your `Directory.Packages.props` and a reference to it in `src/Tessera`, beside `Npgsql`, then write `Database`:

```csharp
// src/Tessera/Modules/Tessera/Infrastructure/Sql/Database.cs (trimmed)
public Task<Result<ImmutableArray<T>, DatabaseError>> Rows<T>(
    Query query, CancellationToken cancel = default) =>
    Run(async connection =>
        (await connection.QueryAsync<T>(Command(query, cancel)).ConfigureAwait(false))
            .ToImmutableArray(), cancel);

private async Task<Result<T, DatabaseError>> Run<T>(
    Func<NpgsqlConnection, Task<T>> work, CancellationToken cancel)
{
    try
    {
        var connection = await _source.OpenConnectionAsync(cancel).ConfigureAwait(false);
        await using (connection.ConfigureAwait(false))
            return Result.Success<T, DatabaseError>(
                await work(connection).ConfigureAwait(false));
    }
    catch (NpgsqlException error)
    {
        return Result.Failure<T, DatabaseError>(Error(error));
    }
}
```

`_source` is Npgsql's `NpgsqlDataSource`, the pool of connections every query borrows from, and `Open`, a static method, makes it from a connection string. `QueryAsync` is Dapper's: it sends the text and the parameters, and turns each row into a `T`. `Run` is the one place the database's exceptions become values.

The worked answer's `Open` also takes the pool's limits, which arrive with N-21 on day 41, and its `Error` sorts each failure into four kinds, which arrive with N-22 on day 42; note both in your comparison. Today write `Open` with the connection string alone, and `DatabaseError` as one record, `public sealed record DatabaseError(string SqlState, string Message);`, and let `Error` read the `SqlState` of a `PostgresException`, or give 08006, 'connection failure', for any other.

Then `CatalogueReader.Load` joins the shell to the pure core of day 5:

```csharp
public static async Task<Result<LayerCatalogue, DatabaseError>> Load(
    Database database, CancellationToken cancel = default) =>
    (await database.Rows<CatalogueRow>(Queries.Layers, cancel).ConfigureAwait(false))
        .Map(rows => FromRows(rows));
```

`Map` runs `FromRows` only when the query succeeded; a failure passes through untouched. No route reads the list yet, so the service first loads it on day 11, with its tile route.

7.4 Red, then green: the catalogue as the real role

Story test 4 signs in as the service's role and checks what the catalogue holds. Make `CatalogueTests` in your integration tests, as the worked answer does, on day 3's `DatabaseFixture`.

7.4.1 Make the tables first, as the owner

The story asks the test to make its own tables. xUnit runs a class's `InitializeAsync` before each test, so the stage goes there, sent through the fixture's `Owner`:

```sql
CREATE SCHEMA IF NOT EXISTS stage;
CREATE TABLE IF NOT EXISTS stage.granted (id int PRIMARY KEY, name text,
    geom geometry(Point, 4326));
CREATE TABLE IF NOT EXISTS stage.mercator (id int PRIMARY KEY, geom geometry(Point, 3857));
CREATE OR REPLACE VIEW stage.granted_view AS SELECT id, name, geom FROM stage.granted;
CREATE MATERIALIZED VIEW IF NOT EXISTS stage.stored_view AS
    SELECT id, name, geom FROM stage.granted;
CREATE TABLE IF NOT EXISTS stage.never_granted (id int PRIMARY KEY,
    geom geometry(Point, 4326));
CREATE TABLE IF NOT EXISTS stage.untyped (id int PRIMARY KEY, geom geometry);
CREATE TABLE IF NOT EXISTS stage."odd ""name""" (id int PRIMARY KEY,
    geom geometry(Point, 4326));
CREATE OR REPLACE FUNCTION stage.places_near(z int, x int, y int) RETURNS bytea
    LANGUAGE sql AS 'SELECT NULL::bytea';
GRANT USAGE ON SCHEMA stage TO tessera_reader;
GRANT SELECT ON stage.granted, stage.mercator, stage.granted_view, stage.stored_view,
    stage.untyped, stage."odd ""name""" TO tessera_reader;
GRANT EXECUTE ON FUNCTION stage.places_near(int, int, int) TO tessera_reader;
```

The stage holds one of each kind the story names: a granted table, a granted view, a granted view that stores its rows, a table never granted, a table whose name holds a space and a double quote, and a granted function. Two more test the query's own rules: a table in Web Mercator, SRID 3857, and a granted column with no type. Each step says `IF NOT EXISTS` or `OR REPLACE`, since it runs before every test. The worked answer's stage also holds two rows and a view that sleeps three seconds; leave them out until the tile route (day 11) and N-22 (day 44) need them.

CSHARP

```csharp
// tests/Tessera.IntegrationTests/CatalogueTests.cs, in your copy (rewrapped)
private static async Task<LayerCatalogue> Load(Database reader) =>
    (await CatalogueReader.Load(reader, TestContext.Current.CancellationToken))
        .Match(found => found, error => throw new Xunit.Sdk.XunitException(error.Message));

[Fact]
public async Task Publishes_what_is_granted_and_nothing_else()
{
    await using var reader = Database.Open(database.Reader);
    var ids = (await Load(reader)).Layers.Keys.Order(StringComparer.Ordinal).ToArray();

    // c1, c2 and c4 publish; c3 (never granted), c5 (a function) and an untyped column do not.
    Assert.Equal(["stage.granted", "stage.granted_view", "stage.mercator",
        "stage.odd \"name\"", "stage.stored_view"], ids);
}
```

`database.Reader` is the fixture's connection string for `tessera_reader`. The worked answer expects six ids, since its stage holds day 44's sleeping view; yours expects five. The class fails to compile until `CatalogueReader.Load` exists, then passes. Make it fail for the right reason once: take `stage.mercator` out of the grant, watch the list lose it, and put it back. Then add the worked answer's two neighbours. `Gives_each_layer_its_srid_and_type` reads `(4326, "Point")` for `stage.granted` and `(3857, "Point")` for `stage.mercator` (c1). `Gives_each_table_its_key_and_each_view_none` finds `"id"` as the key of `stage.granted` and none for the view: US-24's test 2, early, since the key comes from today's query. The three prove c1, c2, c3 and c5, and the first half of c4: the awkward name is present, and its tile waits for day 11.

> WATCH OUT Sign the test in as `tessera_reader`, never as the database's owner. The owner may read every table, so `stage.never_granted` would appear, and a test that checked only the granted tables would pass for the wrong reason: the risk the story names under 'Critical to success'.

> IN INK Commit at each green, with messages such as `US-03: name each column of the catalogue query after a field` and `US-03: load the catalogue as tessera_reader, through one Database class`, and push.

> THE PROOF `Publishes_what_is_granted_and_nothing_else` and its two neighbours pass against the image Testcontainers builds from `db/Dockerfile`, and `npm test` passes. US-03 is done: 5 points of the 7 due by Friday; its c4 tile and c6 withdrawal wait for day 11. Tomorrow's first test: `Loads_the_shipped_seed_once`.

8

Day 7 · Tuesday 27 October: US-24, each seed script once

> IN SHORT Today the program learns a second command, `seed`, which runs each new SQL script of places once, signed in as the publisher, and records it. A test then seeds a fresh database twice and finds the second run does nothing. This is the first of two days for US-24, worth 2 points.

A seed script runs once, and the database keeps the record of it, so a second run changes nothing. US-24 publishes the area's places in nine layers, in three parts: a Python pipeline that turns a file of places into a SQL script, a seed that runs each new script once, and a grant that publishes the tables. Today you read the pipeline and run its tests, then build the seed, test first; tomorrow, the grant.

EXHIBIT 8.1 Day 7 ends when a second seed runs nothing

Goal	`Tessera seed`, through DbUp, as `tessera_publisher`; a test of the shipped script
Story	US-24 c1 to c3 (read and run), c4 (build); story tests 1 and 5
In the worked answer	`src/Tessera/Program.cs`; `Modules/Tessera/Infrastructure/Seeding/SeedScripts.cs`; `Infrastructure/Sql/Queries.cs`, `JournalSchema`; `Tessera.csproj`, the seed scripts' line; `tests/Tessera.IntegrationTests/SeedingTests.cs`; `etl/tests`; `docs/adr/0009-seed-with-dbup.md`
Look for	Why the seed reads its own variable, `TESSERA_PUBLISHER_DATABASE_URL`, and starts no web server; where DbUp keeps its record; how DbUp knows a script
Leave for later	The grant and `npm run publish` (day 8); a failed script (day 8); the pipeline's filters (US-36, Part F)
Done when	`Loads_the_shipped_seed_once` passes

Source: `docs/user-stories/user-stories.md`, *US-24;* `docs/requirements/functional-requirements.md`, *FR-10 to FR-12 and FR-19*

8.1 The pipeline writes the scripts: read it and run its tests

The places come in nine layers of 100 places each, in four schemas, one for each group: `food` (`cafes`, `restaurants`, `markets`), `outdoors` (`parks`, `playgrounds`), `culture` (`museums`, `libraries`) and `transport` (`trains`, `buses`). The shipped script, in your copy since day 3, drops and makes each table with a numbered `fid` and a geometry column in SRID 4326, inserts its rows, adds a spatial index and runs `ANALYZE`.

Criteria 1 to 3 belong to the pipeline in `etl/`, a second program, in Python, which reads a file of places through GDAL and writes one SQL script. It holds 253 of the worked answer's tests, and US-24 has two days, so your copy takes it as it stands: copy `etl/` from the worked answer unchanged, a planned difference until US-36 returns to it on day 57. Then run the tests that prove the three criteria:

```bash
cd ~/src/tessera-rebuild/etl
uv run pytest -q tests/test_table_sql.py tests/test_golden.py tests/test_pipeline.py
```

Then give `scripts/test.ts` the pipeline's four checks, as the worked answer's has them: `uv run` pyright, ruff, lint-imports and pytest, in `etl/`, before the build.

EXHIBIT 8.2 Five of the pipeline's tests prove US-24's first three criteria

CRITERION	TEST	WHAT IT CHECKS
c1	`test_writes_a_table_as_GDAL_wrote_it`	INSERT statements, a `fid SERIAL` primary key, SRID 4326, the spatial index and `ANALYZE`, with GDAL's own `BEGIN` and `COMMIT` taken out
c2	`test_extracts_one_layer_flat_and_in_longitude_and_latitude`	The arguments handed to GDAL, which turn any coordinate system into longitude and latitude; no file in EPSG:3857 is read (FR-11, in part)
c3	`test_refuses_a_table_with_no_rows`	A table left with no rows stops the run
c3	`test_matches_its_recorded_outcome`	Its case `output-exists` refuses a script name already in use
Test 5	`test_refuses_a_misspelt_setting`	A misspelt pipeline file is refused

Source: `etl/tests/test_table_sql.py`; `etl/tests/test_pipeline.py`; `etl/tests/test_golden.py`

Of the five tests in Exhibit 8.2, read the first in full: it lists the statements GDAL wrote for one café.

8.2 Red: a second seed runs nothing

The seed is a command of the program, whose work lives in one static class, so the test calls that class. Make `SeedingTests` in your integration tests, beside yesterday's class, with the same fixture. Its helper `Run` calls `SeedScripts.Run` and throws if the seed fails, so a test stops with the script's own error:

```csharp
// tests/Tessera.IntegrationTests/SeedingTests.cs (trimmed and rewrapped)
[Fact]
public async Task Loads_the_shipped_seed_once()
{
    Run(database.Publisher, SeedScripts.Folder);

    Assert.Equal(900L, await database.OwnerValue<long>("""
        SELECT (SELECT count(*) FROM food.cafes) + (SELECT count(*) FROM food.restaurants)
             -- … + the other seven tables
        """));
    Assert.Equal(100L,
        await database.OwnerValue<long>("SELECT count(*) FROM transport.buses"));
    // … nine spatial indexes, read from pg_indexes
    Assert.Equal("0001-macau-places.sql", await database.OwnerValue<string>(
        "SELECT scriptname FROM tessera.schemaversions WHERE scriptname LIKE '0001-macau%'"));
    Assert.Empty(Run(database.Publisher, SeedScripts.Folder));
}
```

Copy the test whole. `database.Publisher` signs in as `tessera_publisher`, which owns the places; `OwnerValue` reads as the owner, who sees every table before any grant. The last line is criterion 4. The class fails to compile: `SeedScripts` does not exist yet.

8.3 Green: DbUp runs each script once

DbUp is a library that runs SQL scripts against a database, each once, in the order of their names, and records each in a table of its own, its ***journal***. Add `dbup-postgresql` 7.0.1 to your `Directory.Packages.props`, and a reference to it in `src/Tessera`. Then write `SeedScripts` in `Infrastructure/Seeding/`:

CSHARP

```csharp
// src/Tessera/Modules/Tessera/Infrastructure/Seeding/SeedScripts.cs (trimmed)
public static string Folder => Path.Combine(AppContext.BaseDirectory, "seed");

public static Result<IReadOnlyList<string>, string> Run(string connection, string folder)
{
    // … first sends Queries.JournalSchema: DbUp makes its table, but not the schema
    var upgrade = DeployChanges.To.PostgresqlDatabase(connection)
        .WithScriptsFromFileSystem(folder)
        .WithTransactionPerScript()
        // DbUp would otherwise read $name$ in a script as a value to fill in.
        .WithVariablesDisabled()
        .JournalToPostgresqlTable(JournalSchema, JournalTable)
        .LogToNowhere()
        .Build();
    var result = upgrade.PerformUpgrade();
    return result.Successful
        ? Result.Success<IReadOnlyList<string>, string>(
            [.. result.Scripts.Select(script => script.Name)])
        : Result.Failure<IReadOnlyList<string>, string>(
            $"{result.ErrorScript?.Name ?? "The seed"} failed: {result.Error.Message}");
}
```

Each line of the chain sets one thing, as Exhibit 8.3 sets out, and `Run` returns a `Result`: the names of the scripts it ran, or the script that failed and why. The query that makes the journal's schema goes in `Queries`, beside every other SQL text.

EXHIBIT 8.3 Each step of the seed's set-up, and what goes wrong without it

STEP	WHAT IT DOES	WITHOUT IT
`WithScriptsFromFileSystem`	Reads every `.sql` file in the folder and sorts them by name	No scripts to run
`WithTransactionPerScript`	Runs each script in a transaction of its own	A script that fails half-way leaves half its changes (day 8 proves it)
`WithVariablesDisabled`	Leaves `$name$` in a script as typed	DbUp could change a place's text
`JournalToPostgresqlTable`	Keeps the record in `tessera.schemaversions`, apart from the four schemas of places	The record lands in `public`, mixed with other tables
`LogToNowhere`	Silences DbUp's own log; the program prints a short report	A long log on every seed

Source: `src/Tessera/Modules/Tessera/Infrastructure/Seeding/SeedScripts.cs`; `docs/adr/0009-seed-with-dbup.md`

The scripts must travel with the program, so `SeedScripts.Folder` finds them beside it: copy the worked answer's `<None Include="../../db/seed/*.sql" …>` line into `src/Tessera/Tessera.csproj`, which copies each script into `seed/` beside the program. Run the class again, and `Loads_the_shipped_seed_once` passes. Day 3's `Ships_the_seed_made_from_the_places` may now read the script from `SeedScripts.Folder`, as the worked answer's does.

> FROM SCHOOL A ***transaction*** is a group of changes that the database makes all together or not at all: the 'atomic' in the rules for databases you met at school. PostgreSQL can undo even a `DROP TABLE` inside one, so a script that fails half-way leaves the database as it was: US-24 c6.

> WATCH OUT DbUp knows a script by its name alone. Change the SQL of `0001-macau-places.sql` after it has run, and DbUp never runs it again; rename it, and DbUp runs it a second time. That is why the seed's second line reads 'Never change its SQL after it has run': new data goes in a new script with the next number.

8.4 A second command for the program

The test calls `SeedScripts` directly; the office needs a command. Add `["seed"] => Seed()` to day 2's `switch`, and copy the worked answer's `Seed`, whose chain reads the variable, parses it with `ConnectionString.FromUrl`, names a failure with `MapError`, runs `SeedScripts.Run` through `Bind`, and ends in `Match(Report, …)`: day 4's railway. Copy `Report`, `Fail` and `Usage` with it. One line must differ: `Settings.AddFile` arrives with N-21 on day 40, so until then write `configuration.AddEnvironmentVariables();` in its place.

`["seed"]` matches exactly one argument, the word `seed`. Three choices keep the two commands apart. The seed builds no web application, so nothing listens on a port. It reads one variable alone, `TESSERA_PUBLISHER_DATABASE_URL` (`DatabaseConnections.PublisherDatabase`), so the service never holds the publisher's right to write. And it parses that URL with day 2's `FromUrl`, so a missing or malformed URL is named. Copy the worked answer's two tests of the command into `StartTests`: `Refuses_to_seed_and_names_the_publishers_variable` and `Seeds_without_reading_the_services_settings`.

> IN INK Commit at each green, with the messages `US-24: take the pipeline as it stands, with its tests` and `US-24: run each seed script once, through DbUp`, and push.

> THE PROOF `Loads_the_shipped_seed_once` passes, the pipeline's tests pass in your copy, and `npm test` passes. Half a story counts as nothing on the burn-up: 5 points of 7. Tomorrow's first test: `Publishes_nothing_until_the_grant`.

9

Day 8 · Wednesday 28 October: US-24, publish by grant

> IN SHORT Today a grant, and nothing else, publishes the nine tables, and a script that fails leaves the database as it was. US-24 ends today: 2 points.

The seed creates the tables; the publisher alone publishes them, with a grant. A library buys a book, but the book reaches the open shelf only when the librarian says so: the seed is the purchase, and the grant the librarian's word.

EXHIBIT 9.1 Day 8 ends when the reader sees the nine layers only after the grant

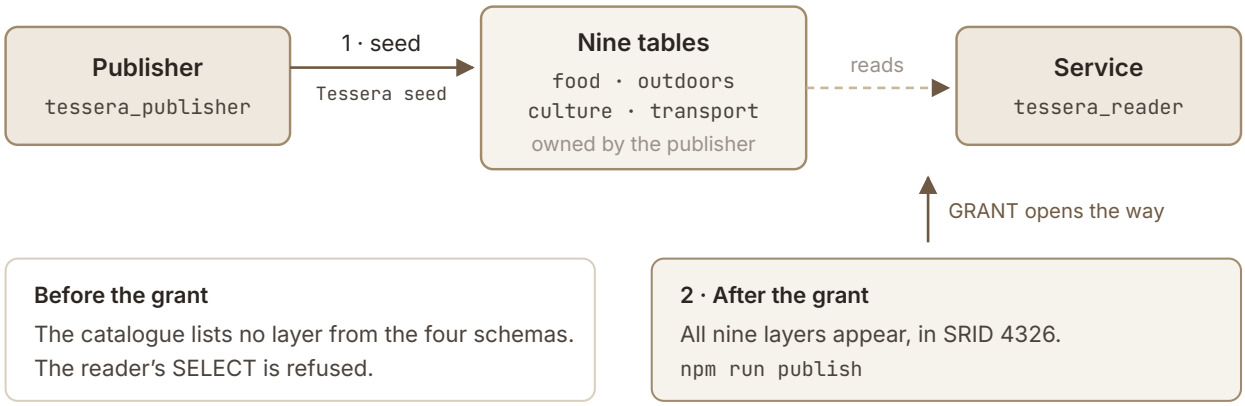
Goal	Three more tests in `SeedingTests`; `npm run publish`; the 900 places published in `rebuild-db`
Story	US-24 c5 and c6; story tests 2 to 4
In the worked answer	`tests/Tessera.IntegrationTests/SeedingTests.cs`; `DatabaseFixture.cs`, `GrantPlaces`; `scripts/publish.ts`; `package.json`, the script `publish`; `db/initdb/20-roles.sql`
Look for	Who owns the tables, and so who may grant them; why the password travels in the environment; why a replaced table loses its grant
Leave for later	The service rereading its list every 30 seconds (N-23); the browser tests over all nine layers (US-23)
Done when	`Publishes_nothing_until_the_grant` and `Undoes_a_script_that_fails` pass, and the reader counts 100 cafés only after the grant

Source: `docs/user-stories/user-stories.md`, *US-24;* `docs/requirements/security-requirements.md`, *SR-04;* `docs/requirements/functional-requirements.md`, *FR-20*

The two roles of `db/initdb/20-roles.sql` divide the work, as Exhibit 9.2 shows. `tessera_publisher` runs the seed, so it owns the schemas and tables it makes, and only an owner may grant them. `tessera_reader`, the service's role, owns nothing and may only read what it is granted. *Sixteen Tiles*, chapter 7, explains roles and grants in full.

> FROM SCHOOL As a file system gives each user rights over each file, a database gives each role a ***privilege*** over each table: `GRANT SELECT` gives the right to read, `REVOKE` takes it back, and `USAGE` on a schema is the right to look inside it at all.

EXHIBIT 9.2 Two roles, two acts: the publisher loads and grants, the reader only reads what is granted



Source: `db/initdb/20-roles.sql`; `scripts/publish.ts`

9.1 Red, then green: nothing is published before the grant

Story test 2 seeds the shipped script, looks at the catalogue as the reader, grants, and looks again. Add two helpers first. In `DatabaseFixture`, `GrantPlaces` sends the publisher's grant, as the owner:

CSHARP

```
// tests/Tessera.IntegrationTests/DatabaseFixture.cs
public Task GrantPlaces() => Owner("""
    GRANT USAGE ON SCHEMA food, outdoors, culture, transport TO tessera_reader;
    GRANT SELECT ON ALL TABLES IN SCHEMA food, outdoors, culture, transport TO tessera_reader;
    """);
```

In `SeedingTests`, `Published` loads the catalogue as `tessera_reader` and returns its ids, sorted, as day 6's `Load` did. Then write the test:

CSHARP

```
// US-24 c5: the seed never grants. The nine layers appear only after the publisher's grant,
// each in longitude and latitude.
[Fact]
public async Task Publishes_nothing_until_the_grant()
{
    Run(database.Publisher, SeedScripts.Folder);
    Assert.DoesNotContain("food.cafes", await Published());

    await database.GrantPlaces();
    await using var reader = Database.Open(database.Reader);
    var catalogue = (await CatalogueReader.Load(reader, TestContext.Current.CancellationToken))
        .Value;
    Assert.Equal((4326, "Point"), catalogue.Find("transport.buses") is { } buses
        ? (buses.Srid, buses.GeometryType) : default);
    Assert.Equal(["\"fid\""], catalogue.Find("food.cafes")!.Order.Select(name => name.Sql));
}
```

Watch it fail for the right reason: run it without the call to `GrantPlaces`. The check before the grant passes, `Load` finds no `transport.buses`, and the test fails on the next line. Put the grant back, and it passes. The last line is the rest of story test 2: the tile's features keep the order of `fid`, the key the catalogue query found.

> **WATCH OUT** The check before the grant looks only for `food.cafes`. The class's database is shared by its tests, and xUnit runs them in any order, so another test may have granted its own tables first. A check that the whole catalogue is empty would fail for a reason unrelated to your places.

9.2 Two edges of the seed

Story tests 3 and 4 need no new code. Copy them from the worked answer; each writes its own scripts into a fresh folder, through the helper `Folder`, with a schema of its own.

EXHIBIT 9.3 The seed's two edges, each proved by a script the test writes for itself

TEST	ITS SCRIPTS	WHAT IT EXPECTS
`Undoes_a_script_that_fails` (c6, test 3)	`0001-trial-a.sql` makes a table with one row; `0002-trial-a-broken.sql` drops it, then calls a function that does not exist	The error starts '0002-trial-a-broken.sql failed'; the table keeps its row; the journal holds no line for the broken script
`Withdraws_a_replaced_table_until_the_grant` (test 4)	`0001-trial-b.sql` makes a table, which the test grants; `0002-trial-b-again.sql` drops it and makes it again	The second seed runs that script alone, and the table leaves the reader's list until the next grant

Source: `tests/Tessera.IntegrationTests/SeedingTests.cs`

Both tests in Exhibit 9.3 pass at their first run, so make the first fail on purpose: delete the line `.WithTransactionPerScript()` from `SeedScripts` and run it with `--filter-method Tessera.IntegrationTests.SeedingTests.Undoes_a_script_that_fails`. Without its transaction, DbUp runs the broken script's `DROP TABLE` for good, and the count of rows fails. Put the line back, and the test passes again.

9.3 Green: the publisher's command

The tests prove the grant works; the office needs a command that sends it. The worked answer's is `npm run publish`, a short zx script:

```ts
// scripts/publish.ts (trimmed)
const container = 'tessera-db';
const password = process.env['TESSERA_PUBLISHER_PASSWORD'];
if (!password) {
  console.error('TESSERA_PUBLISHER_PASSWORD: missing or empty');
  process.exit(1);
}
// … grant: the same two GRANT lines as GrantPlaces
// `--env PGPASSWORD` with no value keeps the password off the command line.
// … docker exec --env PGPASSWORD ${container} psql … --username tessera_publisher … ${grant}
```

Write `scripts/publish.ts` in your copy, naming your container, `rebuild-db`, and add `"publish": "node --env-file-if-exists=.env scripts/publish.ts"` to `package.json`. The flag reads `.env` only if the file exists; your copy has none until day 34, when you copy `.env.example` to it, so until then give the password on the command line, as the Try it below does. The worked answer's script also waits for the database and reads its name, in `scripts/lib/database.ts`; your copy gains both on day 34.

WATCH OUT `GRANT SELECT ON ALL TABLES IN SCHEMA` grants only the tables that exist at that moment, and a dropped table takes its grants with it. So a script that replaces a table leaves it unpublished until the next `npm run publish` (test 4); the seed's report ends 'A new table waits for its grant: npm run publish.'

TRY IT Start `rebuild-db` afresh, as on day 3, with day 3's passwords. Seed, count as the reader, publish and count again:

BASH

```
URL=postgresql://tessera_publisher:publisher-local@127.0.0.1:5434/tessera
TESSERA_PUBLISHER_DATABASE_URL=$URL dotnet run --project src/Tessera -- seed
count() { docker exec -e PGPASSWORD=reader-local rebuild-db psql -h 127.0.0.1 \
  -U tessera_reader -d tessera -c 'SELECT count(*) FROM food.cafes'; }
count; TESSERA_PUBLISHER_PASSWORD=publisher-local npm run publish; count
```

The seed prints 'Ran 0001-macau-places.sql'. The first count fails, since the reader may not yet use the schema `food`; the second answers 100. Seed a second time and it prints 'The seed is up to date.'; `SELECT scriptname FROM tessera.schemaversions`, as the owner, lists `0001-macau-places.sql` once.

IN INK Commit at each green, with messages such as `US-24: publish the nine layers by grant, and only then`, and push. Friday's first task: a clean run of `npm test`.

THE PROOF `Publishes_nothing_until_the_grant`, `Undoes_a_script_that_fails` and `Withdraws_a_replaced_table_until_the_grant` pass, `npm test` passes, and the reader counts 100 cafés in `rebuild-db` only after `npm run publish`. US-24 is done: 7 points of 7, the line the plan draws for the end of week 2.

10

Day 9 · Thursday 29 October: US-07 begun, the tile's address

IN SHORT Today US-07 begins with the tile's address: three whole numbers that must name a square on the grid before the database hears of the request. The route, the query and the errors follow on Tuesday 3 November. This is the first of US-07's 2 points.

A tile request names its square by three numbers, and the service checks them in memory, so a request for a square that does not exist costs the database nothing. US-07, the heart of the product, has two days, split by the review and the holiday of 2 November. Today you build the part that needs no server and no database: the record that holds an address, and the rule that refuses a bad one. The short day leaves time for the week's comparison notes.

EXHIBIT 10.1 Day 9 ends when an address off the grid names no tile

Goal	`TileAddress`, with `Of`, and its three tests
Story	US-07, the domain half of story test 1: a bad request is refused before any query runs
In the worked answer	`src/Tessera/Modules/Tessera/Domain/Tiles.cs`; `tests/Tessera.UnitTests/TileTests.cs`, its first three tests
Look for	Why the constructor is private; why `Of` returns null and not an exception; why level 30 is the ceiling; where a part such as `x` or `1.5` is refused
Leave for later	The route, the tile query, the 404s and the problem document: US-07 c1 to c5, day 11
Done when	`Accepts_a_tile_that_exists` and `Refuses_a_tile_outside_the_grid` pass, and the property test `Gives_only_addresses_on_the_grid` with them

Source: `docs/user-stories/user-stories.md`, *US-07;* `docs/guides/delivery-plan.md`, *iteration 1*

10.1 Read the story first

US-07 asks for five things, and its two days share them, as Exhibit 10.2 shows.

EXHIBIT 10.2 US-07's criteria, split across its two days

CRITERION	WHAT IT ASKS	DAY
Test 1	A bad request is refused before any query runs: today the address, Tuesday the layer	9 and 11
c1	A published layer's tile answers 200 with `application/vnd.mapbox-vector-tile` (FR-21)	11
c2	The same request ending `.mvt` answers the same bytes (FR-22)	11
c3	Features sit at the right positions in the tile's grid (FR-23)	11

c4	An unknown layer answers 404 (FR-24)	11
c5	Every error is a problem document, as RFC 9457 sets it (SR-13)	11

Source: `docs/user-stories/user-stories.md`, *US-07;* `docs/requirements/functional-requirements.md`, *FR-21 to FR-24;* `docs/requirements/security-requirements.md`, *SR-13*

10.2 Red: an address off the grid names no tile

A ***tile address*** names one square of the map by three whole numbers: its zoom level `z`, its column `x` and its row `y`. At level z the world is cut into 2^z by 2^z squares, so x and y each run from 0 to $2^z - 1$. Make `TileTests` in your unit tests, and copy the worked answer's first two tests into it:

CSHARP

```csharp
// tests/Tessera.UnitTests/TileTests.cs (rewrapped)
// Web-map levels run from 0 to 22 (UI-09), and level z has 2^z columns and rows, numbered
// from 0; so 4194303, 2^22 - 1, is the last column at level 22.
[Theory]
[InlineData(0, 0, 0)]
[InlineData(15, 27154, 14280)]
[InlineData(22, 4194303, 0)]
public void Accepts_a_tile_that_exists(int z, int x, int y) =>
    Assert.NotNull(TileAddress.Of(z, x, y));

[Theory]
[InlineData(-1, 0, 0)]
[InlineData(23, 0, 0)]
[InlineData(1, 2, 0)]
[InlineData(1, 0, -1)]
public void Refuses_a_tile_outside_the_grid(int z, int x, int y) =>
    Assert.Null(TileAddress.Of(z, x, y));
```

A `[Theory]` runs once for each `[InlineData]` row. Run the class; it fails to compile:

BASH

```bash
dotnet test --project tests/Tessera.UnitTests --filter-class Tessera.UnitTests.TileTests
```

FROM SCHOOL The seven rows are the three kinds of test data from your lessons. `0, 0, 0` and a tile at level 15 are normal. `22, 4194303, 0` sits on a boundary: the last column of the last level. `1, 2, 0` sits one step past it: at level 1 the grid is 2 by 2, so column 1 exists and column 2 does not. `-1` and `23` are levels that no map has.

10.3 Green: a record only `Of` can make

Write `TileAddress` in the domain, in `Domain/Tiles.cs`, where no framework may reach, so its tests need no server and no database:

CSHARP

```csharp
// src/Tessera/Modules/Tessera/Domain/Tiles.cs (trimmed)
public sealed record TileAddress
{
    public int Z { get; }
    public int X { get; }
    public int Y { get; }

    private TileAddress(int z, int x, int y) => (Z, X, Y) = (z, x, y);

    public static TileAddress? Of(int z, int x, int y, int minZoom = 0, int maxZoom = 22) =>
        z < minZoom || z > maxZoom || z > 30 || x < 0 || y < 0 || x >= 1 << z || y >= 1 << z
            ? null
            : new TileAddress(z, x, y);
}
```

`1 << z` is 2^z, since each shift left doubles a binary number. Three choices keep the record safe. The constructor is private, so `Of` is the one way to make an address, and every address that exists lies on the grid: a smart constructor, like day 4's `SqlName`. Without `init`, no `with` expression can copy one off the grid. `Of` keeps the level between the lowest and highest the caller allows, 0 and 22 unless it says otherwise, and never above 30: `1 << 31` no longer fits a positive C# `int`. And `Of` returns `TileAddress?`, null for a bad address, so the compiler, with nullable checks on, makes every caller deal with the bad case.

Both tests pass. Then add a property test, like day 4's: FsCheck hands `Of` a hundred sets of made-up numbers, and the test checks one rule for each.

CSHARP

```csharp
[FsCheck.Xunit.Property]
public bool Gives_only_addresses_on_the_grid(int z, int x, int y) =>
    TileAddress.Of(z, x, y) is not { } tile
    || (tile.Z is >= 0 and <= 22 && tile.X >= 0 && tile.Y >= 0
        && tile.X < 1 << tile.Z && tile.Y < 1 << tile.Z);
```

A part such as `x`, `1.5` or `9999999999` never reaches `Of`: on day 11 the route's `{z:int}` constraint refuses any part that is not a whole number in an `int`'s range, and the request answers 404.

> TRY IT Add two rows of your own. Macau's home point, 113.55, 22.19, lies in the square 12/3339/1788. Its column is the whole part of (113.55 + 180) ÷ 360 × 4,096, about 3,339.9. Add `[InlineData(12, 3339, 1788)]` to the first test, and `[InlineData(12, 4096, 0)]` to the second, since at level 12 the columns run from 0 to 4,095. Note them as your own.

> IN INK Commit at each green, with the message `US-07: name a tile only on the grid`, and push.

> THE PROOF The three tests pass, and `npm test` passes. US-07 counts nothing until Tuesday: 7 points of 7. Tuesday's first test: `Serves_a_tile_as_a_mapbox_vector_tile`.

11

Day 10 · Friday 30 October: the review, and M1

> IN SHORT Today you prove the first two weeks' work from a clean start, show milestone M1 in both folders, and plan iteration 2. You write no new code. M1 reads 'The places are published by grant'.

A review answers three questions in order: does the work pass from nothing, does the milestone hold, and is every criterion traced and the next plan sound? The delivery plan gives the review the whole of the iteration's last working day. It needs that time: a clean run is slow, the milestone is shown twice, and the plan for the next two weeks must account for a holiday and a story that needs the office's network.

EXHIBIT 11.1 Day 10 ends with M1 shown twice and tagged in ink

Goal	A clean run; M1 shown in both folders; the trace and comparison notes finished; iteration 2 planned
Guide	`docs/guides/delivery-plan.md`: 'The weekly and daily rhythm', 'Tracking progress with a burn-up', 'Risks to delivery'
In the worked answer	`scripts/start.ts`; `scripts/publish.ts`; `.env.example`; `Infrastructure/LiveCatalogue.cs`, its log line
Look for	How the worked answer's scripts do what you did by hand on day 8
Done when	`npm test` passes from a clean start; M1 is shown in both folders; the burn-up reads 7 of 7; the tag `m1` is pushed

Source: `docs/guides/delivery-plan.md`

11.1 The work passes from nothing

A ***clean run*** starts with nothing left over on the machine, so it proves that nothing from earlier days makes the tests pass.

```bash
cd ~/src/tessera-rebuild
docker rm --force rebuild-db
docker image rm tessera-rebuild-db-test
git clean -xdf
npm ci
npm test
```

`git clean -xdf` deletes every file Git does not track: each `bin` and `obj` folder, `node_modules`, the pipeline's packages and anything else a build left, so `npm ci` installs the scripts' packages again from `package-lock.json`. Removing the test image makes Testcontainers build it again from your

`db/Dockerfile`, so a change you forgot to commit there cannot hide. If `npm test` fails, the review has found its first task: fix it before anything else, in a commit of its own.

11.2 M1 holds in both folders

In your copy, start `rebuild-db` afresh, as on day 3, and repeat the try-it of day 8: the seed, the reader's count, `npm run publish`, and the count again. The reader's count fails before the grant and answers 100 after it.

In the worked answer, run the same story through its scripts. `npm start` builds everything, starts the database, runs the seed and starts two copies of the service behind Nginx. It reads five settings from a file named `.env`: copy `.env.example` to `.env`, give each password a value of your own, and set `TESSERA_OFFICE_NETWORK` to your home network, such as `192.168.1.0/24`. Leave it running and use a second terminal for the rest:

```bash
cd ~/src/tessera
npm start                  # first terminal: database, seed, two copies, Nginx
npm run publish            # second terminal: the publisher's grant
docker exec tessera-db psql -U postgres -d tessera -c '\dp food.*'
```

Watch the first terminal. The seed prints 'Ran 0001-macau-places.sql', and each copy of the service logs 'The service publishes 0 layers.' when it first reads its list. Within 30 seconds of the grant, the interval `Catalogue.RefreshSeconds` sets, each logs 'The service publishes 9 layers.' `\dp` lists who may use each table: each of the three tables in `food` shows `tessera_reader=r/tessera_publisher`, so the reader may read (`r`) because the publisher granted it. From day 11 your copy reads its list once, at start-up, and needs a restart to see a new grant: a planned difference until N-23. When you have seen it, stop the copies with Ctrl+C, then the containers with `docker stop tessera-nginx tessera-db`.

11.3 Every criterion has a test, and the plan is sound

11.3.1 The trace

Each criterion of the four stories should have a test, or a line in your notes that names the day or story that proves it. Git finds each story's commits: `git log --oneline --grep 'US-24'`. Exhibit 11.2 gives the trace for the two stories of this week.

EXHIBIT 11.2 US-03 and US-24: each criterion, the worked answer's test, and your copy's

CRITERION	THE WORKED ANSWER	YOUR COPY
US-03 c1, c2, c3, c5	`Publishes_what_is_granted_and_nothing_else`, `Gives_each_layer_its_srid_and_type`, `Lists_tables_and_views_alone` (FR-06, FR-07, FR-09, SR-02)	The same, day 6
US-03 c4	`Serves_the_awkward_name` (FR-08)	Present: day 6; served: day 11
US-03 c6	`Answers_404_for_a_table_revoked_after_start_up` (SR-03)	Day 11, once tiles exist
US-24 c1 to c3	`etl/tests` (FR-10 to FR-12; FR-11 in part)	The same, copied on day 7

US-24 c4	`Loads_the_shipped_seed_once` (FR-19, in part: no test checks the order of names)	The same, day 7
US-24 c5	`Publishes_nothing_until_the_grant`, `Withdraws_a_replaced_table_until_the_grant` (SR-04)	The same, day 8
US-24 c6	`Undoes_a_script_that_fails` (FR-20)	The same, day 8

Source: `docs/requirements/functional-requirements.md`; `docs/requirements/security-requirements.md`; `tests/`

One gap stands out: FR-19 asks that DbUp run scripts in the order of their names, and no test checks it. Close it if the day leaves time.

> TRY IT **Beyond the worked answer.** In `SeedingTests`, write `Runs_the_scripts_in_the_order_of_their_names`. With the helper `Folder`, write `0002-trial-c-second.sql` first and `0001-trial-c-first.sql` second: the first script makes a table `trial_c.places`, the second inserts a row into it. Seed the folder, and expect `["0001-trial-c-first.sql", "0002-trial-c-second.sql"]` from `Run`, and one row in the table. Written in the other order, the insert would find no table and the seed would fail.

Then read the week's comparison notes. Keep each difference that has a reason, such as the single `DatabaseError` record, the pipeline copied whole, or your two rows of the tile at level 12, until its story comes; remove any that has none.

11.3.2 The burn-up

Only finished stories count, so US-07, begun yesterday, adds nothing yet. The plan expects 7 points by the end of week 2; if you are on time, 7 are done. Write the real figure in your notes, in an 'actual' column beside the planned line of Exhibit 1.5; chapter 1 says what a gap calls for.

11.3.3 Ink for the milestone

A ***tag*** is a name fixed to one commit. Tag the commit that reaches M1 with `git tag -a m1 -m 'M1: the places are published by grant'`, and push it with `git push origin m1`.

11.3.4 Plan the next two weeks

Iteration 2 runs from 2 to 13 November and ends with M2, 'The map opens in the office, on safe tiles'. Monday 2 November is All Souls' Day, a holiday in Macau, so the iteration holds nine working days: eight for US-07's second day, S-01, S-06, US-35 and US-23, as chapter 1 sets out, and one for the review.

One story depends on someone else: US-35 needs the office machine, in mirrored networking, with a reserved address, and the plan's risks ask for both by the end of iteration 1. Check today that they have come. Then write Tuesday's first test in your notes: `Serves_a_tile_as_a_mapbox_vector_tile`.

> THE PROOF `npm test` passes from a clean start; both folders show the reader's count fail before the grant and answer 100 after it; the burn-up reads 7 of 7; and the tag `m1` is on GitHub.

12

Day 11 · Tuesday 3 November: US-07, a tile through one query

IN SHORT Today the service answers its first real tile: one query asks PostGIS to cut a layer's places to a square and pack them, and every way the request can fail comes back as a problem document of one shape. This finishes US-07, worth 2 points.

A request for a tile passes two checks and one query, and leaves as a tile or as a problem document. After the holiday of All Souls' Day, the iteration starts on a Tuesday. Day 9 began US-07 with `TileAddress.Of`; today adds the route, the query, the tests through the real database and the shape of every error.

EXHIBIT 12.1 Day 11 ends when the real service serves a tile and answers every error as a problem document

Goal	The tile route, the tile query and `Problems`, with their tests through the real service; `TesseraModule` and `TileSettings`
Story	US-07 c1 to c5; tests 1 to 3. US-03 c6, which waited for a tile. US-00's run-command test, which waited for one too.
In the worked answer	Under `src/Tessera/`: `Modules/Tessera/Features/Tiles.cs`; `Modules/Tessera/TesseraModule.cs`; `TileSettings` in `Modules/Tessera/Domain/Settings.cs`; `Queries.Tile` and `Database.Single` under `Infrastructure/Sql/`; `Common/Api/Problems.cs`; `TesseraHost.cs`. Under `tests/Tessera.IntegrationTests/`: `ServiceTests.cs` and `CatalogueTests.cs`
Look for	Why the query turns the tile's square into the layer's system, never the other way round; why the route checks before it queries; why no refusal repeats the name a stranger sent; why `Problems.Shape` removes `traceId`
Leave for later	The 503 before the list of layers loads (N-23); the lines for `Cache-Control` and the tag (N-18, N-19); the four kinds of `DatabaseError` (N-22); the approved query text (S-01, day 12); Nginx (S-06, day 14)
Done when	`Serves_a_tile_as_a_mapbox_vector_tile`, `Serves_the_same_body_for_mvt`, `Places_each_feature_where_it_lies`, `Answers_an_unknown_layer_with_a_problem_document`, `Answers_a_tile_outside_the_grid_with_404`, `Answers_an_unknown_address_with_a_problem_document`, `Answers_404_for_a_table_revoked_after_start_up` and `Starts_from_the_run_command_and_answers` pass

Source: `docs/user-stories/user-stories.md`, *US-07;* `docs/guides/delivery-plan.md`, *iteration 2*

12.1 Ask the real service for a real tile

The first test starts your whole service on a free port, over a fresh copy of the packaged database, and asks it for a tile through real HTTP.

Create `tests/Tessera.IntegrationTests/ServiceTests.cs`, with two classes. `ServiceFixture` readies the stage once, in the release's order: it starts the database through day 3's `DatabaseFixture`, seeds as the

publisher, grants the places, then starts the service with `TesseraHost.Build` on a free port, with the reader's database URL. `ServiceTests` asks the running service for tiles.

CSHARP

```csharp
// tests/Tessera.IntegrationTests/ServiceTests.cs (shortened)
_app = TesseraHost.Build(new Dictionary<string, string?>
{
    ["Urls"] = $"http://127.0.0.1:{Port}",
    ["DATABASE_URL"] = Database.ReaderUrl,
    ["Tiles:Buffer"] = "64",
});
await _app.StartAsync();
Client = new HttpClient { BaseAddress = new Uri($"http://127.0.0.1:{Port}") };
```

The worked answer's fixture then waits while tiles answer 503, since its list loads in the background (N-23); yours loads its list before it listens, so leave the loop out until day 45. Now write `Serves_a_tile_as_a_mapbox_vector_tile`: it asks for the level-15 tile that holds the café `cafes-001`, at longitude 113.577677 and latitude 22.158784, and expects 200 and the ***content type*** `application/vnd.mapbox-vector-tile`, the label that tells a client what kind of bytes follow. A helper, `TileOf`, works out the tile's column and row from the café's position, so the test hard-codes neither. Leave out the worked answer's line for `Cache-Control`, which belongs to N-18. Run the test with `--filter-method`: it fails with 404, since no route knows the address.

> FROM SCHOOL `TileOf` is the formula every web map uses: the column is (longitude + 180) ÷ 360 × 2^z, rounded down, and the row comes from the Web Mercator formula. For `cafes-001` at level 15 the sums give 26,722.09 and 14,314.81, so the tile is 15/26722/14314.

For green, `TileServer.Answer`, in `Features/Tiles.cs`, checks in a fixed order and returns at the first check that fails:

CSHARP

```csharp
// src/Tessera/Modules/Tessera/Features/Tiles.cs (shortened)
public async Task<IResult> Answer(string layer, int z, int x, int y, CancellationToken cancel)
{
    // … the worked answer first answers 503 until its list of layers loads (N-23)
    if (layers.Find(layer) is not { } found)
        return Problems.NotFound("No published layer has that name.");
    if (TileAddress.Of(z, x, y, settings.MinZoom, settings.MaxZoom) is not { } tile)
        return Problems.NotFound("No tile has those coordinates.");

    var built = await read(Queries.Tile(found, tile, settings), cancel).ConfigureAwait(false);
    // … the bytes become a tile; a refusal becomes a problem document
}
```

Your copy's `TileServer` takes the `LayerCatalogue` itself, loaded once at start-up, where the worked answer takes a function that may give null: a difference until N-23. `read` is a `TileReader`, a delegate: the type of a function, here one that takes a query and gives the tile's bytes, so a test can pass a stand-in. The parameters take the route's names, so ASP.NET Core fills them from the address. `TileRoutes.MapTiles` adds two routes to one method:

```csharp
// src/Tessera/Modules/Tessera/Features/Tiles.cs (shortened)
string[] read = [HttpMethods.Get, HttpMethods.Head];
routes.MapMethods("/{layer}/{z:int}/{x:int}/{y:int}.pbf", read, server.Answer);
routes.MapMethods("/{layer}/{z:int}/{x:int}/{y:int}.mvt", read, server.Answer);
```

The ending `.pbf` is the one pg_tileserv uses, so its clients work unchanged, and `.mvt` is the format's own name. `{z:int}` lets the route match only a whole number.

The route needs settings and a place in the program. Write `TileSettings` in `Domain/Settings.cs`, the worked answer's record without `CacheSeconds`, which N-18 adds: `Resolution` 4,096, `Buffer` 256, `MaxFeatures` 10,000, `MinZoom` 0 and `MaxZoom` 22, each with its `[Range]`. Then make the module's one door, `Modules/Tessera/TesseraModule.cs`, with three methods. `AddSettings` binds the section `Tiles` to the record in one line, `services.AddOptions<TileSettings>().BindConfiguration("Tiles").ValidateDataAnnotations().ValidateOnStart()`, so `Tiles__Buffer` in the environment reaches it and a value out of range stops the start; day 40 gathers such lines into `Settings.Section`. `AddTessera` calls `AddSettings`, takes over day 2's `DatabaseConnections.Register` from `TesseraHost`, and registers one `Database`, opened with the reader's connection string. `MapTessera` loads the catalogue once with `CatalogueReader.Load`, a failure stopping the start with the database's message, then calls `MapTiles` with that catalogue, the `Database` and the tile settings. `TesseraHost.Build` calls `builder.AddTessera()` before it builds, and `app.MapTessera()` after. Two sections can now fail at once, and the start-up check then throws an `AggregateException` holding one `OptionsValidationException` for each, which day 2's `catch` lets through as a crash; give `Serve` a second `catch` for it, and print the `Failures` of each exception inside.

Each check returns a 404 problem document at once, so a request for an unknown layer or a tile off the grid never reaches the database: US-07's first test, 'refused before any query runs'.

`Queries.Tile` fills a fixed text with the layer's quoted names, around one SQL statement:

```sql
-- src/Tessera/Modules/Tessera/Infrastructure/Sql/Queries.cs, Queries.Tile
SELECT ST_AsMVT(tile.*, @layer, @resolution, 'tessera_shape') FROM (
  SELECT ST_AsMVTGeom(ST_Transform(t.{layer.Geometry.Sql}, 3857), ST_TileEnvelope(@z, @x, @y),
                      @resolution, @buffer, true) AS tessera_shape{properties}
  FROM {layer.Table.Sql} AS t
  WHERE t.{layer.Geometry.Sql} && ST_Transform(ST_TileEnvelope(@z, @x, @y,
                                  margin => @margin), @srid){order}
  LIMIT @limit
) AS tile
```

Read it from the inside out. `ST_TileEnvelope` gives the tile's square in Web Mercator. The `WHERE` clause turns that square, widened by a margin, into the layer's own system, such as SRID 4326, and keeps the places inside it. `ST_AsMVTGeom` moves each place into the tile's grid of 4,096 by 4,096 cells, and `ST_AsMVT` packs the rows into one tile. The words in braces are holes in a C# interpolated string, filled with names; the words after `@` are values, which day 12 examines.

> WATCH OUT Write the `WHERE` clause the other way round, as `ST_Transform(t.geom, 3857) && ST_TileEnvelope(…)`, and every test still passes. But PostGIS must then convert every place in the table before it can compare, so the spatial index never applies and each tile reads the whole table. The story's 'Critical to success' names this trap: transform the tile's box, never the table's shapes.

Last, add `Single` to `Database`, beside day 6's `Rows`: it runs a query that gives one value, or null for no row, through Dapper's `QuerySingleOrDefaultAsync` and the same `Run` that turns each failure into a `Result`. `MapTiles` passes `database.Single<byte[]>` as the `TileReader`. On success your copy answers `TypedResults.Bytes(bytes ?? [], ContentType)`: an empty tile is still a tile, 200 with no bytes. The test passes.

Now write the test day 2 left, the worked answer's `Starts_from_the_run_command_and_answers`, in `ServiceTests`. It starts the built `Tessera.dll` as a process of its own, with `Urls` on a free port and `DATABASE_URL` set to the fixture's `Database.ReaderUrl`, asks for the same tile every 100 ms until it answers, and expects 200 (US-00 c2 and test 1). Leave out its last two lines, which read the program's log for 'Request starting': without `tessera.json`'s `Logging` section your copy logs each request, so they wait for day 40. Then delete day 2's `Answers_a_request_on_its_port`: your start now loads the list of layers before it listens, so with its database on port 1 the program stops before it answers, and the new test proves criterion 2 in full.

12.2 Where each café sits

Criterion 3 asks that each feature sit at the right place in the tile's grid, and the worked answer proves it by reading the tile back. First write `Serves_the_same_body_for_mvt`, which fetches the same tile with both endings and expects equal, non-empty bytes. It passes at once, since one method serves both routes: criterion 2 costs no code.

Then write `Places_each_feature_where_it_lies`, a theory with two rows: `food.cafes`, stored in WGS 84, and `stage.cafes_3857`, the same cafés in Web Mercator, which the fixture makes and grants as the database's owner. Add the package `NetTopologySuite.IO.VectorTiles.Mapbox` to the integration tests, at the version the worked answer's `Directory.Packages.props` names, 1.1.0; its `MapboxTileReader` turns the bytes back into points in longitude and latitude. The test finds `cafes-001` among the features and expects its point within 0.00001 degrees of the café's longitude and latitude: one unit of the 4,096 grid at level 15 is about 0.3 metres. Both rows pass: the `WHERE` clause takes each layer's own SRID, and `ST_Transform` brings both into Web Mercator before the grid.

12.3 Three kinds of 'not found', and a grant withdrawn

Each refusal is a 404 with its own detail, and none repeats the name the stranger sent. `Answers_an_unknown_layer_with_a_problem_document` asks for `/food.zoos/0/0/0.pbf` and reads the body: `type` 'about:blank', `title` 'Not Found' and `status` 404, with the content type `application/problem+json` and `Cache-Control: no-store`. `Answers_a_tile_outside_the_grid_with_404` asks for `/food.cafes/1/5/0.pbf`: level 1 has two columns, 0 and 1, so column 5 names no tile; it checks the status and the content type. The first test fails until the next section gives every error its shape.

Today also closes US-03 c6, which asks for 404 once a grant is withdrawn, and needed a tile to ask for. In `tests/Tessera.IntegrationTests/CatalogueTests.cs`, add a helper `Server`, which builds a `TileServer` over a fixed catalogue and the reader's `Database`, and write `Answers_404_for_a_table_revoked_after_start_up`. Your copy's `Database.Open` takes the connection string alone, so write `Database.Open(database.Reader)` where the worked answer adds `new DatabaseSettings()`; the pool's settings arrive with N-21, on day 41. The test makes and grants `stage.revoked`, loads the catalogue, revokes the grant as the owner, then asks for a tile:

```csharp
// tests/Tessera.IntegrationTests/CatalogueTests.cs (shortened)
var catalogue = await Load(reader);
await database.Owner("REVOKE SELECT ON stage.revoked FROM tessera_reader;");

var answer = await Server(catalogue, reader).Answer("stage.revoked", 0, 0, 0,
    TestContext.Current.CancellationToken);

Assert.Equal((404, "application/problem+json"), await Http.StatusAndType(answer));
```

The catalogue still lists the layer, but PostgreSQL refuses the query with the code 42501, 'insufficient privilege'. `Http.StatusAndType`, a helper in `tests/Tessera.IntegrationTests/Http.cs`, runs the answer with no web server and gives its status and content type. Your copy's answer reads the code from day 6's `DatabaseError`:

```csharp
// your copy's Features/Tiles.cs, until N-22 (the book's own)
return built.Match(
    bytes => TypedResults.Bytes(bytes ?? [], ContentType),
    // 42501: the grant was withdrawn; 42P01: the table has gone (US-03 c6).
    error => error.SqlState is "42501" or "42P01"
        ? Problems.NotFound("No published layer has that name.")
        : Problems.Fault());
```

The worked answer sorts every code into one of four kinds, once, in `DatabaseError.Of`, and switches on the kind; N-22 brings that on day 42. The codes stay as text because `Features` may not use Npgsql, whose class `PostgresErrorCodes` names them: `BoundaryTests`' rule `Only_Infrastructure_talks_to_the_database` forbids it.

> NOTE **Beyond the worked answer.** `Answers_an_unknown_layer_with_a_problem_document` proves the 404, but not that no query ran. In `TileTests`, write `Refuses_an_unknown_layer_before_any_query_runs`: build a `TileServer` over a catalogue of one layer, with a stand-in `TileReader` that counts its runs, ask for `food.zoos`, and expect 404, read through `IStatusCodeHttpResult`, and a count of 0. That is why `TileReader` is a delegate: a test can swap the database for a function of three lines.

12.4 Every error has one shape

Every error the HTTP API answers must be a ***problem document***: a short report in JSON, in the shape RFC 9457 sets, so every client reads every failure alike. ASP.NET Core writes these documents itself; `Problems` picks the status and gives each one Tessera's shape.

12.4.1 Red: ASP.NET Core's own shape

Write `src/Tessera/Common/Api/Problems.cs` with `NotFound`, in two forms, with and without a detail, and `Fault`, each a call to `TypedResults.Problem`, and register the service with `builder.Services.AddProblemDetails()` in `TesseraHost.Build`. Run the two 404 tests. The status is right, but the body is ASP.NET Core's default:

```json
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.5","title":"Not Found","status":404,
 "detail":"No published layer has that name.","traceId":"00-4ae7598dad9bce7dd6765db5fd0f3756-…"}
```

The `type` is not 'about:blank', the answer carries no `Cache-Control`, and the `traceId` tells a stranger something of the service's insides.

12.4.2 Green: one function shapes them all

`Problems.Shape` runs on every problem document on its way out:

CSHARP

```
// src/Tessera/Common/Api/Problems.cs (shortened)
public static void Shape(ProblemDetailsContext context)
{
    context.ProblemDetails.Type = "about:blank";
    context.ProblemDetails.Extensions.Remove("traceId");
    context.HttpContext.Response.Headers.CacheControl = "no-store";
    // … a 503 adds Retry-After (N-22)
}

// src/Tessera/TesseraHost.cs
builder.Services.AddProblemDetails(options => options.CustomizeProblemDetails = Problems.Shape);
```

With `type` as 'about:blank', RFC 9457 makes the `title` the status's own phrase, such as 'Not Found', which ASP.NET Core fills in. `no-store` forbids a browser to keep any error. The tests pass. From now on every refusal goes through `Problems`: the story's 'Critical to success' says no handler writes an error body of its own.

12.4.3 An address no route knows

A route's refusals now have one shape, but an address that matches no route still gets an empty 404. Write `Answers_an_unknown_address_with_a_problem_document`, a theory with one row today, `/nothing-here`; US-23 adds two more on day 16. It expects 404, `application/problem+json` and no `detail`. It fails, since nothing writes a body. Add two steps to `TesseraHost.Build`, before `app.UseRouting()`:

CSHARP

```
// src/Tessera/TesseraHost.cs (shortened)
// The first catches a crash and the second an empty error, such as a 404 for an address no
// route knows; both answer with a problem document.
app.UseExceptionHandler();
app.UseStatusCodePages();
app.UseRouting();
```

Both lean on `AddProblemDetails`, so each writes its document through `Shape`, and the test passes.

> TRY IT Start your copy by hand and run `curl -i http://127.0.0.1:8080/food.cafes/12/3339/1788.pbf`, the example tile, then again with `-X POST`: 405, with `Allow: GET, HEAD` and a problem document.

> IN INK Commit at each green, with messages such as `US-07: serve a tile through one query, checked before it runs` and `US-03: answer 404 once a grant is withdrawn`, and push.

> THE PROOF The eight tests of Exhibit 12.1 pass, with your own, and so do `npm test` and CI. US-07 is done: 9 points of 29. Tomorrow's first test: `Sends_the_layer_and_the_tile_as_parameters`.

13

Day 12 · Wednesday 4 November: S-01, values beside the query

> IN SHORT Today you prove that no part of a tile's address enters the text of a query: each value travels beside it as a parameter, and one approved text serves every tile. This is the first day of S-01, worth 2 points.

A request can change the values a query reads, but never the words the query says. S-01 guards the tile query against SQL injection, which day 4 met: SQL hidden inside a request in the hope that the database runs it. Two defences stop it. Today you prove the first, for values; tomorrow, the second, for names.

EXHIBIT 13.1 Day 12 ends when every tile address gives the same approved text, and the address travels as parameters alone

Goal	Three tests of the tile query's text and values, an approved snapshot, and one door to the database
Story	S-01 c1 and c2; tests 1 and 2
In the worked answer	`Queries.Tile` and `Query` in `src/Tessera/Modules/Tessera/Infrastructure/Sql/`; `tests/Tessera.UnitTests/TileTests.cs`; `tests/Tessera.UnitTests/approved/tile-query.sql`; `Tessera.UnitTests.csproj`; `tests/Tessera.ArchitectureTests/`, its project file and `BoundaryTests.cs`
Look for	How an anonymous object's member names match the `@z` names in the SQL; why the property test folds random numbers onto the grid; why the test makes line endings alike before it compares
Leave for later	The quoting of names and the attack through the real database (S-01 c3 and c4, day 13)
Done when	`Sends_the_layer_and_the_tile_as_parameters`, `Matches_the_approved_tile_query` and `Keeps_every_tile_address_out_of_the_text` pass, and the last two have failed once on purpose; `Tessera.ArchitectureTests` exists, and its six rules pass

Source: `docs/user-stories/user-stories.md`, *S-01;* `docs/requirements/security-requirements.md`, *SR-05 to SR-07*

> FROM SCHOOL Your lessons named SQL injection as a threat and input validation as one defence. A ***bound parameter*** is the stronger defence. Think of a paper form: its printed questions are the query's text, and what you write in a box is a value. Whatever you write in the box, it never becomes a new printed question.

13.1 A query is a text and an object of values

Day 6 wrote `Query`, a fixed text with the values that travel beside it, and `Database`, which hands both to Dapper; Dapper sends each member of the values object to PostgreSQL as a parameter of the same name. Yesterday's tile query is the first to carry values, which `Queries.Tile` builds as an anonymous object, a record with no name made on the spot:

```csharp
// src/Tessera/Modules/Tessera/Infrastructure/Sql/Queries.cs, the end of Queries.Tile
new
{
    layer = layer.Id, resolution = limits.Resolution, z = tile.Z, x = tile.X, y = tile.Y,
    buffer = limits.Buffer, margin = (double)limits.Buffer / limits.Resolution,
    srid = layer.Srid, limit = limits.MaxFeatures,
}
```

So `z` fills `@z`. Of a request's parts, the level, column and row travel beside the text as `@z`, `@x` and `@y`, after `TileAddress.Of` has checked them; the layer's name never enters the text, since it is only a key to the catalogue, whose quoted name fills the table's hole (day 13).

13.2 Prove the address stays out of the text

Three unit tests read the query the service would send, with no database at all, so each runs in a fraction of a second.

13.2.1 The values beside the text

In `TileTests`, add a test layer, `Cafes`, built from a `CatalogueRow` with three properties and the key `id`, and fixed settings, `Limits`, with a buffer of 64, apart from the shipped 256, so a change to that default leaves the test unchanged. Then write `Sends_the_layer_and_the_tile_as_parameters`:

```csharp
// tests/Tessera.UnitTests/TileTests.cs
var query = Queries.Tile(Cafes, TileAddress.Of(15, 27154, 14280)!, Limits);

Assert.DoesNotContain("food.cafes", query.Text, StringComparison.Ordinal);
Assert.Equal(new { layer = "food.cafes", resolution = 4096, z = 15, x = 27154, y = 14280,
                   buffer = 64, margin = 0.015625, srid = 4326, limit = 10000 }.ToString(),
             query.Parameters?.ToString());
```

An anonymous object's `ToString` writes every member and its value, such as `{ layer = food.cafes, resolution = 4096, … }`, so comparing the two strings compares every value. Two anonymous objects made in different projects are different types, so the test compares their text, not the objects.

13.2.2 The approved text

S-01's second test compares the whole text with an ***approved snapshot***: a file that holds the output a person has read and accepted. Create `tests/Tessera.UnitTests/approved/tile-query.sql` with the text the query gives for `Cafes`:

```sql
SELECT ST_AsMVT(tile.*, @layer, @resolution, 'tessera_shape') FROM (
  SELECT ST_AsMVTGeom(ST_Transform(t."geom", 3857), ST_TileEnvelope(@z, @x, @y),
                      @resolution, @buffer, true) AS tessera_shape, "group", "layer", "name"
  FROM "food"."cafes" AS t
  WHERE t."geom" && ST_Transform(ST_TileEnvelope(@z, @x, @y, margin => @margin), @srid)
  ORDER BY t."id"
  LIMIT @limit
) AS tile
```

Add one line to `Tessera.UnitTests.csproj`, `<None Include="approved/**" CopyToOutputDirectory="PreserveNewest" />`, so the build copies the file beside the tests. Then write

`Matches_the_approved_tile_query`, which reads the file and compares it with `Queries.Tile(…).Text`. Both sides pass through `ReplaceLineEndings().TrimEnd()`, so a copy checked out on Windows, which ends each line with `\r\n`, compares equal.

13.2.3 Every address, one text

Criterion 1 asks for a property test, the kind day 4 wrote for `SqlName`: a rule that FsCheck tries on a hundred made-up inputs. Write `Keeps_every_tile_address_out_of_the_text`:

CSHARP

```
// tests/Tessera.UnitTests/TileTests.cs
[FsCheck.Xunit.Property]
public bool Keeps_every_tile_address_out_of_the_text(byte level, uint column, uint row)
{
    // Folds any random number onto a real tile: a level from 0 to 22, then a column and row
    // within that level's 2^z.
    var z = level % 23;
    var size = 1L << z;
    var tile = TileAddress.Of(z, (int)(column % size), (int)(row % size))!;
    return Queries.Tile(Cafes, tile, Limits).Text.ReplaceLineEndings().TrimEnd() == Approved;
}
```

FsCheck makes a hundred sets of three numbers and passes when every set returns true. Most random numbers name no tile, so the test folds each onto the grid with `%`, the remainder you met in school as MOD, and can never fail for the wrong reason, a tile that does not exist.

13.2.4 Red on purpose

All three tests pass at once, since yesterday's code is already right. Break it on purpose: in `Queries.Tile`, replace the first `@x` with `{tile.X}`, so the column enters the text, and run the class again.

TEXT

```
failed Tessera.UnitTests.TileTests.Keeps_every_tile_address_out_of_the_text
  Falsifiable, after 1 test (1 shrink) …
failed Tessera.UnitTests.TileTests.Matches_the_approved_tile_query
```

The property fails on its first try, for the right reason. `Sends_the_layer_and_the_tile_as_parameters` still passes: it searches the text for the layer's name alone, so it cannot see a column there. That is why the story asks for the property. Undo the change, and all three pass.

> WATCH OUT When `Matches_the_approved_tile_query` fails after a change you meant, the cure is to copy the new text over the approved file. Read it line by line first. The test proves only that nothing changed unseen; the person who approves the new text is the check.

13.3 One door to the database

Criterion 2 asks that every SQL text live in `Queries.cs` and that only `Database` send it, save the seed's journal schema, so a reviewer reads every query in one file.

An ***architecture test***, a test that reads the built program and checks which part may use which, proves the larger half. It needs the third test project, which day 2 left for today:

BASH

```
dotnet new xunit -o tests/Tessera.ArchitectureTests
dotnet sln add tests/Tessera.ArchitectureTests
```

> WATCH OUT Under your `Directory.Packages.props`, the template's own versions stop the restore: `error NU1008: The following PackageReference items cannot define a value for Version: coverlet.collector, Microsoft.NET.Test.Sdk, xunit, xunit.runner.visualstudio.` Replace the four lines as on day 2, with `xunit.v3` and `TngTech.ArchUnitNET.xUnitV3`, whose version, 0.13.4, goes in `Directory.Packages.props`.

Copy the worked answer's `BoundaryTests.cs` whole: its helpers and six rules. Rule 2, `Only_Infrastructure_talks_to_the_database`, is S-01's; the other five guard the borders between the layers that ADR 0010 sets, and pass on your copy as on the worked answer. Add `'ArchitectureTests'` first in the list of suites `scripts/test.ts` runs, and its step to your CI file.

CSHARP

```csharp
// tests/Tessera.ArchitectureTests/BoundaryTests.cs
// Rule 2: one gate to the database. Every SQL text and every connection lives in the
// Infrastructure (S-01 c2).
[Fact]
public void Only_Infrastructure_talks_to_the_database() =>
    Types().That().ResideInAssembly(Ours)
        .And().DoNotResideInNamespaceMatching(@"^Modules\.Tessera\.Infrastructure(\..+)?$")
        .Should().NotDependOnAny(Database)
        .Check(Program);
```

`Database` here names the libraries that talk to PostgreSQL: Npgsql, Dapper and DbUp. Break the rule on purpose: add a field of type `Npgsql.NpgsqlDataSource` to `TileServer`, run `dotnet test --project tests/Tessera.ArchitectureTests`, and read the message, which names the type and what it depends on. Remove the field, and the rule passes.

The rule keeps the database's libraries inside the Infrastructure, not every text inside `Queries.cs`; `docs/requirements/security-requirements.md` marks SR-06 'in part' for that reason. The criterion leaves the rest to a search at each review:

BASH

```bash
grep -rnw --include=*.cs -E 'SELECT|INSERT|CREATE|GRANT|DROP' src | grep -v Sql/Queries.cs
```

In the worked answer it finds two lines, both comments: `SqlName.cs`, line 5, and `Database.cs`, line 128. Record your copy's result in your notes. The seed's journal is the one exception the criterion names, and even its schema's text, day 7's `Queries.JournalSchema`, lives in `Queries.cs`.

> IN INK Commit at each green, with messages such as `S-01: prove the tile address travels as parameters alone` and `S-01: approve the tile query's text`, then push. No commit holds a break made on purpose.

> THE PROOF The three tests of `TileTests` pass, the two snapshot tests have each failed once for the right reason, and the six rules of `BoundaryTests` pass; `npm test` passes, architecture first. S-01 has one day left. Write tomorrow's first test in your notes: `Answers_an_attack_with_404_and_keeps_every_row`.

14

Day 13 · Thursday 5 November: S-01, names from the catalogue alone

> IN SHORT Today you prove that every name in a query comes from the catalogue, quoted by one function, and that an attack named as a layer harms nothing. S-01 ends: 2 points.

A name cannot travel as a parameter, so it enters the text, and one function makes it safe to enter. PostgreSQL binds values, never names, so names come only from the catalogue, through `SqlName`. Four proofs stand behind it: the quoting, the layer, the compiler and the database.

EXHIBIT 14.1 Day 13 ends when an attack in a layer's name meets 404 and changes no row, and an awkward name is served

Goal	Proof that only quoting makes a `SqlName`; proof through the real database that an attack does nothing
Story	S-01 c3 and c4; test 3. US-03 c4, the awkward name served.
In the worked answer	`Domain/SqlName.cs`; `SqlTests.cs`; in the integration tests, `ServiceTests.cs` and `CatalogueTests.cs`
Look for	Why the private constructor needs no architecture rule; why the attack never reaches the database
Leave for later	Nothing of S-01
Done when	`Answers_an_attack_with_404_and_keeps_every_row` and `Serves_the_awkward_name` pass

Source: `docs/user-stories/user-stories.md`, *S-01;* `docs/requirements/security-requirements.md`, *SR-07 and SR-08*

14.1 Two of the four proofs are already in ink

S-01 c3's first proofs are day 4's and day 5's tests: `Quotes_a_name_and_doubles_each_inner_quote` and `Quotes_every_name_safely` prove `SqlName.Quote`, and `Turns_a_row_into_a_layer_whose_id_is_schema_dot_name` proves that `ToLayer` quotes the table and each property. In yesterday's snapshot every name stands in quotes, an inner quote doubled, so none can end its quotes early.

14.2 Only quoting makes a `SqlName`

The compiler keeps this rule on every build, so no test needs to: only `Quote` and `Qualified` can make a `SqlName`, as a single-case union would guard it in F#. Break the rule where it matters most: in `ToLayer`, write `new SqlName(row.GeometryColumn)` for `SqlName.Quote(row.GeometryColumn)`, build, and undo it.

```
Catalogue.cs(28,13): error CS0122: 'SqlName.SqlName(string)' is inaccessible due to its
protection level
```
TEXT

14.3 An attack through the real database

Criterion 4 asks for a 404 problem document for a layer named `x; DROP TABLE cafes`, with every table keeping its rows; story test 3 adds `food.cafes'; --`. In `ServiceTests`, write `Answers_an_attack_with_404_and_keeps_every_row`:

CSHARP

```
// tests/Tessera.IntegrationTests/ServiceTests.cs
// S-01 c4 and test 3: an attack changes nothing.
[Theory]
[InlineData("x;%20DROP%20TABLE%20cafes")]
[InlineData("food.cafes'%3B%20--")]
public async Task Answers_an_attack_with_404_and_keeps_every_row(string layer)
{
    using var answer = await Get($"/{layer}/0/0/0.pbf");

    Assert.Equal(HttpStatusCode.NotFound, answer.StatusCode);
    Assert.Equal(100L, await service.Database.OwnerValue<long>("SELECT count(*) FROM food.cafes"));
}
```

Each name travels percent-encoded (`%20` a space, `%3B` a semicolon), and `OwnerValue` counts rows as the owner. The test passes at once (swap 404 for 200 to see it fail): the name is only a lookup key, so `Find` answers null and the route answers 404 before any query exists.

NOTE **Beyond the worked answer.** SR-08 is proved only 'in part': the attack test counts the rows of `food.cafes` alone, and never reads the content type. Add `Answers_an_attack_as_a_problem_document_and_keeps_all_nine_tables` to `ServiceTests`, with the same two rows: expect 404 and `application/problem+json`, and 900 rows in US-24's nine tables, counted as the owner:

CSHARP

```
Assert.Equal(900L, await service.Database.OwnerValue<long>("""
  SELECT count(*) FROM (
    SELECT 1 FROM food.cafes UNION ALL SELECT 1 FROM food.restaurants
    -- … UNION ALL the other seven tables
    ) AS places
  """));
```

14.3.1 The awkward name is served

US-03 c4 asks that a table whose name holds a space and a double quote be served; day 6's stage already grants `stage."odd ""name"""`. Write `Serves_the_awkward_name` in the form of day 11's `Answers_404_for_a_table_revoked_after_start_up`, asking `Server(catalogue, reader)` for tile 0/0/0 of `stage.odd "name"` and expecting 200. `SqlName.Qualified` quotes it as one name.

IN INK Commit `S-01: refuse an attack, keep every row` and `US-03: serve the awkward name`; push.

THE PROOF Today's tests and your own pass, with `npm test` and CI. S-01 is done: 11 points of 29. Tomorrow's first check: the address ending `%zz`, sent through Nginx.

15

Day 14 · Friday 6 November: S-06, refuse malformed addresses

IN SHORT Today Nginx enters the build and stands in front of the service. Each malformed address meets a refusal as a problem document that never repeats the address, and you find and close one disguise that slips through. This is S-06, worth 1 point, and the day ends with the weekly check.

No layer of the service should repair an address: each refuses what it cannot read, and the routes match an address only as it was sent. A ***malformed URL*** is an address disguised so that it looks clean only after a web server tidies it, such as a path that climbs above the root, or `..` hidden as `%2e%2e`. Three doors stand between a stranger and a route: Nginx, then Kestrel, ASP.NET Core's own web server, then the routes.

EXHIBIT 15.1 Day 14 ends when five malformed addresses meet a problem document through Nginx, and a clean one gets its tile

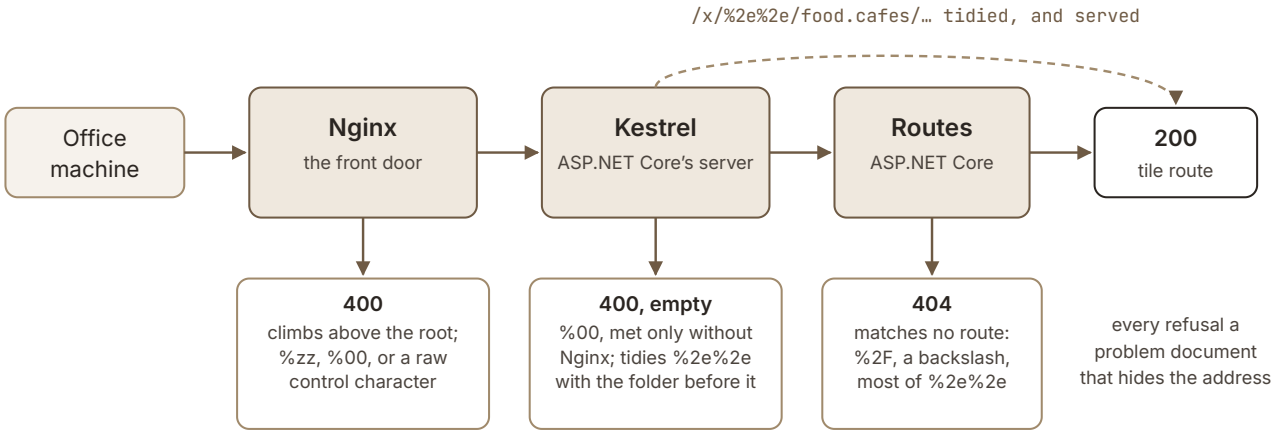
Goal	Nginx in front of the service; the checks of S-06 through Nginx; a sixth target the worked answer lets through
Story	S-06 c1 to c4; tests 1 to 4
In the worked answer	`nginx/Dockerfile` and `nginx/default.conf.template`; `scripts/lib/edge.ts` and `scripts/e2e.ts`; `Answers_an_unknown_address_with_a_problem_document` in `ServiceTests.cs`
Look for	Why the checks write requests byte for byte; why `/malformed` is `internal`; why Nginx passes the address on unchanged; what Kestrel does to `%2e%2e`
Leave for later	Nginx's office-only rule (S-07), its limits (S-30), its tile store (N-19), gzip (N-20) and the second copy (N-29); the browser tests (US-23)
Done when	`npm run e2e` prints that the Nginx checks passed, with the sixth target you add, and `Answers_an_unknown_address_with_a_problem_document` still passes

Source: `docs/user-stories/user-stories.md`, *S-06;* `docs/guides/delivery-plan.md`, *iteration 2*

15.1 Behind Nginx, the service already answers in one shape

The service's own door needs no new code today. Day 11's `UseStatusCodePages` answers an address that matches no route with a 404 problem document, and `Answers_an_unknown_address_with_a_problem_document` proves it: S-06's test 3. Kestrel refuses `%00` itself with an empty 400, no problem document; Nginx refuses that target first, so a browser behind Nginx never meets Kestrel's answer. Exhibit 15.2 shows where each target stops.

EXHIBIT 15.2 Three doors, each refusing what it cannot read; Kestrel tidies one disguise, and that one reaches a route



Source: S-06 c1 to c4 in `docs/user-stories/user-stories.md`; `nginx/default.conf.template`; each answer seen on a scratch copy of the worked answer, 17 October 2026

15.2 Nginx enters the build

Nginx is a ***reverse proxy***: a server that stands in front of another, receives every request first, and passes on the ones it accepts. From today every request from the office meets it before the service. *Sixteen Tiles*, chapter 4, explains Nginx in full; today builds only what S-06 asks.

Write `nginx/Dockerfile`, one line that holds Nginx to release 1.29 on Alpine, as the worked answer does: `FROM nginx:1.29-alpine`. Then write a cut-down `nginx/default.conf.template` of the book's own. The worked answer's holds rules for six later stories; yours holds three parts today, each in the worked answer's words:

NGINX

```
# nginx/default.conf.template: the book's cut-down version for day 14
upstream tessera {
    server 127.0.0.1:${TESSERA_SERVICE_PORT};
}

server {
    listen ${TESSERA_HTTPPORT};

    location / {
        proxy_pass http://tessera;
    }

    error_page 400 /malformed;
    location = /malformed {
        internal;
        default_type application/problem+json;
        add_header Cache-Control no-store always;
        return 400 '{"type":"about:blank","title":"Bad Request","status":400}';
    }
}
```

Nginx refuses an address it cannot read with 400, before any `location` runs; `error_page` changes only the words of that refusal, from an HTML page to a problem document. The word `internal` means only Nginx's own `error_page` reaches `/malformed`; a browser that asks for it by name gets 404. The Nginx image fills

each `${…}` in the template from the container's environment when it starts, so one file serves the office on port 80 and the tests on a free port.

> WATCH OUT Write `proxy_pass http://tessera;` with no path after the name. Nginx then passes the address exactly as it arrived. With a path, Nginx would pass its own tidied form, and a disguised address could reach a route repaired: the one thing the story forbids.

15.3 Check the doors through Nginx

The checks of S-06 are a script, not a C# test, because they need a real Nginx in front of the real service. Write `scripts/lib/edge.ts` with its function `checkEdge` and its helper `raw`, which opens a socket and writes each request byte for byte, so nothing tidies an address before Nginx sees it. Today it holds two of the worked answer's checks. The first asks for `/food.cafes/0/0/0.pbf` and expects 200 and the tile's content type: a clean address reaches the route unchanged. The second sends five targets:

EXHIBIT 15.3 Five targets that must each meet 400 or 404 as a problem document, and the door that refused each through the cut-down Nginx

TARGET	WHAT IT HIDES	ANSWER, AND FROM WHERE
`/food.cafes/%2e%2e/1/2/3.pbf`	`..`, encoded	404, the routes: Kestrel tidies it to `/1/2/3.pbf`
`/food.cafes/../../1/2/3.pbf`	A path that climbs above the root	400, Nginx
`/food.cafes%2F15/1/2.pbf`	An encoded slash	404, the routes
`/food.cafes/15/1/2.pbf%zz`	An invalid `%`	400, Nginx
`/food.cafes/15/1/2.pbf%00`	An encoded null	400, Nginx

Source: `scripts/lib/edge.ts`, the check marked S-06; the answers were seen on a scratch copy, 17 October 2026

TS

```ts
// scripts/lib/edge.ts (shortened)
const answer = await raw(base, target);
assert.match(answer, /^HTTP\/1\.1 40[04] /, target);
assert.match(answer, /application\/problem\+json/, target);
assert.doesNotMatch(answer.split('\r\n\r\n')[1] ?? '', /food\.cafes/, target);
```

The worked answer's `scripts/e2e.ts` starts a fresh database in Docker, seeds and grants it, starts two copies of the built service and Nginx on free ports, runs the browser tests, then calls `checkEdge`. Your copy has no map yet and one copy of the service, so write a cut-down `e2e.ts` of the book's own, with one copy and no Playwright step until US-23 adds one on day 18. Copy the worked answer's way of starting each part, and give its containers names of their own, so a test run never touches `rebuild-db`. Add `"e2e"` to `package.json` and `node scripts/e2e.ts` as the last step of `scripts/test.ts`, run `npm run build`, then `npm run e2e`. Without `error_page 400 /malformed`, the check fails at `../..`, the first target Nginx refuses, since Nginx's own 400 page is HTML. With it, every check passes.

> FROM SCHOOL An address carries any byte as `%` and two hexadecimal digits, the base-16 numbers you met beside binary: `%20` is a space, 32 in denary, and `%2e` a full stop, 46. `%zz` names no byte, since z is no hexadecimal digit, so Nginx cannot read the address and refuses it.

15.4 A disguised path reaches a route

The worked answer's checks pass, yet it has a known fault, which *The World in Squares* names on S-06's page: a disguised path reaches a route. Kestrel decodes `%2e%2e` and removes the dot segment before routing, as a browser would. The check's own target tidies to `/1/2/3.pbf`, which matches no route; but `/x/%2e%2e/food.cafes/0/0/0.pbf` tidies into a tile address, and on a scratch copy of the worked answer it answered 200 with the tile, directly and through the cut-down Nginx above. So S-06 c3 holds only for the target `checkEdge` sends. Do not copy the fault: the story's open question, 'whether Nginx should refuse the disguised forms with 400 too', now has an answer.

NOTE **Beyond the worked answer.** Add `'/x/%2e%2e/food.cafes/0/0/0.pbf'` to the list of targets in your `edge.ts`, and run `npm run e2e`: it fails, with 200. Then make Nginx refuse any target that holds `%2e`, in either case, inside `location /`:

NGINX

```
location / {
    # Beyond the worked answer: a dot hidden as %2e is a disguise, so refuse it.
    if ($request_uri ~* "%2e") {
        return 400;
    }
    proxy_pass http://tessera;
}
```

`$request_uri` is the target as the client sent it, and `~*` ignores case, so `%2E` meets the rule too; the 400 passes through `error_page` as the same problem document. Run the checks again; all six pass. Keep the rule inside the `location`: at the level of `server`, the internal redirect to `/malformed` would meet it again, and Nginx would send its own HTML page. When the tiles gain a `location` of their own with N-19 (day 38), copy the rule into it. The other cure, in the service, refuses a request whose raw target, `RawTarget` on `IHttpRequestFeature`, differs from the tidied path.

TRY IT `security-requirements.md` names two more gaps: the checks accept 400 or 404, so they cannot tell Nginx's refusal from the service's, and send no raw control character (SR-09), and no test sends a backslash (SR-11). Beyond the worked answer, add two targets to `edge.ts`: `/food.cafes/0/0/` with a raw tab before `0.pbf`, which must meet exactly 400 from Nginx, and `/food.cafes\0\0\0.pbf`, which must meet 404 as a problem document. Both answered so on the scratch copy. Then record which door refused each target, as Exhibit 15.3 does.

IN INK Commit at each green, with messages such as `S-06: put Nginx in front, and refuse malformed addresses` and `S-06: refuse a dot hidden as %2e`, and push. Then the weekly check: set the burn-up's actual figure beside the plan's 12, as on day 5; a gap of one day would use the day in hand, Friday 4 December.

THE PROOF `npm run e2e` prints its line of passed checks, the sixth target among them, and `npm test` passes. S-06 is done: 12 points of 29, on the planned line. Push before the weekend. Write Monday's first test in your notes: 'asks nothing more of a machine already in mirrored mode'.

16

Day 15 · Monday 9 November: US-35, the office reaches port 80

> IN SHORT Today every office machine reaches the service on port 80, by the Windows machine's office name, and Nginx sees each machine's address. US-35: 1 point.

Only a request from another office machine can prove this story, so tests check the steps and the real machine the result. In ***mirrored networking***, WSL2's mode for sharing the network, Ubuntu takes the Windows machine's address, so a request on port 80 reaches Nginx.

EXHIBIT 16.1 Day 15 ends when the steps' tests pass and a tile answers 200 from another office machine

Goal	`npm run office-port`, which checks WSL2's mode and prints the firewall steps; port 80 open to the office
Story	US-35 c1 to c6; tests 1 to 3
In the worked answer	`scripts/`: `lib/office.ts`, `lib/office.test.ts`, `office-port.ts`
Look for	Why the steps are text a person runs; why both firewalls need a rule; why no step sets up a port proxy
Leave for later	The office-only rule (S-07); `npm start` (day 34)
Done when	The three tests in `office.test.ts` pass, and checks 2 and 3 pass on the office machine

Source: `docs/user-stories/user-stories.md`, *US-35;* `docs/guides/delivery-plan.md`, *iteration 2*

> FROM SCHOOL A browser tries port 80 when an address names none. A ***firewall*** decides which requests may enter a machine; Windows and its Hyper-V, which runs WSL2, each keep one.

16.1 The steps, as pure functions

The steps need an administrator on Windows, so the script prints them and a test reads them. Red first: write the worked answer's three tests in `scripts/lib/office.test.ts`:

```ts
// scripts/lib/office.test.ts (shortened)
test('asks nothing more of a machine already in mirrored mode', () => {
  assert.deepEqual(networkingSteps('mirrored\n'), []);
});

test('tells a machine in NAT mode, or an older WSL, how to switch', () => {
  for (const mode of ['nat', '']) {
    const steps = networkingSteps(mode).join('\n');
    assert.match(steps, /networkingMode=mirrored/);
    assert.match(steps, /wsl --shutdown/);
  }
});
```

The third, 'opens the one port in both firewalls, and never through a port proxy', calls `firewallSteps('80')` and expects a `New-NetFirewallRule` with `-LocalPort 80`, a `New-NetFirewallHyperVRule` with `-LocalPorts 80`, and no `portproxy` anywhere. Run them; they fail, since `office.ts` does not exist.

```bash
node --test scripts/lib/office.test.ts
```

16.1.1 Green: `office.ts`

`networkingSteps` takes the mode, as `wslinfo --networking-mode` reports it, and returns the steps to switch it, or none:

```ts
// scripts/lib/office.ts (shortened)
export function networkingSteps(mode: string): readonly string[] {
  if (mode.trim() === mirrored) return [];
  return [
    // … a line that names the mode found and the mode needed
    '  1. Add these two lines to %UserProfile%\\.wslconfig:',
    '       [wsl2]',
    `       networkingMode=${mirrored}`,
    '  2. In PowerShell, run "wsl --shutdown", then open Ubuntu again and rerun this script.',
    '',
  ];
}
```

`firewallSteps` returns the two rules for PowerShell and the two checks to make from another machine. The Windows rule opens the port only on the office's own networks; the Hyper-V rule names WSL's fixed `-VMCreatorId`, so it opens the port to WSL alone. Then write `scripts/office-port.ts`, which runs only inside WSL2 (it checks `/proc/version` for Microsoft's name), reads the mode, prints both sets of steps, and ends with 1 when the mode must change first. Add `"office-port": "node --env-file-if-exists=.env scripts/office-port.ts"` and `"test:scripts": "node --test \"scripts/**/*.test.ts\""` to `package.json`, and the same `node --test` line to `scripts/test.ts`, after `tsc`, so the scripts' own tests run in every `npm test`.

```bash
node --test scripts/lib/office.test.ts
npm run typecheck
```

> WATCH OUT Many guides to WSL2 open a port with a Windows port proxy, `netsh interface portproxy`. It works, but every request then seems to come from the Windows machine itself, so Nginx can never tell one office machine from another, which S-07 and S-30 need. The third test forbids the proxy for this reason.

16.2 The checks on the office machine

Tests 2 and 3 need the office machine itself, with Windows 11 22H2 or later and WSL 2.0 or later for mirrored networking. There, inside Ubuntu, run your copy's script and follow what it prints:

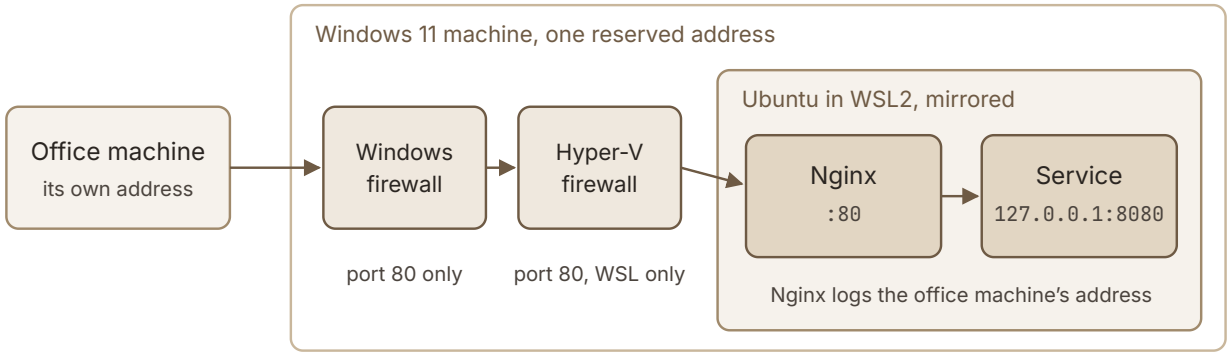
```bash
npm run office-port
```

Then start two things by hand, since `npm start` arrives only on day 34: your service on port 8080, over `rebuild-db`, and your Nginx in front of it on port 80. The line below is the book's own, shaped like `nginxArguments` in the worked answer's `plan.ts`, with two of its four variables:

```bash
docker build --tag tessera-nginx nginx
docker run --detach --name tessera-nginx --restart unless-stopped --network host \
  --env TESSERA_HTTPPORT=80 --env TESSERA_SERVICE_PORT=8080 \
  --volume "$PWD/nginx/default.conf.template:/etc/nginx/templates/default.conf.template:ro" \
  tessera-nginx
```

`--network host` lets Nginx share Ubuntu's network, so it opens port 80 itself and reaches the service at 127.0.0.1. Exhibit 16.2 shows a request's path through two firewalls, each with its own rule.

EXHIBIT 16.2 One tile request from an office machine, through two firewalls, to your service



Source: `scripts/lib/office.ts`; `scripts/lib/plan.ts`; `docs/user-stories/user-stories.md`, *US-35; port 8080 is your copy's until N-21*

EXHIBIT 16.3 Three checks prove the story, and only the first runs without the office

STORY TEST	WHERE	PASSES WHEN
1 The steps, without Windows	Any machine	The three tests in `office.test.ts` pass
2 Port 80 serves	Another office machine	`curl -I http://<office name>:80/food.cafes/0/0/0.pbf` gets 200, and `docker logs --tail 5 tessera-nginx` shows that machine's address

3 A restart changes nothing | The office machine | After `wsl --shutdown` and a fresh start, check 2 passes again

Source: US-35 'Tests to write first'; `firewallSteps` in `scripts/lib/office.ts`

Nginx begins each line of its log with the sender's address, so check 2 reads that column: it must hold the other machine's address, not the Windows machine's. In check 3, start the service again by hand; `--restart unless-stopped` brings Nginx back.

NOTE The requirements files mark US-35's criteria 1, 3 and 4 (FR-55 to FR-57) and 2 (SR-14) 'No test yet': only the real machine can show them. SR-15, the two firewalls, is proved in part, since the test reads the printed steps. Record each check's result and date in your comparison note, and list the inbound firewall rules once, to know no other port stands open. Criterion 3 also needs a reserved address for the Windows machine: ask whoever runs the office network.

IN INK Commit `US-35: print the steps that open port 80 to the office`, and push. Checks 2 and 3 leave no test behind, so your note is their record.

THE PROOF `office.test.ts` and `npm test` pass, and from another office machine a tile answers 200 on port 80, before and after a restart of WSL2. US-35 is done: 13 points of 29. Write tomorrow's first test in your notes: `Writes_the_area_block_before_the_head_closes`.

17

Day 16 · Tuesday 10 November: US-23, the area in the page and the nine layers

IN SHORT Today the service writes the office's area into the map's page, and the map learns the nine layers from its own code. Both are pure functions with fast tests; this is the first of three days on US-23, worth 3 points.

Before the map can open anywhere, it must know two things without asking: where the area lies, and which layers exist. The service answers the first by writing the area into the page it sends. The map answers the second from a list in its own code, so it never loads a list of layers. Tomorrow builds the model on these two facts; Thursday puts it in a browser.

US-23, the first story of Part B, builds on US-07's tiles and US-35's office. The day has four steps: the service fills in the page, serves it, the page reads the area back, and the map names its layers.

EXHIBIT 17.1 Day 16 ends when the page carries the area and the map knows its nine layers

Goal	`AreaSettings`, `MapSettings`, `MapRoutes.Render`, `LayerOf`, `Answer` and `UseMap` in the service; `areaOf` and the nine layers in the map; their tests
Story	US-23, criteria 1, 2, 5 and 7, in `docs/user-stories/user-stories.md`
In the worked answer	`src/Tessera/Modules/Tessera/Features/Map.cs` and `Features/Settings.cs`; `AreaSettings` in `Domain/Settings.cs`; `TesseraModule.cs`, `AddSettings` and `MapTessera`; `tests/Tessera.UnitTests/MapTests.cs`; `ServiceTests.cs`; `web/src/shared/area.ts`; `web/src/layers/layers.ts` and `layers.test.ts`; `docs/adr/0006-the-map-and-the-service.md`
Look for	Why the area travels inside the page and not in a second request; why the JSON sits in a `<script>` block that never runs; why one 404 says why and another says nothing
Leave for later	The model and the update (day 17); MapLibre, the browser tests and the test of the built page (day 18); the zoom buttons (UI-09, day 20)
Done when	The five tests of `MapTests`, two new rows of `Answers_an_unknown_address_with_a_problem_document`, the four tests of 'US-23 the area comes with the page' and the three of 'US-23 the map names the nine layers in its own code' pass, with your own `Names_the_same_nine_layers_as_the_map`

Source: `docs/user-stories/user-stories.md`, *US-23;* `docs/requirements/functional-requirements.md`, *FR-25 to FR-32*

17.1 The service writes the area into the page

The page must open on the office's own area at once, so the service writes the area into the page; a second request would cost a round trip and a moment of blank map. The area is a setting: a name, a box of four numbers in degrees, and a home view of a centre and a level. Copy the record `AreaSettings` from the worked answer's `Domain/Settings.cs`, whole: its defaults are Macau's, and its `Validate` refuses a box turned inside out. Bind the section `Area` in `TesseraModule.AddSettings` with the same line as day 11's

`Tiles`. Your copy has no `tessera.json` until day 40, so the defaults serve, and a variable such as `Area__Name` can change one.

17.1.1 Red: the area travels inside the page

Create `tests/Tessera.UnitTests/MapTests.cs` and write two tests. `Writes_the_area_block_before_the_head_closes` hands `MapRoutes.Render` a small page and Macau's settings, finds the block with the id `tessera-area`, checks that it ends before `</head>`, and reads its JSON back: the name 'Macau' and the box `[113.52, 22.10, 113.61, 22.22]`. `Matches_the_block_in_adr_0006` checks the whole block, letter for letter, against the text ADR 0006 prints:

JSON

```
{"name":"Macau","bounds":[113.52,22.1,113.61,22.22],"home":{"center":
{"lng":113.55,"lat":22.19},"zoom":15}}
```

The compiler refuses, since `MapRoutes` does not exist: today's first red.

17.1.2 Green: Render, a pure function

Create `src/Tessera/Modules/Tessera/Features/Map.cs` with a static class `MapRoutes`, and write `Render`. It turns the area into JSON with `System.Text.Json` and puts the block in front of the head's end:

CSHARP

```
// src/Tessera/Modules/Tessera/Features/Map.cs (shortened)
public static string Render(string html, AreaSettings area)
{
    var json = JsonSerializer.Serialize(new
    {
        name = area.Name,
        bounds = new[] { area.MinLongitude, area.MinLatitude, area.MaxLongitude, area.MaxLatitude },
        home = new { center = new { lng = area.HomeLongitude, lat = area.HomeLatitude },
                     zoom = area.HomeLevel },
    });
    var head = html.IndexOf("</head>", StringComparison.OrdinalIgnoreCase);
    var script = $"""<script id="tessera-area" type="application/json">{json}</script>""";
    return head < 0 ? html : html[..head] + script + html[head..];
}
```

The `new { … }` is an anonymous type, as on day 12, whose member names become the JSON's keys exactly as written. A page with no `</head>` is not the built map's page, so it goes out untouched rather than broken.

The type `application/json` tells the browser the block holds data, so it never runs. The page's ***Content Security Policy***, the header that says which scripts a page may run, therefore lets it through: the policy forbids scripts written into the page, and this block is no script.

> WATCH OUT Write `html.IndexOf("</head>")` with no second argument, and the build stops: Meziantou's rule MA0074, which the worked answer's `Directory.Build.props` turns on, asks for a `StringComparison`. Without one, .NET compares by the rules of the machine's language, which differ from office to office. `OrdinalIgnoreCase` compares character codes alone, and still finds `</HEAD>` in capitals.

17.2 A page for each known layer, and a 404 that says why

The service opens a page only for one of the nine layers, and answers 404 with a reason when it has no page to show. Both rules are pure functions, so `MapTests` calls them with no web server.

17.2.1 Red, then green: one of the nine, letter for letter

Write two theories. `Opens_a_page_named_for_one_of_the_nine_layers` expects `food.cafes.html` to give `food.cafes` and `transport.buses.html` to give `transport.buses`. `Refuses_any_other_name` expects null for `index.html`, `food.zoos.html`, `food.cafes` and `food.cafes.htm`. Then write the set `LayerIds` of nine names, in the four groups of US-24, and `LayerOf`:

```csharp
public static string? LayerOf(string file) =>
    file.EndsWith(".html", StringComparison.Ordinal)
        && LayerIds.Contains(file[..^".html".Length])
        ? file[..^".html".Length]
        : null;
```

`file[..^".html".Length]` is a range: every character up to the fifth from the end, so 'food.cafes.html' loses its '.html'. `/index.html` answers 404 too, so each view of the map has one address.

17.2.2 Red, then green: the preview switched off

Criterion 5 says that with the preview switched off in the settings, `/` answers 404. Write `Answers_404_when_there_is_no_page_to_show`, a theory with two rows: the preview off, and a release that carries no page. Each expects a `ProblemHttpResult` with status 404 and its own detail. Add the record `MapSettings`, with one setting, `ShowPreview`, true by default, to `Features/Settings.cs`, and bind the section `Map` beside `Area`. Then write `Answer`:

```csharp
internal static IResult Answer(string? page, MapSettings settings) =>
    !settings.ShowPreview ? Problems.NotFound("The map preview is switched off.")
    : page is null ? Problems.NotFound("This release carries no map.")
    : TypedResults.Text(page, "text/html; charset=utf-8");
```

`Answer` is `internal`, so only its own project and its friends, the test projects that day 2's `InternalsVisibleTo` lines name, may call it.

> KEY IDEA Two 404s, two jobs. A page named for no layer answers with no detail, so a stranger learns nothing; a preview switched off is the office's own choice, so its 404 says why.

17.3 The service serves the page

`UseMap` joins the three functions to the web server: it reads the page once at start-up, serves its files, and adds two routes. Neither the page nor the area changes while the program runs, so `Render` runs once, not once a request.

```csharp
// src/Tessera/Modules/Tessera/Features/Map.cs (shortened: the files under /assets left out)
var file = Path.Combine(folder, "index.html");
var page = File.Exists(file) ? Render(File.ReadAllText(file), area) : null;
// …
var pages = app.MapGroup("");
pages.AddEndpointFilter(PageHeaders);
string[] read = [HttpMethods.Get, HttpMethods.Head];
pages.MapMethods("/", read, () => Answer(page, settings));
pages.MapMethods("/{file}", read, (string file) =>
    LayerOf(file) is null ? Problems.NotFound() : Answer(page, settings));
```

`PageHeaders` is an ***endpoint filter***, a step ASP.NET Core runs around each route of a group. When the answer is the page, it adds four headers: `Cache-Control: no-cache`, the Content Security Policy in `MapRoutes.Policy`, `X-Content-Type-Options: nosniff` and `Referrer-Policy: no-referrer`. A 404 keeps the problem document's own headers, and the group keeps all four off the tile route. The page's scripts and styles under `/assets` go out through `UseStaticFiles`, with `public, max-age=31536000, immutable`: each file's name carries a hash of its content, so a browser may keep it for a year.

Last, call `app.UseMap(…)` in `TesseraModule.MapTessera`, with the area, the map's settings and the folder `wwwroot` beside the program, as the worked answer does.

17.3.1 Red, then green: names that open no page

Your `ServiceTests` already holds `Answers_an_unknown_address_with_a_problem_document`, with the row `/nothing-here`, from day 11. Add two rows, `/index.html` and `/food.zoos.html`. Before `UseMap`, both pass by luck, since no route knows them; so run them once with `UseMap` in place and the `is null` check in the `/{file}` route left out, and watch the two rows go red. Then put the check back.

The worked answer's `Serves_the_map_page_with_the_area_written_in` waits until Thursday: it fetches the built page and the script the page names, and your copy has no built map until then. Today `/` answers 404, 'This release carries no map.'

17.4 The page reads the area back

Your copy has no `web/` yet, so make it first. Copy the worked answer's project files, `web/package.json`, `package-lock.json`, the two `tsconfig` files, `vite.config.ts`, `eslint.config.js`, `index.html` and `.gitignore`, and its shared kernel, `web/src/shared/` without `area.ts`: `result.ts`, `pipe.ts`, `json.ts`, `problem.ts`, and `geo.ts`, whose `Level` the area's home view needs and day 20 explains. Run `npm ci` in `web/`. Then give `scripts/test.ts` the map's step, `npm ci` and `npm test` in `web/`, before the build, and `scripts/build.ts` the worked answer's two lines that build the map before `dotnet build`; `Tessera.csproj` takes the built map on day 18.

The map reads the block with one pure function, `areaOf`, in `web/src/shared/area.ts`, and falls back on Macau when the block is missing or wrong, which happens only under `vite`, the development server that sends the page without the service.

In `web/src/shared/shared.test.ts`, write the four tests of 'US-23 the area comes with the page': 'reads the area the service sends', 'refuses an area with no name, a box turned inside out, or a home level outside 0 to 22', 'reads the page's text into an area, or opens on Macau when the text holds none it can read' and 'reads a box only as four numbers in degrees, west before east and south before north'. Then write the schema with valibot and the reader, `areaOf`, one `pipe`: parse the JSON, check it against the schema with `parseArea`, and fall back on `MACAU`.

> KEY IDEA The C# anonymous type in `Render` and the TypeScript `Area` describe the same JSON from two sides. `Matches_the_block_in_adr_0006` fixes the text the service writes, and `shared.test.ts` fixes the text the map reads; if either side changes the shape, one test goes red.

17.5 The map names the nine layers in its own code

Criterion 7 asks the map to know its layers without asking the service, so the list lives in `web/src/layers/layers.ts`, and the page builds every tile's address on its own origin.

In `web/src/layers/layers.test.ts`, write the three tests of 'US-23 the map names the nine layers in its own code':

1. 'names the nine layers of US-24, in four groups, in the map's order';
2. 'builds each tile's address on the page's own origin', which also checks that an origin ending in '/' makes no '//';
3. 'reads the layer a page's path names, and only one of the nine', which refuses `/public.roads.html`, `/food.cafes.json` and a tile's own path.

Then write `LAYERS`, `tileUrl` and `layerFromPath`:

```ts
// web/src/layers/layers.ts (shortened)
export function tileUrl(origin: string, id: LayerId): string {
  return `${origin.replace(/\/+$/, "")}/${id}/{z}/{x}/{y}.pbf`;
}

export function layerFromPath(pathname: string): LayerId | undefined {
  const match = /^\/([^/]+)\.html$/.exec(pathname);
  const name = match?.[1];
  return name !== undefined && isLayerId(name) ? name : undefined;
}
```

`{z}`, `{x}` and `{y}` stay as written: MapLibre, the library that draws the map, fills in the level, column and row for each tile it fetches. `LAYERS` ends with `as const satisfies`, so `LayerId` is the union of nine exact names, and a misspelt layer fails to compile.

NOTE **Beyond the worked answer: two lists that must agree.** The service names the nine layers in `MapRoutes.LayerIds`, and the map names them in `LAYERS`. No test in the worked answer compares the two, though both `Map.cs` and *The World in Squares* say they must match. Write one in a new file, `tests/Tessera.UnitTests/LayerListTests.cs`. It climbs from the test's folder until it finds `web/src/layers/layers.ts`, reads each `id:` from it, and compares the two lists, each sorted:

CSHARP

```csharp
// Beyond the worked answer: US-23 c7.
public sealed partial class LayerListTests
{
    private static readonly string[] Layers = ["web", "src", "layers", "layers.ts"];

    [GeneratedRegex("""id: "(?<id>[a-z]+\.[a-z]+)",""",
        RegexOptions.ExplicitCapture, 1000)]
    private static partial Regex LayerId();

    [Fact]
    public void Names_the_same_nine_layers_as_the_map()
    {
        var folder = new DirectoryInfo(AppContext.BaseDirectory);
        while (!File.Exists(Path.Combine([folder.FullName, .. Layers])))
            folder = folder.Parent
                ?? throw new FileNotFoundException("web/src/layers/layers.ts");
        var text = File.ReadAllText(Path.Combine([folder.FullName, .. Layers]));
        var map = LayerId().Matches(text).Select(m => m.Groups["id"].Value);

        Assert.Equal(MapRoutes.LayerIds.Order(StringComparer.Ordinal),
                     map.Order(StringComparer.Ordinal));
    }
}
```

Change `food.cafes` to `food.cafe` in `layers.ts` and it fails, the collections differing at index 2; put it back, and it passes.

IN INK Commit at each green, with the messages `US-23: write the area into the map's page`, `US-23: open a page for the nine layers alone, and 404 with the preview off`, `US-23: serve the map page with its headers`, `US-23: name the nine layers in the map's own code` and `US-23: check the service and the map name the same layers`; push.

THE PROOF `MapTests`, your `LayerListTests` and `ServiceTests` pass in `dotnet test`, and `shared.test.ts` and `layers.test.ts` pass in vitest. `npm test` passes, and CI passes on GitHub. US-23 is open: no points count until Thursday. Write tomorrow's first test in your notes: `opens on the area's home view with the nine layers on, and asks for no list of layers`.

18

Day 17 · Wednesday 11 November: US-23, one model and one pure update

> IN SHORT Today you write the heart of the map: one model that holds everything the page shows, and one pure update that makes each new model from the old one and a message. This is the second of US-23's 3 points.

The map keeps all it knows in one value, and only one pure function may replace it. That rule, US-23 c3, lets a test check every change of the page by calling a function and reading what it returns. The map itself sits behind ***ports***, functions the update asks to move the map, which a test replaces with fakes.

EXHIBIT 18.1 Day 17 ends when the model, its messages and its update pass with values alone

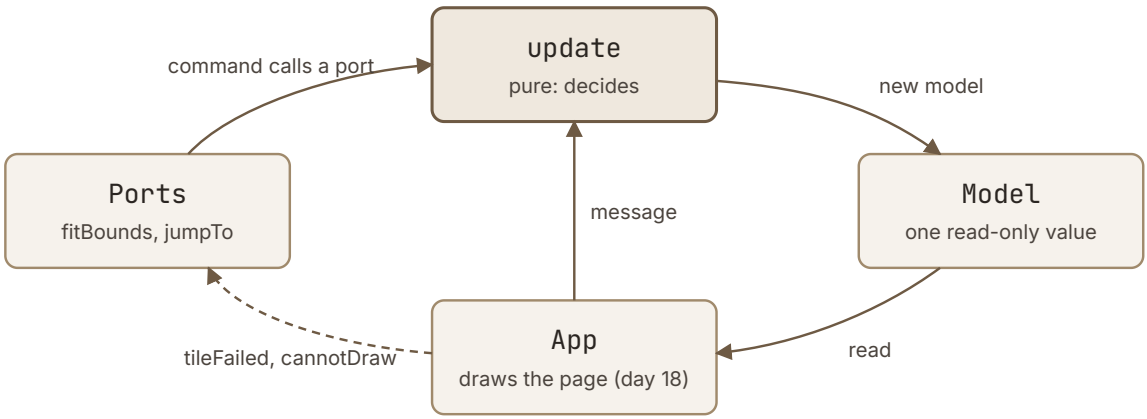
Goal	`Model`, `Message`, `Msg`, `init`, `update` and `Ports` in `map/map.ts`; `tileProblem` in `map/tiles.ts`
Story	US-23, criteria 1, 2, 3, 8 and 9; tests 2 and 3
In the worked answer	`web/src/map/map.ts`; `map/tiles.ts`; `map/map.test.ts`; `app/index.ts`; `app/app.test.ts`; `web/src/architecture.test.ts`
Look for	Why a handler returns only the fields that change; why `init` at `/` returns no command; how `cmd.ofNone` calls a port
Leave for later	The messages `zoomIn`, `zoomOut` and `viewChanged` (UI-09), `toggleUnits` and `pointerMoved` (UI-19), `clusterOpened` (UI-10) and `pinsLanded` (UI-15)
Done when	The six tests of 'US-23 open the map of the area' and 'US-23 criterion 9: a browser that cannot draw the map' in `map.test.ts`, 'starts from one model and changes it only through the update's handlers' in `app.test.ts`, and `architecture.test.ts` pass

Source: `docs/user-stories/user-stories.md`, *US-23;* `docs/guides/reading-the-frontend.md`, *steps 3 and 4*

18.1 The Elmish loop

The map follows the ***Elmish loop***, named after the Elm language and supplied by the library react-elmish: four parts turn in a circle, and only the update decides anything.

EXHIBIT 18.2 One message travels once round the loop; the update alone decides



Source: `web/src/map/map.ts`; `web/src/app/App.tsx`

As Exhibit 18.2 shows, the page and the map send messages, the update returns the new model and perhaps a command, and a command reaches the map only through a port. Today you build the top half; day 18 builds the page and the map that close the circle.

18.2 Red, then green: the page opens on the area

In `web/src/map/map.test.ts`, write 'opens on the area's home view with the nine layers on, and asks for no list of layers'. It runs `init` through react-elmish's helper `initAndExecCmd`, with two fake ports made by `vi.fn()`, and expects the home view, all nine layers, no messages and no call to either port.

```bash
cd web && npx vitest run src/map/map.test.ts
```

It fails to compile: nothing exports `init`. Write the model and `init`, with only the fields US-23 needs:

```ts
// web/src/map/map.ts (shortened: the fields of UI-09, UI-15 and UI-19 left out)
export interface Model {
  readonly view: View;
  readonly visible: readonly LayerId[];
  readonly notice: string | null;
  readonly drawable: boolean;
}

export function init(props: Props): InitResult<Model, Message> {
  const model: Model = {
    view: props.area.home,
    visible: props.openLayer === undefined ? LAYER_IDS : [props.openLayer],
    notice: null,
    drawable: true,
  };
  return props.openLayer === undefined ? [model] : [model, frame(props, props.area.bounds)];
}
```

`visible` comes from `LAYER_IDS`, yesterday's list, so the page needs no request to learn it: the test's 'no list of layers'.

18.3 Red, then green: a layer's page frames its layer

At `/food.cafes.html` the map must show the cafés alone, all in view. Write 'opens /food.cafes.html with the cafés alone, framing the area that holds them', which expects `visible` to hold `food.cafes` alone and the port `fitBounds` to be called with Macau's box and `FRAME_MAX_LEVEL`.

The update may not move the map itself, so `init` returns a command that calls the port:

```ts
// web/src/map/map.ts (shortened)
export const FRAME_MAX_LEVEL = level(18);

const frame = <M extends AnyMessage>(props: Props, bounds: Bounds): Cmd<M> =>
  cmd.ofNone(props.ports.fitBounds, bounds, FRAME_MAX_LEVEL);
```

`cmd.ofNone` calls the function later and sends no message back. Level 18 limits the frame: a layer of a few places never zooms closer, so one café never fills the screen.

> KEY IDEA A port is a promise of an effect, not the effect. The update names what should happen; the code outside the update carries it out. So the test can ask the fake 'were you called, and with what?', and no map has to exist.

18.4 Red, then green: a problem shows

Criterion 8 asks the page to show a failed tile's `detail`, or its `title` where it has none, in three tests:

1. 'shows a problem document's detail, or its title where it has none', which sends `Msg.tileFailed` and calls `noticeOf`;
2. 'leaves a failed tile's detail in the model, read from the answer's body', which is the story's test 3;
3. 'makes words from the status alone when the body is no problem document', for an HTML page from a proxy and for an empty status text, which gives 'HTTP 500'.

Write the handler, `noticeOf`, and `tileProblem` in `map/tiles.ts`:

```ts
// web/src/map/map.ts and web/src/map/tiles.ts (shortened)
export const noticeOf = (problem: Problem): string => problem.detail ?? problem.title;

tileFailed({ problem }) {
  return [{ notice: noticeOf(problem) }];
},

export const tileProblem = (status: number, statusText: string, body: string): Problem =>
  pipe(
    parseJson(body),
    flatMap((json) => readProblem(json, status)),
    defaultWith(() => problemOfStatus(status, statusText)),
  );
```

The handler returns `{ notice: … }`, the one field that changes; react-elmish joins it to the old model to make the new one. `readProblem` comes from the shared kernel, and reads the problem documents the service writes with day 11's `Problems`.

18.5 One model, one pure update

The story's test 2 asks for proof that the model changes only through the update and that the old model stays as it was. In `web/src/app/app.test.ts`, write 'starts from one model and changes it only through the update's handlers'. The worked answer runs the handler `toggleUnits`, which arrives with UI-19; yours runs `tileFailed` and checks that the old model's `notice` is still `null`, a planned difference.

Then add the page's module, `web/src/app/index.ts`, which passes the map's model, messages and update on unchanged, and copy `web/src/architecture.test.ts`. Its list `MAY_IMPORT` lets `app` import `shared`, `design`, `layers` and `map`, and lets `map` import neither `app` nor anything above it: it guards the map's walls, as `Tessera.ArchitectureTests` guards the service's.

18.6 Red, then green: the browser may not draw

Criterion 9 needs a field that says whether the browser can draw. Write the test of 'US-23 criterion 9: a browser that cannot draw the map', 'opens able to draw, records that it cannot, and records that it can again'. Two messages, `cannotDraw` and `canDrawAgain`, set `drawable` to `false` and back; day 18 sends them from MapLibre's events.

```bash
cd web && npm test   # types, lint, then every vitest test
```

IN INK Commit at each green, with the messages `US-23: open the map on the area's home view, all nine layers on`, `US-23: frame one layer through a port`, `US-23: keep a failed tile's problem in the model` and `US-23: guard the walls between the map's modules`, and push.

THE PROOF Every test of `map.test.ts` you wrote today, `app.test.ts` and `architecture.test.ts` pass in `npm test` inside `web/`, with no browser and no map. Your model has four fields, the worked answer's seven. Write tomorrow's first test in your notes: `fills the window and opens on the area's home view, with the nine layers' tiles on the page's own origin`.

19

Day 18 · Thursday 12 November: US-23, the page in a browser

> IN SHORT Today the model meets the screen: a React page draws it, MapLibre draws the tiles, the service ships the built map inside its release, and browser tests prove what a visitor sees. By evening the map opens at `/` and at `/food.cafes.html`; US-23 is done, worth 3 points.

Three kinds of test finish the story: tests in *jsdom*, a pretend browser with no screen, check what the page asks of the map; a test of the built service checks that the map ships inside it; and tests in a real browser check what the map drew. The first kind runs in seconds with a fake MapLibre. The last needs the built service, Nginx and the 900 places, and runs through `npm run e2e`.

EXHIBIT 19.1 Day 18 ends when the map opens in a real browser, and every US-23 test passes

Goal	`App`, `mount`, the map handle and MapCanvas; the map inside the release; the browser tests of US-23
Story	US-23, criteria 1, 2, 4, 6, 7, 8 and 9; tests 1, 2 and 4
In the worked answer	`web/src/main.tsx`; `app/App.tsx` and `App.test.tsx`; `app/mount.tsx` and `mount.test.tsx`; `map/map-handle.ts`; `map/maplibre.ts`; `testing/fake-maplibre.ts`; `src/Tessera/Tessera.csproj`; `ServiceTests.cs`; `web/e2e/map-page.ts`, `map.spec.ts` and `cannot-draw.spec.ts`; `web/playwright.config.ts`
Look for	How the handle keeps a move until the map exists; how one `<Content>` line puts the map inside the release; why `map.spec.ts` writes the nine layers out again; why `cannot-draw.spec.ts` is a file of its own
Leave for later	The zoom buttons and their labels (UI-09); the scale bar and coordinates (UI-19); the dropping pins (UI-15); the circle feed layers and the clusters (UI-10)
Done when	The tests of 'US-23 open the map of the area' and 'US-23 criterion 9: a browser that cannot draw the map' in `App.test.tsx`, 'replaces a page that throws with words that say what to do' in `mount.test.tsx`, `Serves_the_map_page_with_the_area_written_in`, and `npm run e2e -- --grep "US-23"` pass

Source: `docs/user-stories/user-stories.md`, *US-23;* `docs/guides/reading-the-frontend.md`, *steps 1, 2, 6 and 15*

19.1 The page in jsdom, with a fake map

MapLibre draws with WebGL, the browser's way to draw with the graphics card, and jsdom has none. So `App.test.tsx` replaces the library with the worked answer's fake in one line, `vi.mock("maplibre-gl", …)`, and its helper `page` draws `App` and reads back what the page asked of the fake map.

Copy `web/src/testing/fake-maplibre.ts`, which is test code, not the story. Then write the five tests of 'US-23 open the map of the area' in `App.test.tsx`, one at a time, each red before its code: the one `h1` reads 'Map of Macau'; the home view opens at centre `[113.55, 22.19]`, MapLibre zoom 14, with nine tile addresses on `http://tessera.test`; `/food.cafes.html` asks for one layer and calls `fitBounds` with

Macau's box and `maxZoom: 17`; a failed tile's problem detail shows over the map; and a missing image raises no alert.

> WATCH OUT Level 15 on the page is zoom 14 in MapLibre, and level 18 is `maxZoom: 17`. MapLibre draws the world 512 pixels wide at its zoom 0, where the web map's level 0 draws it 256 wide, so MapLibre's count runs one lower (`map/levels.ts`). The service's block says `"zoom":15`, a web-map level; only the map converts it. Mix the two and every view is off by a factor of two.

Write `App` to pass them. It runs the loop with `useElmish`, and draws a hidden heading, the map, and the notice while the map can draw:

```ts
// web/src/app/App.tsx (shortened: class names, and the controls of later stories, left out)
const [model, dispatch] = useElmish<Props, Model, Message>(
  { name: "Tessera", props, init, update });
return (
  <main>
    <h1 className="sr-only">Map of {area.name}</h1>
    <MapView.MapCanvas
      visible={model.visible} origin={origin} initialView={model.view} handle={handle}
      onTileFailed={(problem) => dispatch(Msg.tileFailed(problem))}
      onCannotDraw={() => dispatch(Msg.cannotDraw())}
      onCanDrawAgain={() => dispatch(Msg.canDrawAgain())} />
    {!model.drawable && <p role="alert">{MapView.CANNOT_DRAW}</p>}
    {model.drawable && model.notice !== null && <p role="alert">{model.notice}</p>}
  </main>
);
```

Behind `MapCanvas`, `map/maplibre.ts` turns MapLibre's events into these calls. `watchTileErrors` listens for errors of sources named `tiles:`, reads the answer's body, and passes it through yesterday's `tileProblem`. The map handle, `map/map-handle.ts`, carries yesterday's ports: `/food.cafes.html` asks to frame the area before MapLibre has started, so the handle keeps the latest move and runs it once a map attaches.

Then write the two tests of 'US-23 criterion 9: a browser that cannot draw the map': 'says so in words, in place of the controls, when MapLibre cannot start' and 'says so when the browser takes the map's drawing away, and draws again once it gives it back'. The fake fails on request (`FakeMap.failNext = true`) and fires `webglcontextlost` and `webglcontextrestored`.

```bash
cd web && npx vitest run src/app
```

19.2 A page that fails says so

An error that no part of the page catches makes React remove the whole page, leaving a blank window, which criterion 9 forbids. Write 'replaces a page that throws with words that say what to do' in `web/src/app/mount.test.tsx`: it mounts a component that throws and waits for the alert `PAGE_FAILED`. Then write `mount`:

```ts
// web/src/app/mount.tsx (shortened)
export function mount(element: HTMLElement, page: ReactNode): Root {
  const root = createRoot(element, {
    onUncaughtError: (error) => {
      console.error(error);
      root.render(<PageFailed />);
    },
  });
  root.render(page);
  return root;
}
```

Last, write `main.tsx`: it reads the block `tessera-area` with `areaOf`, names the tab 'Map of Macau', reads the layer from `location.pathname` with `layerFromPath`, and mounts `App`. Every piece it joins has a test of its own, so `main.tsx` has none; the browser tests below run it.

19.3 The map ships inside the service

Criterion 4 asks for one release that holds the service and the map, so the service's project copies the built map beside the program. `npm run build` in `web/` writes the map to `web/dist`; one line in `src/Tessera/Tessera.csproj` then copies every file of it into `wwwroot` in the build's output, where Tuesday's `UseMap` looks:

```xml
<Content Include="../../web/dist/**" Link="wwwroot/%(RecursiveDir)%(Filename)%(Extension)"
         CopyToOutputDirectory="PreserveNewest" CopyToPublishDirectory="PreserveNewest" />
```

Add `<StaticWebAssetsEnabled>false</StaticWebAssetsEnabled>` to the same file's `PropertyGroup`, so the files go out as Vite built them and the SDK's own pipeline for web files stays off.

Now write the test Tuesday left, `Serves_the_map_page_with_the_area_written_in`, in `ServiceTests`. It is a theory with two rows, `/` and `/food.cafes.html`, and checks the whole contract of the page through real HTTP: 200, `text/html`, the block `tessera-area` that begins `{"name":"Macau"`, the map's own `<div id="root"></div>`, the four headers of `PageHeaders`, and the script the page names under `/assets/`, fetched in turn as `text/javascript`.

The integration tests' project refers to the service's project, so its build copies `wwwroot` beside the tests too. Build the map first, then run the test:

```bash
npm run build    # web/: npm ci and vite build into web/dist; then dotnet build in Release
dotnet test --project tests/Tessera.IntegrationTests --filter-class '*ServiceTests'
```

Watch it fail first: delete `web/dist` and the test projects' `bin` folders, run the test, and see 404 where it expects 200, with Tuesday's detail, 'This release carries no map.'

19.4 The real browser

Only a real browser shows what MapLibre drew and what the page asked the service for. Playwright drives Chromium against Nginx and the built service, with the 900 places loaded and published, as `web/playwright.config.ts` sets out: one window of 1,280 by 800, and SwiftShader, Chrome's software drawing, on a machine with no graphics card.

Copy the helpers of `web/e2e/map-page.ts` you need today: `openMap`, `settle`, `view`, `drawn` and `onScreen`. `openMap` sets `window.__tesseraTest` before the page loads, so `main.tsx` hands the test the map as `window.__tesseraMap`. The worked answer's `openMap` also waits for the pins to land, which arrives with UI-15 on day 23; yours waits for `settle` alone, a planned difference.

Then write the five tests of 'US-23 open the map of the area' in `web/e2e/map.spec.ts`:

EXHIBIT 19.2 Five browser tests, each proving what only a real browser can show

TEST	CRITERION	WHAT IT PROVES
opens / on the area's home view with the nine layers, asking only for the page, its files and tiles	1, 7	The centre is 113.55, 22.19; every request goes to the page's own origin; tiles arrive from all nine layers; the policy refuses nothing
fills the window with the map, on a plain background	1	The canvas covers 1,280 by 800 from the top left
opens /food.cafes.html with the cafés alone, all in view	2	The view holds Macau's box; every pin drawn is in the group `food`; only `food.cafes` tiles are asked for
treats a pin as a sign: a click on it opens nothing and moves nothing	6	The zoom and centre stay; no dialog and no side panel appear
shows a tile's problem document over the map	8	A café tile routed to a 503 problem leaves its `detail` as an alert, and axe finds no fault

Source: `web/e2e/map.spec.ts`

As Exhibit 19.2 shows, each test reads what was drawn or asked for; `map.spec.ts` writes the nine layers out again, so it checks the page against the story, not against itself.

> WATCH OUT The worked answer's second test expects the style's layer types to be `background`, `circle` and `symbol`. The `circle` layers are invisible feed layers, of radius 0, that keep MapLibre loading each layer's tiles once the pins move to the clustered source with UI-10 (day 28). Until then, your copy's test expects `background` and the pins' `symbol` layer alone. Note it as a planned difference.

Copy `web/e2e/cannot-draw.spec.ts` too: its browser starts with WebGL turned off, so the test lives alone in its file.

Run `npm run build`, then `npm run e2e -- --grep "US-23"`: `scripts/e2e.ts` passes whatever follows `--` on to Playwright, so it runs the six browser tests whose names hold 'US-23'.

> IN INK Commit at each green, with the messages `US-23: draw the page from the model, with a fake map in jsdom`, `US-23: say so in words when the page fails or cannot draw`, `US-23: ship the built map inside the service's release` and `US-23: prove the map in a real browser`, and push.

> THE PROOF `npm test` and CI pass, the browser tests of US-23 among them. Open the map by hand, as on day 8's seed and day 15's start, at `http://127.0.0.1:8080/`. US-23 is done: 16 points of 16. Tomorrow is the review.

20

Day 19 · Friday 13 November: the review, US-35 on the office machine, and M2

> IN SHORT Today you prove iteration 2 from a clean start, check US-35 where only the office machine can check it, and show milestone M2. You write no new code. M2 reads 'The map opens in the office, on safe tiles'.

M2 joins two halves: safe tiles, which tests prove on any machine, and an office that can reach them, which only the office machine can prove. The review answers four questions, in this order. Does the work pass from nothing? Do US-35's checks still hold on the office machine? Does M2 hold, from another office machine? Is every criterion traced, and is the next plan sound?

EXHIBIT 20.1 Day 19 ends with M2 shown from another office machine and tagged in ink

Goal	A clean run; US-35's checks on the office machine; M2 shown; the comparison notes finished; iteration 3 planned
Guide	`docs/guides/delivery-plan.md`, 'The weekly and daily rhythm' and 'Tracking progress with a burn-up'
In the worked answer	`docs/requirements/functional-requirements.md`, FR-55 to FR-58; `security-requirements.md`, SR-14 and SR-15; your day 15 notes
Look for	Which checks a test can make anywhere, and which only the office can make; what a port proxy would hide
Done when	`npm test` passes from a clean copy; a tile answers 200 from another office machine, before and after `wsl --shutdown`; the burn-up reads 16 of 16; the tag `m2` is pushed

Source: `docs/guides/delivery-plan.md`; `docs/user-stories/user-stories.md`, *US-35*

20.1 The work passes from nothing

A clean run proves that nothing left over from earlier days makes the tests pass. As on day 10:

BASH

```
cd ~/src/tessera-rebuild
git status -sb                                  # nothing to commit
git clean -xdf
npm ci
npm test
```

`git clean -xdf` removes every build's output, the built map and the packages, so the run must fetch, build and copy them all again. Every test of US-07, S-01, S-06 and US-23 is in that run: the 'safe tiles' half of M2. It proves nothing about the office, since a request from this machine never crosses either firewall.

20.2 US-35 on the office machine

Half of US-35's proof needs the office machine, so the review repeats day 15's checks 2 and 3 (Exhibit 16.3) on today's build, with the whole of iteration 2 behind Nginx. Start your service and your Nginx by hand, as on day 15, since `npm start` arrives only on day 34. From another office machine, ask for a tile on port 80 and read the first column of `docker logs --tail 5 tessera-nginx` for that machine's address; then run `wsl --shutdown` on the office machine, start both again, and ask once more. Last, check that whoever runs the office network has reserved the Windows machine's address (criterion 3).

> TRY IT Turn day 15's notes into a checklist of US-35's four checks, with a line for the date, the status curl printed and the address Nginx logged. It stands in for the tests that FR-55 to FR-57 and SR-14 lack, and the release on 7 December repeats it.

20.3 M2 holds in the office

Show the milestone where its words say: in the office. From another office machine, open `http://<office name>/` in a browser. The map fills the window over Macau, at level 15, with pins from the nine layers. Open `http://<office name>/food.cafes.html` and the cafés appear alone, all in view. Then ask for a malformed tile, such as `/food.cafes/15/1/2.pbf%zz`, and see Nginx refuse it with a 400 problem document that names no address: S-06's guard, at the office's own door, with day 14's rule for `%2e` in place.

20.4 Every criterion has a test, and the plan is sound

1. **The trace.** Each criterion of US-07, S-01, S-06, US-35 and US-23 should have a test, or a line in your notes. Git finds each story's commits: `git log --oneline --grep 'US-23'`. US-35's criteria 1 to 4 point to today's checklist, and its criteria 5 and 6 to `scripts/lib/office.test.ts`, which checks the printed steps, not the firewalls themselves (SR-15, in part). US-23's criterion 5 points to `Answers_404_when_there_is_no_page_to_show`, and criterion 7 also to your own `Names_the_same_nine_layers_as_the_map`.
2. **The differences.** Read the week's comparison notes, and keep each difference that has a reason: the shorter model, `tileFailed` in place of `toggleUnits`, `openMap` without the pins' landing, a style without the circle feed layers. Each closes when its story arrives (Appendix A).
3. **The burn-up.** The plan asked for 16 points by the end of week 4, and 16 are done: US-07, S-01, S-06, US-35 and US-23 on top of M1's 7. Write the actual figure beside the plan in your notes.
4. **Ink for the milestone.** Tag the commit that reaches M2, and push the tag:

```bash
git tag -a m2 -m 'M2: the map opens in the office, on safe tiles'
git push origin m2
```

5. **Plan iteration 3.** UI-09, UI-15, UI-12 and UI-19 fill 16 to 24 November, and UI-10 begins on Wednesday 25 November; M3 falls on Friday 27 November. Read steps 5 and 12 of `docs/guides/reading-the-frontend.md` before Monday. Write Monday's first test in your notes: `doubles and halves the scale one level at a time, through the map's port`.

> THE PROOF `npm test` passes from a clean start; the office checklist is filled in; the map opens from another office machine; the burn-up reads 16 of 16; the tag `m2` is on GitHub.

21

Day 20 · Monday 16 November: UI-09, levels that double

IN SHORT The map's model learns to count zoom in web-map levels and to move one whole level at a time. This is the first of UI-09's two days, worth 2 points.

Today the map's model counts zoom as every web map does, in levels from 0 to 22, and the '+' and '−' messages move it one whole level. A type of its own keeps the level apart from MapLibre's zoom, which runs one lower, so the compiler refuses any mix of the two.

Iteration 3 opens today. Its review, on Friday 27 November, checks milestone M3, 'The map zooms, keeps calm and tells places apart'. UI-09 takes days 20–21, UI-15 days 22–23, UI-12 day 24, UI-19 days 25–26 and UI-10 days 27–28, ending on day 30; the plan expects 23 of 29 points at the review. Each story changes the browser alone, so you work in `web/` throughout and change no line of C#.

EXHIBIT 21.1 Day 20 ends with six tests that need no browser

Goal	A model whose level stays within 0 to 22 and moves one whole level for each '+' or '−'
Story	UI-09 c1, with the ends of c2; tests 1 and 2
In the worked answer	`web/src/shared/geo.ts`; `web/src/map/levels.ts`; `metresPerPixel` in `web/src/map/scale.ts`; `zoomBy` in `web/src/map/map.ts`; `web/src/map/readouts.test.ts`; `web/src/map/map.test.ts`
Look for	Why MapLibre's zoom runs one lower; why the update clamps a view the map reports
Leave for later	The buttons and the world's edge (day 21); the scale bar (day 25)
Done when	In `readouts.test.ts`: 'doubles and halves the scale one level at a time', 'speaks web-map levels, one above MapLibre's zoom, which draws the world 512 pixels wide', 'keeps the two scales apart, as F#'s units of measure would: the compiler refuses one for the other', 'reads MapLibre's zoom as a level within 0 to 22'; in `map.test.ts`: 'doubles and halves the scale one level at a time, through the map's port', 'keeps the level within 0 to 22'

Source: `docs/user-stories/user-stories.md`, *UI-09;* `docs/guides/reading-the-frontend.md`, *step 5*

21.1 Red, then green: the scale doubles

Read UI-09 in *The World in Squares* and step 5 of `reading-the-frontend.md` first; both explain the fact the day turns on. The stories count ***web-map levels***: level 0 shows the world in one square 256 pixels wide, and each level up doubles the detail. MapLibre draws the world 512 pixels wide at its zoom 0, so its zoom reads one lower. Create `web/src/map/readouts.test.ts` with the rule every web map shares:

```ts
const LAT = MACAU.home.center.lat; // 22.19° N

describe("UI-09 levels", () => {
  it("doubles and halves the scale one level at a time", () => {
    const at15 = metresPerPixel(15, LAT);
    expect(metresPerPixel(16, LAT)).toBeCloseTo(at15 / 2, 9);
    expect(metresPerPixel(14, LAT)).toBeCloseTo(at15 * 2, 9);
  });
```

Run `cd web && npx vitest run src/map/readouts.test.ts`: red. Write `metresPerPixel` in `web/src/map/scale.ts`, which grows into the scale bar on day 25:

```ts
const EQUATOR_METRES = 2 * Math.PI * 6378137;

export function metresPerPixel(level: number, latitude: number): number {
  return (EQUATOR_METRES * Math.cos((latitude * Math.PI) / 180)) / (256 * 2 ** level);
}
```

> FROM SCHOOL Each level doubles the width of the world: 256 pixels at level 0, 256 × 2^{15} = 8,388,608 pixels at level 15, as each extra bit doubles a binary range. Divide the equator, about 40,075 km, by that width, and shrink the result by the cosine of the latitude, since circles of latitude grow shorter towards the poles. Over Macau one pixel at level 15 covers about 4.42 m.

21.2 Red, then green: two counts that never mix

A level and MapLibre's zoom are both numbers, so one can slip into the other's place and leave the map one level out. A ***branded type*** closes the gap: a number carrying a mark that exists for the compiler alone (ADR 0004). Write the three tests named in Exhibit 21.1. The second marks with `// @ts-expect-error` two lines the compiler must refuse: `const wrong: MapZoom = level(15);` and `toMapZoom(15);`. The level has lived in the shared kernel's `geo.ts` since day 16, because the area's home view uses it; MapLibre's zoom goes in the map module:

```ts
// shared/geo.ts
export type Level = number & { readonly unit: "level" };
export const level = (n: number): Level => n as Level;
export function clampZoom(zoom: number): Level {
  return level(Math.min(MAX_ZOOM, Math.max(MIN_ZOOM, zoom)));
}

// map/levels.ts
export type MapZoom = number & { readonly unit: "mapZoom" };
export function toMapZoom(webLevel: Level): MapZoom { return mapZoom(webLevel - 1); }
export function toLevel(zoom: MapZoom): Level { return level(zoom + 1); }
export const levelOf = flow(mapZoom, toLevel, clampZoom);
```

`flow`, from `shared/pipe.ts`, joins three functions into one that runs them in the order written, as F#'s `>>` does. Export the new names through `web/src/map/index.ts`, since the test imports from `"."`.

> WATCH OUT Vitest strips the types without checking them, so `npx vitest run` passes the second test whatever the compiler thinks. The test bites under `npm test` in `web/`, whose first step, `tsc --noEmit`, fails on an `@ts-expect-error` line that raises no error.

21.3 Red, then green: the update moves one whole level

The buttons come tomorrow, but the messages they send must move the map today. Add a `UI-09 levels` block to `web/src/map/map.test.ts`, whose fake ports from `vi.fn()` record each move the update asks for:

```ts
it("keeps the level within 0 to 22", () => {
  expect(updateFn(...args(Msg.zoomIn(), { view: { ...MACAU.home, zoom: level(22) } }))[1])
    .toBeUndefined();
  // … the same for zoomOut at level 0
  expect(updateFn(...args(Msg.viewChanged({ ...MACAU.home, zoom: level(24) })))[0].view?.zoom)
    .toBe(22);
});
```

Its twin sends `zoomIn` at level 16.4 and expects the fake `jumpTo` to receive 17.4, then `zoomOut` at 15 and expects 14. One function answers both buttons in `map.ts`:

```ts
function zoomBy(step: 1 | -1, model: Model, props: Props)
  : UpdateReturnType<Model, Message> {
  const allowed = step > 0 ? canZoomIn(model) : canZoomOut(model);
  return allowed
    ? [{}, jump(props, { ...model.view, zoom: clampZoom(model.view.zoom + step) })]
    : [{}];
}
```

The handlers `zoomIn` and `zoomOut` call `zoomBy(1, …)` and `zoomBy(-1, …)`. The view keeps its centre, so the scale doubles or halves about it, as criterion 1 asks. The map reports where it ended through `viewChanged`, whose handler returns `[{ view: { ...view, zoom: clampZoom(view.zoom) } }]`.

> KEY IDEA The update clamps the view the map reports rather than trust the sender, so the model never holds a level outside 0 to 22. In test terms, 0 and 22 are boundary data and 24 is erroneous data.

> IN INK Run `cd web && npm test`, then commit at each green, with the message `UI-09: count levels 0 to 22 and move one whole level`, and push.

> THE PROOF The six tests of Exhibit 21.1 pass under `npm test`. Compare `levels.ts` and `map.ts` with the worked answer's. UI-09 stays open. Tomorrow's first test: `states the current level in each label, and nowhere on screen`.

22

Day 21 · Tuesday 17 November: UI-09, buttons that say the level

> IN SHORT Two zoom buttons speak the level and grey out at either end, and the map stops before its edge. This finishes UI-09, worth 2 points.

Today '+' and '−' appear at the top right, tell a screen reader the level and turn grey at either end, and the map stops before it shows the world's edge. A pretend browser proves the buttons; a real Chrome proves the edge.

Yesterday's model moves; today the visitor gets buttons to move it, and the map gets its limits.

EXHIBIT 22.1 Day 21 ends with UI-09 proved in a pretend browser and in Chrome

Goal	Two buttons that state the level; a map that never shows its edge
Story	UI-09 c1 to c5; test 3
In the worked answer	`web/src/map/labels.ts`; `ZoomButton` in `web/src/map/MapControls.tsx`; `web/src/app/App.tsx`; `baseOptions` in `web/src/map/maplibre.ts`; `web/src/map/controls.test.tsx`; `web/e2e/map.spec.ts`
Look for	Why the button uses `aria-disabled`, not `disabled`; why `WORLD` stops short of 180°
Leave for later	The compass (day 26); a cluster's click to a level such as 16.4 (UI-10, day 30)
Done when	'says the level only in the zoom buttons' labels, to one decimal place between levels'; 'states the current level in each label, and nowhere on screen'; 'turns the button grey at the end of the range, and it does nothing there'; in Chrome, 'moves one level per button, and says so in the labels' and 'keeps the edge of the map out of view'

Source: `docs/user-stories/user-stories.md`, *UI-09;* `docs/requirements/functional-requirements.md`, *FR-33 to FR-37; NFR-62*

22.1 Red, then green: words for a level

A screen reader, the program that reads a page aloud, hears the level in each button's label, and nothing else on the page shows it (criterion 3). A cluster that opens may leave the map between levels (criterion 4), so the words keep one decimal place. Test them in the `UI-09 levels` block of `readouts.test.ts`:

```ts
expect(zoomLabel("in", 15)).toBe("Zoom in, now at level 15 of 22");
expect(zoomLabel("out", 16.44)).toBe("Zoom out, now at level 16.4 of 22");
expect(levelText(17.96)).toBe("18");
```

```ts
// map/labels.ts
export function levelText(level: number): string {
  const tenths = Math.round(level * 10) / 10;
  return Number.isInteger(tenths) ? String(tenths) : tenths.toFixed(1);
}

export function zoomLabel(direction: "in" | "out", level: number): string {
  return `Zoom ${direction}, now at level ${levelText(level)} of ${MAX_ZOOM}`;
}
```

22.2 Red, then green: the buttons

A component test draws the buttons in jsdom, a pretend browser inside the test runner, and finds each as a screen reader would: by its role and its ***accessible name***, the words a screen reader says for it. Create `web/src/map/controls.test.tsx`; its first line, `// @vitest-environment jsdom`, asks for jsdom:

```ts
it("states the current level in each label, and nowhere on screen", async () => {
  const { container } = controls(15);
  expect(screen.getByRole("button", { name: "Zoom in, now at level 15 of 22" })).toBeTruthy();
  expect(screen.getByRole("button", { name: "Zoom out, now at level 15 of 22" }))
    .toBeTruthy();
  expect(container.textContent).not.toMatch(/15/);
  expect(await violations(container)).toEqual([]);
});
```

`controls(level)` draws `MapControls` with fake handlers; `violations`, from `web/src/testing/axe.ts`, runs axe, a library that checks a page against the web's accessibility rules. The second test draws level 22, clicks '+', and expects `aria-disabled="true"` and no call, then clicks '−' and expects one. One component serves both buttons, so they cannot drift apart:

```ts
<button
  type="button"
  aria-label={zoomLabel(direction, level)}
  aria-disabled={!enabled}
  onClick={() => enabled && onPress()}
  className="state-layer map-button"
>
  <Icon name={direction === "in" ? "add" : "remove"} />
</button>
```

> WATCH OUT `disabled` greys a button but also takes it out of the Tab order, so a screen reader never reaches its label. `aria-disabled` keeps it in reach; `onClick` checks `enabled` itself, and `.state-layer[aria-disabled="true"]` in `styles.css` greys it. `App.tsx` turns each press into `dispatch(Msg.zoomIn())` or `dispatch(Msg.zoomOut())`.

22.3 Red, then green: the edge stays out of view

Criterion 5 needs a real browser, since only MapLibre knows what it drew. Add a `UI-09 levels` block to `web/e2e/map.spec.ts`. One test clicks '+' and reads the new labels; the other jumps below the lowest level over the world's top-left corner and reads the map's bounds:

```ts
await page.evaluate(() => window.__tesseraMap?.jumpTo({ zoom: -1, center: [-179, 84] }));
await settle(page);
// … read west, south, east and north from getBounds()
expect(west).toBeGreaterThanOrEqual(-180);
expect(north).toBeLessThanOrEqual(85.06);
```

```bash
npm run build                    # at the root: the map into web/dist, then dotnet build
npm run e2e -- --grep "UI-09"    # database, service and Nginx, then Playwright
```

`dotnet build` copies `web/dist` beside the program as `wwwroot/`, so the browser test meets today's map. The test fails until the map has limits, which MapLibre takes as start-up options in `baseOptions`:

```ts
export const WORLD: [[number, number], [number, number]] = [
  [-179.9999, -85.051129],
  [179.9999, 85.051129],
];
// in baseOptions:
minZoom: toMapZoom(MIN_ZOOM),
maxZoom: toMapZoom(MAX_ZOOM),
maxBounds: WORLD,
renderWorldCopies: false,
```

`renderWorldCopies: false` draws one world, not a row of copies. The edges stop a ten-thousandth of a degree short of 180°, since MapLibre 6.13 failed to start with bounds spanning exactly 360°; the comment above `WORLD` says nobody has checked the 6.14 that `web/package.json` pins.

> FROM SCHOOL Web Mercator, the flat projection web maps use, stretches the globe more and more towards the poles. It cuts the world off at about 85.05° north and south, where the flattened world becomes a square; beyond lies the blank edge that the test keeps out of view.

> TRY IT Beyond the worked answer: criterion 6, pins that keep their size, has no test (FR-37, 'No test yet'). MapLibre draws an icon at a fixed size on screen unless the style scales it with `icon-size`. On day 24, in the `UI-12` block of `style.test.ts`, add a test of your own, 'keeps each pin's size at every level (UI-09, criterion 6)': `expect(symbol(LAYER.pins).layout?.["icon-size"]).toBeUndefined()`. It proves the size on screen, not that the pins move in step with the map.

> IN INK Run `cd web && npm test`, then commit at each green, with the message `UI-09: zoom buttons that state the level and stop at the ends`, and push.

> THE PROOF The three Vitest tests and the two browser tests of Exhibit 22.1 pass. UI-09 is done: 2 points, 18 of 29. Tomorrow's first test: `keeps the theme in styles.css`.

23

Day 22 · Wednesday 18 November: UI-15, one theme for the page

> IN SHORT Every colour moves into one theme, and every icon comes from one font. This is the first of UI-15's two days, worth 2 points.

Today every colour on the page moves into one block of `styles.css`, built from Material 3's roles, and a test that reads the source refuses a colour written anywhere else. The icons come from one font, Material Symbols, and the map keeps a plain background.

UI-15 follows the first zoom for the plan's fourth reason: it sets the colours, movement and icons that every later control follows, so a control built before it would be built twice. Today takes the colours and icons; tomorrow, the movement.

EXHIBIT 23.1 Day 22 ends when no file but the theme names a colour

Goal	One theme of colours; group colours read from it; icons from one font; a plain background
Story	UI-15 c1, c2 and c9; tests 1 and 3
In the worked answer	`web/src/styles.css`; `web/src/design/theme.ts`; `Icon` in `web/src/design/components.tsx`; `web/src/rules.test.ts`; `web/src/design/design.test.ts`; `web/src/map/style.test.ts`
Look for	`--color-*: initial;`; why MapLibre and the canvas read colours at run time
Leave for later	Movement, elevation, corners and state layers (day 23); group shapes (day 24)
Done when	In `rules.test.ts`: 'keeps the theme in styles.css', 'lists every colour written outside the theme's Material roles, and finds none', 'draws every icon from Material Symbols', 'draws no icon any other way: the only drawings are the group shapes and the compass'; 'takes each group's colour from the theme, never from code'; 'draws nothing beneath the places but the plain background'; in Chrome, 'draws the zoom icons from Material Symbols, one glyph each'

Source: `docs/user-stories/user-stories.md`, *UI-15; NFR-49, NFR-50, NFR-57*

23.1 Red: a test that reads the source

No module owns the rule 'one theme', so the worked answer checks the text of every shipped file under `web/src`. Create `web/src/rules.test.ts`. It reads each file, blanks out comments, skips the `@theme` block of `styles.css`, the one place a colour may be written, and hunts for colours:

```ts
it("lists every colour written outside the theme's Material roles, and finds none", () => {
  const literal = new RegExp([
    String.raw`#[0-9a-fA-F]{3,8}\b`,
    String.raw`\b(?:rgba?|hsla?|hwb|lab|lch|oklab|oklch)\(`,
    String.raw`(?:["'\x60]|:\s*)(?:${NAMED_COLOURS.join("|")})\b`,
  ].join("|"));
  expect(LINES.filter(({ line }) => literal.test(line))
    .map(({ where, line }) => `${where}: ${line.trim()}`)).toEqual([]);
});
```

Run `npx vitest run src/rules.test.ts`. The failure lists each colour by file and line, such as a hex value you wrote for US-23; clearing the list is the morning's work.

> KEY IDEA A test can read code as text as well as run it. A whole-code test states a rule for every file at once, and its failure names each line that breaks it.

23.2 Green: Material 3's colour roles in one block

Material 3, Google's design system, builds a set of ***colour roles*** from one seed colour: `primary` for actions, `surface` for backgrounds, `on-surface` for text on them. The seed is the action blue `#1F5FA8`. The first test, 'keeps the theme in styles.css', expects that line and `--color-*: initial;` in the block Tailwind reads:

```text
@theme static {
  --color-*: initial;
  --color-primary: #1F5FA8; /* 5.8:1 on surface-low, as text */
  --color-surface: #F9F9FF;
  --color-on-surface: #1A1B20; /* 14.1:1 on surface-high */
  /* … */
  --color-group-food: #B5452B; /* circle · 4.9:1 on surface-low */
  --color-north: #B3261E;
}
```

`--color-*: initial;` removes Tailwind's own palette, so no class such as `bg-red-500` brings a colour from outside the roles; `static` writes out every variable, used or not, since code outside CSS reads them by name. Replace each colour the test listed with its role, such as `bg-surface-low`.

> FROM SCHOOL `#1F5FA8` is a colour in hexadecimal: one byte each for red, green and blue. 1F is 31 in denary, 5F is 95 and A8 is 168, so blue leads.

23.3 Colours for code that cannot read CSS

MapLibre's paint and the canvas that draws the pins take plain colour values, not CSS variables, so they read the theme at run time through one function that takes a reader of variables. The page passes its own reader; a test passes a fake:

```ts
export function readTheme(root: Element = document.documentElement): Theme {
  const style = getComputedStyle(root);
  return themeFrom((variable) => style.getPropertyValue(variable).trim());
}
```

Start a `UI-12 shapes as well as colours` block in `design/design.test.ts` with ‘takes each group’s colour from the theme, never from code’. It reads each `--color-…` line of `styles.css` into `TOKENS`, builds a theme with `themeFrom`, and expects each group’s colour to equal the file’s.

23.4 Icons from one font, on a plain background

Every icon comes from Material Symbols, rounded (criterion 2). `Icon` writes the icon’s name as text, and the font draws it as one glyph, by a ligature:

```ts
<span aria-hidden="true" className={`icon ${className}`} style={{ fontSize: `${size}px` }}>
  {name}
</span>
```

The whole rounded font is 5.4 MB, so the worked answer ships a cut of 2.6 kB with the map’s two icons, `add` and `remove`; copy `web/src/assets/fonts/material-symbols-rounded-subset.woff2`. The two icon tests of `rules.test.ts` check every name a `.tsx` file passes against the `material-symbols` package’s list, and allow an `<svg>` in `design/components.tsx` and `map/MapControls.tsx` alone.

> WATCH OUT A name the cut font lacks shows as text: the button reads ‘add’ in place of a plus. The browser test ‘draws the zoom icons from Material Symbols, one glyph each’ catches it, since it expects each icon 24 pixels wide, one glyph.

Criterion 9 keeps the background plain. The style is a value, so ‘draws nothing beneath the places but the plain background’, in the `UI-15 a plain background` block of `style.test.ts`, expects the bottom layer to be `{ id: "background", type: "background", paint: { "background-color": "var(--color-surface)" } }`. Its fake theme passes each variable’s name, so the test checks where a colour goes, not what it is.

> IN INK Run `cd web && npm test`, then `npm run build && npm run e2e -- --grep "Material Symbols"` at the root; commit at each green, with the message `UI-15: one Material 3 theme, and icons from Material Symbols`, and push.

> THE PROOF The tests of Exhibit 23.1 pass. UI-15 stays open. Tomorrow’s first test: `makes every transition and animation last 300 ms or less`.

24

Day 23 · Thursday 19 November: UI-15, movement that stays calm

> IN SHORT Every movement stays short, nothing loops, and the pins drop in once. This finishes UI-15, worth 2 points.

Today every movement on the page lasts 300 ms or less, nothing loops, and a visitor who has asked their system to reduce motion sees nothing move. The pins drop in once as the page opens, then rest.

Movement lives in three places: the page's CSS, MapLibre's camera and the pins' drop. The day takes each in turn, then meets three criteria that the worked answer leaves untested.

EXHIBIT 24.1 Day 23 ends when no duration passes 300 ms and reduced motion stops everything

Goal	Short, eased movement that never loops, and none on request
Story	UI-15 c6 to c8; tests 2 and 4; c3 to c5 beyond the worked answer
In the worked answer	`web/src/design/motion.ts`; `:root` and `@media (prefers-reduced-motion: reduce)` in `styles.css`; `motion()` in `web/src/map/map-handle.ts`; `web/src/map/landing.ts`; `useLanding` in `web/src/map/canvas-hooks.ts`
Look for	How the test finds every duration in the source; why the handle marks a move `essential`
Leave for later	Nothing: UI-15 closes today
Done when	In `rules.test.ts`: 'finds the durations it checks', 'makes every transition and animation last 300 ms or less', 'lets nothing loop', 'stops all movement when the visitor asks for reduced motion'; in `design.test.ts`: 'moves for no longer than 300 ms, and not at all for a visitor who asks for less', 'eases as Material's emphasised curve: fast away, slow to rest' and the three drop tests; the seven tests of `landing.test.ts`; in Chrome, 'drops the pins in as the page opens, then hands over to the map's own' and 'stops all movement when the visitor asks for reduced motion'

Source: `docs/user-stories/user-stories.md`, *UI-15; NFR-51 to NFR-56*

24.1 Red, then green: every duration in the source

The ceiling of 300 ms covers every file, so it joins yesterday's whole-code tests. Add a `UI-15 calm motion` block to `rules.test.ts` that collects every time written in CSS, such as `300ms`, and every Tailwind class such as `duration-300`:

```ts
const durations = LINES.flatMap(({ where, line }) => [
  ...[...line.matchAll(/(?<![\w.-])(\d+(?:\.\d+)?)(ms|s)\b/g)]
    .map((m) => ({ where, ms: Number(m[1]) * (m[2] === "s" ? 1000 : 1) })),
  ...[...line.matchAll(/\b(?:duration|delay)-(\d+)\b/g)]
    .map((m) => ({ where, ms: Number(m[1]) })),
]);

it("makes every transition and animation last 300 ms or less", () => {
  expect(durations.filter((d) => d.ms > 300)).toEqual([]);
});
```

'finds the durations it checks' expects at least four, since a search that found none would pass every rule; 'lets nothing loop' refuses `infinite`, `@keyframes` and `animation-iteration-count`. Green comes from Material's three durations, written once in the `:root` of `styles.css` as `--motion-short: 150ms`, `--motion-exit: 200ms` and `--motion-medium: 300ms`, and once for code:

```ts
export const MOTION = { short: 150, exit: 200, medium: 300 } as const;
export function movementMs(reduced: boolean): number {
  return reduced ? 0 : MOTION.medium;
}
```

24.2 Red, then green: nothing moves on request

A visitor can ask their operating system to reduce motion, and the browser passes the wish on as ***prefers-reduced-motion***. CSS answers it with one rule for every element, which 'stops all movement when the visitor asks for reduced motion' looks for:

```text
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation: none !important;
    transition: none !important;
    scroll-behavior: auto !important;
  }
}
```

MapLibre moves outside CSS, so the map's handle reads the wish itself:

```ts
const motion = () => {
  const duration = movementMs(prefersReducedMotion());
  return { duration, easing: emphasised, essential: duration > 0 };
};
```

By its own rule MapLibre skips a move for such a visitor unless the move is `essential`. The page has already applied the wish, so it marks its moves essential, and MapLibre does not apply the wish twice. Names and pins fade by `fadeDuration: movementMs(reducedMotion)` in `baseOptions`; the browser test reads that fade from MapLibre's private `_fadeDuration`, since MapLibre offers no public way to read it. `emphasised`, Material's easing, starts fast and settles slowly; read its comment on Newton's method, then type it and its test.

24.3 The pins drop in once

Criterion 6 is the one movement no action of the visitor starts: as the page opens, the pins drop into place one after another, each in 0.3 seconds. Test the spacing first, in a `UI-15 the pins drop in as the page opens` block of `design.test.ts`:

```ts
it("drops each pin in 300 ms, one after another, 30 ms apart", () => {
  expect(dropDelays(4, false)).toEqual([0, 30, 60, 90]);
  expect(dropMs(dropDelays(4, false))).toBe(90 + 300);
});
```

```ts
export function dropDelays(count: number, reduced: boolean): readonly number[] {
  if (reduced || count <= 0) return [];
  const step = count <= 1 ? 0 : Math.min(DROP_STEP_MS, DROP_SPREAD_MS / (count - 1));
  return Array.from({ length: count }, (_, i) => Math.round(i * step));
}
```

101 pins 30 ms apart would start the last at 3,000 ms, so the step shrinks until the last starts by 600 ms. `landing.ts` decides when the drop runs. It is a ***state machine***: a value naming the stage, `waiting`, `dropping` or `landed`, and a function, `step`, that moves it on when an event happens. Type its seven tests in `landing.test.ts`, then the file. The model gains `landed` and `pinsLanded` ('opens with the pins still to land, and records their landing', in `map.test.ts`); the style hides the map's own pins until then ('keeps the map's own pins and names unseen until the dropping pins land', whose lines for clusters and names wait for days 28 and 31); `useLanding` runs the timers, with its own test in `canvas-hooks.test.tsx`; and `openMap` in `web/e2e/map-page.ts` now waits for the pins to land.

> TRY IT Beyond the worked answer: elevation, corners and the state layer (criteria 3 to 5; NFR-51 to NFR-53) read 'No test yet'. The worked answer's block 'UI-15 named surfaces' checks that `.card` and `.map-button` are named once in `styles.css`, but not their shadows or corners. `styles.css` meets the criteria, so add this test to your `rules.test.ts`, and note it as your own. The name tooltips' 4-pixel corners come with `drawTooltip`, in `design/canvas.ts`, on day 31.
>
> ```ts
> it("follows Material's elevation, corners and state layers (UI-15, c3 to c5)", () => {
>   expect(STYLES).toMatch(/\.card \{\s*@apply [^;]*rounded-medium[^;]*shadow-level-1/);
>   expect(STYLES).toMatch(/\.map-button \{\s*@apply [^;]*rounded-medium[^;]*shadow-level-2/);
>   expect(STYLES).toContain("--radius-medium: 12px;");
>   expect(STYLES).toContain("--state-hover: 0.08;");
>   expect(STYLES).toContain("--state-pressed: 0.12;");
> });
> ```

> IN INK Run `cd web && npm test`, then `npm run build && npm run e2e -- --grep "UI-15"` at the root; commit at each green, with the message `UI-15: calm movement, and none for a visitor who asks for less`, and push.

> THE PROOF The tests of Exhibit 24.1 pass, with your own beside them. UI-15 is done: 2 points, 20 of 29. Tomorrow's first test: `gives each group its own shape`.

25

Day 24 · Friday 20 November: UI-12, shapes as well as colours

> IN SHORT Each group gains a shape, and each group colour proves its contrast. This is UI-12, worth 1 point; the afternoon checks the burn-up.

Today each group gains a white shape inside its pin, a circle, a triangle, a square or a diamond, so a visitor who cannot tell the colours apart can still tell the groups apart. A second test proves each group's colour stands out from white by at least 4.5 to 1.

EXHIBIT 25.1 Day 24 ends with four shapes and every group colour at 4.5:1 or more

Goal	One shape for each group; contrast of 4.5:1 for each group colour on white
Story	UI-12 c1 and c2; tests 1 and 2
In the worked answer	`web/src/design/shapes.ts`; `GROUP_LOOKS` in `web/src/design/theme.ts`; `drawPin` in `web/src/design/canvas.ts`; `readImageName` in `web/src/map/images.ts`; `web/src/testing/contrast.ts`
Look for	One path string drawn two ways; why the contrast test uses `expect.soft`
Leave for later	The cluster's ring of colours (day 28)
Done when	'gives each group its own shape'; 'meets 4.5:1 for text in each group colour on white'; the fourteen 'meets 4.5:1 for … on …' pairs, 'lists each text pair once' and 'measures contrast as WCAG does'; 'draws each place with its group's pin'

Source: `docs/user-stories/user-stories.md`, *UI-12; NFR-64, NFR-65*

25.1 Red, then green: four shapes

The first test names each group's shape and checks that the four drawings differ. Put it first in the `UI-12` block of `design.test.ts`:

```ts
expect(Object.fromEntries(GROUP_KEYS.map((g) => [g, GROUP_LOOKS[g].shape]))).toEqual({
  food: "circle", outdoors: "triangle", culture: "square", transport: "diamond",
});
expect(new Set(GROUP_KEYS.map((g) => SHAPE_PATHS[GROUP_LOOKS[g].shape])).size).toBe(4);
```

Each shape is SVG path data, a string of drawing moves in a 24 by 24 box. A canvas, through `Path2D`, and an SVG `<path>` draw the same string, so the map's pins and the pins that drop in never drift apart. Copy `SHAPE_PATHS` from `shapes.ts`: four short strings, such as `"M12 6.8l5.2 9H6.8z"` for the triangle. `GROUP_LOOKS` in `theme.ts` pairs each group with its shape and colour variable, such as `food: { shape: "circle", colour: "--color-group-food" }`; its type, `Record<GroupKey, GroupLook>`, will not compile with a group left out.

25.2 Red, then green: contrast of 4.5 to 1

Criterion 2 borrows WCAG's rule for text: a ***contrast ratio*** of at least 4.5 to 1 against the background, where 1 to 1 is one colour on itself and 21 to 1 is black on white:

```ts
it("meets 4.5:1 for text in each group colour on white", () => {
  for (const g of GROUP_KEYS) {
    expect.soft(contrastRatio(token(`group-${g}`), token("surface-lowest")), g)
      .toBeGreaterThanOrEqual(4.5);
  }
});
```

```ts
// testing/contrast.ts
function luminance([r, g, b]: readonly [number, number, number]): number {
  const linear = (c: number): number => {
    const s = c / 255;
    return s <= 0.04045 ? s / 12.92 : ((s + 0.055) / 1.055) ** 2.4;
  };
  return 0.2126 * linear(r) + 0.7152 * linear(g) + 0.0722 * linear(b);
}
// contrastRatio: (lighter + 0.05) / (darker + 0.05)
```

`expect.soft` records a failure and carries on, so one run names every group that falls short. On white the four group colours range from about 5.5:1, for food, to 7.4:1, for culture. The worked answer adds the page's fourteen pairs of text and background, run by `it.each`; 'lists each text pair once', since a pair listed twice hides one left out; and 'measures contrast as WCAG does', which pins the sum: black on white gives 21, and `#777777` on white 4.48, under the line.

25.3 The pin on the map

`drawPin` fills the pin's outline in the group's colour, then the group's shape in white. The style names each pin's picture by its group, such as `pin:food`, and MapLibre asks the page to draw any picture it lacks. Add 'draws each place with its group's pin' to a `UI-12` block of `style.test.ts`, with day 21's pin-size test beside it:

```ts
expect(symbol(LAYER.pins).layout?.["icon-image"])
  .toEqual(["concat", "pin:", ["get", "group"]]);
expect(readImageName("pin:transport")).toEqual({ kind: "pin", group: "transport" });
expect(readImageName("pin:bakeries")).toBeNull();
```

The worked answer's test also reads a cluster's name, `cluster:3-0-1-0`; leave that line for day 28.

> IN INK Run `cd web && npm test`, then commit at each green, with the message `UI-12: a shape for each group, and contrast of 4.5:1`, and push.

> THE PROOF The tests of Exhibit 25.1 pass. UI-12 is done: 1 point, 21 of 29, the plan's figure for this Friday. Write your own beside it, and read any gap in days, as on day 14. Monday's first test: `chooses 500 m at level 15 and 50 m at level 18`.

26

Day 25 · Monday 23 November: UI-19, the scale bar

> IN SHORT A scale bar shows a round length that fits in 140 pixels, and a click switches its units. This is the first of UI-19's two days, worth 2 points.

Today a scale bar appears at the bottom left: the longest round length, 1, 2 or 5 of a unit, that fits in 140 pixels, which reads 500 m at level 15 over Macau and 50 m at level 18. A click switches it to feet and miles, and one record feeds both the drawing and the words a screen reader hears.

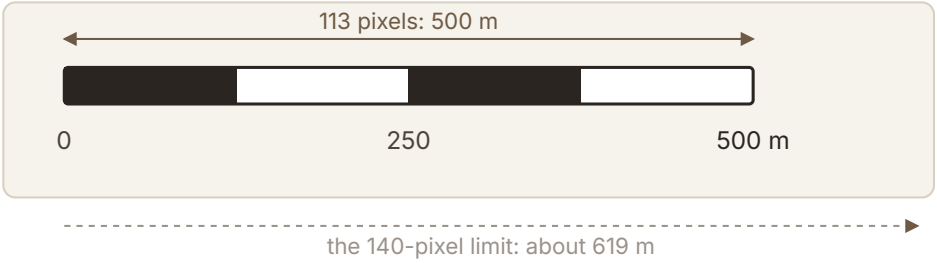
EXHIBIT 26.1 Day 25 ends when the bar reads 500 m at level 15 and switches units on a click

Goal	A bar of four sections whose length and figures follow the level, in metric or imperial units
Story	UI-19 c1 to c4; tests 1 and 2
In the worked answer	`web/src/map/scale.ts`; `ScaleBar` in `web/src/map/ScaleBar.tsx`; `.scale-sections` in `styles.css`; `toggleUnits` in `web/src/map/map.ts`
Look for	Why one value, `ScaleReading`, holds both the drawing and the spoken label
Leave for later	The compass and the coordinates (day 26)
Done when	In `readouts.test.ts`: 'chooses 500 m at level 15 and 50 m at level 18', 'takes the longest length of 1, 2 or 5 of a unit that fits', 'switches between metric and imperial on a click, and names the units it will switch to'; in `controls.test.tsx`: 'shows '0', the halfway figure and the full length, and names its length and the other units' and 'switches between metric and imperial on a click', a name `map.test.ts` shares

Source: `docs/user-stories/user-stories.md`, *UI-19; FR-49 to FR-51, NFR-60*

Work the bar out on paper first, so you know what the test should expect. Exhibit 26.2 works level 15; at level 18 a pixel covers an eighth as much, 0.553 m, so 140 pixels cover about 77 m, and the bar shows 50 m in 90 pixels.

EXHIBIT 26.2 At level 15 the bar shows 500 m in 113 of its 140 pixels



4.42 m a pixel
× 140 = 619 m
round down: 500 m
÷ 4.42 = 113 px

Source: `web/src/map/scale.ts`; `web/src/map/readouts.test.ts`, *at latitude 22.19° N*

26.1 Red, then green: 500 m and 50 m

The story names two figures, and so does the test, in a `UI-19 the scale bar` block of `readouts.test.ts`:

```ts
it("chooses 500 m at level 15 and 50 m at level 18", () => {
  const at15 = scaleBar(15, LAT, "metric");
  const at18 = scaleBar(18, LAT, "metric");
  expect([at15.half, at15.full]).toEqual(["250", "500 m"]);
  expect([at18.half, at18.full]).toEqual(["25", "50 m"]);
  for (const bar of [at15, at18]) expect(bar.widthPx).toBeLessThanOrEqual(140);
  expect(at15.widthPx).toBe(Math.round(500 / metresPerPixel(15, LAT)));
});
```

```ts
export function roundLengthBelow(value: number): number {
  const power = 10 ** Math.floor(Math.log10(value));
  const leading = value / power;
  return (leading >= 5 ? 5 : leading >= 2 ? 2 : 1) * power;
}
```

```ts
export function scaleBar(level: number, latitude: number, units: Units): ScaleReading {
  const perPixel = metresPerPixel(level, latitude);
  const { length, unit, metres } = roundLength(perPixel * SCALE_MAX_PX, units);
  const [one, many] = UNIT_WORDS[unit];
  const other = units === "metric" ? "imperial" : "metric";
  return {
    widthPx: Math.round(metres / perPixel),
    half: figure(length / 2),
    full: `${figure(length)} ${unit}`,
    // … label: 'Scale bar: 500 metres. Switch to imperial units'
  };
}
```

> FROM SCHOOL `Math.log10` counts digits. For 619 it gives 2.79; the floor is 2, so `power` is 100 and `leading` is 6.19. The selection picks 5, and 5 × 100 is 500. The same three branches serve 0.9 m or 730 km: the test tries 0.9, 1, 1.99, 2, 4.99, 5, 9.99 and 730, the boundaries on either side of each choice.

`roundLength` turns metres into kilometres from 1,000 and feet into miles from one mile, and counts a foot as exactly 0.3048 m, its legal definition, so both bars measure the same ground. The block's other tests expect '2 km' at level 12, '2,000 ft' at level 15 and '5 mi' at level 11, and the label 'Scale bar: 1 mile. Switch to metric units' at level 13, with 'mile' for one.

26.2 The bar on the page

The bar is a button: a click switches the units, and its label gives the length in words and the units a click switches to. A screen reader skips the figures beneath, or it would read '0 250 500 m' after the label. The component test in `controls.test.tsx` finds the button by the name 'Scale bar: 500 metres. Switch to imperial units' and expects its text to be `0250500 m`, the three figures joined:

```ts
<button type="button" aria-label={bar.label} onClick={onToggleUnits}
  className="state-layer card …">
  <span aria-hidden="true" className="scale-sections … duration-300 ease-emphasized"
    style={width} />
  <span aria-hidden="true" className="relative block h-4 …" style={width}>
    <span className="absolute left-0">0</span>
    <span className="absolute left-1/2 -translate-x-1/2">{bar.half}</span>
    <span className="absolute right-0 …">{bar.full}</span>
  </span>
</button>
```

The four sections, filled and empty in turn, are one CSS gradient on `.scale-sections`. The model gains `units`, and the update answers `toggleUnits` in one line: `[{ units: model.units === "metric" ? "imperial" : "metric" }]`. `App.tsx` passes the bar `model.view.zoom` and the latitude of the map's centre, `model.view.center.lat`.

TRY IT Beyond the worked answer: FR-50 names `controls.test.tsx` and the browser test for the four sections (criterion 2), but both read the three figures, not the sections; NFR-60, the change within 0.3 seconds (criterion 3), is proved in part, by the rule that caps every written duration at 300 ms. jsdom draws nothing, but it reads the sections' class and width. Add to your `controls.test.tsx`, and note it as your own:

```ts
it("draws four sections that change within 0.3 s (UI-19, c2, c3)", () => {
  const { container } = render(<ScaleBar level={15} latitude={LAT} units="metric"
    onToggleUnits={vi.fn()} />);
  const sections = container.querySelector<HTMLElement>(".scale-sections");
  expect(sections?.style.width).toBe("113px");
  expect(sections?.className).toContain("duration-300");
});
```

IN INK Run `cd web && npm test`, then commit at each green, with the message `UI-19: a scale bar of 1, 2 or 5 of a unit, in metric or imperial`, and push.

THE PROOF The tests of Exhibit 26.1 pass. UI-19's first browser test waits for tomorrow, since its last line looks for the compass. Tomorrow's first test: `names north for screen readers, as a sign rather than a button`.

27

Day 26 · Tuesday 24 November: UI-19, the compass and the coordinates

> IN SHORT A compass names north, and a readout gives the point under the pointer. This finishes UI-19, worth 2 points.

Today a compass that points north joins the top of the controls, and a readout beside the scale bar gives the latitude and longitude under the pointer, such as '22.19° N, 113.55° E'. The readout goes when the pointer leaves the map, with no code of its own to hide it.

EXHIBIT 27.1 Day 26 ends when the readout follows the pointer and goes over a control

Goal	A compass that names north; the point under the pointer, latitude first, to two decimal places
Story	UI-19 c5 to c9; tests 3 and 4
In the worked answer	`Compass` in `web/src/map/MapControls.tsx`; `web/src/map/coordinates.ts`; `Coordinates` in `web/src/map/ScaleBar.tsx`; `attachEvents` in `web/src/map/maplibre.ts`
Look for	Why a value that rounds to zero takes N or E; why a zoom button hides the readout without code of its own
Leave for later	A compass that turns the map back to north, which the story leaves open
Done when	'names north for screen readers, as a sign rather than a button'; 'shows the point under the pointer, silent to screen readers, and goes when the pointer leaves'; 'writes the point under the pointer as 22.19° N, 113.55° E, latitude first, to two decimal places'; 'names the southern and western hemispheres, and takes a value that rounds to zero as north or east'; 'follows the pointer, and forgets it when the pointer leaves'; 'reports the pointer's position, and null when it leaves the map'; in Chrome, both tests of 'UI-19 distance, direction and position'

Source: `docs/user-stories/user-stories.md`, *UI-19; FR-52 to FR-54, NFR-66, NFR-67*

27.1 Red, then green: a compass that is a sign

Add a `UI-19 the compass` block to `controls.test.tsx`. It finds the compass as a picture with a name, and proves that no button of that name exists:

```ts
controls(15);
expect(screen.getByRole("img", { name: "Compass: north is up" })).toBeTruthy();
expect(screen.queryByRole("button", { name: /compass/i })).toBeNull();
```

The map never turns, so the compass is a sign, not a button: a `div` with `role="img"` and that label, around a small SVG of a ring, a red needle and a red 'N'. `MapControls` draws it above the zoom buttons, and `baseOptions` sets `dragRotate: false`, so north always points up.

TRY IT Beyond the worked answer: FR-52, the red needle and 'N' of criterion 5, is proved in part: the test above checks the compass's name, not its colour. Add to its block, as your own:

```ts
it("points north with a red needle and a red 'N' (UI-19, criterion 5)", () => {
  controls(15);
  const compass = screen.getByRole("img", { name: "Compass: north is up" });
  expect(compass.querySelectorAll(".fill-north")).toHaveLength(2);
  expect(compass.querySelector("text")?.textContent?.trim()).toBe("N");
});
```

27.2 Red, then green: the point in words

The readout writes latitude first, to two decimal places, with a letter for each hemisphere. The two tests of a `UI-19 coordinates` block in `readouts.test.ts` expect '22.19° N, 113.55° E' for Macau, '33.87° S, 0.13° W' for a point south and west, and '0.00° N, 0.00° E' for `{ lng: -0.001, lat: -0.004 }`:

```ts
function coordinate(value: number, positive: string, negative: string): string {
  const rounded = Math.abs(value).toFixed(2);
  return `${rounded}° ${value < 0 && rounded !== "0.00" ? negative : positive}`;
}

export function formatPosition({ lng, lat }: LngLat): string {
  return `${coordinate(lat, "N", "S")}, ${coordinate(lng, "E", "W")}`;
}
```

−0.001 rounds to 0.00, a point on the line, which lies on neither side; the check `rounded !== "0.00"` gives it the positive letter.

27.3 Red, then green: the readout follows the pointer

The position travels from MapLibre to the model and back to the page, and each step has its test. `attachEvents`, the one function that wires MapLibre's events, turns `mousemove` into a position and `mouseout` into `null`; `maplibre.test.ts` fires both on the fake map of `web/src/testing/fake-maplibre.ts`. `App.tsx` turns each report into `Msg.pointerMoved`; the model gains `pointer`; `Coordinates` draws it, or nothing for `null`, with `aria-hidden="true"`, which its component test checks.

The browser test moves the mouse over Macau's centre, reads '22.19° N, 113.55° E', hovers over '+' and expects the readout gone. No code hides it: the controls sit beside the map's container, not inside it, so the pointer leaves the map and MapLibre fires `mouseout`. FR-54 notes a gap, a pointer on the scale bar or the compass, which a second `hover()` would close.

IN INK Run `cd web && npm test`, then `npm run build && npm run e2e -- --grep "UI-19"` at the root; commit at each green, with the message `UI-19: a compass that names north, and the point under the pointer`, and push.

THE PROOF The tests of Exhibit 27.1 pass. UI-19 is done: 2 points, 23 of 29, the figure the plan sets for M3. Tomorrow's first test: `merges pins that sit closer than the distance for the level`.

28

Day 27 · Wednesday 25 November: UI-10 begins, where pins merge

IN SHORT The style gains a clustered source, and two tests fix where pins merge. This is the first of UI-10's three days, worth 3 points; the third falls on Monday 30 November, in iteration 4.

Today pins that sit closer than 48 pixels on screen merge into one cluster, a distance that halves on the ground at each level up, and from level 16 every place stands apart. The browser does the clustering, as ADR 0005 decides, so the service and its tables stay as they are.

A ***cluster*** is one round mark that stands for several places close together. MapLibre clusters only a GeoJSON source, but the service sends vector tiles; ADR 0005 settles how the map bridges the two.

EXHIBIT 28.1 Day 27 ends with the merge distance proved by the library MapLibre clusters with

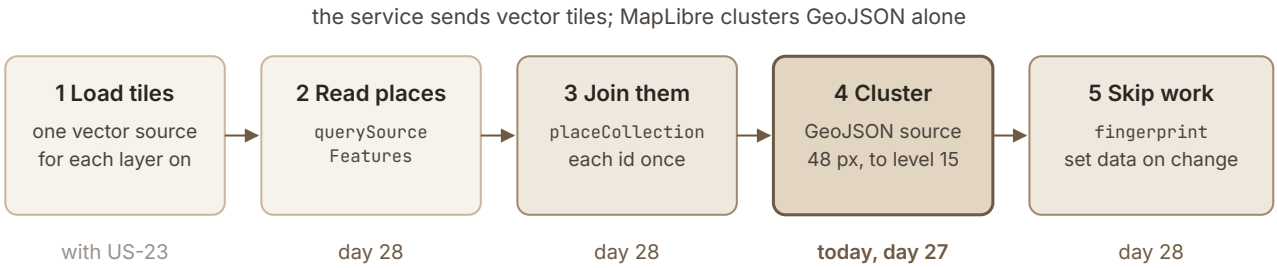
Goal	A clustered source with a radius of 48 pixels, clustering up to level 15
Story	UI-10 c1, the merging; tests 1 and 2; UI-17 c5, the last cluster level
In the worked answer	`docs/adr/0005-cluster-in-the-browser.md`; `CLUSTER_RADIUS_PX` and `sources()` in `web/src/map/style.ts`; `LAST_CLUSTER_LEVEL` in `levels.ts`; `mergeDistanceMetres` in `labels.ts`; `web/src/map/clusters.test.ts`
Look for	Why the test builds its own Supercluster; why `extent` is 512
Leave for later	Feeding the places and the ring (day 28); a click that opens a cluster, its label, the pins that move out (day 30)
Done when	'merges pins that sit closer than the distance for the level'; 'halves that distance at each level up'; 'keeps every place apart from level 16 (UI-17, criterion 5)'

Source: `docs/user-stories/user-stories.md`, *UI-10;* `docs/adr/0005-cluster-in-the-browser.md`; *FR-38, FR-47*

28.1 Read ADR 0005: the browser clusters

The worked answer clusters in the browser, from the tiles it has already loaded. Exhibit 28.2 shows the five steps of the bridge; today builds step 4, and tomorrow steps 2, 3 and 5.

EXHIBIT 28.2 Five steps carry places from vector tiles into MapLibre's clusters



Source: `docs/adr/0005-cluster-in-the-browser.md`, *'Decision'*

KEY IDEA The ADR weighs four options and records the cost of its choice: the map clusters only the places in the tiles it has loaded, so a cluster at the screen's edge can grow as the visitor pans. Read a decision's costs as closely as its gains.

28.2 Red: the distance at which pins merge

The test must cluster as MapLibre does, without a browser. MapLibre clusters with Supercluster, which the map's development packages bring in through `@maplibre/geojson-vt`, so the test builds one with the map's radius and last level. Create `web/src/map/clusters.test.ts`:

```ts
function clustersAt(at: Level, features: Points): number {
  const index = new Supercluster({ radius: CLUSTER_RADIUS_PX, extent: 512,
    maxZoom: toMapZoom(LAST_CLUSTER_LEVEL) });
  index.load(features);
  return index.getClusters([-180, -85, 180, 85], toMapZoom(at)).length;
}

it("merges pins that sit closer than the distance for the level", () => {
  const distance = mergeDistanceMetres(14, LAT);
  expect(clustersAt(level(14), pair(distance * 0.8))).toBe(1);
  expect(clustersAt(level(14), pair(distance * 1.2))).toBe(2);
});
```

`pair(metres)` places two points that far apart, due east of each other on Macau's latitude. `extent: 512` matches the 512-pixel tiles against which MapLibre measures its radius. Each pair sits 20 per cent inside or outside the distance, so rounding cannot tip a result. 'halves that distance at each level up' expects the 0.8 pair apart at level 15, and a 0.4 pair merged.

28.3 Green: a radius, a distance and a last level

```ts
// map/style.ts
export const CLUSTER_RADIUS_PX = 48;
// map/labels.ts
export function mergeDistanceMetres(level: number, latitude: number): number {
  return CLUSTER_RADIUS_PX * metresPerPixel(level, latitude);
}
// map/levels.ts
export const LAST_CLUSTER_LEVEL = level(15);
```

At level 14 over Macau a pixel covers about 8.85 m, so pins closer than about 425 m merge; at level 15, about 212 m. Give the style its clustered source beside US-23's vector sources:

```ts
[SOURCE.places]: {
  type: "geojson",
  data: empty(),
  cluster: true,
  clusterMaxZoom: toMapZoom(LAST_CLUSTER_LEVEL),
  clusterRadius: CLUSTER_RADIUS_PX,
  clusterProperties: CLUSTER_PROPERTIES,
},
```

> WATCH OUT `clusterMaxZoom` is MapLibre's zoom, one below the level. Write `15` there and the map clusters through level 16, where UI-17 needs every place apart. `toMapZoom` gives 14, and its branded type refuses a plain number.

The third test, 'keeps every place apart from level 16 (UI-17, criterion 5)', expects two places one metre apart to stay two at level 16. `CLUSTER_PROPERTIES` and the source's data come tomorrow; until then, write `clusterProperties: {}`.

> IN INK Run `cd web && npm test`, then commit at each green, with the message `UI-10: merge pins closer than 48 pixels, up to level 15`, and push.

> THE PROOF The three tests of Exhibit 28.1 pass. UI-10 stays open and adds nothing to the burn-up until it is done. Tomorrow's first test: `orders names between layers by the map's order, then by id, whatever order the tiles bring`.

29

Day 28 · Thursday 26 November: UI-10, places fed and ringed

IN SHORT The loaded places reach the clusters, and each cluster wears its groups' colours. This is the second of UI-10's three days, worth 3 points.

Today the places of the loaded tiles reach the clustered source, each kept once, and each cluster wears a ring of its groups' colours in proportion. A pure join does the work, a fingerprint skips it when nothing has changed, invisible layers keep the tiles loading, and the cluster rests still once drawn.

Today follows the bridge of Exhibit 28.2: join the places, skip work when nothing has changed, keep the tiles loading, then draw each cluster.

EXHIBIT 29.1 Day 28 ends when every place reaches the clusters once and each cluster shows its groups

Goal	One list of places from the loaded tiles; a ring of group colours on each cluster; a cluster at rest
Story	UI-10 c1, the number and the ring, and c5
In the worked answer	`placeCollection` and `fingerprint` in `web/src/map/places.ts`; `feedPlaces` in `maplibre.ts`; `feedLayer` and `clusterImage` in `style.ts`; `readImageName` in `images.ts`; `drawCluster` in `web/src/design/canvas.ts`
Look for	Why a place can arrive twice; why the counts travel inside the picture's name
Leave for later	A click that opens a cluster (c3), its label (c4), the pins that move out (c2): day 30
Done when	'orders names between layers by the map's order, then by id, whatever order the tiles bring', without its last line; 'drops features that are not points or carry no id'; 'loads the tiles of each layer that is on through a layer that draws nothing'; 'draws each place with its group's pin', with its cluster line; 'lets nothing loop'; your own test of the fingerprint

Source: `docs/user-stories/user-stories.md`, *UI-10;* `docs/adr/0005-cluster-in-the-browser.md`; *FR-38, NFR-59*

29.1 Red, then green: each place once

A place near a tile's edge arrives in each tile whose margin holds it, so the join keeps each place once: a `Map` keyed by id holds one entry for each key. The worked answer files the join's tests under UI-17, since they also prove the fixed order of names; write them today, under that name, in `style.test.ts`. Leave the first test's last line, which asks the style for the sort key, for day 33:

```ts
const loaded = [
  { layerId: "food.cafes",
    features: [feature("cafes-002"), feature("cafes-001"), feature("cafes-002")] },
  { layerId: "outdoors.parks", features: [feature("parks-001")] },
] as const;
const places = placeCollection(loaded).features
  .map((f) => [f.properties.id, f.properties.sort, f.properties.one]);
expect(places).toEqual([["cafes-001", 0, "Café"], ["cafes-002", 1, "Café"],
  ["parks-001", 1_000_000, "Park"]]);
```

```ts
export function placeCollection(loaded: readonly LoadedLayer[]): PlaceCollection {
  const features = loaded.flatMap(({ layerId, features: raw }, layerIndex) => {
    const layer = layerOf(layerId);
    if (layer === undefined) return [];
    const unique = new Map(raw.flatMap((f) => {
      const point = pointOf(f);
      const values = valuesOf(f);
      return point === null || values.id === ""
        ? [] : [[values.id, { point, values }] as const];
    }));
    return [...unique.values()].toSorted(byId)
      .map(({ point, values }, rank): PlaceFeature => ({
      type: "Feature",
      geometry: { type: "Point", coordinates: point },
      properties: { ...values, layerId, group: layer.group, one: layer.one,
        sort: layerIndex * LAYER_STEP + rank },
    }));
  });
  return { type: "FeatureCollection", features };
}
```

29.2 Skip work: the fingerprint

Each new list makes MapLibre cluster again, so `feedPlaces` sets the source's data only when the places change. It compares a ***fingerprint***, the places' ids joined into one string, with the last it set:

```ts
export function fingerprint(collection: PlaceCollection): string {
  return collection.features.map((f) => f.properties.id).join("|");
}
```

`feedPlaces` runs when a tile source finishes loading and when the map comes to rest, once a frame however many tiles arrive in it. It reads each layer's places with `map.querySourceFeatures`, joins them and compares fingerprints: a shell around the pure join, with no rule of its own. Type it from the worked answer.

TRY IT Beyond the worked answer: the fingerprint has no test. Add this beside the join's tests, importing `fingerprint` from `"."`, and note it as your own:

```ts
it("changes the fingerprint only when the places change (ADR 0005, step 5)", () => {
  const once = placeCollection([{ layerId: "food.cafes",
    features: [feature("cafes-001")] }]);
  const twice = placeCollection([{ layerId: "food.cafes",
    features: [feature("cafes-001"), feature("cafes-001")] }]);
  expect(fingerprint(twice)).toBe(fingerprint(once));
  expect(fingerprint(placeCollection([]))).not.toBe(fingerprint(once));
});
```

29.3 Tiles that load, with nothing drawn

Today the pins and names move from the vector sources to the clustered source. No visible layer reads the tiles any longer, and MapLibre loads no tile for a source that no layer reads. The worked answer gives each layer that is on a ***feed layer***: a `circle` layer of radius 0 and opacity 0, whose one job is to make MapLibre load the layer's tiles for `feedPlaces` to read:

```ts
function feedLayer(id: LayerId): LayerSpecification {
  return {
    id: LAYER.feed(id),
    type: "circle",
    source: SOURCE.tiles(id),
    "source-layer": id,
    paint: { "circle-radius": 0, "circle-opacity": 0, "circle-stroke-opacity": 0 },
  };
}
```

Its test, 'loads the tiles of each layer that is on through a layer that draws nothing', sits in the `UI-15 a plain background` block of `style.test.ts` and expects no feed layer for a layer that is off. The circle layers close day 18's planned difference: put `"circle"` back into the browser test 'fills the window with the map, on a plain background', which now expects `background`, `circle` and `symbol`, as the worked answer's does.

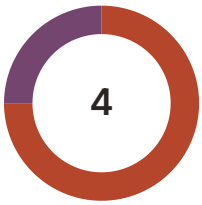
29.4 The ring of colours

A cluster shows its number on a white disc, ringed in its groups' colours in proportion (criterion 1). MapLibre counts each group's places in each cluster through `clusterProperties`: for each group, add 1 for a place in it, else 0. The style names the cluster's picture by its four counts, such as `cluster:3-0-1-0`, so each mix of groups gets a picture of its own:

```ts
const CLUSTER_PROPERTIES = Object.fromEntries(
  GROUP_KEYS.map((g) => [g, ["+", ["case", ["==", ["get", "group"], g], 1, 0]]]));
```

Add the cluster line to day 24's test 'draws each place with its group's pin', expecting `readImageName("cluster:3-0-1-0")` to give `{ kind: "cluster", counts: { food: 3, outdoors: 0, culture: 1, transport: 0 } }`, then teach `readImageName` to read four whole counts after `cluster:`.

EXHIBIT 29.2 The picture `cluster:3-0-1-0` holds four places: three in food, one in culture



The ring starts at twelve o'clock and runs clockwise, in the map's order of groups.

Food and drink: 3 of 4 places, three quarters of the ring.

Culture: 1 of 4 places, the last quarter.

Outdoors and transport hold no place here, so they draw nothing.

```
drawCluster(theme, { food: 3, outdoors: 0, culture: 1, transport: 0 })
```

Source: `drawCluster` in `web/src/design/canvas.ts` (outer 48 px, white disc 34 px), drawn here at twice the size

As Exhibit 29.2 shows, `drawCluster` paints one slice for each group with places, then the white disc and the number, with Material's level-2 shadow. jsdom has no canvas, so no Vitest test draws a cluster, and FR-38 reads 'in part: no test checks the ring'; the browser tests see the clusters on the map. `provideImages`, in `maplibre.ts`, draws each picture the first time MapLibre asks for its name.

29.5 A cluster that rests still

Criterion 5 asks for no pulse or loop. Day 23's whole-code test 'lets nothing loop' already refuses `infinite`, `@keyframes` and `animation-iteration-count` in every file. `drawCluster` draws each picture once, and the clusters layer takes no landing paint, so a cluster shows at once and never moves. Add the clusters' line to day 23's test 'keeps the map's own pins and names unseen until the dropping pins land', which expects no paint on that layer. NFR-59 counts this proof as partial, since the test reads the page's styles, not the clusters MapLibre draws. Three criteria wait for Monday: criterion 3, 'zooms at least to level 16'; criterion 4, the label; and criterion 2, whose NFR-58 reads 'No test yet'.

> IN INK Run `cd web && npm test`, then commit at each green, with the message `UI-10: feed the loaded places to the clusters, ringed by group`, and push.

> THE PROOF The tests of Exhibit 29.1 pass, your fingerprint test among them. Start `rebuild-db` and your service as on day 18, and open `http://127.0.0.1:8080/` ; zoom out to level 13 over Macau, and numbered, ringed clusters appear. Monday's first test: `zooms at least to level 16`.

30

Day 29 · Friday 27 November: the review, and M3

> IN SHORT A clean run proves the fortnight's work, and the review shows M3. You write no new code; the burn-up should read 23 of 29, with UI-10 two days in.

Today you prove the fortnight's work from a clean start, show milestone M3, 'The map zooms, keeps calm and tells places apart', and plan the six days before the release. The review also lists the criteria the worked answer leaves without a full test, and decides by the plan's rule whether to cut.

EXHIBIT 30.1 Day 29 ends with M3 shown, the gaps written down and the tag `m3` pushed

Goal	A clean run; M3 shown; the untested criteria listed; iteration 4 planned
Guide	`docs/guides/delivery-plan.md`: 'The weekly and daily rhythm', 'Tracking progress with a burn-up', 'When time runs short'
In the worked answer	`scripts/test.ts`; the column 'Checked by' of `docs/requirements/functional-requirements.md` and `non-functional-requirements.md`
Done when	`npm test` passes from a clean start; M3 shown in Chrome; the burn-up reads 23 of 29; the tag `m3` is pushed

Source: `docs/guides/delivery-plan.md`

30.1 The work passes from nothing, and M3 holds

A clean run proves that the tests pass without help from anything earlier days left on your machine. As on day 10, remove every file Git does not track, then install the scripts' packages and run the lot:

```bash
cd ~/src/tessera-rebuild
git status -sb                          # nothing to commit
git clean -xdf
npm ci
npm test
```

The worked answer's `npm test` runs `scripts/test.ts`: `tsc` and `node --test` for the scripts, the pipeline's four checks through `uv run`, `npm ci` and `npm test` in `web/`, the build, the three .NET test projects through `dotnet run`, then the browser tests through Nginx. Yours runs the same steps for the projects you have made. The milestone rests on its tests: UI-09's browser tests show that the map zooms, UI-15's that it keeps calm and UI-19's that it reads well; UI-12's 'gives each group its own shape' shows, without a browser, that it tells places apart.

30.2 The criteria that still lack a full test

The fortnight's stories hold eleven requirements that the requirement files mark 'No test yet' or prove 'in part'. Your own tests cover six; one more needs an optional `hover()`; three rest on what MapLibre draws, or on the type scale, and stay open; one waits for day 30. Copy Exhibit 30.2 into your notes.

EXHIBIT 30.2 Eleven requirements the worked answer leaves wholly or partly untested

REQUIREMENT	CRITERION	WORKED ANSWER	YOUR OWN TEST
FR-37	UI-09 c6: pins keep their size	No test yet	Days 21 and 24
NFR-50	UI-15 c2: Material's type scale	In part	Open
NFR-51 to NFR-53	UI-15 c3 to c5: elevation, corners, state layers	No test yet	Day 23
NFR-60	UI-19 c3: the bar changes within 0.3 seconds	In part	Day 25
FR-52	UI-19 c5: the red needle and 'N'	In part	Day 26
FR-54	UI-19 c8: the readout over the scale bar or compass	In part	Day 26, a second `hover()`
FR-38, NFR-59	UI-10 c1, the ring; c5, a cluster at rest	In part	Open: MapLibre draws them
NFR-58	UI-10 c2: pins that move out	No test yet	Wait for day 30

Source: `docs/requirements/functional-requirements.md`; `docs/requirements/non-functional-requirements.md`

> WATCH OUT A requirement file can claim more than its tests prove: FR-50, the scale bar's four sections, names two tests that read the three figures alone. Read each named test before you trust the word 'full'.

30.3 The burn-up, the cut, and iteration 4

Mark your own figure for each Friday beside the plan's line in Exhibit 1.5, which reaches 23 points today, at M3, and 29 by the release.

The plan's rule decides today. If your line stands two or more points below 23, cut UI-17 at once, give its two days to UI-10 and UI-11, and record the cut with its reason; a gap of one point uses the day in hand.

Iteration 4 has six working days, as Exhibit 1.3 sets out: day 30 finishes UI-10, day 31 builds UI-11, days 32 and 33 build UI-17, day 34 is the day in hand for S-07, and day 35 holds the review and the release, M4. The burn-up climbs from 23 to 29, since UI-10's three points count only when it is done.

> IN INK Tag M3 with `git tag -a m3 -m 'M3: the map zooms, keeps calm and tells places apart'`, and push the tag.

> THE PROOF `npm test` passes from a clean start, your notes hold the gaps, and `m3` is on GitHub. Iteration 3 is done: 7 points. Monday's first test: `zooms at least to level 16`.

31

Day 30 · Monday 30 November: UI-10, opening a cluster

> IN SHORT A cluster opens on a click and names its places for a screen reader. This finishes UI-10, worth 3 points, and opens iteration 4.

Today a click on a cluster zooms the map far enough to open it, and no further, and a screen reader hears the places inside.

EXHIBIT 31.1 Day 30 ends with three new unit tests and two browser tests passing

Goal	A click that opens a cluster; a label that names what the cluster holds
Story	UI-10 c3 and c4; c5 closed on day 28; c2 has no test in the worked answer
In the worked answer	In `web/src/map/`: `map.ts` (`clusterOpened`), `levels.ts`, `targets.ts` (`openCluster`, `nameClusters`), `labels.ts` (`clusterLabel`), `clusters.test.ts`; `web/e2e/map.spec.ts`
Look for	Why the update, not MapLibre, decides the level
Leave for later	Names beside the pins (UI-11, tomorrow); the order of names (UI-17)
Done when	The blocks 'UI-10 opening a cluster' and 'UI-10 clusters', and 'UI-10 c4: names a cluster's places…', pass

Source: `docs/user-stories/user-stories.md`, *UI-10;* `docs/requirements/`, *FR-38, FR-39, NFR-58 and NFR-63*

31.1 Red: a click that zooms too little, or too far

Criterion 3 sets two limits on a click: at least level 16, where names appear tomorrow, and no further than the cluster needs, or the visitor loses the street they meant to see. Add the block 'UI-10 opening a cluster' to `web/src/map/clusters.test.ts`; its tests run the pure update with fake ports:

```ts
// web/src/map/clusters.test.ts, in describe("UI-10 opening a cluster")
  it("zooms at least to level 16", async () => {
    const p = props();
    await execCmd(updateFn(...args(Msg.clusterOpened(center, level(15.2)), {}, p))[1]);
    expect(p.ports.jumpTo).toHaveBeenCalledWith({ center, zoom: 16 });
  });
```

Its twin, 'zooms no further than the cluster needs to open', sends `level(19)` and expects `zoom: 19`. Run `npx vitest run src/map/clusters.test.ts` in `web/`: both fail to compile, which counts as red.

31.2 Green: the update takes the larger level

One message and one line of update pass both tests. The message carries the cluster's centre and its ***expansion level***, at which it breaks apart; the update takes the larger of that and 16, within UI-09's range:

```ts
// web/src/map/levels.ts
/** UI-10, criterion 3: a click on a cluster zooms at least to level 16. */
export const OPEN_CLUSTER_LEVEL = level(16);

// web/src/map/map.ts (trimmed)
  | { name: "clusterOpened"; center: LngLat; expansionLevel: Level }
// …
  clusterOpened({ center, expansionLevel }, _model, props) {
    return [{}, jump(props,
      { center, zoom: clampZoom(Math.max(OPEN_CLUSTER_LEVEL, expansionLevel)) })];
  },
```

The update stays pure. `openCluster`, in `targets.ts`, asks MapLibre's `getClusterExpansionZoom` and sends the answer through `levelOf`; without `levelOf`, every cluster opens one level short.

31.3 Red, then green: a label that names the places

Criterion 4 has a screen reader say what a cluster holds and how to see it:

```ts
// web/src/map/clusters.test.ts
  it("UI-10 c4: names a cluster's places and says that zooming in shows them", () => {
    expect(clusterLabel(3, ["A", "B", "C"]))
      .toBe("3 places: A, B, C. Zoom in to see them.");
    // … and for 40 places, ten names, then "and 30 more"
  });
```

It fails. Write `clusterLabel` in `labels.ts`: the first ten names (`NAMED_IN_CLUSTER`), then 'and N more' and 'Zoom in to see them.' `nameClusters`, in `targets.ts`, fetches them while the map rests.

31.4 The browser tests

In the block 'UI-10 clusters' of `web/e2e/map.spec.ts`, 'opens a cluster on a click, at least to level 16' clicks a cluster at level 13 over Macau and expects level 16 or more; 'names a cluster's places for screen readers' expects a button labelled `/^\d+ places: .+\. Zoom in to see them\.$/`. Run `npm run build`, then `npm run e2e -- --grep "UI-10"`.

> TRY IT Beyond the worked answer: NFR-58, criterion 2, pins that part from a cluster in 0.3 s, reads 'No test yet'. Write 'UI-10 c2 (beyond the worked answer): parts a cluster within 300 ms' in `map.spec.ts`: read MapLibre's private `_fadeDuration`, as UI-15's reduced-motion test does, and expect at most 300; then expect clusters at level 15 over `BUSIEST` and none at 16. The test is fragile: a private name may change.

> IN INK Run `npm test`, then commit at each green, with the messages `UI-10: open a cluster at least to level 16` and `UI-10: name a cluster's places`, and push.

> THE PROOF The new tests and `npm test` pass. UI-10 is done: 3 points, 26 of 29. Tomorrow's first test: `shows names from level 16, and none inside a cluster`.

32

Day 31 · Tuesday 1 December: UI-11, names from level 16

> IN SHORT One new layer in the style draws names from level 16, skips clusters and sets each name in light text on a dark tooltip. This is UI-11, worth 1 point.

Today every pin gains its name in a dark tooltip, from level 16 up and never inside a cluster. UI-11 builds on UI-10, since a name must stay hidden while its pin sits inside a cluster.

EXHIBIT 32.1 Day 31 ends with two style tests and one browser test passing

Goal	Names beside the pins from level 16; none below; none inside a cluster
Story	UI-11, criteria 1 to 3, in `docs/user-stories/user-stories.md`
In the worked answer	In `web/src/map/`: `levels.ts` (`NAMES_FROM_LEVEL`), `style.ts` (`placeLayers`, `nameLayout`), `images.ts`, `style.test.ts`; `web/src/design/canvas.ts`; `web/e2e/map.spec.ts`
Look for	How one filter keeps names out of clusters; why the tooltip is an image that stretches
Leave for later	Names that clash (UI-17, tomorrow and Thursday)
Done when	'shows names from level 16, and none inside a cluster' and 'writes names in light text on a dark tooltip, in the self-hosted Roboto' pass in `style.test.ts`, and 'shows names from level 16, and none below' passes in `map.spec.ts`

Source: `docs/user-stories/user-stories.md`, *UI-11; FR-40 to FR-42*

32.1 Red: the style has no names

The first test asks three things of the names' layer in the style that `buildStyle` returns. The style is plain data, so the test needs no map and no browser. Add the block 'UI-11 names' to `web/src/map/style.test.ts`:

```ts
// web/src/map/style.test.ts
describe("UI-11 names", () => {
  it("shows names from level 16, and none inside a cluster", () => {
    const names = symbol(LAYER.names);
    expect(names.minzoom).toBe(toMapZoom(NAMES_FROM_LEVEL));
    expect(names.layout?.["text-field"]).toEqual(["get", "name"]);
    expect(names.filter).toEqual(["!", ["has", "point_count"]]);
  });
});
```

Run `cd web && npx vitest run src/map/style.test.ts`. It fails: `symbol`, the helper at the top of the file, throws 'no symbol layer' and names `LAYER.names`.

32.2 Green: one layer, one level, one filter

Three settings on one new layer pass the test. `minzoom` hides the layer below level 16; the filter keeps it to single places, since only a cluster has a `point_count`; `text-field` reads each place's name, which ADR 0005 requires every tile to carry:

```ts
// web/src/map/levels.ts
/** UI-11, criterion 1: names show from level 16. */
export const NAMES_FROM_LEVEL = level(16);

// web/src/map/style.ts (trimmed)
const notCluster: ExpressionSpecification = ["!", ["has", "point_count"]];
// …
    {
      id: LAYER.names,
      type: "symbol",
      source: SOURCE.places,
      filter: notCluster,
      // UI-11, criterion 2: below level 16 no name shows.
      minzoom: toMapZoom(NAMES_FROM_LEVEL),
      layout: nameLayout(),
      paint: { "text-color": theme.inverseOnSurface, ...paintOf(LAYER.names, landed) },
    },
```

FROM SCHOOL The filter is a Boolean expression written as a list: `["!", ["has", "point_count"]]` means NOT (has point_count). MapLibre works it out for each feature, as a selection statement would, and draws a name only where it comes out true.

WATCH OUT Write `toMapZoom(NAMES_FROM_LEVEL)`, never `16`. MapLibre's `minzoom` speaks MapLibre's zoom, one below the web-map level, so a bare 16 would hide names until level 17.

32.3 The tooltip: light text on a dark box

Criterion 1 asks for Material's plain dark tooltip, and the second test fixes its look. The name's colour is the theme's `inverse-on-surface`; the box is an image, `tooltip`, that MapLibre stretches to fit each name through `icon-text-fit`; the letters come from the Roboto the page serves itself, so the map asks no outside server for glyphs. The test 'writes names in light text on a dark tooltip, in the self-hosted Roboto' checks all three, and `readImageName("tooltip")` too.

Make it pass with the first lines of `nameLayout` in `style.ts`: `text-field`, `text-font`, a 12-pixel `text-size`, the `tooltip` image and its padding of 4 by 8 pixels. Then write `drawTooltip` in `design/canvas.ts`: a dark box with the 4-pixel corners UI-15's criterion 4 sets for name tooltips. Last, add the names' line to day 23's test 'keeps the map's own pins and names unseen until the dropping pins land'.

32.4 The browser test

The style cannot say whether MapLibre draws names at 16 and none at 15. The browser test asks MapLibre what it drew in the names' layer over the sample's busiest street, `BUSIEST` in `web/e2e/map-page.ts`:

```ts
// web/e2e/map.spec.ts
test.describe("UI-11 names", () => {
  test("shows names from level 16, and none below", async ({ page }) => {
    await openMap(page);
    await view(page, 15, BUSIEST);
    expect(await drawn(page, ["names"])).toEqual([]);
    await view(page, 16, BUSIEST);
    expect((await drawn(page, ["names"])).length).toBeGreaterThan(0);
  });
});
```

Run `npm run build`, then `npm run e2e -- --grep "UI-11"`.

IN INK Run `npm test`, then commit at each green, with the messages `UI-11: show names from level 16, and none inside a cluster` and `UI-11: draw each name on Material's dark tooltip`, and push.

THE PROOF The two style tests, the browser test and `npm test` pass. UI-11 is done: 1 point, 27 of 29. Tomorrow's first test: `gives the names four anchors, and lets a name give way while its pin stays`.

33

Day 32 · Wednesday 2 December: UI-17, names that give way

IN SHORT Names gain four anchors, the layers stack so icons claim their space before names, and places stay apart from level 16. This is the first of UI-17's two days, worth 2 points.

Today each name learns to try four sides of its pin and to give way when none has room, while every pin stays on the map. MapLibre does the placing; your style sets the rules.

EXHIBIT 33.1 Day 32 ends with two new style tests passing, and day 27's cluster test still green

Goal	Names that never cover a name or an icon; pins that always show
Story	UI-17, criteria 1, 3, 4 and 5, in `docs/user-stories/user-stories.md`; criterion 2 and the images come tomorrow
In the worked answer	`web/src/map/style.ts` (`nameAnchors`, `nameLayout`, `placeLayers` and the file's opening comment); `web/src/map/levels.ts` (`LAST_CLUSTER_LEVEL`); `web/src/map/style.test.ts`; `web/src/map/clusters.test.ts`
Look for	Why names and pins sit in separate layers; why the names' layer comes first in the list
Leave for later	Which name wins a clash (criterion 2), and the screenshots (tomorrow)
Done when	'gives the names four anchors…' and 'places every icon before any name…' pass, and day 27's 'keeps every place apart from level 16 (UI-17, criterion 5)' still passes

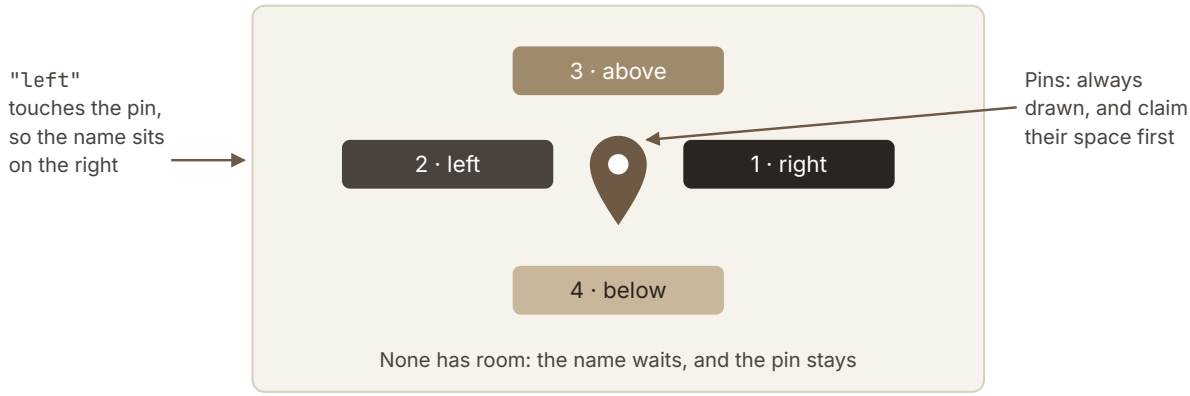
Source: `docs/user-stories/user-stories.md`, *UI-17;* `docs/requirements/functional-requirements.md`, *FR-43 and FR-45 to FR-47*

33.1 How MapLibre places a name

MapLibre places symbols one at a time. Each claims a ***collision box*** of screen, and MapLibre leaves out a later symbol whose box would overlap a claimed one. Your style decides which symbols may overlap anyway, which give way, and the order in which they claim.

A name may move before it gives way. Each ***anchor*** is a side to try, in the order the style lists them, as Exhibit 33.2 shows. Read the names backwards: MapLibre names the side of the text that touches the pin.

EXHIBIT 33.2 A name tries four sides of its pin, in order, and gives way if none has room



Source: `web/src/map/style.ts`, `nameAnchors`

33.2 Red: four anchors, and a pin that never hides

The first test fixes criteria 3 and 4. Add the block 'UI-17 names that never overlap' to `web/src/map/style.test.ts`:

```ts
// web/src/map/style.test.ts
  it("gives the names four anchors, and lets a name give way while its pin stays", () => {
    const anchors = symbol(LAYER.names).layout?.["text-variable-anchor-offset"];
    // MapLibre names the side of the text that touches the place:
    // 'left' puts the name on the right.
    expect(Array.isArray(anchors) && anchors.filter((a) => typeof a === "string"))
      .toEqual(["left", "right", "bottom", "top"]);
    const pins = symbol(LAYER.pins);
    expect(pins.layout).toMatchObject({ "icon-allow-overlap": true });
    expect(pins.layout?.["text-field"]).toBeUndefined();
    expect(symbol(LAYER.names).layout?.["text-allow-overlap"]).toBeUndefined();
    expect(symbol(LAYER.names).layout)
      .toMatchObject({ "icon-ignore-placement": true, "text-padding": 9 });
  });
```

It fails on the first `expect`: yesterday's names have no anchors.

> WATCH OUT The story's list of tests calls this one 'gives every place layer four anchors and optional text', a name left from an older design in which each layer of places carried its own names. `style.test.ts` uses the name above and checks one layer of names and one of pins. Use the file's name; the story's idea stands.

33.3 Green: the anchors and the padding

`nameAnchors` returns the four sides, each with an offset in ems that sets the name clear of the pin's image. Add it and the rest of `nameLayout`:

```ts
// web/src/map/style.ts (trimmed)
function nameAnchors(): VariableAnchorOffsetCollectionSpecification {
  // … em(), clear, halfWidth, top and bottom: the pin's image in pixels, turned to ems …
  return [
    "left", [em(halfWidth + clear), -em(PIN_HEAD_ABOVE_TIP)],
    "right", [-em(halfWidth + clear), -em(PIN_HEAD_ABOVE_TIP)],
    "bottom", [0, -em(top + clear)],
    "top", [0, em(bottom + clear)],
  ];
}
// … in nameLayout():
    "text-variable-anchor-offset": nameAnchors(),
    "icon-allow-overlap": true,
    "icon-ignore-placement": true,
    "text-padding": NAME_PADDING,
```

`NAME_PADDING`, 9, is the tooltip's widest padding plus one, so two tooltips never touch; the box claims nothing, and the padded text claims for it. The pins' layer gets `icon-allow-overlap` and no `text-field`: a pin always draws, and only a name gives way.

33.4 Red, then green: icons claim first

Criterion 1 forbids a name over an icon, and the order of the layers enforces it: MapLibre places symbols from the top layer down, so the layer listed last claims first. The second test, 'places every icon before any name, so no name covers an icon', expects the last three layer ids to be `[LAYER.names, LAYER.pins, LAYER.clusters]`.

If your `placeLayers` already lists names, then pins, then clusters, the test passes at once. A test you have never seen fail proves nothing, so swap the names and the pins, watch it fail, and swap them back. The opening comment of the worked answer's `style.ts` says why names and pins sit apart: in one layer, a later pin, which always draws, could cover an earlier name.

33.5 Apart from level 16: day 27's test

Criterion 5 joins UI-17 to UI-10: below level 16 crowded places stay in clusters, and from 16 every place shows its own pin. Day 27 wrote the test, 'keeps every place apart from level 16 (UI-17, criterion 5)'; today it counts for UI-17. It holds while `LAST_CLUSTER_LEVEL` in `levels.ts` is `level(15)`: set it to 16 once, see red, and restore it.

> IN INK Run `npm test`, then commit at each green, with the messages `UI-17: give each name four sides to try, and keep every pin on show` and `UI-17: let icons claim their space before names`, and push.

> THE PROOF The two new tests pass, with day 27's, and `npm test` passes. UI-17 is half built and adds nothing to the burn-up yet. Tomorrow's first test: `orders names between layers by the map's order, then by id, whatever order the tiles bring`.

34

Day 33 · Thursday 3 December: UI-17, a fixed order and three approved images

> IN SHORT Every place gets a sort key, the tile query keeps its order, and three images at levels 16, 18 and 20 are approved. This finishes UI-17, worth 2 points, and with it every story of Parts A and B.

Today the same name wins every clash, every time, and three approved screenshots show no name overlapping another. The tile query orders each table by its key, and the map ranks places by layer, then by id, so a reload shows the same street.

EXHIBIT 34.1 Day 33 ends with UI-17's last tests passing in TypeScript, in C# and in the browser

Goal	A fixed order of names; three approved images; UI-17 done
Story	UI-17, criterion 2 and tests 2 and 3, in `docs/user-stories/user-stories.md`
In the worked answer	`web/src/map/style.ts` (`sortKey`); `web/src/map/places.ts` (`byId`); `src/Tessera/Modules/Tessera/Infrastructure/Sql/Queries.cs` (`Tile`); `tests/Tessera.UnitTests/TileTests.cs` and `approved/tile-query.sql`; `web/e2e/map.spec.ts`; `web/playwright.config.ts`; `web/e2e/__screenshots__/`
Look for	Why `byId` compares code by code and not by language; why the screenshot tolerance is 1 pixel in 100
Leave for later	Criterion 6, the names at a tile's edge, which has no test in the worked answer
Done when	'orders names between layers by the map's order…', with its new line for the style, `Matches_the_approved_tile_query` and the three 'matches the approved image at level …' tests pass

Source: `docs/user-stories/user-stories.md`, *UI-17;* `docs/requirements/functional-requirements.md`, *FR-44 and FR-48*

34.1 Red: the style ignores the rank

Day 28's test 'orders names between layers by the map's order, then by id, whatever order the tiles bring' already gives every place a ***sort key***: layers a million apart, then each place's rank by id. MapLibre still places names in the order it meets them. Add the test's last line, which asks the style to read the key:

```ts
// web/src/map/style.test.ts, the end of 'orders names between layers …'
    expect(symbol(LAYER.names).layout)
      .toMatchObject({ "symbol-sort-key": ["get", "sort"], "symbol-z-order": "source" });
```

It fails: the layer of names has no `symbol-sort-key`.

34.2 Green: the style reads the rank

Add `"symbol-sort-key": sortKey` and `"symbol-z-order": "source"` to the names and the pins: the lower key places first and wins. `byId` compares ids code by code, never with `localeCompare`, which ranks text by the browser's language and could give two visitors two winners. Run the block with `npx vitest run src/map/style.test.ts -t "UI-17"` in `web/`.

34.3 The service's half: the tile query's order

The story's second test is day 12's `Matches_the_approved_tile_query`, in `tests/Tessera.UnitTests/TileTests.cs`. It compares `Queries.Tile`'s text with `approved/tile-query.sql`, whose sixth line reads `ORDER BY t."id"`: the test's layer of cafés has the key `id`, and the query orders by it. A view has no key, and `Tile` then writes no `ORDER BY` at all. Run it alone:

```bash
dotnet test --project tests/Tessera.UnitTests \
  --filter-method "*Matches_the_approved_tile_query"
```

The order arrived with US-24 and S-01, so the test should pass already. If your query lacks it, add `ORDER BY` to the approved file first, see red, then write the order in `Tile`.

No test in the worked answer checks the view's half. Add one of your own to `TileTests.cs`, and watch it fail once by removing the `IsDefaultOrEmpty` check:

```csharp
// tests/Tessera.UnitTests/TileTests.cs: beyond the worked answer
// UI-17 test 2: a view has no key, so its query sets no order.
[Fact]
public void Leaves_a_view_without_an_order()
{
    var view = CatalogueReader.ToLayer(
        new CatalogueRow("food", "slow_cafes", "geom", 4326, "POINT", """["name"]"""));
    Assert.DoesNotContain("ORDER BY",
        Queries.Tile(view, TileAddress.Of(15, 27154, 14280)!, Limits).Text,
        StringComparison.Ordinal);
}
```

The row leaves out its last field, the key columns, which default to `"[]"`: the empty list a view's row carries.

34.4 The approved images

Playwright shows MapLibre's collision boxes, takes a screenshot at levels 16, 18 and 20 over the busiest street, and compares each with an ***approved image*** kept in the repository. A name that moves or overlaps changes far more pixels than the 1 in 100 that `playwright.config.ts` allows for drawing in software. Copy that file's fixed window, 1,280 by 800, and its SwiftShader settings, or the pixels differ from machine to machine:

```ts
// web/e2e/map.spec.ts (trimmed): one test for each of levels 16, 18 and 20
      await openMap(page);
      // … set window.__tesseraMap.showCollisionBoxes = true …
      await view(page, level, BUSIEST);
      await expect(page).toHaveScreenshot(`UI-17-level-${level}.png`);
```

The first run has no images, so after `npm run build` ask Playwright to write them: `npm run e2e -- --grep "approved image" --update-snapshots`. Before you commit each file, open it and look for any name over a name or a pin: an approved image proves only what you checked.

> WATCH OUT The automatic build on GitHub runs `npm run e2e -- --grep-invert "approved image"` and never compares these images, since its Chrome draws other pixels from the Chromium that wrote them. Only `npm test` on your own machine proves criterion 1. The delivery plan places the images in `web/e2e/screenshots`; the folder is `web/e2e/__screenshots__`, as `playwright.config.ts` sets.

> TRY IT Beyond the worked answer: FR-48, a name near a tile's edge giving way as one in the middle does, reads 'No test yet'. Names draw from one clustered source, so no name meets an edge; but a place near an edge arrives in two tiles, and `placeCollection` reads both in its layer's one load. Write 'UI-17 c6 (beyond the worked answer): keeps a place from two tiles once' in `style.test.ts`: feed `placeCollection` one load of `food.cafes` holding `feature("cafes-001")` twice, and expect one feature with `sort` 0. MapLibre's own handling of the edge stays unproved.

> IN INK Run `npm test`, then commit at each green, with `web/e2e/__screenshots__` and the messages `UI-17: let the style read each place's rank`, `UI-17: prove a view's tile query sets no order` and `UI-17: approve the names at levels 16, 18 and 20 over the busiest street`, and push.

> THE PROOF `npm test` passes, and UI-17 is done: 29 of 29 points. Tomorrow's first test: 'names every missing variable at once'.

35

Day 34 · Friday 4 December: S-07, office machines alone

IN SHORT One setting, `TESSERA_OFFICE_NETWORK`, names the office; `npm start` checks it, and Nginx refuses any machine outside it. This is S-07, worth 1 point, built on the day in hand.

The day in hand is free, so today your copy gains `npm start`, and Nginx learns to answer office machines alone and to refuse every other with 403. Had a story slipped, the day would finish it instead, and S-07, first on the cut list, would move to the second plan.

EXHIBIT 35.1 Day 34 ends with the settings checked by pure tests, and the door checked through Nginx

Goal	`npm start` in your copy; 403 for a stranger, before any other rule; the tile for an office machine and for this machine; no start with a network that lets in the world
Story	S-07, criteria 1 to 4, in `docs/user-stories/user-stories.md`
In the worked answer	`scripts/start.ts`; `scripts/lib/plan.ts` (`readStartSettings`, `isOfficeNetwork`, `databaseUrl`, `runArguments`, `nginxArguments`); `scripts/lib/plan.test.ts`; `scripts/lib/database.ts`; `nginx/default.conf.template` (`geo $office`, `@forbidden`); `scripts/lib/edge.ts`; `scripts/e2e.ts`; `.env.example`
Look for	Why the refusal comes before every other rule; how one machine plays both a stranger and an office machine
Leave for later	The cap on each visitor (S-30), kept tiles (N-19), `/health` (N-25) and the second copy (N-29): the second plan
Done when	'names every missing variable at once' and 'accepts an office network, and refuses one too wide or misspelt' pass under `node --test`, and the S-07 checks in `scripts/lib/edge.ts` pass in `npm run e2e`

Source: `docs/user-stories/user-stories.md`, S-07; `docs/requirements/security-requirements.md`, SR-16 to SR-19

FROM SCHOOL An IPv4 address is 32 bits, four bytes. A network is an address, a slash and a ***prefix***, the count of leading bits its addresses share: `192.168.10.0/24` leaves 256 addresses, `/8` about sixteen million, and `/0` every address there is.

35.1 First, one command to start it all

Criterion 4 says `npm start` refuses a bad network, but your copy has no `npm start`: since day 15 you have started the database, the program and Nginx by hand. The morning goes to a task outside the story, which the release needs anyway, and it starts with tests. In `scripts/lib/plan.test.ts`, type three of the worked answer's:

- ‘names every missing variable at once’, but expect four problems for an empty environment, not five: `TESSERA_DB_PASSWORD`, `TESSERA_READER_PASSWORD`, `TESSERA_PUBLISHER_PASSWORD` and `TESSERA_HTTPPORT`. The fifth, the office’s network, comes this afternoon;
- ‘percent-encodes a password that holds an @’, since the service and the seed each read the database as one URL, and an `@` in a password would end the user’s part early;
- ‘places the database on this machine unless told otherwise’.

Run `node --test scripts/lib/plan.test.ts` and watch all three fail. Then write `scripts/lib/plan.ts`: `readStartSettings`, which returns the map’s own `Result` from `web/src/shared/result.ts`; `readDatabasePlace`; `databaseUrl`; `runArguments`, which takes your image’s name, `tessera-rebuild-db`, beside the container’s; and `nginxArguments`. Yours takes one service port, not two, and your `StartSettings` has no service port at all, since your copy listens on 8080 until N-21.

Last, write `scripts/start.ts`, which runs, in order, the steps you have run by hand:

1. `node scripts/build.ts`, which builds the map into `web/dist` and then the service in Release, with the map copied beside the program as `wwwroot/`;
2. `rebuild-db`, left alone if it runs, started if it stopped, made with `runArguments` the first time; then `waitForDatabase`, from the worked answer’s `scripts/lib/database.ts`, which day 8 left for today: type it now, and let `publish.ts` call it too;
3. the seed, as the publisher;
4. the service, as the reader, on port 8080;
5. Nginx in front, as on day 15.

Steps 3 and 4 run the one program your build writes, `Tessera.dll`, in its two ways. The seed reads one variable, `TESSERA_PUBLISHER_DATABASE_URL`; the service reads `DATABASE_URL`, and `Urls`, the address ASP.NET Core listens on:

```ts
// scripts/start.ts, the book's own (trimmed): steps 3 and 4
const program = resolve('src/Tessera/bin/Release/net10.0/Tessera.dll');
const servicePort = '8080';
// …
// 3. The seed, as the publisher: DbUp runs each new script once.
const publisher =
  databaseUrl('tessera_publisher', settings.publisherPassword, settings.database);
const asPublisher = $({ env: { ...process.env, TESSERA_PUBLISHER_DATABASE_URL: publisher } });
await asPublisher`dotnet ${program} seed`;

// 4. The service, as the reader, in the background.
const reader = databaseUrl('tessera_reader', settings.readerPassword, settings.database);
const service = $({
  nothrow: true,
  stdio: 'inherit',
  env: { ...process.env, DATABASE_URL: reader, Urls: `http://127.0.0.1:${servicePort}` },
})`dotnet ${program}`;
```

The seed finishes before the service starts; the service runs in the background while step 5 starts Nginx, and the script waits on it last, so Ctrl+C stops both. Copy `.env.example` to `.env`, fill in day 3’s passwords, set `TESSERA_DB_PORT=5434`, and add the worked answer’s `"start"` line to `package.json`: `node --env-file-if-exists=.env scripts/start.ts`. This `start.ts` is the book’s own: it runs one copy under your container’s name, restarts nothing that stops, and waits for no `/health`, until N-25 and N-29 complete it (Appendix A).

> WATCH OUT The worked answer's `tessera.json` sets `Urls` to port 7800; your copy gains that file with N-21. Until then the variable alone sets the port, and a service started without it listens on ASP.NET Core's own default, port 5000, where Nginx never looks.

35.2 Red: a network too wide or misspelt

Criterion 4 is a pure decision, so its test needs no Docker and no Nginx. In `scripts/lib/plan.test.ts`, type `office`, the worked answer's set of every required variable, then the story's first test, with four good networks and six bad:

```ts
// scripts/lib/plan.test.ts
test('accepts an office network, and refuses one too wide or misspelt', () => {
  for (const good of ['192.168.10.0/24', '10.0.0.0/8', '127.0.0.4/30', '192.168.10.7/32'])
    assert.ok(isOfficeNetwork(good), good);
  for (const bad of ['0.0.0.0/0', '10.0.0.0/7', '192.168.10.0', '192.168.10.0/33',
                     '192.168.300.0/24', 'office'])
    assert.ok(!isOfficeNetwork(bad), bad);
  const read = readStartSettings({ ...office, TESSERA_OFFICE_NETWORK: '0.0.0.0/0' });
  assert.match(read.ok ? '' : read.error.join(), /TESSERA_OFFICE_NETWORK/);
});
```

Run `node --test scripts/lib/plan.test.ts`. It fails: `isOfficeNetwork` does not exist.

35.3 Green: the check, and the refusal to start

A regular expression reads the five numbers, and two comparisons judge them: each byte must be at most 255, and the prefix from 8 to 32:

```ts
// scripts/lib/plan.ts
export function isOfficeNetwork(text: string): boolean {
  const parts = /^(\d{1,3})\.(\d{1,3})\.(\d{1,3})\.(\d{1,3})\/(\d{1,2})$/.exec(text);
  if (parts === null) return false;
  const [, ...numbers] = parts.map(Number);
  const prefix = numbers[4] ?? -1;
  return numbers.slice(0, 4).every(octet => octet <= 255) && prefix >= 8 && prefix <= 32;
}
```

Then add `'TESSERA_OFFICE_NETWORK'` to the names `readStartSettings` requires, with a problem that names it when the check fails, and change this morning's four problems to five. `nginxArguments` passes the network to Nginx with `--env`. Both tests pass, and `npm start` with `TESSERA_OFFICE_NETWORK=0.0.0.0/0` prints one line and stops:

```text
TESSERA_OFFICE_NETWORK: '0.0.0.0/0' is not a network such as 192.168.10.0/24, with a prefix
from 8 to 32
```

35.4 Nginx: refuse first

Two parts of `nginx/default.conf.template` do the work. A `geo` block sets `$office` to 1 for this machine or an address inside the network, and 0 for any other; the server block's first rule refuses a 0 before any other runs:

NGINX

```nginx
# nginx/default.conf.template (trimmed)
geo $office {
    default 0;
    127.0.0.1 1;
    ${TESSERA_OFFICE_NETWORK} 1;
}
server {
    if ($office = 0) {
        return 403;
    }
    error_page 403 = @forbidden;
    # …
    location @forbidden {
        default_type application/problem+json;
        add_header Cache-Control no-store always;
        return 403 '{"type":"about:blank","title":"Forbidden","status":403}';
    }
}
```

`127.0.0.1 1` meets criterion 3: this machine is always inside. `@forbidden` turns the bare 403 into a problem document, as US-07 requires, and `no-store` stops anyone keeping it. The refusal comes first for the reason S-07's 'Critical to success' gives: once Nginx keeps tiles (N-19), a check after the cache would hand a stranger a kept tile. From today the template needs `TESSERA_OFFICE_NETWORK`, so day 15's hand-typed `docker run` line no longer starts it; `npm start` passes it for you.

35.5 Red, then green: a stranger and a neighbour on one machine

The story's second test runs through Nginx. Linux answers every `127.x.x.x` address on the machine itself, so `scripts/e2e.ts` sets the office to `127.0.0.4/30`, the addresses 127.0.0.4 to 127.0.0.7. `127.0.0.2` is then a stranger, and `127.0.0.5` an office machine:

TS

```ts
// scripts/lib/edge.ts, in checkEdge
  const stranger = await ask(tile, {}, 'GET', '127.0.0.2');
  assert.equal(stranger.status, 403);
  assert.equal(stranger.headers['content-type'], 'application/problem+json');
  assert.equal(stranger.headers['cache-control'], 'no-store');
  assert.equal((await ask(tile, {}, 'GET', '127.0.0.5')).status, 200);
```

`ask` gains a parameter, `from`, which `node:http` takes as `localAddress`. In your `e2e.ts`, start Nginx with `nginxArguments` and the office `127.0.0.4/30`, as the worked answer's does. Run the checks before the `geo` block exists: the stranger gets 200, and the first `assert` fails. Add the block, then run `npm run build && npm run e2e` again. Playwright asks from 127.0.0.1, so the run proves criterion 3 too.

> IN INK Run `npm test`, then commit at each green, with the messages `Release: start the database, the seed, the service and Nginx with npm start`, `S-07: refuse to start with an office network too wide or misspelt` and `S-07: answer office machines alone, and strangers with 403`, and push.

> THE PROOF The S-07 tests and `npm test` pass, and `npm start` brings up your copy behind Nginx. S-07 is done: 30 points, and the second plan starts a point lighter. Monday's first step: clone a clean copy, and run `npm test`.

36

Day 35 · Monday 7 December: the review and the release, M4

> IN SHORT No story today: you review iteration 4, release Parts A and B and register the start-up task, reaching milestone M4, 'Parts A and B released to the office'.

Today you prove the build from a clean copy, release it to the office machine, set it to start with Windows and write your first weekly check. M4 holds when another office machine opens the map.

EXHIBIT 36.1 Day 35 ends with the map open on another office machine, and the tag m4 pushed

Goal	A release the office uses, which comes back by itself after a reboot
Story	None: the review and milestone M4, in `docs/guides/delivery-plan.md`; the release never slips past today, since 8 December is a holiday
In the worked answer	`scripts/test.ts`; `scripts/build.ts`; `scripts/start.ts`; `src/Tessera/Tessera.csproj` (the map as `wwwroot`); `scripts/boot-task.ts`, `scripts/boot.ts`, `scripts/lib/boot.ts` and `scripts/lib/boot.test.ts`; `README.md`, its table of commands
Look for	What a release is here: no file to send, but a commit the office machine builds; what a weekly check must ask while Tessera has no runbook
Leave for later	`/health` (N-25), two copies (N-29), the cache (N-19) and the nightly backup: the second plan, and after
Done when	`npm test` passes in a clean clone; the map opens from another office machine; the start-up task brings Tessera back after a restart; the tag `m4` is pushed

Source: `docs/guides/delivery-plan.md`, *iteration 4 and the burn-up;* `docs/study/05-deployment-and-operations.md`

36.1 The review: every test from a clean start

A clean clone proves the build lives in the repository, not in your machine's leftovers:

```bash
git clone https://github.com/YOUR-NAME/tessera-rebuild.git ~/src/m4-check
cd ~/src/m4-check
npm install
npm test
```

`npm test` runs the steps of day 29's clean run, so a file missing from the repository fails here, not on the office machine. Then read the iteration against the burn-up, Exhibit 1.5: the plan's line reached 29 points on day 33, and S-07, on the day in hand, took it to 30. Set your own column beside it. Record each difference from the worked answer you mean to keep, with the tests you wrote beyond it: UI-10's criterion 2, UI-17's criterion 6 and the view's tile query.

36.2 The start-up task, before the tag

A ***start-up task*** makes Windows run `npm start` when the office machine switches on, so the map returns after a reboot with no one signed in. The stories place it with N-29, in the second plan, but the office machine will restart before 18 January. So, before you tag the release, bring it forward as a release task outside the 30 stories, and note it as a planned difference.

Begin, as every story does, with a test. Type the worked answer's `scripts/lib/boot.test.ts` and watch it fail. Then type `scripts/lib/boot.ts`, whose pure functions build the PowerShell; `scripts/boot.ts`, which runs `npm start` into `start.log`; `scripts/boot-task.ts`, which prints the steps; and the entry `"boot-task"` in `package.json`.

```bash
node --test scripts/lib/boot.test.ts
# 'registers a task that starts with Windows, runs for ever and tries again', and four more
```

`registerSteps` prints the PowerShell for a task that runs at start-up, a minute after Windows, so WSL2 and the network are up before Ubuntu and Docker start. The task finds its programs by the folders your shell uses, which `linuxPath` keeps, so `dotnet` must be on the shell's `PATH` when you run `npm run boot-task`. Commit, run `npm test` once more, and push.

36.3 The release: a tag the office machine builds

Here a ***release*** is a tagged commit that the office machine clones and builds, not a file you send. `npm start` first runs `scripts/build.ts`: the map into `web/dist`, then `dotnet build --configuration Release`, which copies the map beside the program as `wwwroot/`, so one build ships both. The office machine therefore needs the .NET 10 SDK inside Ubuntu, beside Node.js and Docker.

Push the tag `m4` (see Ink), then set the release up on the office machine:

```bash
git clone --branch m4 https://github.com/YOUR-NAME/tessera-rebuild.git ~/src/tessera-rebuild
cd ~/src/tessera-rebuild
npm install
cp .env.example .env
nano .env              # passwords, TESSERA_HTTPPORT=80, TESSERA_OFFICE_NETWORK
npm run office-port    # US-35: checks mirrored networking, and prints the firewall steps
npm start
```

Open `http://<office machine>/` from another machine in the office, and zoom in until the clusters open and the names appear. That view is milestone M4.

> WATCH OUT The worked answer's `npm start` runs two copies, starts either again when it stops, and waits for `/health`; yours runs one, with no cache and no limit for each visitor, until the second plan. The plan accepts this for a pilot in one office.

Then register the start-up task: stop `npm start`, run `npm run boot-task`, and paste what it prints into PowerShell run as administrator. Prove it: restart Windows, wait two minutes, and open the map from another office machine. If it fails, `Get-ScheduledTaskInfo -TaskName 'Tessera'` and `tail -n 100 start.log` show why.

36.4 The first weekly check

A short weekly check finds most trouble before the office does. Tessera has no runbook to give you one, as `docs/study/05-deployment-and-operations.md` says, so write your own, four lines long, and run it today, so next Friday's check has a record to compare with. Exhibit 36.2 suggests one.

EXHIBIT 36.2 A weekly check of the book's own, as the release of 7 December can answer it

LINE OF THE CHECK	TODAY, ON YOUR RELEASE
Ask the service through Nginx	`/health` arrives with N-25. Until then, ask for a tile: `curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1/food.cafes/0/0/0.pbf` prints `200`
Look at the map from another office machine	Yes: open `http://<office machine>/` and zoom in until the places show
Look at the backups	None exist. The worked answer has `npm run backup` and `npm run backup-task`, listed in its `README.md`, but no story builds them. List the backup as a risk in the second plan
Look at the disk: `df -h /` and `docker system df`	Yes: note the free space, and act before it falls below a fifth of the disk

Source: the book's own; `README.md`, the table of commands

Note the results; the next check falls on Friday 11 December, day 38. Last, sketch the second plan for Parts C to F: 22 points in thirteen stories, from 9 December to 18 January 2027, in build order, with the same rule for cuts. Day 36 writes it out.

> IN INK Commit the start-up task with the message `Release: bring the start-up task forward from N-29`; run `npm test`, push, then tag the release with `git tag -a m4 -m 'M4: Parts A and B released to the office'` and push the tag.

> THE PROOF A clean clone passes `npm test`; another office machine opens the map; Tessera returns after a restart; `m4` marks the release. M4 holds: Parts A and B, 29 points in 16 stories, and S-07, released before the holiday.

37

Day 36 · Wednesday 9 December: the second plan, and N-18, a tile the browser keeps

> IN SHORT This morning you plan the thirteen stories the release left behind; this afternoon every tile learns to tell the browser how long to keep it. N-18 is worth 1 point.

Plan first, then build the smallest story, so the plan starts on the day it is written. The delivery plan leaves no day for planning, so the plan takes the morning and N-18, one header from one setting, the afternoon.

EXHIBIT 37.1 Day 36 ends with a plan on GitHub and the header on every tile

Goal	A second plan for Parts C to F, 13 stories and 22 points; a tile carries `Cache-Control: public, max-age=60`, an error `no-store`
Story	Steps 17 to 30 of `docs/user-stories/user-stories.md`, less S-07, built on 4 December; today N-18 c1 to c5
In the worked answer	`docs/guides/delivery-plan.md`, 'After 8 December'; `Features/Tiles.cs`, `TileServer.CacheControl` and `TileResult`; `Domain/Settings.cs`, `TileSettings`; `Common/Api/Problems.cs`, `Shape`; `web/e2e/map.spec.ts`
Look for	Each story's points and 'Builds on'; why 0 gives `no-cache` and not `max-age=0`
Leave for later	The tag and Nginx's store (N-19); `tessera.json` itself (N-21)
Done when	`docs/second-plan.md` is committed; `Tells_the_browser_and_nginx_how_long_to_keep_a_tile` passes, with two new assertions in `ServiceTests` and the browser test 'asks once for a tile when the visitor pans away and back within the minute'

Source: `docs/guides/delivery-plan.md`; `docs/user-stories/user-stories.md`, *N-18*

37.1 Count the days: exactly enough, with none to spare

The plan starts with a sum, and the sum leaves no slack. Twenty-nine weekdays run from Wednesday 9 December to Monday 18 January; four holidays (22, 24 and 25 December, 1 January) and three reviews leave twenty-two days for twenty-two points. The worked answer's delivery plan sketches this plan and leaves the sum to you.

Chapter 1's Exhibit 1.4 gives every day of the plan, and Exhibit 37.2 the iterations to copy into `docs/second-plan.md`.

EXHIBIT 37.2 Three iterations, each closed by a review

ITERATION	DAYS	STORIES	STORY DAYS	GOAL ITS REVIEW CHECKS
5	9–18 Dec (36–43)	N-18 to N-21; N-22 begun	7	'A tile costs one query a minute, and travels packed', Fri 18 Dec
6	21 Dec–8 Jan (44–54)	N-22 to N-26, N-28, S-30	10	'The service stays steady when the database is not', Fri 8 Jan
7	11–18 Jan (55–60)	N-29, US-36	5	'Nginx guards the door, and the places arrive clean', Mon 18 Jan

Source: `docs/user-stories/user-stories.md`*; the delivery plan names four milestones alone, so these reviews check goals and are tagged* `review-c`, `review-d` *and* `review-ef`

Add four risks to the plan's table: no day in hand, so a story that overruns is cut from the bottom, US-36 first, never by dropping a test; holidays, so write the next test down before each; Docker and Nginx, which most proofs start; and no backup, which day 35's check found and no story builds.

37.2 Turn a number into a header

N-18's rule is a pure function from seconds to text, so it gets the first test and needs no server. HTTP's ***Cache-Control*** header tells a browser what it may keep, and its ***max-age*** says for how many seconds. In `tests/Tessera.UnitTests/TileTests.cs`:

CSHARP

```
// N-18 test 1: the header follows the setting.
[Theory]
[InlineData(60, "public, max-age=60")]
[InlineData(0, "no-cache")]
public void Tells_the_browser_and_nginx_how_long_to_keep_a_tile(int seconds, string header) =>
    Assert.Equal(header, TileServer.CacheControl(seconds));
```

Red: the build fails, since `TileServer` has no `CacheControl`. Green is one expression in `TileServer`, `seconds > 0 ? $"public, max-age={seconds}" : "no-cache"`. `public` lets any store on the way, Nginx included, keep the tile; `no-cache` means 'keep it, but ask before each use', which c2 wants when the office sets the time to 0.

37.3 Wire the header into every answer

The number joins `TileSettings`, day 11's record for the section `Tiles`, as `[Range(0, int.MaxValue)] public int CacheSeconds { get; init; } = 60;`. Your copy has no `tessera.json` until N-21, on day 40, so the default of 60 serves until then. ASP.NET Core's `TypedResults.Bytes` writes no `Cache-Control` of its own, so the tile's answer becomes a small record that sets the header, then hands over the bytes. Day 37 adds the tag:

```csharp
// Features/Tiles.cs, inside TileServer: the book's step towards the worked answer
public sealed record TileResult(ReadOnlyMemory<byte> Bytes, string CacheControl) : IResult
{
    public Task ExecuteAsync(HttpContext httpContext)
    {
        httpContext.Response.Headers.CacheControl = CacheControl;
        return TypedResults.Bytes(Bytes, ContentType).ExecuteAsync(httpContext);
    }
}
```

In `Answer`, the tile's arm becomes `bytes => new TileResult(bytes ?? [], CacheControl(settings.CacheSeconds))`. Errors keep the opposite rule (c3): day 11's `Problems.Shape` already sets `no-store` on every problem document. Prove both through the real service, with one assertion in each of two `ServiceTests`; the first stays red until the wiring is in:

```csharp
// in Serves_a_tile_as_a_mapbox_vector_tile (N-18 c1 and test 2)
Assert.Equal("public, max-age=60", answer.Headers.CacheControl?.ToString());
// in Answers_an_unknown_layer_with_a_problem_document (N-18 c3)
Assert.Equal("no-store", answer.Headers.CacheControl?.ToString());
```

37.4 Prove the map asks once, and the refusals

Only a browser shows whether the header changes what it asks for. Copy into `web/e2e/map.spec.ts` the block `N-18 a browser keeps a tile` and its test 'asks once for a tile when the visitor pans away and back within the minute': it opens the home view, pans to Coloane, far enough south that no home tile stays in sight, pans back, and expects one request for each home tile. Run `npm run build`, then `npm run e2e -- --grep "asks once"`. Criterion 4 has no test (NFR-04): one would wait out a full minute, so check it by hand on day 38. Criterion 2 is proved in part (NFR-02): nothing checks that Nginx then keeps nothing.

Criterion 5 is cheap to prove: the binder cannot turn 'soon' into a whole number, so the start fails and names the key. The test needs settings as JSON, so first swap day 2's helper in `SettingsTests.cs` for the worked answer's three, `Host`, `Settings<T>` and `Problems(json, url)`, which build a host on given JSON with `TesseraModule.AddSettings` and report the start-up check. They lean on `Check` and `Problems` from the worked answer's `Common/Infrastructure/Settings.cs`: copy those two now; day 40 adds the rest. Day 2's theory then calls `Problems("{}", url)`. Write the theory `Refuses_values_that_disagree(json, problem)`, which expects `Problems(json)` to contain the problem, with the worked answer's row `{"Tiles":{"CacheSeconds":"soon"}}` and `CacheSeconds`. It passes at once; change `"soon"` to `60`, watch it fail, and put it back. A time below zero meets the `[Range]`, which day 40 proves among three mistakes at once.

> IN INK Commit `docs/second-plan.md` with the message `Plan Parts C to F: three iterations, 22 points, none to spare`, and each green with `N-18: tell the browser how long to keep a tile, and keep no error`, and push.

> THE PROOF The plan is on GitHub, and today's tests pass. The burn-up reads 31 of 52. Tomorrow's first test: `TileTests.Tags_a_tile_by_its_bytes`.

38

Day 37 · Thursday 10 December: N-19, a tag made from the tile's bytes

> IN SHORT Today each tile gets a label made from its own bytes, which a browser can send back to ask whether its copy still holds. This is the first of two days of N-19, worth 2 points.

A tag computed from the bytes needs no memory: the same bytes give the same tag on any machine and after any restart. Today builds the tag and puts it on every tile; tomorrow uses it for 304, and puts Nginx's store in front.

EXHIBIT 38.1 Day 37 ends with a weak tag on every tile

Goal	Every 200 carries `ETag: W/"…"`, the same for the same bytes
Story	N-19 c1, test 1
In the worked answer	`Features/Tiles.cs`, `TileServer.Tag` and `TileResult`; `TileTests.cs`; `ServiceTests.cs`
Look for	Why the tag is weak; how many of the hash's bytes it keeps
Leave for later	304, Nginx's store, `X-Cache-Status` and the 64 MB limit (day 38)
Done when	`Tags_a_tile_by_its_bytes` passes, in the unit tests and in the integration tests, with `Answers_head_with_the_headers_alone`

Source: `docs/user-stories/user-stories.md`, *N-19*

38.1 Make the tag from a hash

An ***ETag*** is a short label for one version of an answer. A ***hash*** of the bytes, a fixed-size fingerprint like the one US-01 keeps of the places' source, makes a label that changes whenever a byte does. The worked answer uses SHA-256 and keeps the first 16 of its 32 bytes, written as 32 hexadecimal digits.

38.1.1 Red: the same bytes, the same tag

CSHARP

```
// N-19 c1: equal bytes give an equal tag, and the tag is weak.
[Fact]
public void Tags_a_tile_by_its_bytes()
{
    Assert.Equal(TileServer.Tag([1, 2, 3]), TileServer.Tag([1, 2, 3]));
    Assert.NotEqual(TileServer.Tag([1, 2, 3]), TileServer.Tag([1, 2, 4]));
    Assert.StartsWith("W/\"", TileServer.Tag([]).ToString(), StringComparison.Ordinal);
}
```

38.1.2 Green: hash, cut, wrap

```csharp
// Features/Tiles.cs, inside TileServer; a line break moved to fit the page
public static EntityTagHeaderValue Tag(ReadOnlySpan<byte> bytes) =>
    new($"\"{Convert.ToHexStringLower(SHA256.HashData(bytes).AsSpan(0, 16))}\"",
        isWeak: true);
```

`EntityTagHeaderValue`, from `Microsoft.Net.Http.Headers`, is ASP.NET Core's own type for a tag, so the framework can compare it later. The `isWeak: true` writes `W/`, which marks a ***weak tag***: it promises the same content, not the same bytes. N-20 will pack each tile with gzip on its way out, and the packed bytes differ from the plain ones; a weak tag still matches both. The tag of an empty tile, which the map meets over the sea, reads `W/"e3b0c44298fc1c149afbf4c8996fb924"`: the first half of the well-known SHA-256 of no bytes.

38.2 Put the tag on every tile

One argument does it. `TypedResults.Bytes` takes the tag as `entityTag`, writes the `ETag` header, and from tomorrow compares it with the browser's; change the last line of `TileResult.ExecuteAsync`:

```csharp
return TypedResults.Bytes(Bytes, ContentType, entityTag: Tag(Bytes.Span))
    .ExecuteAsync(httpContext);
```

38.2.1 Red, then green: through the running service

Two tests in `ServiceTests.cs` prove the wiring. The first asks for one tile twice:

```csharp
// N-19 c1: a tile carries a weak tag, made from its bytes, which Nginx compares (ADR 0008).
[Fact]
public async Task Tags_a_tile_by_its_bytes()
{
    using var first = await Get($"/food.cafes/{Zoom}/{Tile.X}/{Tile.Y}.pbf");
    using var second = await Get($"/food.cafes/{Zoom}/{Tile.X}/{Tile.Y}.pbf");

    Assert.True(first.Headers.ETag?.IsWeak);
    Assert.Equal(first.Headers.ETag, second.Headers.ETag);
}
```

The second, `Answers_head_with_the_headers_alone`, sends `HEAD`, as a watcher checking a tag may, and expects 200, the same `ETag` as a `GET`, and an empty body. The tile routes answer both methods, through `MapMethods` with `[HttpMethods.Get, HttpMethods.Head]`; if yours use `MapGet`, change them now.

Without `entityTag` the first assertion fails; with it, both pass.

> IN INK Commit at each green, with the message `N-19: tag each tile by its bytes`, and push.

> THE PROOF `Tags_a_tile_by_its_bytes` passes twice, in `TileTests` and in `ServiceTests`, and so does `Answers_head_with_the_headers_alone`. N-19 stays open, so the burn-up still reads 31 of 52. Tomorrow's first test: `ServiceTests.Answers_a_matching_tag_with_304`.

39

Day 38 · Friday 11 December: N-19, 304 and Nginx's store

> IN SHORT Today a browser that sends the tag back gets 304 with no body, and Nginx keeps each tile as long as the service allows, answers the next request from its store, and says so in a header. This finishes N-19, worth 2 points.

Nginx turns a crowd's requests into one query a tile a minute, by obeying the header N-18 wrote. You prove the service's own 304, add four lines to the template, prove them with the edge checks, and write a test for the two rules no check reaches.

EXHIBIT 39.1 Day 38 ends with 304 from the service and a HIT through Nginx

Goal	A matching tag gets 304; a second request through Nginx says `X-Cache-Status: HIT`
Story	N-19 c2 to c5, tests 2 and 3
In the worked answer	`ServiceTests.cs`; `nginx/default.conf.template`; `scripts/lib/edge.ts`, the function `checkEdge`; `scripts/e2e.ts`
Look for	Why the C# holds no `if` for `If-None-Match`; where the store's size is set; why the template sets no time of its own
Leave for later	The lock for a crowd (N-24); stale tiles while the database is down (N-28)
Done when	`Answers_a_matching_tag_with_304` passes, `npm run e2e` reads `X-Cache-Status: HIT`, and your new `scripts/lib/nginx.test.ts` passes

Source: `docs/user-stories/user-stories.md`, *N-19*

39.1 Let ASP.NET Core answer 304

Yesterday's tag already does the work: `TypedResults.Bytes` compares the request's `If-None-Match` with the answer's tag and sends 304 itself, so the code adds no selection. The test proves it in `ServiceTests.cs`:

```csharp
// N-19 c2: a browser that sends the tag back, with its copy still good, gets 304
// and no body. The answer keeps its Cache-Control, so the browser keeps the copy
// for another max-age.
[Fact]
public async Task Answers_a_matching_tag_with_304()
{
    using var first = await Get($"/food.cafes/{Zoom}/{Tile.X}/{Tile.Y}.pbf");
    // … the same request again, with first.Headers.ETag in If-None-Match

    Assert.Equal(HttpStatusCode.NotModified, again.StatusCode);
    Assert.Equal("public, max-age=60", again.Headers.CacheControl?.ToString());
    Assert.Empty(await again.Content.ReadAsByteArrayAsync(
        TestContext.Current.CancellationToken));
}
```

It passes at once; remove `entityTag` for a moment and watch it fail, then put it back. The 304 still costs a query, since the service must build the tile to know its tag; Nginx's store saves even that.

39.2 Write the check first

The proof reads Nginx's own word, not a stopwatch: the story's 'Critical to success' says timing alone proves nothing. In `scripts/lib/edge.ts`, at the top of `checkEdge`, write:

```ts
// N-19: the first request builds the tile; the second comes from Nginx's store.
const first = await ask(tile, { 'Accept-Encoding': 'identity' });
assert.equal(first.status, 200);
assert.equal(first.headers['content-type'], 'application/vnd.mapbox-vector-tile');
// 60 seconds is Tiles.CacheSeconds' default; Nginx keeps the tile that long.
assert.equal(first.headers['cache-control'], 'public, max-age=60');
const second = await ask(tile, { 'Accept-Encoding': 'identity' });
assert.equal(second.headers['x-cache-status'], 'HIT');
assert.deepEqual(second.body, first.body);

// N-19 c2: a browser that sends the tag back gets 304 and no body.
const tag = String(first.headers['etag']);
assert.match(tag, /^W\//);
const unchanged = await ask(tile, { 'If-None-Match': tag });
assert.equal(unchanged.status, 304);
assert.equal(unchanged.body.length, 0);
```

Run `npm run build` and `npm run e2e`: red, since the second answer carries no `X-Cache-Status`.

39.3 Give Nginx a store

A ***cache*** is a store of answers kept to give again. Nginx's needs a folder, a name and a size, in one line outside the `server` block:

```nginx
proxy_cache_path /var/cache/nginx/tiles levels=1:2 keys_zone=tiles:10m max_size=64m
                 inactive=10m use_temp_path=off;
```

`keys_zone=tiles:10m` keeps the tiles' names in 10 MB of memory, room for about 80,000 of them; `max_size=64m` caps the tiles on disk, and when the store is full the tile used longest ago goes first (c4). Then add a location for tiles alone, inside the `server` block:

```nginx
location ~ \.(pbf|mvt)$ {
    proxy_pass http://tessera;
    proxy_cache tiles;
    # …
    add_header X-Cache-Status $upstream_cache_status always;
}
```

Keep the `proxy_pass` you wrote for S-06, and copy day 14's rule against `%2e` into this location too, since a request that matches it no longer passes through `location /`. Rerun `npm run e2e`: the checks pass. The template sets no time of its own: Nginx keeps a tile as long as the service's `max-age` says, so browser and Nginx share one clock, `Tiles.CacheSeconds`, and new places reach every screen within it (c5).

39.4 Test the two rules no check reaches

Criteria 4 and 5 have no test in the worked answer (NFR-09 and NFR-10, 'No test yet'). Both rules live in the template, so a test beyond the worked answer can read them there, in `scripts/lib/nginx.test.ts`:

```ts
// Beyond the worked answer: N-19 c4 and c5, read from the template.
import { test } from 'node:test';
import assert from 'node:assert/strict';
import { readFileSync } from 'node:fs';

const template = readFileSync('nginx/default.conf.template', 'utf8');

test('keeps at most 64 MB of tiles', () => {
  assert.match(template, /proxy_cache_path [^;]*max_size=64m/);
});

test('keeps a tile no longer than the service allows', () => {
  assert.doesNotMatch(template, /proxy_ignore_headers[^;]*Cache-Control/);
});
```

Run `npm run test:scripts`. Nginx obeys `Cache-Control` unless told to ignore it, hence the second test. Break each rule once, with `max_size=128m` and then `proxy_ignore_headers Cache-Control;`, and watch each test fail; they prove the setting, not Nginx's behaviour.

> TRY IT Watch a tile expire by hand (N-18 c4). Add `Tiles__CacheSeconds=5` to `.env`, run `npm start`, and ask for one tile twice with `curl -s -o /dev/null -D - http://127.0.0.1/food.cafes/0/0/0.pbf`: `MISS`, then `HIT`. Six seconds later your copy says `EXPIRED` (the worked answer, with N-28's lines, `STALE`). Then remove the line.

> IN INK Commit at each green, with messages such as `N-19: keep each tile in Nginx for max-age, and say HIT`, and push.

> THE PROOF Today's tests and checks pass. N-19 is done: 33 of 52. It is Friday: run the weekly check of Exhibit 36.2. Monday's first check: a tile asked for with `Accept-Encoding: gzip` arrives packed.

40

Day 39 · Monday 14 December: N-20, tiles and files travel packed

IN SHORT Today Nginx packs tiles, the page and the map's files with gzip for every browser that accepts it, and leaves them plain for any client that does not. This is N-20, worth 1 point.

Packing is Nginx's job, so N-20 is five lines of template and a handful of checks. The worked answer proves each of the four criteria in part, and none in full; you write the checks that finish all four.

EXHIBIT 40.1 Day 39 ends with packed answers that unpack to the plain bytes

Goal	A packed tile and a packed script, each unpacking to the plain answer's bytes
Story	N-20 c1 to c4
In the worked answer	`nginx/default.conf.template`, the five `gzip` lines; `scripts/lib/edge.ts`
Look for	Why `edge.ts` uses `node:http` and not `fetch`; which types `gzip_types` lists; why the C# service never hears of gzip
Leave for later	Brotli, which the story leaves 'open to talk'
Done when	`npm run e2e` passes, its browser tests and its edge checks, your four new checks among them

Source: `docs/user-stories/user-stories.md`, *N-20*

40.1 Check the bytes on the wire

gzip packs bytes without loss: unpacking gives back every byte. A check must see the packed bytes, and `fetch` unpacks gzip by itself, so `edge.ts`'s `ask` uses `node:http`. Add the worked answer's two checks to `checkEdge`:

```ts
// N-20: packed when the browser accepts gzip, and the same bytes once unpacked.
const packed = await ask(tile, { 'Accept-Encoding': 'gzip' });
assert.equal(packed.headers['content-encoding'], 'gzip');
assert.match(String(packed.headers['vary']), /Accept-Encoding/);
assert.deepEqual(gunzipSync(packed.body), first.body);
// N-20 test 2: the map's main script, found in the page, travels at under half its
// plain size, and unpacks to the plain bytes.
```

Run `npm run e2e`: the first new assertion fails, since nothing packs yet.

40.2 Turn gzip on in the server block

Five directives do the whole story:

```nginx
gzip on;
gzip_proxied any;
gzip_comp_level 1;
gzip_vary on;
gzip_types application/vnd.mapbox-vector-tile application/problem+json application/json
           text/css text/javascript application/javascript;
```

`gzip_proxied any` packs answers to requests that came through another proxy too, such as an office's web proxy. Level 1 is the fastest, and a tile shrinks nearly as much as at level 9 (the story's 'Pack lightly'). `gzip_vary on` adds `Vary: Accept-Encoding`, the header that tells every store on the way that the answer depends on what the client accepts (c3). `gzip_types` lists the kinds worth packing; Nginx always packs `text/html`, so the page needs no entry. The C# service never learns that gzip exists: `TesseraHost` adds no compression of its own.

40.3 Check the criteria the worked answer proves in part

The requirements mark all four of N-20's rules as proved in part: NFR-11 never asks for the page packed, NFR-12 never checks that a plain answer lacks `Content-Encoding`, NFR-13 checks `Vary` on the packed answer alone, and NFR-14 sends back the plain answer's tag alone. These checks go beyond the worked answer; put them in `checkEdge` after the packed tile:

```ts
// Beyond the worked answer. N-20 c2 and c3: plain on request, and Vary still there.
assert.equal(first.headers['content-encoding'], undefined);
assert.match(String(first.headers['vary']), /Accept-Encoding/);
// N-20 c1: the page packs too.
const packedPage = await ask(`${base}/`, { 'Accept-Encoding': 'gzip' });
assert.equal(packedPage.headers['content-encoding'], 'gzip');
// N-20 c4: a browser holding the packed form sends the same weak tag, and gets 304.
const packedUnchanged = await ask(tile, { 'Accept-Encoding': 'gzip', 'If-None-Match': tag });
assert.equal(packedUnchanged.status, 304);
```

Each passes at once, since the template already does the work. Prove each can fail: switch `gzip_vary` off and watch the `Vary` check fail, then switch it on again.

The story's fourth test, 'the map still opens', is the browser tests themselves: a browser always sends `Accept-Encoding`, so every Playwright test now reads packed tiles.

> IN INK Commit at each green, with the message `N-20: pack tiles, the page and the map's files with gzip`, and push.

> THE PROOF `npm run e2e` passes: the browser tests over packed tiles, then the edge checks, yours included. Part C is done, worth 4 points; the burn-up reads 34 of 52. Tomorrow's first test: `SettingsTests.Reads_the_shipped_file`, against a `tessera.json` you write first.

41

Day 40 · Tuesday 15 December: N-21, every setting in one file

> IN SHORT Today `tessera.json` arrives in your copy: five sections of Tessera's own, each bound to one record that refuses a key it does not know and a value outside its limits. This is the first of two days of N-21, worth 2 points.

A setting the maintainer can read without opening the code needs three things: one file, a unit in each name, and a refusal for every mistake. Today proves the file, the defaults and the refusals; tomorrow, the environment's override and the rule against passwords.

EXHIBIT 41.1 Day 40 ends with five sections, each bound to its record and checked at start

Goal	`Database`, `Tiles`, `Map`, `Catalogue` and `Area` in `tessera.json`, beside `Urls` and `Logging`, each read by a record whose defaults match the file
Story	N-21 c1, c2 and c4; tests 1, 2 and 4
In the worked answer	`tessera.json`; `Common/Infrastructure/Settings.cs`; `TesseraModule.cs`, `AddSettings`; the records in `Domain/Settings.cs`, `Infrastructure/Settings.cs` and `Features/Settings.cs`; `SettingsTests.cs`
Look for	`ErrorOnUnknownConfiguration`; `ValidateOnStart`; how the unit tests read the shipped file
Leave for later	Environment variables, the shipped file's safety and the limits in action (day 41)
Done when	`Reads_the_shipped_file` and `Refuses_a_key_it_does_not_know` pass, with `Gives_each_key_left_out_its_default`, `Leaves_a_misspelt_section_at_its_defaults`, `Names_every_value_out_of_range_at_once`, `Refuses_values_that_disagree` and your `Keeps_five_sections_and_a_unit_in_each_time`

Source: `docs/user-stories/user-stories.md`, *N-21*

41.1 Write the file, then prove it reaches its records

Copy the worked answer's `tessera.json` into the root of your copy, but keep `"Urls": "http://127.0.0.1:8080"` until tomorrow. Each length of time carries its unit in its name, so no one reads seconds as milliseconds:

```json
{
  "Urls": "http://127.0.0.1:8080",
  "Logging": { "LogLevel": { "Default": "Information", "Microsoft": "Warning" } },
  "Database": { "PoolSize": 4, "ConnectionLifetimeSeconds": 3600,
                "QueryTimeoutSeconds": 10, "ConnectionWaitSeconds": 3 },
  "Tiles": { "Resolution": 4096, "Buffer": 256, "MaxFeatures": 10000,
             "MinZoom": 0, "MaxZoom": 22, "CacheSeconds": 60 },
  "Map": { "ShowPreview": true },
  "Catalogue": { "RefreshSeconds": 30 },
  "Area": { "Name": "Macau", "MinLongitude": 113.52, "…": "…", "HomeLevel": 15 }
}
```

Laid out tightly here, with one `LogLevel` line cut and `Area` shortened; copy the real file.

Three sections are bound already: `Tiles` since day 11, `Area` and `Map` since day 16. Two are new. `Database` is read by `DatabaseSettings`, in `Infrastructure/Settings.cs`, four whole numbers with `[Range]` limits; `Catalogue` by `CatalogueSettings`, whose one number, `RefreshSeconds`, waits for N-23. Copy both records from the worked answer. One line in `src/Tessera/Tessera.csproj` and one in the unit tests' project copy `tessera.json` beside the program and the tests: `<None Include="../../tessera.json" Link="tessera.json" CopyToOutputDirectory="PreserveNewest" />`, so the tests read the real file, not a copy that could drift. The first test reads it through the property `Shipped`:

```csharp
[Fact]
public void Reads_the_shipped_file()
{
    Assert.Equal(60, Settings<TileSettings>(Shipped).CacheSeconds);
    Assert.Equal(15, Settings<AreaSettings>(Shipped).HomeLevel);
    Assert.Equal(30, Settings<CatalogueSettings>(Shipped).RefreshSeconds);
}
```

It is red until `CatalogueSettings` exists and is bound; `Settings<T>` is day 36's helper.

41.1.1 Green: one file, one door, one method a section

ASP.NET Core reads the file, and each module binds its own sections. `Settings.AddFile` adds `tessera.json` from beside the program, then the environment again, so a variable still wins over the file; `TesseraHost.Build` calls it on the builder's configuration, and `Program.Seed` on its own, in place of day 7's `AddEnvironmentVariables`.

```csharp
// Common/Infrastructure/Settings.cs
public static void AddFile(IConfigurationBuilder configuration) =>
    configuration
        .AddJsonFile(Path.Combine(AppContext.BaseDirectory, "tessera.json"), optional: true)
        .AddEnvironmentVariables();

public static void Section<T>(IServiceCollection services, string name) where T : class =>
    services.AddOptions<T>()
        .BindConfiguration(name, binder => binder.ErrorOnUnknownConfiguration = true)
        .ValidateDataAnnotations()
        .ValidateOnStart();
```

ASP.NET Core's ***binder*** copies each key of a section into the record's property of the same name, letter case aside: `Tiles:CacheSeconds` fills `TileSettings.CacheSeconds`. `TesseraModule.AddSettings` now calls

`Section` once for each of the five sections, in place of the three lines of days 11 and 16, so no class that reads a setting checks it again. `optional: true` lets the program start with no file at all (c2). `Logging` in the file keeps ASP.NET Core's own lines at `Warning`, so add the last two lines of `Starts_from_the_run_command_and_answers`, left out on day 11: the log says 'Now listening on' and holds no 'Request starting'.

> TRY IT After `npm run build`, move `tessera.json` out of `src/Tessera/bin/Release/net10.0` and run `dotnet Tessera.dll` there with only `DATABASE_URL` set. It starts on every default, and listens on `http://localhost:5000`: `Urls` has no default of Tessera's own, so ASP.NET Core picks its own. Put the file back.

41.2 Refuse every mistake, and name it

Criterion 4 wants each mistake to stop the start and name itself. Write the refusals before you add the rules:

CSHARP

```
// A key the service does not read is refused: it is most often a misspelling. A misspelt
// section name is not checked; its keys bind to nothing (ADR 0003).
[Fact]
public void Refuses_a_key_it_does_not_know() =>
    Assert.Contains("'CacheSecond'", Problems("""{"Tiles":{"CacheSecond":60}}"""),
        StringComparison.Ordinal);
```

The second, `Names_every_value_out_of_range_at_once`, writes `{"Tiles":{"CacheSeconds":-1},"Database":{"PoolSize":0},"Area":{"HomeLevel":23}}` with no database URL, and expects all four problems named in one answer. `ErrorOnUnknownConfiguration = true` answers the first: the binder throws an `InvalidOperationException` when a key finds no property, so replace `Serve`'s two `catch` blocks with the worked answer's one, which takes all three kinds and prints `Settings.Problems(error)`. The attributes answer the second, through `ValidateDataAnnotations`, and `ValidateOnStart` makes `Settings.Check` run them as the program starts, not when a request first reads the setting. Then add the worked answer's two other rows to `Refuses_values_that_disagree`: a box turned inside out, `{"Area":{"MinLongitude":113.7}}`, which must name 'west to east', and a zoom range likewise, `{"Tiles":{"MinZoom":10,"MaxZoom":5}}`, which must name 'MaxZoom must not be below MinZoom'. A rule across two values needs code, so `TileSettings` implements `IValidatableObject`:

CSHARP

```
// Domain/Settings.cs, inside TileSettings; a line break moved to fit the page
public IEnumerable<ValidationResult> Validate(ValidationContext validationContext) =>
    MaxZoom >= MinZoom ? [] : [new ValidationResult("MaxZoom must not be below MinZoom.")];
```

Last, `Gives_each_key_left_out_its_default` runs on `{}` and finds a pool of 4 and the area named Macau, and `Leaves_a_misspelt_section_at_its_defaults` writes `{"Tile":{"CacheSeconds":0}}` and still finds 60: no record claims a section called `Tile`. The story lists that gap under 'Open to talk'.

> FROM SCHOOL Validation at school means presence, range, type and format checks on what a user types. The settings check is the same idea run on a file: `[Range(0, 30)]` is a range check, an `int` that refuses 'soon' is a type check, and `[Required]` on the area's name is a presence check.

41.3 Test what the worked answer leaves unproved

Criterion 1 is proved in part (NFR-41): `Reads_the_shipped_file` reads three values, and no test checks the five sections or the units in the names. Write one, beyond the worked answer:

CSHARP

```csharp
// Beyond the worked answer. N-21 c1: five sections of Tessera's own beside
// ASP.NET Core's two, and every length of time named in seconds.
[Fact]
public void Keeps_five_sections_and_a_unit_in_each_time()
{
    using var file = JsonDocument.Parse(Shipped);
    var sections = file.RootElement.EnumerateObject().ToArray();
    var times = sections
        .Where(section => section.Value.ValueKind == JsonValueKind.Object)
        .SelectMany(section => section.Value.EnumerateObject())
        .Select(key => key.Name)
        .Where(name => TimeWords.Any(word => name.Contains(word, StringComparison.Ordinal)));

    Assert.Equal(["Urls", "Logging", "Database", "Tiles", "Map", "Catalogue", "Area"],
        sections.Select(section => section.Name));
    Assert.All(times, name => Assert.EndsWith("Seconds", name, StringComparison.Ordinal));
}

// The words that mark a length of time in a key's name.
private static readonly string[] TimeWords = ["Time", "Wait", "Cache", "Refresh"];
```

Rename `QueryTimeoutSeconds` to `QueryTimeout` in the file and watch it fail; the record would then refuse the key too. Criterion 2 is proved in part as well (NFR-42): no test starts the program with no file, so the Try it above is your proof.

> IN INK Commit at each green, with the message `N-21: read five sections from tessera.json, refuse unknown keys, keep defaults`, and push.

> THE PROOF `Reads_the_shipped_file`, `Refuses_a_key_it_does_not_know` and every other test of `SettingsTests` pass, yours included. N-21 stays open; tomorrow's first test is `Lets_a_variable_override_a_key`.

42

Day 41 · Wednesday 16 December: N-21, the environment, the limits, and no password

> IN SHORT Today an environment variable overrides any key, the section `Database` reaches the pool, the shipped file is pinned safe, and a misspelt key is named with every other problem. This finishes N-21, worth 2 points.

The file holds every setting but the secrets, and the environment can change any of them without an edit. You prove the override, hand the pool its limits, move your copy to port 7800, and mend the worked answer's one fault in this story.

EXHIBIT 42.1 Day 41 ends with the file whole, overridable and safe to commit

Goal	`Tiles__CacheSeconds` beats the file; `Database.Open` takes the four limits; no password; `Urls` on 127.0.0.1:7800
Story	N-21 c3, c4 and c5; tests 3 and 5
In the worked answer	`SettingsTests.cs`; `Infrastructure/Sql/Database.cs`, `Open`; `.env.example`'s last paragraph on variables
Look for	Which source wins when two set one key; which of Npgsql's names carry a unit
Leave for later	What the limits do when the database fails (N-22)
Done when	`Lets_a_variable_override_a_key` and `Ships_a_file_with_no_password_that_listens_on_this_machine_alone` pass, with your `Lets_a_variable_with_two_underscores_override_the_file` and `Names_a_misspelt_key_with_every_other_problem`, and your copy answers through Nginx from port 7800

Source: `docs/user-stories/user-stories.md`, *N-21*

42.1 Let the environment win

ASP.NET Core lets the last source added win. `Settings.AddFile` adds the file and then the environment, so a variable beats the file. The worked answer's test lays its value over both, in memory, under the name the configuration uses inside, with a colon for each step down:

CSHARP

```csharp
// Tiles__CacheSeconds in the environment reaches the configuration as Tiles:CacheSeconds,
// and wins over the file.
[Fact]
public void Lets_a_variable_override_a_key() =>
    Assert.Equal(0, Settings<TileSettings>(Shipped,
        new() { ["Tiles:CacheSeconds"] = "0" }).CacheSeconds);
```

That proves the order, not the spelling (NFR-43, in part): no test sets a real variable spelt `Tiles__CacheSeconds`, with two underscores for each step down, as `.env.example` tells the maintainer to. A test cannot safely change its own process's environment while other test classes run beside it, so this one, beyond the worked answer, starts the built program with a variable of its own. It joins `tests/Tessera.IntegrationTests/StartTests.cs`, whose helper `Run` starts `Tessera.dll` with only the variables it is given:

```csharp
// Beyond the worked answer. N-21 c3: Tiles__CacheSeconds reaches
// Tiles:CacheSeconds and wins over the shipped file's 60. With no DATABASE_URL
// the program stops and names every problem; CacheSeconds is among them only
// if the variable arrived.
[Fact]
public async Task Lets_a_variable_with_two_underscores_override_the_file()
{
    var (code, errors) = await Run(
        new Dictionary<string, string> { ["Tiles__CacheSeconds"] = "-1" });

    Assert.Equal(1, code);
    Assert.Contains("CacheSeconds", errors, StringComparison.Ordinal);
}
```

Spell it with one underscore, `Tiles_CacheSeconds`, and it fails: the program names `DATABASE_URL` alone.

42.2 Hand the pool its limits

Part D needs limits on the database; yesterday put them in the section `Database`, and today they reach the pool. `Database.Open` takes the record and copies each value into Npgsql's connection string:

```csharp
// Infrastructure/Sql/Database.cs (shortened)
public static Database Open(string connection, DatabaseSettings pool) =>
    new(NpgsqlDataSource.Create(new NpgsqlConnectionStringBuilder(connection)
    {
        ApplicationName = "tessera",
        MaxPoolSize = pool.PoolSize,
        ConnectionLifetime = pool.ConnectionLifetimeSeconds,
        CommandTimeout = pool.QueryTimeoutSeconds,
        // Covers both opening a connection and waiting for a free one in the pool.
        Timeout = pool.ConnectionWaitSeconds,
        // …
    }.ConnectionString));
```

The ***connection pool*** keeps four connections open for requests to share; a request waits up to three seconds for a free one, and a query runs at most ten. Npgsql's `Timeout` and `CommandTimeout` are both in seconds, and their names do not say so; Tessera's, `ConnectionWaitSeconds` and `QueryTimeoutSeconds`, put the unit where a reader looks. In `TesseraModule.AddTessera`, build the one `Database` from `IOptions<DatabaseSettings>`, and let your tests that open their own pool pass `new DatabaseSettings()`, the defaults. *Sixteen Tiles*, chapter 6, explains the pool and its waits.

42.3 Pin the shipped file, and move to port 7800

Change `Urls` in `tessera.json` to `"http://127.0.0.1:7800"`. From today your service listens on 7800, not 8080: change the port your `start.ts` hands Nginx and gives the service as `Urls`, and the address you

open by hand. The shipped file is committed, so it must hold no password and listen on this machine alone, since Nginx is the only door (ADR 0008):

```csharp
// ADR 0003: the shipped tessera.json holds no password, and listens on this machine alone.
[Fact]
public void Ships_a_file_with_no_password_that_listens_on_this_machine_alone()
{
    Assert.DoesNotContain("assword", Shipped, StringComparison.Ordinal);
    Assert.DoesNotContain("postgresql://", Shipped, StringComparison.Ordinal);
    Assert.Contains("\"Urls\": \"http://127.0.0.1:7800\"", Shipped, StringComparison.Ordinal);
}
```

'assword' catches 'Password' and 'password' alike. The password reaches the service inside `DATABASE_URL`, which `npm start` builds from `.env`, and Git ignores `.env`.

> KEY IDEA A rule that guards a file belongs in a test, not a code review: a password pasted into `tessera.json` at midnight fails the build before it reaches GitHub.

42.4 Mend a fault: a misspelt key that stops the start alone

The worked answer has a known fault here: the binder names a misspelt key and nothing else, so N-21 c4's 'every such problem at once' holds for values, not for a misspelt key. Started with `Tiles__CacheSecond=5`, `Database__PoolSize=0` and no `DATABASE_URL`, it prints one line, the binder's complaint about `'CacheSecond'`, and stops: the binder throws before any value is checked. Write the test that shows it, beyond the worked answer and red there:

```csharp
// Beyond the worked answer, and red there. N-21 c4: a misspelt key is named
// with every other problem, not alone.
[Fact]
public void Names_a_misspelt_key_with_every_other_problem()
{
    var problems = Problems("""{"Tiles":{"CacheSecond":60,"MinZoom":10,"MaxZoom":5}}""",
        url: null);

    Assert.Contains("'CacheSecond'", problems, StringComparison.Ordinal);
    Assert.Contains("MaxZoom must not be below MinZoom", problems, StringComparison.Ordinal);
    Assert.Contains("DATABASE_URL", problems, StringComparison.Ordinal);
}
```

To mend it, let the binder pass unknown keys, and refuse them in a check of your own that runs with the others. Every check of a record, the attributes and yours alike, is an `IValidateOptions<T>`, and the start-up check gathers all their failures into one list:

CSHARP

```csharp
// Common/Infrastructure/Settings.cs: the book's own, beyond the worked answer
public static void Section<T>(IServiceCollection services, string name) where T : class
{
    services.AddOptions<T>()
        .BindConfiguration(name)
        .ValidateDataAnnotations()
        .ValidateOnStart();
    // An unknown key joins the other problems, rather than stopping the start alone
    // (N-21 c4).
    services.AddSingleton<IValidateOptions<T>>(provider =>
        new KnownKeys<T>(provider.GetRequiredService<IConfiguration>().GetSection(name)));
}

/// <summary>Refuses each key of a section that its record does not read, by name.</summary>
private sealed class KnownKeys<T>(IConfigurationSection section) : IValidateOptions<T>
    where T : class
{
    public ValidateOptionsResult Validate(string? name, T options)
    {
        var known = typeof(T).GetProperties().Select(property => property.Name)
            .ToHashSet(StringComparer.OrdinalIgnoreCase);
        var unknown = section.GetChildren().Select(key => key.Key)
            .Where(key => !known.Contains(key)).ToArray();
        return unknown.Length == 0
            ? ValidateOptionsResult.Success
            : ValidateOptionsResult.Fail(
                unknown.Select(key => $"'{key}' is not a setting of {section.Path}."));
    }
}
```

Rerun `SettingsTests`: the new test passes, and so does `Refuses_a_key_it_does_not_know`, since the message still quotes `'CacheSecond'`. Started as above, your program now prints three lines: the pool, `'CacheSecond' is not a setting of Tiles.`, and the missing address. Two gaps stay, as in the worked answer: a misspelt section, and a value of the wrong type such as ‘soon’, which the binder still refuses alone. Note the difference in your comparison.

> IN INK Commit at each green, with the messages `N-21: let the environment override any key, hand the pool its limits, and ship no password` and `N-21: name a misspelt key with every other problem`, and push.

> THE PROOF Every test of `SettingsTests` and `StartTests` passes, and `npm test` passes whole, through Nginx to port 7800. N-21 is done; the burn-up reads 36 of 52, six of the second plan’s 22, the number Friday’s review expects. Tomorrow’s first test: `DatabaseErrorTests.Asks_the_browser_to_wait_for_a_failure_that_may_pass`, with 08006 as its first row.

43

Day 42 · Thursday 17 December: N-22, four kinds of database failure

> IN SHORT Today every failure the database can give becomes a value of one of four kinds, sorted once by its code, so the tile route can answer each kind its own way. This is the first of two days of N-22, worth 2 points.

Whether a failure deserves 503 or 500 depends on one question, 'will waiting cure it?', and PostgreSQL's code answers it. Today builds the sorting and the one door through which every failure passes; Monday, after the review, turns each kind into an answer.

EXHIBIT 43.1 Day 42 ends with every failure sorted into one of four kinds

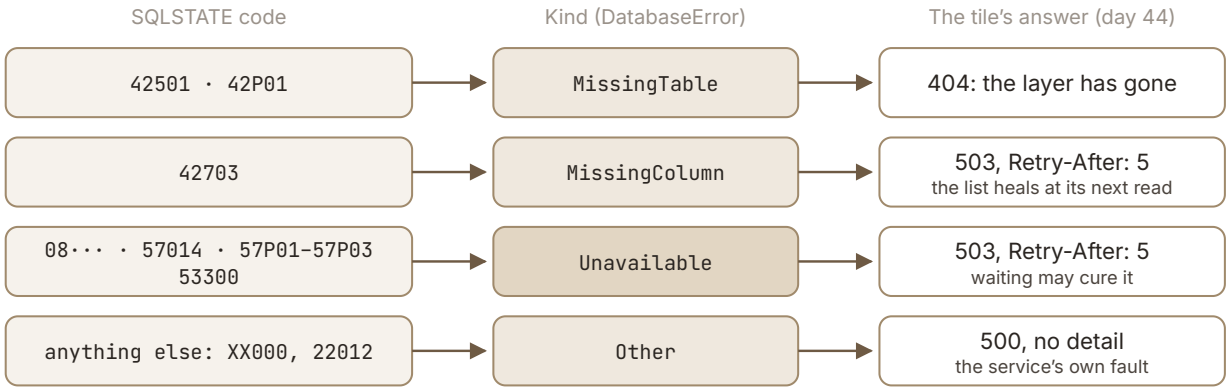
Goal	`DatabaseError`, a closed set of four records, in place of day 6's single record
Story	N-22 test 1, for c1, c2 and c4
In the worked answer	`Infrastructure/Sql/Database.cs`, the record `DatabaseError` and the method `Error`; `tests/Tessera.UnitTests/DatabaseErrorTests.cs`; `Tessera.csproj`, the analyser
Look for	Why a timed-out wait counts as 57014; why `Of` is the only place a code is read
Leave for later	The answers 503 and 500, and the time limits in action (day 44)
Done when	`Asks_the_browser_to_wait_for_a_failure_that_may_pass` and `Sorts_any_other_failure_into_its_kind` pass, with `Lists_every_kind_as_a_case`

Source: `docs/user-stories/user-stories.md`, *N-22*

43.1 Sort a code into a kind

PostgreSQL labels every refusal with a five-character ***SQLSTATE*** code, and its first two characters give the class: `08` is a broken connection, `42` a mistake in the query or a missing grant. The story names which codes wait and which do not, and Exhibit 43.2 shows where each goes.

EXHIBIT 43.2 Each code is sorted once into a kind, and each kind has one answer



Source: `Infrastructure/Sql/Database.cs`, `DatabaseError.Of`; `Features/Tiles.cs`, `TileServer.Answer`

43.1.1 Red: one row a code

A theory runs once for each `InlineData` row, so eight rows prove eight cases in a few lines. In `tests/Tessera.UnitTests/DatabaseErrorTests.cs`:

CSHARP

```csharp
// 08006 a lost connection, 57014 a query past its time, 57P01 a server stopping, 53300 no
// room for another connection: each may pass, so the tile answers 503 with Retry-After.
[Theory]
[InlineData("08006")]
[InlineData("57014")]
[InlineData("57P01")]
[InlineData("53300")]
public void Asks_the_browser_to_wait_for_a_failure_that_may_pass(string sqlState) =>
    Assert.IsType<DatabaseError.Unavailable>(DatabaseError.Of(sqlState, ""));

// 42501 a missing grant and 42P01 a missing table mean the layer has gone; 42703 a dropped
// column; XX000 an internal fault, which waiting cannot cure.
[Theory]
[InlineData("42501", typeof(DatabaseError.MissingTable))]
[InlineData("42P01", typeof(DatabaseError.MissingTable))]
[InlineData("42703", typeof(DatabaseError.MissingColumn))]
[InlineData("XX000", typeof(DatabaseError.Other))]
public void Sorts_any_other_failure_into_its_kind(string sqlState, Type kind) =>
    Assert.IsType(kind, DatabaseError.Of(sqlState, ""));
```

Red: the build fails, since day 6's `DatabaseError` has no kinds and no `Of`.

43.1.2 Green: a closed set of records and one switch

Day 6's record becomes an abstract record with four sealed records inside it. F# would write a union, `MissingTable | MissingColumn | Unavailable | Other`; C# 14 has none, so the attribute `[Closed]` names the cases, and the ExhaustiveMatching analyser stops the build when a `switch` over a failure misses one. Add `<PackageReference Include="ExhaustiveMatching.Analyzer" />` to `src/Tessera/Tessera.csproj` and its version, 0.5.0, to `Directory.Packages.props`, as the worked answer does.

CSHARP

```csharp
// Infrastructure/Sql/Database.cs (shortened; line breaks moved to fit the page)
[Closed(typeof(MissingTable), typeof(MissingColumn), typeof(Unavailable), typeof(Other))]
public abstract record DatabaseError(string SqlState, string Message)
{
    public sealed record MissingTable(string SqlState, string Message)
        : DatabaseError(SqlState, Message);
    // … MissingColumn, Unavailable and Other, written the same way

    public static DatabaseError Of(string sqlState, string message) => sqlState switch
    {
        PostgresErrorCodes.InsufficientPrivilege or PostgresErrorCodes.UndefinedTable
            => new MissingTable(sqlState, message),
        PostgresErrorCodes.UndefinedColumn => new MissingColumn(sqlState, message),
        PostgresErrorCodes.QueryCanceled or PostgresErrorCodes.AdminShutdown
            or PostgresErrorCodes.CrashShutdown or PostgresErrorCodes.CannotConnectNow
            or PostgresErrorCodes.TooManyConnections => new Unavailable(sqlState, message),
        _ when sqlState.StartsWith("08", StringComparison.Ordinal)
            => new Unavailable(sqlState, message),
        _ => new Other(sqlState, message),
    };
}
```

Npgsql's `PostgresErrorCodes` names each code. Each code that will pass is `Unavailable`, and the guard `when sqlState.StartsWith("08", …)` catches the whole class of broken connections in one line; 42501, a withdrawn grant, keeps US-03's 404, and anything unknown is `Other`, the service's own fault.

The analyser checks each `switch` against `[Closed]`, but cannot see a fifth record that derives from `DatabaseError` and is missing from the list. The worked answer's third test, `Lists_every_kind_as_a_case`, makes the two agree: it reads the cases `[Closed]` names, finds every type whose base is `DatabaseError`, and expects the same set.

Day 6's `Database.Run` already catches every `NpgsqlException` and passes it to `Error`, so no caller ever writes `try` (ADR 0004). From today `Error` gives a kind, by three rules:

CSHARP

```csharp
// Infrastructure/Sql/Database.cs; line breaks moved to fit the page
private static DatabaseError Error(NpgsqlException error) => error switch
{
    PostgresException refusal => DatabaseError.Of(refusal.SqlState, refusal.MessageText),
    { InnerException: TimeoutException } =>
        DatabaseError.Of(PostgresErrorCodes.QueryCanceled, error.Message),
    _ => DatabaseError.Of(PostgresErrorCodes.ConnectionFailure, error.Message),
};
```

The middle rule is the subtle one: when Npgsql gives up on a query or a wait for a connection, its exception carries no code, only a `TimeoutException` inside, and the worked answer counts it as 57014, so it waits. Anything else that never reached the database is 08006.

> IN INK Commit at each green, with the message `N-22: sort every database failure into one of four kinds`, and push.

> THE PROOF The three tests of `DatabaseErrorTests` pass. N-22 stays open across the review; Monday's first test: `Answers_503_when_the_database_cannot_be_reached`.

44

Day 43 · Friday 18 December: the second plan's first review

> IN SHORT Today you prove the first iteration's goal from a clean copy, read the burn-up, and plan iteration 6 around four holidays. No story moves today; the iteration's six points are counted, and N-22, half-built, counts nothing.

A review proves the iteration's goal with evidence a stranger could repeat, then turns the burn-up into next week's decisions. The goal of iteration 5 is 'a tile costs one query a minute, and travels packed', and four headers show it.

EXHIBIT 44.1 Day 43 ends with the goal shown, tagged and the next iteration planned

Goal	The goal shown through Nginx; the burn-up's actual column filled in; iteration 6 planned
Story	N-18, N-19, N-20 and N-21, all done; N-22 open, as US-07 was at the first plan's first review
In the worked answer	`docs/guides/delivery-plan.md`, 'The weekly and daily rhythm'; your `docs/second-plan.md`
Look for	Any criterion you proved by hand alone; any difference from the worked answer you mean to keep
Leave for later	N-22's answers, on Monday
Done when	`npm test` passes from a clean copy, and the tag `review-c` is on GitHub

Source: `docs/guides/delivery-plan.md`, *'Each iteration ends with a review'*

44.1 Run every test from a clean copy

A clean copy proves that nothing on your machine alone makes the tests pass, such as a file Git ignores. Clone your own repository into a fresh folder and run everything:

```bash
git clone https://github.com/YOUR-NAME/tessera-rebuild.git /tmp/review-c
cd /tmp/review-c && npm ci && npm test
```

`npm test` runs the scripts' types and tests, the pipeline's checks, the map's tests, the build, the three .NET test projects with `dotnet run`, and then the browser tests and edge checks. It takes a while; read your comparison notes while it runs.

44.2 Show the goal through Nginx

Start the service with `npm start` in your own copy and ask for one tile twice, as an office machine would. Each line of Exhibit 44.2 is a criterion you can point to.

```bash
curl -s -o /dev/null -D - -H 'Accept-Encoding: gzip' http://127.0.0.1/food.cafes/0/0/0.pbf
curl -s -o /dev/null -D - -H 'Accept-Encoding: gzip' http://127.0.0.1/food.cafes/0/0/0.pbf
```

EXHIBIT 44.2 Four headers on the second answer prove the goal

HEADER	WHAT IT SHOWS	STORY
`Cache-Control: public, max-age=60`	The browser keeps the tile a minute	N-18
`ETag: W/"…"`	A held copy costs a 304, not a tile	N-19
`X-Cache-Status: HIT`	Nginx answered; no query ran	N-19
`Content-Encoding: gzip` and `Vary: Accept-Encoding`	The tile travelled packed, and no store will mix the forms	N-20

Source: the answers of your own `npm start`; `scripts/lib/edge.ts` checks the same headers

N-21 shows in the file itself: open `tessera.json` and read its five sections aloud, unit by unit, beside `Urls` and `Logging`. Then tag the review. The first plan tagged its milestones `m1` to `m4`; the delivery plan names no more, so the second plan tags each review by the parts it closes:

```bash
git tag -a review-c -m 'First review of the second plan: Part C'
git push origin review-c
```

44.3 Read the burn-up, and plan iteration 6

Add the actual column to the plan's burn-up and read the gap in days, since one point is about one day. The plan's line stands at 33 of 52 on 11 December and 36 today; only finished stories count.

- **No gap.** Iteration 6 starts on Monday with N-22's second day, as planned.
- **A gap of one day.** The plan has no day in hand to absorb it. Finish the open story first on Monday, record the day lost, and decide at the review of 8 January whether US-36 waits for a third plan.
- **A gap of two days or more.** Cut from the bottom now, not in January: US-36, the last story, moves to a third plan, and its three days go to the stories above it.

Run the weekly check of Exhibit 36.2, as every Friday. Then write iteration 6's plan in your notes, as the delivery plan's review asks: its stories, N-22's second day to S-30; its first test, `Answers_503_when_the_database_cannot_be_reached`; and the risk that has grown. Iteration 6 holds four holidays, 22, 24 and 25 December and 1 January; a story that crosses one needs its next test written down before you stop.

> IN INK Commit the burn-up and the iteration's plan in `docs/second-plan.md`, with the message `First review: six points, iteration 6 planned`, and push.

> THE PROOF `npm test` passed from a clean copy, the tag `review-c` is on GitHub, and the plan records the gap, if any, in days. The burn-up reads 36 of 52. Write Monday's first test in your notes: `Answers_503_when_the_database_cannot_be_reached`.

45

Day 44 · Monday 21 December: N-22, ask the browser to wait

IN SHORT Today each kind of failure becomes an answer: 503 with `Retry-After: 5` for a failure that will pass, 500 for one that will not, and neither kept. This finishes N-22, worth 2 points.

A 503 with `Retry-After` tells the map 'try again in five seconds'; a 500 tells it 'something is broken'. The code is one switch, proved against a real database that is gone and then slow. The 500 has no such proof in the worked answer, and leaves no trace in its log, so you write the test and mend the code.

EXHIBIT 45.1 Day 44 ends with each kind answered, and proved against a real database

Goal	503 with `Retry-After: 5` and `no-store` for `Unavailable`; 500 with no detail for `Other`, with its cause in the log
Story	N-22 c1 to c4, tests 2 and 3
In the worked answer	`Features/Tiles.cs`, `TileServer.Answer`; `Common/Api/Problems.cs`; `tests/Tessera.IntegrationTests/CatalogueTests.cs`; `TileTests.cs`
Look for	Where the five seconds are written, once; why the 500 carries no detail
Leave for later	The list of layers while the database is down (N-23); Nginx's kept tiles in an outage (N-28)
Done when	`Answers_503_when_the_database_cannot_be_reached` and `Answers_503_when_a_query_runs_past_its_time` pass, with `Answers_503_for_a_dropped_column` and your `Answers_500_and_logs_the_cause`

Source: `docs/user-stories/user-stories.md`, *N-22*

45.1 Set a stage that misbehaves

The integration tests' `CatalogueTests` sets its stage in `InitializeAsync`, as the database's owner. Add a view that takes three seconds over every row:

```sql
CREATE OR REPLACE VIEW stage.slow AS SELECT g.id, g.geom FROM stage.granted g
    CROSS JOIN LATERAL (SELECT pg_sleep(3)) AS pause;
```

Grant it with the rest, and add it to the list `Publishes_what_is_granted_and_nothing_else` expects: six ids now, as the worked answer's.

45.2 Write the two tests, red

Both call the tile route's own code through the class's helper `Server`, with no web server, and read the answer with the helpers in `Http.cs`; new today is `StatusAndHeaders`, which runs an answer with

`Problems.Shape` in place, as the service would, and returns the headers a browser would see. The first points a pool at a port nothing holds:

```csharp
// N-22 c1 and test 2: a database that cannot be reached answers 503 with Retry-After, not
// 500.
[Fact]
public async Task Answers_503_when_the_database_cannot_be_reached()
{
    // … the catalogue, loaded as the reader; then a pool on a port FreePort found unused,
    // with ConnectionWaitSeconds = 1 to keep the test short
    var answer = await Server(catalogue, gone).Answer("stage.granted", 0, 0, 0,
        TestContext.Current.CancellationToken);
    var (status, headers) = await Http.StatusAndHeaders(answer);

    Assert.Equal(503, status);
    Assert.Equal("5", headers.RetryAfter.ToString());
    Assert.Equal("no-store", headers.CacheControl.ToString());
}
```

The second, `Answers_503_when_a_query_runs_past_its_time`, opens a pool with `QueryTimeoutSeconds = 1`, asks for a tile of `stage.slow`, and expects 503 in under three seconds, long before the view wakes (c2).

Both are red, answering 500: Thursday's kinds exist, but nothing answers them yet.

45.3 Answer each kind

One `switch` in `TileServer.Answer` turns each kind into its answer, and the analyser refuses the build if a kind is missing. C# still asks for a last arm, since it cannot see that the set is closed; `ExhaustiveMatch.Failed` throws there, and never runs.

```csharp
// Features/Tiles.cs, inside Answer (line breaks moved to fit the page)
return built.Match(
    bytes => new TileResult(bytes ?? [], CacheControl(settings.CacheSeconds)),
    error => error switch
    {
        DatabaseError.MissingTable _ =>
            Problems.NotFound("No published layer has that name."),
        DatabaseError.MissingColumn _ =>
            Problems.Unavailable("The layer has changed. Try again shortly."),
        DatabaseError.Unavailable _ =>
            Problems.Unavailable("The database cannot be reached. Try again shortly."),
        DatabaseError.Other _ => Problems.Fault(),
        _ => throw ExhaustiveMatch.Failed(error),
    });
```

`Problems` writes the five seconds once, as `public const int RetryAfterSeconds = 5;`, and `Shape` adds `Retry-After` to every 503:

```csharp
// Common/Api/Problems.cs, inside Shape; line breaks moved to fit the page
if (context.ProblemDetails.Status == StatusCodes.Status503ServiceUnavailable)
    context.HttpContext.Response.Headers.RetryAfter =
        RetryAfterSeconds.ToString(System.Globalization.CultureInfo.InvariantCulture);
```

`Problems.Unavailable(detail)` answers 503 with a detail, and `Problems.Fault()` answers 500 with none, so a stranger learns nothing of the service's insides. Every problem already carries `no-store`, so no one keeps a failure (c3). Rerun `CatalogueTests`: both pass, and the slow query answers in about one second. The unit test `Answers_503_for_a_dropped_column`, in `TileTests`, proves the `MissingColumn` arm with a stand-in reader that returns 42703; copy it.

45.4 Mend a fault: a 500 that leaves no trace

Criterion 4 is proved in part (NFR-21): the tests sort the codes, and none sends a 500 through the route. The worked answer also has a known fault here, no log on a 500: `TileServer.Answer` answers `DatabaseError.Other` with a 500 and logs nothing, though `Problems.Fault`'s comment says 'the log holds the cause', so the maintainer learns as little as the stranger. Write the test beyond the worked answer, in `TileTests.cs`; it does not compile there, since the worked answer's `TileServer` takes no log:

CSHARP

```
// Beyond the worked answer. N-22 c4: a failure that waiting cannot cure, here 22012, a
// division by zero, answers 500 with no detail, and leaves its cause in the log.
[Fact]
public async Task Answers_500_and_logs_the_cause()
{
    var log = new Notes();
    var server = new TileServer(
        () => CatalogueReader.FromRows(
            [new CatalogueRow("food", "cafes", "geom", 4326, "POINT", "[]")]),
        (_, _) => Task.FromResult(Result.Failure<byte[]?, DatabaseError>(
            DatabaseError.Of("22012", "division by zero"))),
        Limits,
        log);
    var answer = await server.Answer("food.cafes", 15, 27154, 14280,
        TestContext.Current.CancellationToken);
    var problem = Assert.IsType<ProblemHttpResult>(answer);
    Assert.Equal((500, null), (problem.StatusCode, problem.ProblemDetails.Detail));
    Assert.Contains("22012", Assert.Single(log.Lines), StringComparison.Ordinal);
}
```

`Notes` is a small `ILogger` of the test's own whose `Log` adds each formatted line to `Lines`. To mend the code, give `TileServer` a fourth parameter, `ILogger? log = null`, so the tests that pass three still compile. The last arm becomes `DatabaseError.Other other => Fault(other)`, and the private method `Fault` calls `log?.LogError("A tile failed with {SqlState}: {Message}", error.SqlState, error.Message)` before it returns `Problems.Fault()`. `MapTiles` takes the logger as a new last parameter and hands it on, and `TesseraModule.MapTessera` passes `app.Services.GetRequiredService<ILoggerFactory>().CreateLogger("Tessera.Tiles")`.

> IN INK Commit at each green, with the messages `N-22: answer 503 with Retry-After when the database cannot answer` and `N-22: log the cause of every 500`, and push.

> THE PROOF `CatalogueTests`, `TileTests` and `DatabaseErrorTests` pass, and so does `npm test`. N-22 is done; the burn-up reads 38 of 52, on plan. Tomorrow, 22 December, is a holiday; write Wednesday's first test in your notes: `Holds_no_list_until_the_first_load_succeeds`.

46

Day 45 · Wednesday 23 December: N-23, a list of layers that lives

IN SHORT Today the list of layers becomes a value the service swaps whole, which no request ever sees half done, and which survives a failed reload. This is the first of two days of N-23, worth 2 points.

A list read once at start-up cannot learn of a new grant, and a list reread without care can vanish when the database blinks. Today builds the class that holds the list and its two rules, and loads it once in the background; Monday turns that one load into a loop.

EXHIBIT 46.1 Day 45 ends with a live list that keeps its last good value

Goal	`LiveCatalogue`: no list until the first load; a failed refresh keeps the last good list; tiles answer 503 until a list exists
Story	N-23 c3 and c4, test 1
In the worked answer	`Infrastructure/LiveCatalogue.cs`, `Current` and `Refresh`; `Features/Tiles.cs`, the first line of `Answer`; `tests/Tessera.UnitTests/CatalogueTests.cs`; `ServiceTests.cs`, `ServiceFixture`
Look for	Why the swap uses `Interlocked.Exchange`; when the log speaks
Leave for later	The retries and the refresh every 30 seconds (day 46)
Done when	`Holds_no_list_until_the_first_load_succeeds` and `Keeps_the_last_good_list_through_a_failure` pass, and every test of `ServiceTests` still passes

Source: `docs/user-stories/user-stories.md`, *N-23*

46.1 Test the two rules with a stand-in

The live list takes its loader as a function, so the tests hand it a queue of answers instead of a database. In `tests/Tessera.UnitTests/CatalogueTests.cs`, three small helpers come first: `Ok(catalogue)` and `Down()` make a load that succeeds and one that fails with 08006, and `Ids(live)` reads the current list's ids, or `["none"]` before the first load, so an assertion can tell 'no list' from 'an empty list'.

```csharp
[Fact]
public async Task Holds_no_list_until_the_first_load_succeeds()
{
    var answers = new Queue<Func<Task<Result<LayerCatalogue, DatabaseError>>>>(
        [Down, () => Ok(With("cafes"))]);
    using var live = Live(_ => answers.Dequeue()());

    await live.Refresh(CancellationToken.None);
    Assert.Equal(["none"], Ids(live));

    await live.Refresh(CancellationToken.None);
    Assert.Equal(["food.cafes"], Ids(live));
}
```

Its twin, `Keeps_the_last_good_list_through_a_failure` (N-23 c4), queues a good load, then a failure, and still finds `food.cafes` after both refreshes. Today the helper `Live(load)` is `new(load, NullLogger.Instance)`; Monday it gains the two waits. Both tests are red: `LiveCatalogue` does not exist.

46.2 Swap the whole list in one step

Many requests read the list while one strand of work replaces it, so the replacement must happen in one step. `Interlocked.Exchange` swaps a whole list into the field as one ***atomic step***, which no other thread can see half done, and `Volatile.Read` fetches the field afresh on every read. .NET runs the class as a ***hosted service***, a background loop that starts and stops with the program; today its loop runs once:

```csharp
// Infrastructure/LiveCatalogue.cs: today's form, the book's step towards the worked answer
public sealed class LiveCatalogue(
    Func<CancellationToken, Task<Result<LayerCatalogue, DatabaseError>>> load,
    ILogger log) : BackgroundService
{
    private LayerCatalogue? _current;

    public LayerCatalogue? Current => Volatile.Read(ref _current);

    public async Task Refresh(CancellationToken cancel)
    {
        var loaded = await load(cancel).ConfigureAwait(false);
        if (loaded.IsFailure)
        {
            log.LogWarning("Could not read the list of layers ({SqlState}): {Message}",
                loaded.Error.SqlState, loaded.Error.Message);
            return;
        }
        var old = Interlocked.Exchange(ref _current, loaded.Value);
        // … the log speaks only when the set of layers changes
    }

    // Day 45: the list loads once, in the background, as the program starts.
    protected override Task ExecuteAsync(CancellationToken stoppingToken) =>
        Refresh(stoppingToken);
}
```

A failure only logs, with the database's code, so the last good list stays (c4). The swap returns the old list, and `Refresh` logs 'The service publishes N layers.' only when the set of ids differs, so a refresh every 30

seconds leaves the log quiet.

> FROM SCHOOL Two threads that read and write one variable at once can meet a race condition: one reads while the other is half-way through a change. You may have met it as two people booking the last seat. An atomic step has no half-way, so no one can see it there.

46.3 Answer 503 until a list exists

`TileServer` takes a function, `Func<LayerCatalogue?> catalogue`, in place of the list it held, and asks it once, first, with the same `Retry-After` as N-22 (c3):

CSHARP

```
// Features/Tiles.cs, the start of Answer
if (catalogue() is not { } layers)
    return Problems.Unavailable("The list of layers is not ready yet.");
if (layers.Find(layer) is not { } found)
    return Problems.NotFound("No published layer has that name.");
// …
```

In `MapTiles`, build the server with `() => catalogue.Current`. In `TesseraModule.AddTessera`, register one `LiveCatalogue` as a singleton, with `cancel => CatalogueReader.Load(database, cancel)` as its loader and a logger named `Tessera.Catalogue`, and then `builder.Services.AddHostedService(services => services.GetRequiredService<LiveCatalogue>())`, so the loop and the tiles share one object. The integration `CatalogueTests`' helper `Server` becomes `new(() => catalogue, reader.Single<byte[]>, Tiles)`.

The list now arrives a moment after the service starts, so `ServiceFixture` must wait for it. At the end of its `InitializeAsync`, it asks for `/food.cafes/0/0/0.pbf` every 100 ms, up to a hundred times, until the answer is no longer 503, and throws 'The list of layers did not load within ten seconds.' if it never is.

> WATCH OUT Read the list once a request, never twice. `Answer` calls `catalogue()` once and keeps the result in `layers`; a request that read `Current` again for the next step might meet two different lists if a refresh landed between the reads.

> IN INK Commit at each green, with the message `N-23: hold the list of layers in one field, and keep the last good list`, and push.

> THE PROOF `Holds_no_list_until_the_first_load_succeeds` and `Keeps_the_last_good_list_through_a_failure` pass, and `ServiceTests` passes whole. Christmas Eve and Christmas Day are holidays; write Monday's first test in your notes: `Tries_until_the_database_answers`.

47

Day 46 · Monday 28 December: N-23, reread the list on a schedule

IN SHORT Today the list reloads itself: at once at start, every five seconds until the first load succeeds, then every 30 seconds. A grant shows within half a minute, and the service starts with or without the database. This finishes N-23, worth 2 points.

The loop makes one choice on each turn, so stand-ins with short waits test it in milliseconds. A real database then proves the grant and the revoke, and the real program proves the start with no database at all.

EXHIBIT 47.1 Day 46 ends with a grant served within one refresh, and a start with no database

Goal	A table granted after start-up joins the list within `Catalogue.RefreshSeconds`; the program starts with the database down and answers 503
Story	N-23 c1 to c3, tests 2 and 3
In the worked answer	`LiveCatalogue.cs`, `ExecuteAsync`; `TesseraModule.cs`, `AddTessera`; `CatalogueTests.cs` in both test projects; `StartTests.cs`
Look for	Why the retry reuses `Problems.RetryAfterSeconds`; what a zero `RefreshSeconds` means
Leave for later	`/health` and its two checks (N-25, day 48)
Done when	`Sees_a_grant_and_a_revoke_within_one_refresh` passes, with `Tries_until_the_database_answers`, `Loads_once_when_refresh_is_zero` and the first half of `Starts_with_the_database_down_and_answers_503`

Source: `docs/user-stories/user-stories.md`, *N-23*

47.1 Test the loop with short waits

The class gains two parameters, `every`, the wait between refreshes, and `retry`, the wait before the first load succeeds. The helper `Live` becomes `new(load, every, TimeSpan.FromMilliseconds(10), NullLogger.Instance)`, with `every` zero unless a test says otherwise, so a test of the retries ends in a moment:

```csharp
// N-23 c3: the loop tries again until the database answers.
[Fact]
public async Task Tries_until_the_database_answers()
{
    var calls = 0;
    using var live = Live(_ => ++calls < 3 ? Down() : Ok(With("cafes")),
        TimeSpan.FromSeconds(30));

    await live.StartAsync(TestContext.Current.CancellationToken);
    for (var wait = 0; wait < 100 && live.Current is null; wait++)
        await Task.Delay(10, TestContext.Current.CancellationToken);
    await live.StopAsync(TestContext.Current.CancellationToken);

    Assert.Equal(["food.cafes"], Ids(live));
    Assert.True(calls >= 3);
}
```

The second, `Loads_once_when_refresh_is_zero`, starts a loop with `every` zero, waits for `live.ExecuteTask`, the loop's own task, to end, and expects one call. Both are red against Wednesday's single load: the first finds no list after one failure, and the second never builds, since `Live` takes no `every` yet.

47.1.1 Green: one choice a turn

```csharp
// Infrastructure/LiveCatalogue.cs (shortened)
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    try
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await Refresh(stoppingToken).ConfigureAwait(false);
            if (Current is not null && every <= TimeSpan.Zero)
                return;
            await Task.Delay(Current is null ? retry : every, stoppingToken)
                .ConfigureAwait(false);
        }
    }
    catch (OperationCanceledException) when (stoppingToken.IsCancellationRequested)
    {
        // The service is stopping; the loop ends with it.
    }
}
```

Before the first list, wait `retry`; after it, wait `every`, or stop when `every` is zero, which the section `Catalogue` allows as 'read the list once'. `Task.Delay` takes the stop signal, so the program never waits out a refresh to close.

47.2 Wire the waits in the module's door

`TesseraModule.AddTessera` hands the two waits to the class, between the loader and the logger of day 45: `RefreshSeconds` from `IOptions<CatalogueSettings>`, and for the retry `Problems.RetryAfterSeconds`, the five seconds every browser was asked to wait. Nothing at start-up reads the database now, so `Program.Serve` starts whether the database answers or not (c3).

47.3 Prove the promises through a real database and a real program

The first proof joins the integration `CatalogueTests`. It runs a live list against the real database, with waits of 200 and 100 ms in place of 30 and 5 seconds, and watches a table join and leave:

```csharp
// N-23 c1 and c2, and test 2 (a string split, and lines broken, to fit the page)
[Fact]
public async Task Sees_a_grant_and_a_revoke_within_one_refresh()
{
    // … as the owner: make stage.late, and revoke it from tessera_reader
    await using var reader = Database.Open(database.Reader, new DatabaseSettings());
    using var live = new LiveCatalogue(cancel => CatalogueReader.Load(reader, cancel),
        TimeSpan.FromMilliseconds(200), TimeSpan.FromMilliseconds(100), NullLogger.Instance);
    await live.StartAsync(TestContext.Current.CancellationToken);

    await Until(() => live.Current?.Layers.ContainsKey("stage.granted") == true);
    Assert.False(live.Current!.Layers.ContainsKey("stage.late"));

    await database.Owner("GRANT SELECT ON stage.late TO tessera_reader;");
    await Until(() => live.Current?.Layers.ContainsKey("stage.late") == true);

    await database.Owner("REVOKE SELECT ON stage.late FROM tessera_reader;");
    await Until(() => live.Current?.Layers.ContainsKey("stage.late") == false);

    await live.StopAsync(TestContext.Current.CancellationToken);
}
```

`Until` checks its condition every 100 ms for up to five seconds, since no single moment is sure to see a refresh that runs in the background. The revoke's other half, a new tile answering 404, is US-03's `Answers_404_for_a_table_revoked_after_start_up`.

The second proof, in `StartTests.cs`, starts the built program with a `DATABASE_URL` that points at a free port. Write the first half of `Starts_with_the_database_down_and_answers_503`: a loop that asks for `/food.cafes/0/0/0.pbf` every 100 ms until the program listens, then expects 503, `Retry-After` of five seconds, and a program still running (`Assert.False(program.HasExited)`). Day 48 adds its second half. Criterion 3 is proved in part in the worked answer (NFR-24): no test asks for the map while the database is down. Add two lines beyond it, once the tile has answered:

```csharp
// Beyond the worked answer. N-23 c3: the map's page needs no database.
using var page = await client.GetAsync("/", TestContext.Current.CancellationToken);
Assert.Equal(System.Net.HttpStatusCode.OK, page.StatusCode);
```

Run `npm run build` first, since `StartTests` starts the built program.

> IN INK Commit at each green, with the message `N-23: reread the list on a schedule, and start without the database`, and push.

> THE PROOF Both `CatalogueTests` classes and `StartTests` pass, and so does `npm test`. N-23 is done; the burn-up reads 40 of 52. Tomorrow's first check: twenty requests at once for one slow tile all answer 200 in under four seconds.

48

Day 47 · Tuesday 29 December: N-24, one query for a crowd

> IN SHORT Today Nginx lets the first of many requests for one new tile build it, and makes the rest wait for that one answer. This is N-24, worth 1 point.

A store helps only once a copy is in it; Nginx's cache lock covers the seconds before. At nine o'clock a crowd opens the map, and hundreds of requests for the same tile arrive before any copy exists. Two directives turn them into one query, and a check with a view made slow on purpose proves it.

EXHIBIT 48.1 Day 47 ends with twenty requests served by one query

Goal	Twenty requests at once for a two-second tile all answer 200 within four seconds; a browser that leaves stops nothing
Story	N-24 c1 to c3
In the worked answer	`nginx/default.conf.template`, the tiles' location; `scripts/e2e.ts`, the view `food.slow_cafes`; `scripts/lib/edge.ts`, the crowd check
Look for	Why the lock waits 15 seconds; why the slow view sleeps once a query, not once a place
Leave for later	The cap of 32 requests at once for one machine (S-30), which reuses the slow view
Done when	`npm run e2e` passes its crowd check and your new check for a browser that leaves

Source: `docs/user-stories/user-stories.md`, *N-24*

48.1 Make a tile that takes two seconds

A real tile builds in milliseconds, too fast to show a crowd, so `scripts/e2e.ts` creates, after the seed, a view whose every tile takes two seconds, and grants it with the rest, as the publisher:

```ts
const slow = 'CREATE VIEW food.slow_cafes AS SELECT c.fid, c.geom, c.name FROM '
  + '(SELECT pg_sleep(2)) AS pause CROSS JOIN food.cafes AS c;';
const grant = `${slow} GRANT USAGE ON SCHEMA food, outdoors, culture, transport `
  + 'TO tessera_reader; GRANT SELECT ON ALL TABLES IN SCHEMA food, outdoors, culture, '
  + 'transport TO tessera_reader;';
```

Each is one line in the worked answer, split here to fit.

The pause sits in a subquery of one row, so it runs once for each query, not once for each café.

48.2 Write the crowd check, red

The check sends twenty requests at once and times them all. In `checkEdge`, after the N-20 checks:

```ts
// N-24: twenty requests at once for one tile that takes two seconds to build all get it
// in about two.
const crowdTile = `${base}/food.slow_cafes/0/0/0.pbf`;
const began = Date.now();
const crowd = await Promise.all(Array.from({ length: 20 },
  () => ask(crowdTile, { 'Accept-Encoding': 'identity' })));
const took = Date.now() - began;
assert.ok(crowd.every(answer => answer.status === 200),
  `statuses ${crowd.map(answer => answer.status).join(',')}`);
assert.ok(took >= 1800 && took < 4000, `twenty requests took ${took} ms`);
```

Run `npm run e2e`. Without a lock, all twenty requests miss the store and reach the service at once. Your one copy has a pool of four connections, so the queries run four at a time, two seconds a round, and a request that waits over three seconds (`ConnectionWaitSeconds`) for a connection answers 503: the check fails on its statuses.

> WATCH OUT The worked answer's comment speaks of 'the two copies' eight connections', since N-29 later runs two copies; yours has one, with four. With the lock, one query runs whatever the pool.

48.3 Turn the lock on

A ***cache lock*** lets one request fetch a missing copy while the others for the same key wait for it. Add three lines to the tiles' location:

```nginx
location ~ \.(pbf|mvt)$ {
    proxy_pass http://tessera;
    proxy_cache tiles;
    # Many people asking for one new tile at once cost one query (N-24). The others wait
    # as long as the first request may take, 15 seconds, the same as proxy_read_timeout.
    proxy_cache_lock on;
    proxy_cache_lock_timeout 15s;
    # …
    # A browser that leaves does not cancel a tile others may be waiting for.
    proxy_ignore_client_abort on;
    # …
}
```

Fifteen seconds outlasts the service's own worst case, the section `Database`'s three-second wait for a connection and ten-second query, so a waiting request never gives up on a query that will still answer. Rerun `npm run e2e`: the twenty answers arrive together in about two seconds, and the next request for the tile finds the kept copy (c3).

> FROM SCHOOL The lock is mutual exclusion, the idea behind a lock on a shared file: one holder at a time, and the rest queue. Here the queue does not take turns at the work; it shares the one result.

48.4 Check the browser that leaves

Criterion 2 has no test in the worked answer: NFR-16 says 'No test yet'. Without `proxy_ignore_client_abort`, the first browser to give up would cancel the query, and every request waiting behind it would wait again for a new one. This check, beyond the worked answer, goes after the crowd check, with `request` imported from `node:http`:

```ts
// Beyond the worked answer. N-24 c2: the first asker leaves after half a second, and the
// query goes on for the five who stay. A tile no other check asks for, so nothing is kept
// yet.
const leaverTile = `${base}/food.slow_cafes/2/3/1.pbf`;
const leaver = request(leaverTile);
leaver.on('error', () => {});
leaver.end();
setTimeout(() => leaver.destroy(), 500);
const stayedFrom = Date.now();
const stayers = await Promise.all(Array.from({ length: 5 }, () => ask(leaverTile)));
assert.ok(stayers.every(answer => answer.status === 200));
assert.ok(Date.now() - stayedFrom < 3000, 'the stayers waited for a second query');
```

Tile 2/3/1 holds Macau, so the view finds cafés there and the query sleeps its two seconds. With the directive, the five stayers receive the first query's tile at about two seconds. Remove the directive and watch the time grow past 3,000 ms: the cancelled query leaves the lock free, and one of the stayers must start another. Against a stand-in service that took two seconds a tile, the stayers waited about 2.0 seconds with the directive and 4.0 to 4.5 without it; the bound leaves room for Nginx's own waits.

> IN INK Commit at each green, with the message `N-24: serve a crowd asking for one new tile from one query`, and push.

> THE PROOF `npm run e2e` passes: the crowd of twenty in about two seconds, and the five who stayed. N-24 is done; the burn-up reads 41 of 52, on the plan's line. Tomorrow's first test: `Says_up_when_the_service_can_serve_tiles`.

49

Day 48 · Wednesday 30 December: N-25, a health check that says UP only when it means it

> IN SHORT Today `/health` says UP only when the database answers within a second and the list of layers has loaded, and `npm start` waits for that word. This is N-25, worth 1 point, and the last story of this part.

A health check is honest when it counts exactly what a tile needs, and nothing else. ASP.NET Core's health checks do the answering; you write the two checks, choose the words and status codes, keep every answer out of every store, and prove all three words.

EXHIBIT 49.1 Day 48 ends with UP, DOWN and OUT_OF_SERVICE each proved

Goal	`/health` answers 200 `{"status":"UP"}`, or 503 with `DOWN` or `OUT_OF_SERVICE`, never kept
Story	N-25 c1 to c4
In the worked answer	`Infrastructure/HealthChecks.cs`; `Features/Health.cs`, `MapHealth`; `Infrastructure/Sql/Database.cs`, `Ping`; `TesseraModule.cs`; `scripts/lib/database.ts`, `waitForHealth`
Look for	Why the ping keeps a clock of its own; why the answer names no fault
Leave for later	The order in which requests wait for a connection (N-26, day 49)
Done when	`Says_up_when_the_service_can_serve_tiles` and `Reports_degraded_until_the_first_list_loads` pass, with `Starts_with_the_database_down_and_answers_503` whole and your `scripts/lib/database.test.ts`

Source: `docs/user-stories/user-stories.md`, *N-25*

49.1 Prove UP on the running service

A ***health check*** is an address a watcher asks to learn whether the service can do its job; Tessera's, like pg_tileserv's, is `/health`. The first test asks it on the real service, in `ServiceTests.cs`, and fails with 404, since no route answers `/health` yet:

```csharp
// N-25 c1 and test 1: with the list loaded and the database answering, /health says UP.
[Fact]
public async Task Says_up_when_the_service_can_serve_tiles()
{
    using var answer = await Get("/health");

    Assert.Equal(HttpStatusCode.OK, answer.StatusCode);
    Assert.Equal("""{"status":"UP"}""",
        await answer.Content.ReadAsStringAsync(TestContext.Current.CancellationToken));
    Assert.Equal("application/json", answer.Content.Headers.ContentType?.MediaType);
    Assert.True(answer.Headers.CacheControl?.NoStore);
}
```

49.2 Write the two checks, and say the worst

Each check is a small class that answers one question with one expression, and ASP.NET Core keeps the worst result: `Unhealthy` outranks `Degraded`.

```csharp
// Infrastructure/HealthChecks.cs; line breaks moved to fit the page
public sealed class DatabaseAnswers(Database database) : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context, CancellationToken cancellationToken = default) =>
        await database.Ping(cancellationToken).ConfigureAwait(false)
            ? HealthCheckResult.Healthy()
            : HealthCheckResult.Unhealthy();
}
```

`CatalogueReady`, beside it, takes the `LiveCatalogue` and answers `Degraded` while `Current` is null, else `Healthy`. Register both in `TesseraModule.AddTessera` with `builder.Services.AddHealthChecks().AddCheck<DatabaseAnswers>("database").AddCheck<CatalogueReady>("catalogue")`. `Database.Ping` sends `SELECT 1` through the same pool as the tiles, with a clock of its own, since a frozen database can outlast Npgsql's limits: a token that cancels after one second takes the ping out of the pool's queue, and `WaitAsync(PingLimit, stop)` stops waiting at the same moment, so `/health` answers within a second.

49.2.1 Green: one route, three words

`MapHealth`, in `Features/Health.cs`, maps the checks to `/health` with a table of status codes, 200 for healthy and 503 for either of the others, and a writer of its own:

```csharp
// Features/Health.cs, inside the answer's writer
var status = report.Status switch
{
    HealthStatus.Healthy => "UP",
    HealthStatus.Degraded => "OUT_OF_SERVICE",
    _ => "DOWN",
};
// No copy is kept, since a kept 'UP' would hide a failure from the watcher (N-25 c3).
context.Response.Headers.CacheControl = "no-store";
context.Response.ContentType = "application/json";
return context.Response.WriteAsync($$"""{"status":"{{status}}"}""");
```

The body names no fault; when the list fails to load, `LiveCatalogue`'s warning in the log names the database's code (c2). Call `app.MapHealth()` first in `MapTessera`, and the test passes.

49.3 Prove the two other words

The first, in the unit tests' `CatalogueTests`, asks the check itself before and after a load:

CSHARP

```
// N-25: /health says OUT_OF_SERVICE while no list has loaded, and the check turns healthy
// once one has, so a list that loads late heals the service without a restart.
[Fact]
public async Task Reports_degraded_until_the_first_list_loads()
{
    using var live = Live(_ => Ok(With("cafes")));
    var check = new CatalogueReady(live);
    var before = await check.CheckHealthAsync(new(), TestContext.Current.CancellationToken);
    await live.Refresh(TestContext.Current.CancellationToken);
    var after = await check.CheckHealthAsync(new(), TestContext.Current.CancellationToken);

    Assert.Equal((HealthStatus.Degraded, HealthStatus.Healthy),
        (before.Status, after.Status));
}
```

The second finishes Monday's `Starts_with_the_database_down_and_answers_503`. After the tile's 503, it asks the running program for `/health` and expects 503 and exactly `{"status":"DOWN"}` within two seconds, then asks for the tile again and expects 503 within half a second. Criteria 2 and 3 are proved in part (NFR-46, NFR-47): no test reads `OUT_OF_SERVICE` from `/health` itself, and `no-store` is read on `UP` alone. Add one line beyond the worked answer, after the `DOWN` assertion:

CSHARP

```
// Beyond the worked answer. N-25 c3: DOWN is never kept either.
Assert.True(health.Headers.CacheControl?.NoStore);
```

49.4 Let npm start wait for UP

`npm start` says 'Tessera is ready' only once `/health`, asked through Nginx, says UP (c4). `waitForHealth`, in `scripts/lib/database.ts`, asks once a second for up to the seconds it is given, counts a refused connection as 'not yet', and returns `'up'` or `'not yet'`. Add it to your `database.ts`, and call it at the end of your `start.ts`, in place of the tile your weekly check has asked for since day 35.

NFR-48 says 'No test yet', so write `scripts/lib/database.test.ts`, beyond the worked answer, which runs the function against a stand-in:

TS

```
// Beyond the worked answer: N-25 c4. standIn(statuses) is a small node:http server on a
// free port that answers /health with each status in turn, 200 for UP and 503 otherwise.
test('says up only once /health says UP', async () => {
  const health = await standIn(['OUT_OF_SERVICE', 'UP']);
  try {
    assert.equal(await waitForHealth(health.url, 3), 'up');
  } finally {
    health.close();
  }
});
```

A second test, 'never counts DOWN as ready', gives `['DOWN']` and two seconds, and expects `'not yet'`. Run `npm run test:scripts`; the second test takes its two seconds.

> IN INK Commit at each green, adding `scripts/lib/database.test.ts`, with the message `N-25: say UP only when the database answers and the list has loaded`, and push.

> THE PROOF `ServiceTests`, both `CatalogueTests`, `StartTests` and `npm run test:scripts` pass. N-25 is done; the burn-up reads 42 of 52, on the plan's line. Tomorrow's first test: `PoolTests.Runs_queries_in_the_order_they_arrive`.

50

Day 49 · Thursday 31 December: N-26, first come, first served

IN SHORT Today you prove that requests wait for a connection in the order they arrive, that the ping gives up after a second, and that a request told to stop leaves the queue at once. N-26 is worth 1 point; tomorrow, 1 January, is a holiday.

N-26 needs no new code in the service: Npgsql's pool keeps the queue, and N-25's ping already carries its limit, so the day is proof. The worked answer proves the story in three integration tests, `PoolTests`, which you type one by one. Each passes at once, so you break the code on purpose to see it fail.

EXHIBIT 50.1 Day 49 ends with each of the pool's rules seen to fail, then pass

Goal	Three tests that prove N-26, and one for the tile query's cancel signal
Story	N-26 c1 to c3 and its Known limit
In the worked answer	`tests/Tessera.IntegrationTests/PoolTests.cs`; `Database.Open` and `Database.Ping`; `docs/adr/0007-limits-in-nginx.md`
Look for	Why ADR 0007 adds no code for the order; which token reaches the wait for a connection
Leave for later	Kept tiles while the service is down (N-28, Monday)
Done when	`Runs_queries_in_the_order_they_arrive`, `Gives_up_the_ping_while_every_connection_is_busy` and `Lets_a_cancelled_request_leave_the_queue` pass; so does your `Hands_the_tile_query_the_requests_own_cancel_signal`, beyond the worked answer

Source: `docs/user-stories/user-stories.md`, *N-26;* `docs/requirements/non-functional-requirements.md`, *NFR-26 to NFR-28*

50.1 Read: who keeps the queue

Npgsql keeps the queue; Tessera sets only its size, four connections, and its wait, three seconds (day 41). Npgsql lends each freed connection to the request that has waited longest: first in, first out. ADR 0007 records this as 'Npgsql's pool (N-26) · Nothing to add', but a rule you borrow still needs a test, or a later Npgsql could change it unseen. The day is three tests and one of your own (Exhibit 50.1); the last proves the worked answer's fourth known fault: N-26's known limit names the wrong signal.

50.2 Set the stage: one connection and two slow views

A pool of four makes an order hard to see, so the tests shrink it to one. Make `tests/Tessera.IntegrationTests/PoolTests.cs` a class fixture over US-01's `DatabaseFixture`, with a stage of its own: one place, and two views of it that hold a connection for a fixed time.

CSHARP

```csharp
// PoolTests.cs; SQL lines broken to fit the page
public sealed class PoolTests(DatabaseFixture database)
    : IClassFixture<DatabaseFixture>, IAsyncLifetime
{
    public async ValueTask InitializeAsync() => await database.Owner("""
        -- … stage.queue_place, a table of one place at 113.54, 22.19, then:
        CREATE OR REPLACE VIEW stage.queue_pause AS SELECT q.id, q.geom
            FROM stage.queue_place q CROSS JOIN LATERAL (SELECT pg_sleep(0.4)) AS pause;
        CREATE OR REPLACE VIEW stage.queue_hold AS SELECT q.id, q.geom
            FROM stage.queue_place q CROSS JOIN LATERAL (SELECT pg_sleep(3)) AS pause;
        -- … USAGE on stage, and SELECT on all three, granted to tessera_reader
        """);

    public ValueTask DisposeAsync() => ValueTask.CompletedTask;

    // One connection, and a wait long enough that no request gives up while the others run.
    private static readonly DatabaseSettings OneConnection =
        new() { PoolSize = 1, QueryTimeoutSeconds = 30, ConnectionWaitSeconds = 30 };
    private static readonly TileSettings Limits = new() { Buffer = 64 };
```

Each view holds one place, so `pg_sleep` runs once a tile query. Each test opens its own pool, with no web service between, and two helpers from the worked answer's file finish the stage: `Layer` finds a layer by its id, and `TileOf` gives the tile that holds a point at a level (day 9's sums).

50.3 Five tiles, one connection

Criterion 1 is the first test: five different tiles of the 0.4-second view, at levels 0 to 4, asked for 50 milliseconds apart, must finish in the order asked.

CSHARP

```csharp
[Fact]
public async Task Runs_queries_in_the_order_they_arrive()
{
    await using var reader = Database.Open(database.Reader, OneConnection);
    var layer = await Layer(reader, "stage.queue_pause");
    var finished = new System.Collections.Concurrent.ConcurrentQueue<int>();

    var asks = new List<Task>();
    for (var zoom = 0; zoom < 5; zoom++)
    {
        var (x, y) = TileOf(113.54, 22.19, zoom);
        var mine = zoom;
        var query = Queries.Tile(layer, TileAddress.Of(zoom, x, y)!, Limits);
        asks.Add(reader.Single<byte[]>(query, TestContext.Current.CancellationToken)
            .ContinueWith(_ => finished.Enqueue(mine), TaskScheduler.Default));
        await Task.Delay(50, TestContext.Current.CancellationToken);
    }
    await Task.WhenAll(asks);

    Assert.Equal([0, 1, 2, 3, 4], finished.ToArray());
}
```

It passes at once, and no code of yours can break it, since the order belongs to Npgsql; so break the test: expect `[0, 1, 2, 4, 3]`, run it, and put it back.

50.4 The ping behind a busy pool

Criterion 3 is the second test: while a tile of the three-second view holds the only connection, the ping must say 'no' within a second and a half, and once the hold ends, 'yes'.

CSHARP

```
[Fact]
public async Task Gives_up_the_ping_while_every_connection_is_busy()
{
    await using var reader = Database.Open(database.Reader, OneConnection);
    var layer = await Layer(reader, "stage.queue_hold");
    var (x, y) = TileOf(113.54, 22.19, 0);
    var hold = reader.Single<byte[]>(Queries.Tile(layer, TileAddress.Of(0, x, y)!, Limits),
        TestContext.Current.CancellationToken);
    await Task.Delay(300, TestContext.Current.CancellationToken);

    var clock = System.Diagnostics.Stopwatch.StartNew();
    var answered = await reader.Ping(TestContext.Current.CancellationToken);

    Assert.False(answered);
    Assert.True(clock.Elapsed < TimeSpan.FromSeconds(1.5), $"took {clock.Elapsed}");
    Assert.False(hold.IsCompleted, "the tile query should still hold its connection");
    await hold;
    Assert.True(await reader.Ping(TestContext.Current.CancellationToken));
}
```

It passes at once, since day 48's `Ping` already carries the limit. Make it fail: set `PingLimit` in `Database.cs` to `TimeSpan.FromSeconds(5)` and rerun; the ping waits out the hold and answers, so `Assert.False(answered)` fails. Put the 1 back.

> KEY IDEA When the code comes first, break it on purpose before you trust the test. Red first is a rule about evidence, not about the order you type in.

50.5 A wait told to stop

Criterion 2 is the third test: a query whose own token is cancelled while it waits for the only connection must leave the queue at once, not after the pool's 30 seconds:

CSHARP

```
[Fact]
public async Task Lets_a_cancelled_request_leave_the_queue()
{
    // … the same hold of the only connection as above, then:
    using var leave = CancellationTokenSource.CreateLinkedTokenSource(
        TestContext.Current.CancellationToken);
    leave.CancelAfter(TimeSpan.FromMilliseconds(200));
    var clock = System.Diagnostics.Stopwatch.StartNew();
    var (x1, y1) = TileOf(113.54, 22.19, 1);

    await Assert.ThrowsAnyAsync<OperationCanceledException>(() =>
        reader.Single<byte[]>(Queries.Tile(layer, TileAddress.Of(1, x1, y1)!, Limits),
            leave.Token));
    Assert.True(clock.Elapsed < TimeSpan.FromSeconds(1), $"took {clock.Elapsed}");
    await hold;
}
```

This one rests on a line you wrote with US-03: `Database.Run` passes the token to `_source.OpenConnectionAsync(cancel)`, so the cancel reaches the wait itself. Break it: change that call to `OpenConnectionAsync(CancellationToken.None)`. The waiting query now leaves only when the hold ends, and the test fails with 'took 00:00:02.8…'. Put `cancel` back.

50.6 Beyond the worked answer: the right cancel signal

N-26's Known limit says a tile query runs on the service's stop signal; the code hands it the request's own `CancellationToken`, which ASP.NET Core cancels when the caller leaves. The worked answer's third test proves the queue lets a cancelled query go, but no test says which token a tile query carries. Add a unit test to `tests/Tessera.UnitTests/TileTests.cs` that catches the token the route hands its `TileReader`:

CSHARP

```csharp
// Beyond the worked answer. N-26 c2: the tile query waits with the request's own cancel
// signal, which ASP.NET Core cancels when the caller leaves, not the service's.
[Fact]
public async Task Hands_the_tile_query_the_requests_own_cancel_signal()
{
    using var request = new CancellationTokenSource();
    CancellationToken seen = default;
    var server = new TileServer(
        () => CatalogueReader.FromRows(
            [new CatalogueRow("food", "cafes", "geom", 4326, "POINT", "[]")]),
        (_, cancel) =>
        {
            seen = cancel;
            return Task.FromResult(Result.Success<byte[]?, DatabaseError>([]));
        },
        Limits);

    await server.Answer("food.cafes", 15, 27154, 14280, request.Token);

    Assert.True(seen == request.Token, "the tile query lost the request's cancel signal");
}
```

Three arguments build `TileServer`, whose logger is optional (day 44). Run `dotnet test --project tests/Tessera.UnitTests --filter-class '*.TileTests'`: it passes. To see it fail, pass `CancellationToken.None` to `read` in `TileServer.Answer`; the message names the lost signal. Put `cancel` back, and note in your comparison that the story names the wrong signal.

> WATCH OUT In effect the Known limit still holds: with `proxy_ignore_client_abort on` (N-24), Nginx keeps its connection to the service open when a browser leaves, so the query runs on and lands in Nginx's store. S-30's limits (days 52–53), not a cancel, cure overload.

> IN INK Commit at each green, adding `tests/Tessera.IntegrationTests/PoolTests.cs`, with messages such as `N-26: prove the pool serves waiting queries in the order they arrive`, and push.

> THE PROOF `PoolTests` passes its three tests and `TileTests` its new one, and you have seen each fail once, for a reason you chose. N-26 is done; the burn-up reads 43 of 52. Monday's first test: `checkEdge`'s new block, 'with the service stopped, a tile never kept answers 503'.

51

Day 50 · Monday 4 January: N-28 begins, kept tiles with the service down

IN SHORT Today Nginx learns to serve the tiles it holds while the service cannot answer, and to answer every other tile with the service's own kind of 503. N-28 is worth 2 points; today is the first.

Two rules in Nginx keep the map whole while the service is down: serve what you hold, and refuse the rest in the service's own words. Today covers the service stopped, criteria 1 and 3, which the worked answer proves at the end of `checkEdge` in `scripts/lib/edge.ts`. Tomorrow covers a database that hangs, where the worked answer proves less.

EXHIBIT 51.1 Day 50 ends when a kept tile still arrives with the service stopped, and a new one answers 503

Goal	Nginx serves its kept tiles while the service is down, and answers any other tile 503 with `Retry-After: 5`
Story	N-28 c1 and c3; its test 1, 'Kept tiles with the service down'
In the worked answer	`nginx/default.conf.template`: `proxy_cache_use_stale`, `error_page 502 504` and `@unavailable`; the last block of `checkEdge` in `scripts/lib/edge.ts`; `stop` in `scripts/e2e.ts`
Look for	Why `http_503` sits in the list of stale reasons; why Nginx turns its own 502 into a 503
Leave for later	An expired tile served at once, and a database that hangs (tomorrow); the second copy of the service (N-29, day 56)
Done when	The N-28 block of `checkEdge` passes in `npm run e2e`: the kept tile arrives byte for byte, and `/food.cafes/22/0/0.pbf` answers 503 as `application/problem+json` with `Retry-After: 5`

Source: `docs/user-stories/user-stories.md`, *N-28;* `docs/requirements/non-functional-requirements.md`, *NFR-29 and NFR-31*

51.1 Read: an old tile beats no tile

A kept tile, even an old one, is almost always right, because the places change only when a seed script runs. A copy older than its `max-age` is a ***stale tile***. Serving it while someone mends the fault keeps the map whole; refusing it would blank every square for a fault no visitor can cure. Exhibit 51.1 sets the day: one check, red first, then two rules.

51.2 Red: stop the service and ask

The check must stop the service part-way through. Your `scripts/e2e.ts` starts it as a background zx process, `dotnet` on the built `Tessera.dll` with its own `Urls`, so it can kill that process and wait for it to exit. Until day 56 your copy runs one copy of the service, so `checkEdge` takes one stop function; the worked answer's second arrives with N-29.

```ts
// In scripts/e2e.ts: stop the copy, and wait until it has gone.
const stop = (copy: ProcessPromise | undefined) => async () => {
  await copy?.kill();
  await copy;
};
```

Then add the check as the last block of `checkEdge`. It asks for `tile`, the first tile the check asked for, and for a tile no check has asked for, at level 22:

```ts
// With the service down, a kept tile still arrives, and a tile never kept asks the browser to
// try again shortly, as a problem document.
await stop();
const kept = await ask(tile, { 'Accept-Encoding': 'identity' });
assert.equal(kept.status, 200);
assert.deepEqual(kept.body, first.body);
const never = await ask(`${base}/food.cafes/22/0/0.pbf`);
assert.equal(never.status, 503);
assert.equal(never.headers['content-type'], 'application/problem+json');
assert.equal(never.headers['retry-after'], '5');
```

Run `npm run build`, then `npm run e2e -- --grep-invert "approved image"`. The check fails on `never.status`, for the right reason: ‘actual: 502, expected: 503’. Nginx found no service behind it and answered ‘Bad Gateway’ with a page of HTML that no map can read.

> WATCH OUT The kept tile passes before you write any rule: the check asked for it within its `max-age` of 60 seconds, so Nginx serves it as a plain hit. The worked answer’s check proves criterion 1 only for a fresh tile, so NFR-29 is ‘in part’; tomorrow you watch a tile past its `max-age`.

51.3 Green: two rules in Nginx

The first rule turns Nginx’s own failure into the service’s kind of answer. `error_page` sends a 502 (no service) or a 504 (a service too slow) to a named location, which writes the same problem document the service writes for N-22:

```nginx
# With the service down and no copy held, the browser gets the service's own kind of answer:
# a problem document that asks it to try again in five seconds (N-22, N-28).
error_page 502 504 = @unavailable;
location @unavailable {
    default_type application/problem+json;
    add_header Cache-Control no-store always;
    add_header Retry-After 5 always;
    return 503 '{"type":"about:blank","title":"Service Unavailable","status":503}';
}
```

The second rule, in the tile location, lets Nginx serve a stale copy when the service fails. Each word names one failure: `error` for a refused connection, `timeout` for one that hangs, and four statuses the service or Nginx may give.

```nginx
# A tile already held is served while the service or the database cannot answer, and an
# expired one is served while a fresh copy is fetched behind it.
proxy_cache_use_stale error timeout updating http_500 http_502 http_503 http_504;
proxy_cache_background_update on;
```

Copy both lines now; `updating` and the background update serve tomorrow's criterion. Run `npm run e2e` again: it passes.

KEY IDEA `http_503` joins N-28 to N-22. When the database is down, the service answers 503, and Nginx reads that answer as a reason to serve its stale copy. Two stories written weeks apart meet in one word.

51.4 Tidy: speak one error language

Compare the body Nginx now writes with the one the service writes. `Problems.Shape` sets the type `about:blank`, `no-store`, and, on a 503, `Retry-After` from `Problems.RetryAfterSeconds`, 5; ASP.NET Core supplies the title 'Service Unavailable'. Nginx now says the same, so the map reads one kind of 503 whoever sent it. Change the five seconds in one, and change it in the other.

TRY IT Run your copy with `npm start`, ask for a tile with `curl -s -o /dev/null -D - http://127.0.0.1/food.cafes/12/3339/1788.pbf`, then stop the service with Ctrl+C and ask again. Nginx and the database keep running in Docker, so the tile still arrives. Ask for `/food.cafes/12/3339/1789.pbf`, which you never asked for: 503. If your office port is not 80, use the one your `.env` names.

IN INK Commit at each green, with the messages `N-28: answer a tile never kept with the service's own 503` and `N-28: serve kept tiles while the service is down`, and push.

THE PROOF `npm run e2e` ends with your edge checks passed, the new block among them. Note in your comparison that your `checkEdge` takes one stop function where the worked answer's takes two. Tomorrow's first check: 'N-28: an expired tile arrives at once while a fresh one is fetched'.

52

Day 51 · Tuesday 5 January: N-28 ends, an old tile and a hanging database

IN SHORT Today you watch what yesterday's rules promise beyond a stopped service: an expired tile served at once, and a database that hangs. You also hold `/health` to the story's own second. N-28 ends with its second point.

Criteria 2, 4 and 5 rest on rules you already have, so today you watch them work, write down what you saw, and tighten one test to the story's number. The worked answer has no test for criterion 2 (NFR-30), proves criterion 4 only against a database that is slow or gone (NFR-32), and allows `/health` two seconds where criterion 5 asks for one (NFR-33).

EXHIBIT 52.1 Day 51 ends when you have watched a stale tile and a paused database, and DOWN arrives within a second

Goal	Evidence for N-28 c2, c4 and c5, beyond what the worked answer proves
Story	N-28 c2, c4, c5, its test 2 ('No database at start') and its Known limit
In the worked answer	`updating` and `proxy_cache_background_update on` in `nginx/default.conf.template`; `ConnectionWaitSeconds` and `QueryTimeoutSeconds` in `tessera.json`; `proxy_read_timeout 15s`; `StartTests.Starts_with_the_database_down_and_answers_503`
Look for	How `X-Cache-Status` tells a stale tile from a fresh one; whether the service's limits end before Nginx's
Leave for later	Limits on each visitor (S-30, tomorrow)
Done when	`Starts_with_the_database_down_and_answers_503`, already in your copy from days 46 and 48, passes with `/health` held to one second; your notes hold the observations below

Source: `docs/user-stories/user-stories.md`, *N-28;* `docs/requirements/non-functional-requirements.md`, *NFR-30, NFR-32 and NFR-33*

52.1 An expired tile arrives at once

The day has three parts (Exhibit 52.1). The first, criterion 2, asks Nginx to serve an expired tile while it fetches a fresh one, so the browser never waits for the database. Two words do it, `updating` and `proxy_cache_background_update on`: Nginx answers the first request after expiry from its old copy, and fetches a fresh one behind it.

To see it, the tile must expire quickly. An environment variable overrides `tessera.json` (N-21), and `npm start` hands its environment to the service, so a short `max-age` needs no change to any file:

```bash
Tiles__CacheSeconds=2 npm start
# in a second terminal:
for i in 1 2 3; do
  curl -s -o /dev/null -D - http://127.0.0.1/food.cafes/12/3339/1788.pbf | grep -i x-cache
  sleep 3
done
```

In a run for this book the first answer said `MISS` and the second, three seconds later, `STALE`, in a few milliseconds; the next request inside the two seconds said `HIT`, from the copy fetched behind it. `EXPIRED` would mean the browser waited for a fresh tile, which criterion 2 forbids. Note the words you saw.

> TRY IT Turn this into a check beyond the worked answer: a second function in `scripts/lib/edge.ts`, run by `scripts/e2e.ts` against a service started with `Tiles__CacheSeconds=2` (the first checks expect `max-age=60`), asserting `STALE` or `UPDATING` and the first answer's body. Stop the service before the second request, and the same check proves criterion 1 past its `max-age`.

52.2 A database that hangs

A database that hangs, unlike one that is down, answers nothing. `docker pause` makes one to order: it freezes every program in a container, so connections stay open and queries never return.

```bash
docker pause rebuild-db                  # your copy's database, as npm start names it
time curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1/food.cafes/12/3339/1790.pbf
time curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1/food.cafes/12/3339/1788.pbf
time curl -s -w '\n' http://127.0.0.1/health
docker unpause rebuild-db
```

The second tile, asked for earlier, arrives at once from Nginx's store (criterion 1). `/health` answers `{"status":"DOWN"}` in about a second (criterion 5); in the run for this book, 1.04 seconds, since the ping's own clock stops at one. The first tile, never asked for, answers 503 (criterion 4), but how long it takes depends on the pool.

EXHIBIT 52.2 A tile never kept, with the database paused: the limit that ends the wait depends on the pool

THE COPY ASKED	WHAT IT WAITS FOR	SEEN	WHO ANSWERS 503
No idle connection	A new connection, up to `ConnectionWaitSeconds`, 3	3.1 s	The service
An idle connection	The query, up to `QueryTimeoutSeconds`, 10, then Npgsql's attempt to cancel it, which the frozen database never answers	16.1 s	The service, alone; through Nginx, Nginx at 15 s

Source: a run for this book, 18 October 2026, a scratch copy of the worked answer with both copies asked directly; through Nginx the same tile took 18.0 s

As Exhibit 52.2 shows, a copy that borrows an idle connection waits longer than its query limit. After ten seconds Npgsql sends PostgreSQL a request to cancel the query, and a frozen database answers that no faster than the query. Nginx's `proxy_read_timeout`, 15 seconds, then runs out first, and its `error_page 502 504` writes the 503 from yesterday. The browser still gets a 503 with `Retry-After: 5`, so criterion 4 holds in effect; the wait is longer than the story suggests. Time your own three requests and note them.

> WATCH OUT The worked answer's template says `proxy_read_timeout` is 'longer than the service's own worst case, 3 seconds' wait for a connection and a 10-second query, so the service answers its own 503 first'. Against a paused database that is not so: the worst case seen was 16 seconds. With two copies (N-29), Nginx counts that timeout against the copy and tries the other, which is why the tile through Nginx took 18 seconds. Write this down as a finding for the comparison; the story's Known limit, 'a wave of slow 503s', describes the effect.

52.3 No database at start, within a second

N-28's test 2 is already in your copy: on days 46 and 48 you wrote `Starts_with_the_database_down_and_answers_503`, which starts the program with nothing listening on its database port, then times `/health` and a second tile. It allows `/health` two seconds; criterion 5 says one. Hold the test to the story's number, beyond the worked answer:

CSHARP

```
// Beyond the worked answer. N-28 c5: DOWN within a second, as the criterion says.
Assert.True(clock.Elapsed < TimeSpan.FromSeconds(1), $"/health took {clock.Elapsed}");
```

BASH

```
dotnet test --project tests/Tessera.IntegrationTests --filter-class '*.StartTests'
```

In a run for this book the tightened test passed three times in three: with nothing listening, the connection is refused at once, and the ping answers 'no' well within its second. A hanging database takes the full second, as the paused run showed, so keep the bound at one and no lower. The test runs `dotnet` by name, so `dotnet` must be on your `PATH`.

> IN INK Commit at each green, with the message `N-28: hold /health to one second with no database at start`, add your notes on the stale tile and the paused database to your comparison file, and push.

> THE PROOF `StartTests` passes with the one-second bound, and your notes record a stale tile, a paused database and a `DOWN` within a second. N-28 is done, and with it Part D; the burn-up reads 45 of 52. Tomorrow's first check: 'S-30: a flood from one address meets 429'.

53

Day 52 · Wednesday 6 January: S-30 begins, a steady rate for each visitor

IN SHORT Today Nginx counts each office machine's tile requests by its address, lets a fresh view's burst through, and answers anything beyond a steady 100 a second with 429. S-30 is worth 2 points; today is the first.

A rate limit shares the service fairly: each office machine may ask for 100 tiles a second, with room for a burst of 300, and the rest wait a second. Part E guards the door. S-07 already shuts it to strangers; S-30 shares the room among the office machines inside, with today's rate and tomorrow's cap on requests at once. Both live in Nginx, as ADR 0007 decides.

EXHIBIT 53.1 Day 52 ends when a flood from one address meets 429, and another address passes

Goal	A rate of 100 tiles a second for each office machine, with a burst of 300, and 429 beyond it as a problem document
Story	S-30 c1, c2 and c4; its tests 1 and 2
In the worked answer	`nginx/default.conf.template`: `map $remote_addr $visitor`, `limit_req_zone`, `limit_req`, `limit_req_status`, `@busy`; the S-30 flood in `checkEdge`; `docs/adr/0007-limits-in-nginx.md`
Look for	Why the key is the address, not the connection; why the flood comes from `127.0.0.6`, an address no other check uses
Leave for later	The cap of 32 at once, and the machine itself going free (tomorrow)
Done when	The S-30 flood block of `checkEdge` passes in `npm run e2e`: at least 300 of 600 answer 200, the rest 429 with `Retry-After: 1` and, beyond the worked answer, `Cache-Control: no-store`; `127.0.0.5` still gets 200

Source: `docs/user-stories/user-stories.md`, *S-30;* `docs/requirements/security-requirements.md`, *SR-20, SR-21 and SR-23*

53.1 Read: why a burst, and why an address

A ***rate limit*** caps how many requests one visitor may make in a second. A fresh view of nine layers asks for about a hundred tiles at once, and steady panning asks for far fewer, so a limit that refused the first hundred would break the map on every opening. Nginx therefore keeps a ***burst***: 300 requests a visitor may send above the steady rate before the limit bites. Past the burst, each further request answers 429, 'Too Many Requests', until the rate falls back. *Sixteen Tiles*, chapter 4, gives the reasons behind each number.

The count follows the machine's address, not the connection, since a browser opens several connections at once and a script could open more. US-35's mirrored networking makes this possible: Nginx sees each office machine's own address, not the Windows machine's. Exhibit 53.1 lists the pieces.

53.2 Red: a flood from one address

Add the flood to `checkEdge`, after the S-07 check. Six hundred requests for a kept tile leave from `127.0.0.6`, over eight connections. Linux answers every `127.x.x.x` address on the machine itself, and your `scripts/e2e.ts` sets the office to `127.0.0.4/30` (day 34), so `127.0.0.4` to `127.0.0.7` all count as office machines.

```ts
const flood = new Agent({ keepAlive: true, maxSockets: 8 });
const answers = await Promise.all(Array.from({ length: 600 },
  () => ask(tile, {}, 'GET', '127.0.0.6', flood)));
flood.destroy();
const busy = answers.filter(answer => answer.status === 429);
assert.ok(busy.length > 0, 'no request met the limit');
assert.ok(answers.filter(answer => answer.status === 200).length >= 300,
  'the burst of 300 did not pass');
assert.equal(busy[0]?.headers['content-type'], 'application/problem+json');
assert.equal(busy[0]?.headers['retry-after'], '1');
// Beyond the worked answer: S-30 c2 asks for no-store too (SR-21).
assert.equal(busy[0]?.headers['cache-control'], 'no-store');
// Another office machine is not slowed by the first one's limit.
assert.equal((await ask(tile, {}, 'GET', '127.0.0.5')).status, 200);
```

Build and run the browser tests as on day 50. The check fails with 'no request met the limit'. Every request is a hit on a kept tile, so Nginx answers all 600 without asking the service, and nothing counts them. The line marked 'beyond the worked answer' reads the one header criterion 2 names that the worked answer's check leaves out, which is why SR-21 is 'in part' there.

53.3 Green: count each visitor

Three pieces make the rate, and each sits where the worked answer puts it. At the top of the template, outside the server block, name the key and the zone that counts by it:

```nginx
# S-30: each office machine is counted by its own address. This machine's own requests, such
# as npm start's health check, have an empty key, and Nginx counts nothing for an empty key.
map $remote_addr $visitor {
    127.0.0.1 "";
    default $binary_remote_addr;
}
# 100 tiles a second for each visitor, with room for a burst of 300: one fresh view of nine
# layers asks for about a hundred tiles at once. 10 MB holds about 160,000 addresses.
limit_req_zone $visitor zone=visitor:10m rate=100r/s;
```

In the server block, choose 429 for a refusal and send it to a named location that writes the problem document:

```nginx
    limit_req_status 429;
    error_page 429 = @busy;
# …
    location @busy {
        default_type application/problem+json;
        add_header Cache-Control no-store always;
        add_header Retry-After 1 always;
        return 429 '{"type":"about:blank","title":"Too Many Requests","status":429}';
    }
```

Last, in the tile location, apply the rate to tiles alone:

```nginx
        # S-30: past the burst, a visitor's tiles answer 429 until the rate catches up.
        limit_req zone=visitor burst=300 nodelay;
```

`nodelay` answers the burst at once rather than spacing it out at the steady rate, so a fresh view fills in one go. Run `npm run e2e` again: it passes.

> **WATCH OUT** The order of checks in `checkEdge` matters. The flood leaves `127.0.0.6` over its limit for a few seconds, so any later check that asked from that address would meet 429 through no fault of its own. Give each limit check an address of its own, as the worked answer does.

> **KEY IDEA** Count by machine, not by connection. The limit follows the address, so opening more connections cannot dodge it.

> **IN INK** Commit at each green, with the message `S-30: hold each office machine to 100 tiles a second, with a burst of 300`, and push.

> **THE PROOF** The flood block passes, with `127.0.0.5` answered straight after it, and every earlier check in `checkEdge` still passes. Tomorrow's first check: 'S-30: forty at once from one address leave eight or more answered 429'.

54

Day 53 · Thursday 7 January: S-30 ends, thirty-two at once

> IN SHORT Today Nginx caps each office machine at 32 requests in hand at once, and you prove that the machine Tessera runs on meets no limit at all. S-30 ends today with its second point, the last story before tomorrow's review.

A rate counts requests over time; a cap counts requests open at the same moment, and S-30 needs both. A browser waiting on slow tiles sends few requests a second, yet holds many open, each a query the database must run. Today's cap closes that gap. The worked answer proves criterion 3; criterion 5, the machine itself going free, has no test there (SR-24), so you write one.

EXHIBIT 54.1 Day 53 ends when forty requests at once meet the cap, and this machine's own flood meets none

Goal	At most 32 requests in hand for each office machine, the 33rd answered 429; no limit on 127.0.0.1
Story	S-30 c3 and c5; its test 3, 'Thirty-two at once'
In the worked answer	`limit_conn_zone`, `limit_conn connections 32` and `limit_conn_status` in `nginx/default.conf.template`; the forty-at-once block of `checkEdge`; the slow view `food.slow_cafes` in `scripts/e2e.ts`
Look for	Why the forty use `keepAlive: false` and room for 64 sockets; why the tile, `1/1/0`, is one no check has asked for
Leave for later	The second copy of the service (N-29, days 55 and 56)
Done when	The forty-at-once block of `checkEdge` passes in `npm run e2e`, and your check that 600 requests from 127.0.0.1 meet no 429 passes

Source: `docs/user-stories/user-stories.md`, *S-30;* `docs/requirements/security-requirements.md`, *SR-22 and SR-24*

54.1 Red: forty at once

Forty requests for one slow tile must each hold a connection of their own, or Nginx sees fewer than forty open. So the check switches off keep-alive, which would let one connection carry request after request, and allows 64 sockets:

```ts
// The tile, 1/1/0, is one no check has asked for, so every request waits on the slow query
// rather than meeting a kept copy; the address, 127.0.0.7, has met no limit yet.
const many = new Agent({ keepAlive: false, maxSockets: 64 });
const forty = await Promise.all(Array.from({ length: 40 },
  () => ask(`${base}/food.slow_cafes/1/1/0.pbf`, {}, 'GET', '127.0.0.7', many)));
many.destroy();
const refused = forty.filter(answer => answer.status === 429).length;
assert.ok(refused >= 8, `${refused} of 40 refused`);
assert.equal(forty.filter(answer => answer.status === 200).length, 40 - refused);
```

Run `npm run e2e`. It fails with '0 of 40 refused'. N-24's lock holds all forty behind one query of two seconds, and then every one of them receives the tile.

54.2 Green: a cap by the same key

The cap counts by `$visitor`, yesterday's key, so this machine's empty key goes free here too:

```nginx
# The count of requests each visitor has open at once, by the same key, so this machine is
# exempt here too; the cap itself is in the server block.
limit_conn_zone $visitor zone=connections:10m;
# …
    # S-30: at most 32 requests at once from one visitor. Past either limit, the answer
    # is 429.
    limit_conn connections 32;
    limit_conn_status 429;
```

The cap sits in the server block, not the tile location, so it covers every address: the map's page and files and `/health` as well as tiles. Yesterday's `error_page 429 = @busy` already writes the problem document. Run the check again; at least eight of the forty now answer 429. The check says 'at least', since a request that finishes early frees its place for another: write the bound the arithmetic guarantees, never the number one run gave. Exhibit 54.1 names the one check left.

54.3 Beyond the worked answer: the machine itself goes free

Criterion 5 says no limit applies to the machine Tessera runs on, so `npm start`'s own checks never meet one. The worked answer's checks ask from 127.0.0.1 often, but never in a flood, so nothing proves the empty key works. Add this block to `checkEdge`, after the first flood and the request from `127.0.0.5`:

```ts
// Beyond the worked answer: S-30 c5 has no check there (SR-24).
const home = new Agent({ keepAlive: true, maxSockets: 8 });
const own = await Promise.all(Array.from({ length: 600 },
  () => ask(tile, {}, 'GET', '127.0.0.1', home)));
home.destroy();
assert.equal(own.filter(answer => answer.status === 429).length, 0,
  'this machine met a limit');
```

See it fail before you trust it: change `127.0.0.1 "";` in the `map` block to `127.0.0.9 "";` and run. In a run for this book the check then failed with 'this machine met a limit': 277 of the 600 answered 429. Put it back.

> IN INK Commit at each green, with the messages `S-30: cap each office machine at 32 requests at once` and `S-30: check this machine's own flood meets no limit`, and push.

> THE PROOF Both S-30 blocks pass in `npm run e2e`, with your exemption check between them, and S-30 is done; the burn-up reads 47 of 52. Tomorrow is the second plan's second review; write Monday's first test in your notes: 'N-29: puts the two copies on ports 7800 and 7801 unless told otherwise'.

55

Day 54 · Friday 8 January: the second plan's second review

> IN SHORT Today you run every test from a clean copy, show that the service stays steady when the database is not, count the points, and plan the last six days. You write no story code.

Part D is done when you can stop the database in front of someone and the map stays. The second review closes the stretch that began on 21 December, across four holidays: the rest of Part D, N-22 to N-28, and S-30 from Part E. It follows the delivery plan's four steps: every test from a clean start, the work shown with its evidence, the fortnight's comparison notes read, and the next stretch planned.

EXHIBIT 55.1 Day 54 ends with Part D shown, the burn-up read, and the last six days planned

Goal	Part D and S-30 shown in your copy; the points counted; the last six days written into your plan
Story	None new; the review covers N-21 to N-28 and S-30
In the worked answer	`docs/guides/delivery-plan.md`, 'The weekly and daily rhythm' and 'Tracking progress with a burn-up'; `docs/requirements/non-functional-requirements.md`; `docs/requirements/security-requirements.md`, SR-20 to SR-24
Look for	Each criterion of these stories marked 'No test yet' or 'in part', and whether your copy now proves it
Leave for later	N-29's two copies and US-36's clean places (the last six days)
Done when	`npm test` passes in a fresh clone; the database goes and the map stays, shown by hand; the plan is committed with your notes; the tag `review-d` is pushed

Source: `docs/guides/delivery-plan.md`; *the second plan*

55.1 The work passes from a clean copy

A clean copy proves that nothing left on your machine from earlier days makes the tests pass. Clone your repository afresh, as on day 43, and run everything; Exhibit 55.1 lists the rest of the day.

```bash
cd ~/src/tessera-rebuild && git status -sb   # nothing to commit, in step with origin
git clone https://github.com/YOUR-NAME/tessera-rebuild.git /tmp/review-d
cd /tmp/review-d && npm ci && npm test
```

`npm test` builds the map and the service, runs the three .NET test projects (with `PoolTests` and the tightened `StartTests`), starts fresh databases for the browser tests, and runs the edge checks last, S-30's limits among them. If it fails, the review has found its first task: fix it in a commit of its own before anything else.

55.2 The database goes, the map stays

Part D's promise is a state anyone can see, so show it by hand as well as by test. Start your copy, open the map, and pan about so Nginx keeps some tiles. Then stop the database and watch each story answer for itself:

```bash
npm start                                  # in one terminal
docker stop rebuild-db                     # in another: the database npm start runs
curl -si http://127.0.0.1/health | head -1  # 503, within a second (N-25, N-28 c5)
curl -s -o /dev/null -D - http://127.0.0.1/food.cafes/12/3339/1788.pbf | head -1
docker start rebuild-db
```

The tile you viewed still arrives, from Nginx's store (N-28); a tile you never viewed answers 503 with `Retry-After: 5` (N-22); `/health` says `DOWN` within a second (N-25). Start the database again and ask `/health` every few seconds: it returns to `UP` with no restart of the service, since the pool opens new connections and the list of layers survived the outage (N-23 c4). The service stays steady when the database is not.

EXHIBIT 55.2 The second review's evidence: Part D's stories and S-30, and the tests that prove each

STORY	IN THE WORKED ANSWER	YOURS BEYOND IT
N-21	`SettingsTests`, `StartTests`	Five sections; a variable with two underscores; a misspelt key named with the rest (days 40–41)
N-22	`DatabaseErrorTests`; the two `Answers_503_…` in `CatalogueTests`	`Answers_500_and_logs_the_cause` (day 44)
N-23	Both `CatalogueTests`; `StartTests`	The map's page with no database (day 46)
N-24	`checkEdge`: twenty requests for one slow tile	The first asker leaves, the rest stay (day 47)
N-25	`Says_up_when_the_service_can_serve_tiles`; `Reports_degraded_until_the_first_list_loads`	`no-store` on `DOWN`; `database.test.ts` (day 48)
N-26	`PoolTests`, three tests	`Hands_the_tile_query_the_requests_own_cancel_signal` (day 49)
N-28	`checkEdge`, with the service stopped; `StartTests`	`/health` held to one second; notes on a stale tile and a paused database (days 50–51)
S-30	`checkEdge`: the flood and the forty	`no-store` on 429; this machine's own flood (days 52–53)

Source: `docs/requirements/non-functional-requirements.md`; `docs/requirements/security-requirements.md`; *this book, days 40 to 53*

Read Exhibit 55.2 against the requirements files row by row. One finding needs a sentence of its own in your notes: day 51's paused database outlasted Nginx's 15 seconds, so the template's comment that the service answers its own 503 first does not hold for a database that hangs.

55.3 Read the burn-up

Only finished stories count. The release carried 30 points; Part C added 4, Part D 11 and S-30 2, so the line stands at 47 of 52 tonight, eleven points since the first review. Set your actual figure beside the plan's. The last stretch holds five points in five days and no day in hand, so a gap found today must be closed by a decision today, recorded with its reason, never by shortening the last review.

55.4 Plan the last six days

Two stories remain, and the last review. Write them into your plan with the first test of each day, as Exhibit 55.3 does:

EXHIBIT 55.3 The last six days: two stories in five, and the third review on the sixth

DAY	DATE	STORY	FIRST TEST
55	Mon 11 Jan	N-29 (1 of 2)	‘puts the two copies on ports 7800 and 7801 unless told otherwise’
56	Tue 12 Jan	N-29 (2 of 2)	With the first copy stopped, six fresh tiles answer 200
57	Wed 13 Jan	US-36 (1 of 3)	`test_runs_steps_in_the_order_written_and_hands_back_the_context`
58	Thu 14 Jan	US-36 (2 of 3)	`test_tidies_text_into_one_form_with_single_spaces`
59	Fri 15 Jan	US-36 (3 of 3)	`test_logs_each_filter_and_each_row_it_touched`
60	Mon 18 Jan	The third review	A clean clone passes `npm test`

Source: the second plan; `docs/user-stories/user-stories.md`

IN INK Commit the plan with the message `Second review: Part D and S-30 shown; the last six days planned`, push, then tag it with `git tag -a review-d -m 'Second review of the second plan: Part D and S-30'` and push the tag.

THE PROOF `npm test` passes in a fresh clone, the database goes and the map stays, the burn-up reads 47 of 52, and the plan for the last six days is on GitHub. Monday’s first test: ‘N-29: puts the two copies on ports 7800 and 7801 unless told otherwise’.

56

Day 55 · Monday 11 January: N-29 begins, the restart loop with a fake clock

> IN SHORT Today you write the parts of N-29 that need no Docker and no service: the two ports, the wait before a restart, and the loop that starts a copy again when it stops, tested with fake copies on a fake clock. N-29 is worth 2 points; today is the first.

Every decision of N-29 is a pure function, so a fake clock tests four restarts in no time at all. N-29 runs two copies of the service, so a crash in one never blanks the map. Today's half decides which ports, how long to wait and when to stop; tomorrow's runs the copies and puts Nginx in front of both. The split follows ADR 0002's first rule: each decision is a pure function in `scripts/lib`, and the script that runs commands carries them out.

EXHIBIT 56.1 Day 55 ends when the ports, the wait and the restart loop pass their tests without Docker

Goal	Two ports from one setting; a restart wait that doubles; a loop that restarts a stopped copy and stops when told
Story	N-29 c1 (the ports), c4 and c7; its tests 1 and 4
In the worked answer	`readStartSettings` and `restartDelay` in `scripts/lib/plan.ts`; `keepRunning` and `Supervision` in `scripts/lib/supervise.ts`; their tests in `plan.test.ts` and `supervise.test.ts`
Look for	Why the first port must lie between 1,024 and 65,534; why a copy that ran for a minute resets the count; how a fake clock lets four restarts take no time
Leave for later	Nginx's second server, `start.ts` running both copies, and the edge checks (tomorrow)
Done when	'puts the two copies on ports 7800 and 7801 unless told otherwise' and 'waits a second before a restart, doubling after each quick failure up to 30' pass in `plan.test.ts`; 'starts a copy again after each stop, doubling the wait after quick failures', 'waits a second again after a copy that ran for over a minute, then doubles afresh' and 'starts nothing again once told to stop' pass in `supervise.test.ts`

Source: `docs/user-stories/user-stories.md`, *N-29;* `docs/requirements/non-functional-requirements.md`, *NFR-34, NFR-37 and NFR-40*

56.1 Two ports from one setting

The first copy listens on `TESSERA_SERVICE_PORT`, 7800 by default, as your copy has since day 41, and the second on the next port. Exhibit 56.1 gives the five tests; write the first in `scripts/lib/plan.test.ts`, where `office` already holds every required variable:

```ts
// N-29: two copies, on the port given and the next one.
test('puts the two copies on ports 7800 and 7801 unless told otherwise', () => {
  const result = readStartSettings(office);
  assert.deepEqual(result.ok ? result.value.servicePorts : [], ['7800', '7801']);
  const moved = readStartSettings({ ...office, TESSERA_SERVICE_PORT: '8100' });
  assert.deepEqual(moved.ok ? moved.value.servicePorts : [], ['8100', '8101']);
  // 80 needs root, 65535 leaves no port for the second copy, and 'abc' is no port at all.
  for (const bad of ['80', '65535', 'abc']) {
    assert.equal(readStartSettings({ ...office, TESSERA_SERVICE_PORT: bad }).ok, false, bad);
  }
});
```

Run `node --test scripts/lib/plan.test.ts`. It fails: your `StartSettings` has no `servicePorts`. Add the field as a pair, `readonly [string, string]`, in place of a single port if your copy holds one, and fill it in `readStartSettings`, with one more problem in its list for a port out of range: `TESSERA_SERVICE_PORT: '80' is not a port from 1024 to 65534`. The first port defaults to `'7800'`, and the second is the first plus one.

> FROM SCHOOL A port is a 16-bit number, so the highest is $2^{16} - 1$, 65,535. The second copy needs the first port plus one, so the first may be 65,534 at most. Ports below 1,024 need root on Linux, which is why Nginx, not the service, answers on 80.

56.2 A wait that doubles

A copy that cannot start, such as one with a broken setting, fails again at once. Restarting it at once would spin, filling the log a thousand times a minute. The cure is a ***back-off***: wait a second, then double the wait after each quick failure, up to 30 seconds. Test the arithmetic alone:

```ts
test('waits a second before a restart, doubling after each quick failure up to 30', () => {
  assert.deepEqual([0, 1, 2, 3, 4, 5, 6, 10].map(restartDelay),
    [1000, 1000, 2000, 4000, 8000, 16000, 30000, 30000]);
});
```

```ts
export function restartDelay(quickFailures: number): number {
  return Math.min(30_000, 1000 * 2 ** Math.max(0, quickFailures - 1));
}
```

56.3 The loop, with a fake world

The loop that keeps a copy running needs five things from outside: start a copy and learn how it ended, know whether to stop, wait, read the clock, and write a line. Real ones would make a test wait minutes, so the loop takes them as one parameter, a record of functions called `Supervision`, and the test passes fakes. Write `scripts/lib/supervise.test.ts` first. Its fake world holds a clock that moves only when a copy 'runs' or the loop 'waits', and it records each wait:

```ts
// world(runs): a fake Supervision whose start() adds the next run's length to a fake clock
// and returns 'code 1', whose wait() records each wait and moves the clock, and whose say()
// keeps each line; stopping() turns true once every run is used.

test('starts a copy again after each stop, doubling the wait after quick failures',
  async () => {
  const fake = world([100, 100, 100, 100, 100]);
  await keepRunning('port 7800', fake.world);

  assert.equal(fake.started(), 5);
  assert.deepEqual(fake.waits, [1000, 2000, 4000, 8000]);
  assert.equal(fake.lines[0],
    'The copy on port 7800 stopped (code 1); starting it again in 1 s.');
});
```

Add the other two tests as the worked answer writes them: `[100, 100, 90_000, 100, 100]` must give waits of `[1000, 2000, 1000, 2000]`, since a copy that ran 90 seconds counts as healthy; and `[100]` must start once and wait never. Run `node --test scripts/lib/supervise.test.ts`: all three fail, since `keepRunning` does not exist. Then write it in `scripts/lib/supervise.ts`:

```ts
export async function keepRunning(name: string, world: Supervision): Promise<void> {
  let quickFailures = 0;
  while (!world.stopping()) {
    const began = world.now();
    const ended = await world.start();
    if (world.stopping()) return;
    quickFailures = world.now() - began > 60_000 ? 1 : quickFailures + 1;
    const wait = restartDelay(quickFailures);
    world.say(
      `The copy on ${name} stopped (${ended}); starting it again in ${wait / 1000} s.`);
    await world.wait(wait);
  }
}
```

WATCH OUT Check `stopping()` again after `start()` returns. Ctrl+C stops the copy, so `start()` resolves; without the second check, the loop would read that stop as a crash and start the copy again, the one thing criterion 7 forbids.

IN INK Commit at each green, adding `scripts/lib/supervise.ts scripts/lib/supervise.test.ts`, with the messages `N-29: put the two copies on the first port and the next`, `N-29: double the wait after each quick failure, up to 30 seconds` and `N-29: restart a stopped copy, and stop when told`, and push.

THE PROOF `npm run test:scripts` passes, with the two new tests in `plan.test.ts` and three in `supervise.test.ts`, with no Docker running. Tomorrow's first check: 'N-29: with the first copy stopped, six fresh tiles answer 200'.

57

Day 56 · Tuesday 12 January: N-29 ends, two copies behind Nginx

IN SHORT Today `npm start` runs and restarts two copies of the service, and Nginx shares requests between them, stepping round one that fails. N-29 ends with its second point, and with it Part E.

Two copies on one machine guard against a failed program: Nginx steps round the one that fails, and `npm start` brings it back. Three files change: Nginx's template gains a second server, `scripts/start.ts` runs one `keepRunning` loop for each copy, and `scripts/e2e.ts` starts two copies so that `checkEdge` can stop them one at a time. Criteria 3 and 5 have no test in the worked answer (NFR-36, NFR-38); the second earns a check of your own.

EXHIBIT 57.1 Day 56 ends when one copy stopped costs nothing, and both stopped still leave the kept tiles

Goal	Two copies behind Nginx, taking turns; a request that meets a failed copy moves to the other; Ctrl+C stops both
Story	N-29 c1 to c7; its tests 2 and 3
In the worked answer	`upstream tessera`, `proxy_next_upstream` and `proxy_connect_timeout` in `nginx/default.conf.template`; `nginxArguments` in `scripts/lib/plan.ts`; `scripts/start.ts`; `scripts/e2e.ts`; the N-29 block of `checkEdge`
Look for	Why only `error` and `timeout` count against a copy; why `stopCopy` catches twice
Leave for later	Nothing: the start-up task, which the story leaves open to talk, came forward on day 35
Done when	`checkEdge`'s N-29 block passes: six fresh tiles answer 200 with `X-Cache-Status: MISS` and `/health` says `UP` with the first copy stopped; with both stopped, the kept tile arrives and a tile never kept answers 503. 'runs nginx on the machine's network, with both ports filled in' passes in `plan.test.ts`, and your outage check passes in `e2e.ts`

Source: `docs/user-stories/user-stories.md`, *N-29;* `docs/requirements/non-functional-requirements.md`, *NFR-34 to NFR-40*

57.1 Red: stop one copy, then the other

The check comes from the story's tests 2 and 3 (Exhibit 57.1). Give `checkEdge` a second stop function: day 50's single stop becomes the worked answer's `stopFirst` and `stopSecond`. With the first copy stopped, six tiles no check has asked for must each arrive, freshly built, and `/health` must still say `UP`:

```ts
await stopFirst();
for (let attempt = 0; attempt < 6; attempt++) {
  const fresh = await ask(`${base}/food.cafes/22/1/${attempt}.pbf`,
    { 'Accept-Encoding': 'identity' });
  assert.equal(fresh.status, 200, `tile ${attempt} with one copy stopped`);
  assert.equal(fresh.headers['x-cache-status'], 'MISS');
}
const health = await ask(`${base}/health`);
assert.equal(health.status, 200);
assert.match(health.body.toString(), /"UP"/);

await stopSecond();
// … day 50's kept tile and the 503 for a tile never kept follow here
```

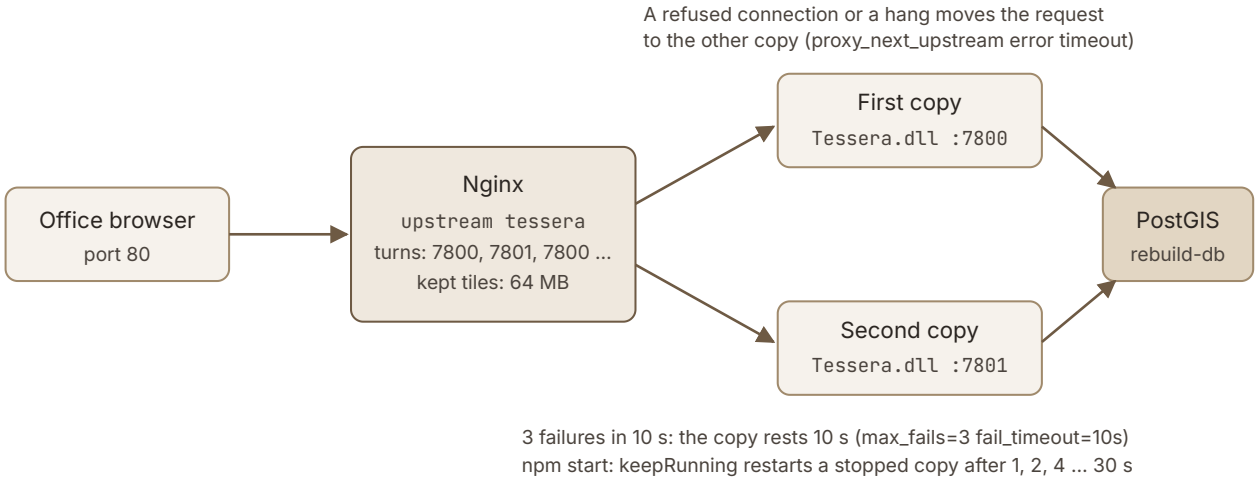
Run `npm run e2e`. Your `e2e.ts` still starts one copy, so the first fresh tile answers 503: with its only copy gone, Nginx has nowhere to send the request, and day 50's `@unavailable` answers.

57.2 Green: a group of two

57.2.1 Nginx

Your template has named its one server `upstream tessera` since day 14. Add the second, and two settings to each. Nginx sends requests to the members in turn (Exhibit 57.2), a plan called ***round robin***, and stops sending to one that fails, a ***passive health check***: it watches real requests rather than asking a health address.

EXHIBIT 57.2 Nginx shares tiles between two copies, and npm start restarts the one that stops



Source: `nginx/default.conf.template`; `scripts/start.ts`; `scripts/lib/supervise.ts`

```nginx
upstream tessera {
    server 127.0.0.1:${TESSERA_SERVICE_PORT} max_fails=3 fail_timeout=10s;
    server 127.0.0.1:${TESSERA_SECOND_SERVICE_PORT} max_fails=3 fail_timeout=10s;
    keepalive 16;
}
# … in the server block:
    proxy_connect_timeout 1s;
    proxy_read_timeout 15s;
    proxy_next_upstream error timeout;
```

Only a crash or a hang counts against a copy: a database outage reaches both copies at once, so `http_503` appears in `proxy_cache_use_stale` but not in `proxy_next_upstream`. `proxy_connect_timeout 1s` is short on purpose: a copy on the same machine connects in far less, so one that has not is down, and the request moves on.

57.2.2 The scripts

`nginxArguments` in `plan.ts` now takes both ports and passes `TESSERA_SECOND_SERVICE_PORT` to the container, and its test, 'runs nginx on the machine's network, with both ports filled in', checks both. `scripts/e2e.ts` starts two copies, one on each free port, and hands `checkEdge` a stop for each. `scripts/start.ts` runs yesterday's loop once for each port, with the real world:

```ts
const runCopy = (port: string) => keepRunning(`port ${port}`, {
  start: async () => {
    const copy = $({ nothrow: true, stdio: 'inherit', env: { ...process.env,
      DATABASE_URL: reader, Urls: `http://127.0.0.1:${port}` } })`dotnet ${program}`;
    copiesNow.set(port, copy);
    const ended = await copy;
    return ended.exitCode === null ? `signal ${ended.signal}` : `code ${ended.exitCode}`;
  },
  stopping: () => stopping,
  wait,
  now: Date.now,
  say: line => console.error(line),
});
```

`Urls` overrides the port in `tessera.json`, as any variable overrides its key (N-21), so one built program serves both ports. Ctrl+C sets `stopping`, cuts short any wait before a restart through an `AbortController`, and stops both copies. Read `stopCopy` in the worked answer: a copy that has already ended makes zx throw at once, and one that ends while the signal is on its way makes the kill fail later, so the function catches both, and one stopped copy can never keep the other running. Run `npm run e2e` again: it passes.

57.3 Beyond the worked answer: a database outage rests no copy

Criterion 5 has no check in the worked answer. The check must ask within the ten seconds a rest would last, so it cannot wait for the database to come back first. Instead it reads who answers `/health` while the database is still down: a copy writes `{"status":"DOWN"}`, while Nginx, with both copies rested, writes its own problem document. Put it in `scripts/e2e.ts` after the browser tests and before `checkEdge`, which stops the copies:

```ts
// Beyond the worked answer: N-29 c5 (NFR-38). A database outage rests neither copy.
await $`docker stop ${container}`;
for (let attempt = 0; attempt < 6; attempt++) {
  const down = await fetch(`${base}/food.cafes/21/0/${attempt}.pbf`);
  if (down.status !== 503) {
    throw new Error(`tile ${attempt} with the database down: ${down.status}`);
  }
}
// A rested copy leaves Nginx no copy to ask, and Nginx writes its own 503.
for (let ask = 0; ask < 2; ask++) {
  const health = await (await fetch(`${base}/health`)).text();
  if (health !== '{"status":"DOWN"}') throw new Error(`/health through Nginx: ${health}`);
}
await $`docker start ${container}`;
for (const port of servicePorts) {
  if ((await waitForHealth(`http://127.0.0.1:${port}/health`, 60)) !== 'up') {
    throw new Error(`the copy on ${port} did not say UP`);
  }
}
```

See it fail: add `http_503` to `proxy_next_upstream` and run. In a run for this book the check then stopped with '/health through Nginx' and Nginx's own problem document, `{"type":"about:blank","title":"Service Unavailable","status":503}`: the six 503s had rested both copies. Put the line back. An earlier draft of this check started the database again first and then asked for a fresh tile; it passed even with the fault, because the restart took longer than the ten-second rest.

> TRY IT Watch criterion 3 rather than test it. With `npm start` running, stop the first copy by its port (`fuser -k 7800/tcp`), then ask for fresh tiles in a loop and read the terminal: `npm start` prints 'The copy on port 7800 stopped' and starts it again after a second, while every tile still arrives.

> IN INK Commit at each green, with messages such as `N-29: put two copies behind Nginx, taking turns`, and push.

> THE PROOF `npm run e2e` ends with the line the worked answer prints: 'Nginx checks passed: the office-only rule, the limit for each visitor, the cap on requests at once, one query for a crowd, kept tiles, 304, gzip, HEAD, malformed addresses, a second copy taking over, and tiles served with both copies down.' Your outage check passes before it. Part E is done; the burn-up reads 49 of 52. Tomorrow's first test: `test_runs_steps_in_the_order_written_and_hands_back_the_context`.

58

Day 57 · Wednesday 13 January: US-36 begins, a chain of filters

IN SHORT Today you take apart the pipeline's cleaning, which you copied whole on day 7, and type it again test first: steps joined with `>>` into a chain, and the four filters that judge a place's shape. US-36 is worth 3 points; today is the first.

A table is cleaned by one chain of pure steps, and a stop is a value the chain carries, not an exception. Part F holds one story, in Python, in `etl/`, which both builds of Tessera share. US-36 is the part of the pipeline that cleans: a chain of ***filters*** between reading a file and writing a seed script, so faults in a publisher's data stop at the pipeline rather than reach the map. For three days you remove that part from your copy, a module at a time, and type it again test first: today the filters that judge the shape, tomorrow the text and the whole table, Friday the log and a run from end to end.

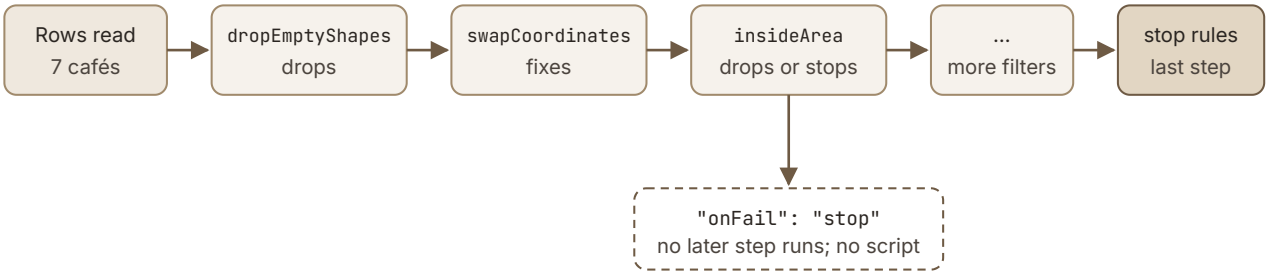
EXHIBIT 58.1 Day 57 ends when a chain of steps runs in order, stops at a stop, and four shape filters pass

Goal	A `Step` that joins with `>>`; `filter_step`, which turns a judge into a step; `dropEmptyShapes`, `repairShapes`, `swapCoordinates` and `insideArea`
Story	US-36 c1 and c2; its test 1, 'Each filter alone', for the shape filters
In the worked answer	`etl/src/tessera_etl/kernel/pipes.py`; `core/steps.py` (`chain`, `filter_step`, `failures`, `fixes`); `core/rows.py`; `core/filters/drop_empty_shapes.py`, `repair_shapes.py`, `swap_coordinates.py`, `inside_area.py`; `etl/tests/test_chain.py`; `etl/tests/test_filters.py`; `etl/tests/helpers.py`
Look for	Why a stop is a field of the context, not an exception; why a filter judges a whole table at once with a mask
Leave for later	The text and duplicate filters, the stop rules and the pipeline file's faults (tomorrow); the log and the geodatabase (Friday)
Done when	`test_runs_steps_in_the_order_written_and_hands_back_the_context`, `test_runs_no_step_after_a_stop`, `test_joins_a_list_of_steps_into_one_chain` and `test_runs_the_second_filter_on_what_the_first_kept_and_joins_their_logs` pass in `test_chain.py`; `test_drops_a_row_with_no_shape_or_an_empty_one`, `test_repairs_an_area_whose_edge_crosses_itself`, `test_swaps_a_point_whose_longitude_and_latitude_changed_places`, `test_keeps_a_line_that_runs_out_of_the_area` and `test_stops_at_the_first_bad_row_when_told_to_and_keeps_every_row` pass in `test_filters.py`

Source: `docs/user-stories/user-stories.md`, *US-36;* `docs/requirements/functional-requirements.md`, *FR-13 and FR-14*

EXHIBIT 58.2 A table passes through its filters in order, and a stop ends the chain

Each step takes a Cleaning (rows, log, stop) and returns a new one; >> joins two steps.



Every fix, drop and stop adds one line to the log, naming the row, the filter and the reason.

Source: `etl/src/tessera_etl/kernel/pipes.py` ; `etl/src/tessera_etl/core/steps.py` ; `etl/tests/test_run.py`

58.1 Read: one step, one context

A step takes one value and returns a changed copy of it, as Exhibit 58.2 draws. In the pipeline that value is a `Cleaning`, a frozen record of a table's rows, its log so far and any reason to stop. Because every step takes and returns the same kind of value, any two steps can be joined, and a chain of eight filters is one step too. This is function composition, *g*(*f*(*x*)), with `>>` borrowed from F# to mean 'then'. Read `kernel/pipes.py`, which knows nothing of places, then `core/steps.py`, which fills it in. Exhibit 58.1 lists the tests the day ends with.

58.2 Red, then green: steps that join

Delete `etl/src/tessera_etl/kernel/pipes.py` and `etl/tests/test_chain.py` from your copy, then write the chain's first tests again. They need no places: a tiny context, `Seen`, records the names of the steps that ran, and a step called `halt` sets its `stop`.

PYTHON

```
def test_runs_steps_in_the_order_written_and_hands_back_the_context() -> None:
    assert (note("a") >> note("b") >> note("c"))(Seen()).names == ("a", "b", "c")
    assert (note("a") >> note("b"))(Seen()).names == ("a", "b")


def test_runs_no_step_after_a_stop() -> None:
    assert (note("a") >> halt("enough") >> note("b"))(Seen()) == Seen(("a",), "enough")
```

BASH

```
cd etl
uv run pytest tests/test_chain.py
```

> WATCH OUT While a module is gone, every test that imports it fails, the golden cases among them. Run only the file you are working on while you type, and run the whole suite, `uv run pytest`, before each commit: commit only when every test passes.

Both fail on the import: there is no `Step`. Write it in `kernel/pipes.py` as a frozen dataclass holding one function, with `__rshift__` to join two:

```python
@dataclass(frozen=True, slots=True)
class Step[C: Stoppable]:
    apply: Callable[[C], C]

    def __call__(self, context: C) -> C:
        return self.apply(context)

    def __rshift__(self, then: "Step[C]") -> "Step[C]":
        def both(context: C) -> C:
            done = self(context)
            return done if done.stop is not None else then(done)

        return Step(both)
```

The stop is a value: a step that stops returns the context with its reason in `stop`, and `both` then skips every later step. No exception leaves the chain, so a test can compare the whole outcome with `==`. It is the same idea as the service's `Result`, a failure carried as a value. Add `chain` to `core/steps.py`, which folds a list of steps into one with `reduce`, starting from `IDENTITY`, the step that changes nothing, and make `test_joins_a_list_of_steps_into_one_chain` pass.

58.3 A filter is a judge and a step

Each filter judges a whole table at once. Its judge returns the table, mended where it could be, and a list of verdicts: `Fixed` for a row it mended, `Failed` for a row it cannot pass. `filter_step`, in `core/steps.py`, turns a judge into a step: it logs each verdict, drops each failed row, or, with `"onFail": "stop"`, ends the run at the first failed row.

```python
for verdict in judged.verdicts:
    match verdict:
        case Fixed(row, did):
            log.append(LogLine(row, name, Act.FIXED, did))
        case Failed(row, reason) if on_fail is OnFail.STOP:
            log.append(LogLine(row, name, Act.STOPPED, reason))
            return replace(cleaning, log=cleaning.log + tuple(log),
                           stop=f"{name} stopped the run at row {row}: {reason}.")
        case Failed(row, reason):
            log.append(LogLine(row, name, Act.DROPPED, reason))
        case _:
            assert_never(verdict)
```

> WATCH OUT `assert_never` earns its place. pyright, in strict mode, checks that every kind of verdict has a case above it; add a third kind one day and forget it here, and the type check fails before any test runs. It does for Python what the ExhaustiveMatching analyser does for `DatabaseError`'s switch in the service (day 42).

58.4 Four shape filters, test first

Take the filters in the order a pipeline file lists them. For each, delete its module in `core/filters/`, its line in `core/filters/__init__.py` and its tests in `etl/tests/test_filters.py`; then write the tests again, watch them fail, and write the module. Keep `tests/helpers.py`: its helpers, such as `place`, `shaped`, `clean`, `kept` and `log_lines`, build a table of a few rows and read the outcome. Exhibit 58.3 gives each filter its test.

EXHIBIT 58.3 Four filters judge a place's shape, each with a test that names what it does

FILTER	WHAT IT DOES	ITS TEST
`dropEmptyShapes`	Fails a row with no shape, or an empty one	`test_drops_a_row_with_no_shape_or_an_empty_one`
`repairShapes`	Rebuilds an invalid shape with shapely's `make_valid`	`test_repairs_an_area_whose_edge_crosses_itself`
`swapCoordinates`	Swaps back a shape outside the area that lands inside it when turned round; fails nothing	`test_swaps_a_point_whose_longitude_and_latitude_changed_places`
`insideArea`	Fails a shape wholly outside the area; keeps a line that runs out of it	`test_keeps_a_line_that_runs_out_of_the_area`; `test_stops_at_the_first_bad_row_when_told_to_and_keeps_every_row`

Source: `etl/src/tessera_etl/core/filters/`; `etl/tests/test_filters.py`

The first is the shortest, and shows the shape of all eight. Its judge is one line of masks, columns of true and false, joined by `|`, OR, row by row:

```python
def create(on_fail: OnFail = OnFail.DROP) -> Step[Cleaning]:
    """Builds the filter from plain values, so a test can make one without a pipeline file."""

    def judge(frame: GeoDataFrame) -> Judged:
        empty = frame.geometry.isna() | frame.geometry.is_empty
        return Judged(frame, failures(empty, "it has no shape"))

    return filter_step(NAME, on_fail, judge)
```

PYTHON

When the four pass alone, make them pass together: `test_runs_the_second_filter_on_what_the_first_kept_and_joins_their_logs` joins `swapCoordinates`, `insideArea` and `requireFields`. It needs `requireFields` from tomorrow's list; write that small filter now, or mark the test to skip until tomorrow, and say which in your notes.

```bash
uv run pytest tests/test_chain.py tests/test_filters.py
```

BASH

IN INK Commit at each green, with the messages `US-36: join pipeline steps with >>, stopping at the first stop`, `US-36: turn a judge into a filter that fixes, drops or stops` and `US-36: add the four shape filters`, and push.

THE PROOF The tests above pass, and so do the pipeline's other three checks: `uv run pyright`, `uv run ruff check .` and `uv run lint-imports`, which proves the kernel and the core still touch no file. Tomorrow's first test: `test_tidies_text_into_one_form_with_single_spaces`.

59

Day 58 · Thursday 14 January: US-36 continues, clean text and the stop rules

> IN SHORT Today the pipeline learns to tidy text, to drop a place with no name or one entered twice, to stop a table that loses too much, and to name every fault in a pipeline file at once. This is US-36's second point.

Rows are judged one by one, but a table must also be judged whole, so the stop rules run last and stop a table that lost too much. Yesterday's filters judged each row's shape; today's judge its text and its neighbours, and the stop rules, the last step of every chain, judge the whole table. The day ends with the pipeline file itself, since a filter misspelt there must stop the run before any row is read.

EXHIBIT 59.1 Day 58 ends when four more filters, the stop rules and the file's faults pass their tests

Goal	`tidyText`, `requireFields`, `dropDuplicateIds`, `dropNearDuplicates`; `stop_when`; every fault in a pipeline file named at once
Story	US-36 c1, c3 and c5; its tests 1 and 2
In the worked answer	`core/filters/tidy_text.py`, `require_fields.py`, `drop_duplicate_ids.py`, `drop_near_duplicates.py`; `core/judge.py`; `stop_when` in `core/steps.py`; `StopWhen` in `core/plan.py`; `core/pipeline.py`; `etl/tests/test_filters.py`; `etl/tests/test_pipeline.py`
Look for	Why text that tidies to nothing becomes null; how `dropNearDuplicates` measures metres from degrees; where a table's own stop rules win over the file's
Leave for later	The log and a whole run from a geodatabase (tomorrow)
Done when	In `test_filters.py`: `test_tidies_text_into_one_form_with_single_spaces`, `test_tidies_only_the_fields_named`, `test_drops_a_row_missing_a_required_field_or_holding_only_spaces`, `test_keeps_the_first_of_two_rows_with_one_id`, `test_drops_a_place_entered_twice_nearby_and_keeps_one_of_the_same_name_far_off`, `test_measures_a_degree_of_latitude_as_about_111_kilometres`. In `test_pipeline.py`: `test_stops_a_table_that_drops_more_than_ten_per_cent_by_default`, `test_lets_a_table_set_its_own_rules`, `test_ends_each_chain_from_the_file_with_the_closest_stop_rules`, `test_runs_filters_in_order_and_ends_at_the_first_stop`, `test_names_every_fault_at_once`

Source: `docs/user-stories/user-stories.md`, *US-36;* `docs/requirements/functional-requirements.md`, *FR-13, FR-15 and FR-17*

59.1 Text and neighbours

Four filters remain, and each test states its filter's rule in one example, as Exhibit 59.2 sets out. Take them in its order: delete each module and its tests from your copy, then write the test again before the module.

Exhibit 59.1 lists every test the day must pass.

EXHIBIT 59.2 Four filters judge text and neighbours; each test holds one example of its rule

FILTER	THE EXAMPLE IN ITS TEST	WHAT THE LOG SAYS
`tidyText`	`" Café \t Uno "` becomes `"Café Uno"`; a note of three spaces becomes null	Row 1: fixed by tidyText: tidied name, note
`requireFields`	Of rows named 'A', ' ' and nothing, only the first stays	name is missing
`dropDuplicateIds`	Ids 7, 8, 7 and none: the second 7 goes	Row 3: dropped by dropDuplicateIds: it repeats id '7' of row 1
`dropNearDuplicates`	'café uno ' about 11 m from 'Café Uno' goes; a 'Café Uno' 1.1 km away stays	Row 2: dropped by dropNearDuplicates: its name matches row 1, 11 m away

Source: `etl/tests/test_filters.py`

PYTHON

```python
# etl/tests/test_filters.py
def test_drops_a_place_entered_twice_nearby_and_keeps_one_of_the_same_name_far_off() -> None:
    # 0.0001 degrees of latitude is about 11 metres; 0.01 is about 1.1 kilometres.
    outcome = clean(
        drop_near_duplicates.create("name", metres=25),
        [
            place(1, 113.55, 22.19, name="Café Uno"),
            place(2, 113.55, 22.1901, name="café  uno "),
            place(3, 113.55, 22.20, name="Café Uno"),
        ],
    )

    assert kept(outcome) == [1, 3]
    assert log_lines(outcome) == [
        "Row 2: dropped by dropNearDuplicates: its name matches row 1, 11 m away"]
```

Run `uv run pytest tests/test_filters.py` after each filter.

> FROM SCHOOL A degree of latitude is about 111 kilometres everywhere on Earth, so 0.0001 degrees is about 11 metres; `test_measures_a_degree_of_latitude_as_about_111_kilometres` pins that number between 111,150 and 111,250 metres. A degree of longitude shrinks towards the poles, so `judge.metres` must work on a sphere, not a flat grid.

59.2 Red: the table that lost too much

A filter that drops rows one by one cannot see the whole table. A publisher's file with a broken column could lose half its places to `requireFields`, each drop correct, and the map would lose half a layer without a word. The ***stop rules*** look at the table after its filters: the run stops when more than `droppedOverPercent` of the rows were dropped, 10 by default, or fewer than `rowsUnder` remain, 1 by default.

The tests use ten places, so each unnamed one is ten per cent of the table, and `requireFields` drops by name:

```python
def test_stops_a_table_that_drops_more_than_ten_per_cent_by_default() -> None:
    report_ = clean_plan(plan_of(None, require_name()), ten(named=8))

    assert report_.stop == (
        "food.cafes: 2 of 10 rows were dropped (20 %), more than the 10 % allowed.")


def test_lets_a_table_set_its_own_rules() -> None:
    assert clean_plan(plan_of(rules(droppedOverPercent=25), require_name()),
                      ten(named=8)).stop is None
    # … and a table that keeps 8 of 10 rows, with rowsUnder 9, stops.
```

Delete `stop_when` from `core/steps.py` and run them with `uv run pytest tests/test_pipeline.py -k "ten_per_cent or own_rules"`: both fail, since nothing measures the table any more.

59.3 Green: one step at the end

`stop_when` is a step like any filter, so it joins the chain with `>>` and needs no special place in the program. A chain a filter stopped never reaches it.

```python
def step(cleaning: Cleaning) -> Cleaning:
    kept = len(cleaning.frame)
    percent = 0.0 if cleaning.read == 0 else 100.0 * cleaning.dropped / cleaning.read
    least, most = rules.rows_under, rules.dropped_over_percent
    if least is not None and kept < least:
        reason = f"only {kept} rows remain, fewer than the {least} required"
    elif most is not None and percent > most:
        reason = (
            f"{cleaning.dropped} of {cleaning.read} rows were dropped "
            f"({round(percent, 1):g} %), more than the {most:g} % allowed"
        )
    else:
        return cleaning
    return replace(cleaning, stop=f"{cleaning.table.table}: {reason}.")
```

Then make `test_ends_each_chain_from_the_file_with_the_closest_stop_rules` pass: `stopWhen` may sit on the whole file or on one table, and the table's own rules win. Last, `test_runs_filters_in_order_and_ends_at_the_first_stop` proves criteria 1 and 2 together: a `requireFields` set to stop ends the run at row 10, and `tidyText`, after it, never runs.

> KEY IDEA Stop by default. A run that drops more than a tenth of a table stops; a publisher who wants more loss must say so in the file, where the next reader will see it.

59.4 Every fault at once

Criterion 5 asks the pipeline to read the whole file before it reads a row, and to name every fault in one answer, so a publisher fixes the file in one pass rather than five. The test's bad file holds five faults: an output named `parks.sql`, with no four-digit number to set its order, a table name in capitals, a filter that does not exist, `dropNearDuplicates` with no `metres`, and `insideArea` with no area to judge by.

```python
found = problems(bad, area=False)

assert len(found) == 5
assert any(problem.startswith("output:") for problem in found)
assert any("schema.table" in problem for problem in found)
assert any("'dropDuplicates' is no filter" in problem for problem in found)
assert "Food.Cafes: dropNearDuplicates: needs metres." in found
assert any("insideArea: needs an area" in problem for problem in found)
```

Keep `core/pipeline.py` in your copy, and read it for how it gathers the faults: each check returns a list of problems rather than raising on the first, and the lists join. The four-digit number matters because DbUp runs the seed scripts in the order of their names (ADR 0009). Since that code came with `etl/` on day 7, the retyped test passes at once; break it on purpose, by making the check for `metres` report nothing, and watch it fail before you mend it. Do the same with the tests of single faults beside it, such as `test_refuses_a_filter_that_lacks_a_setting_it_needs` and `test_refuses_a_misspelt_setting`.

> **WATCH OUT** A misspelt setting must fail, not pass silently. `"feilds"` in place of `"fields"` would otherwise leave `requireFields` with nothing to require. The worked answer's pydantic models forbid any key they do not name (`extra="forbid"` in `core/plan.py`), so `test_refuses_a_misspelt_setting` expects two lines: 'feilds' is not a setting, and the filter needs fields. Compare day 41, where the service's settings stopped at the first misspelt key: the pipeline names them all.

> **IN INK** Commit at each green, with messages such as `US-36: stop a table that loses more than its rules allow`, and push.

> **THE PROOF** `uv run pytest` passes every test in `etl/tests`, and pyright, ruff and import-linter pass beside it. All eight filters exist. Tomorrow's first test: `test_logs_each_filter_and_each_row_it_touched`.

60

Day 59 · Friday 15 January: US-36 ends, the log and a whole run

IN SHORT Today every run of the pipeline writes a log of what it did to each row, and a whole run from a file geodatabase cleans seven cafés into one script or stops with none. US-36 ends today with its third point, and with it the last of the thirty stories.

A dropped row leaves no trace in the script, so the log is the publisher's only account of what was left out and why. Today writes it, then proves the whole pipeline from the outside: the tool runs as a program, on a fixture ***geodatabase*** of faulty places, and the test reads the files it leaves. The afternoon runs the office's own pipeline file.

EXHIBIT 60.1 Day 59 ends when a whole run cleans a geodatabase, logs each row, and stops with no script when told

Goal	A log in `db/seed-logs/` for every run; a geodatabase read by `layer`, with curves made straight, heights dropped and coded values written as their descriptions
Story	US-36 c4 and c6; its test 3, 'A geodatabase, end to end'
In the worked answer	`core/report.py`; `shell/program.py`; `core/seed_script.py` (`extract_arguments`); `etl/tests/test_run.py`; `etl/tests/conftest.py`; `etl/tests/fixtures/`; `data/macau.pipeline.json`
Look for	Why `report.log` takes the time as a parameter; the three exit codes, 0, 1 and 2; why `-dim XY` drops heights
Leave for later	Nothing: Monday is the last review
Done when	`test_logs_each_filter_and_each_row_it_touched` and `test_extracts_one_layer_flat_and_in_longitude_and_latitude` pass in `test_pipeline.py`; `test_cleans_a_geodatabase_into_one_script_and_logs_each_row` and `test_stops_at_a_bad_row_writes_no_script_and_logs_why` pass in `test_run.py`

Source: `docs/user-stories/user-stories.md`, *US-36;* `docs/requirements/functional-requirements.md`, *FR-16 and FR-18*

60.1 The log is a pure function

The log is text made from the run's results, so it can be a pure function: `report.log` in `core/report.py` takes the pipeline file's name, the file, each table's plan and outcome, the result line and the time, and returns the text; the program writes it to a file. Even the time arrives as a parameter, so the test can fix it. This is the first of the day's four tests (Exhibit 60.1).

```python
log = report.log(
    "data/macau.pipeline.json",
    file,
    [(plan, done.outcome)],
    "wrote it.",
    datetime(2026, 10, 15, 13, 0, 0, tzinfo=timezone(timedelta(hours=8))),
)

assert "Tessera ETL log, 2026-10-15 13:00:00 +08:00" in log
assert "  tidyText: kept 10, fixed 0, dropped 0" in log
assert "  requireFields: kept 9, fixed 0, dropped 1" in log
assert "  Row 10: dropped by requireFields: name is missing" in log
assert log.endswith("Result: wrote it.\n")
```

That is the heart of `test_logs_each_filter_and_each_row_it_touched`. Delete `core/report.py` from your copy, write the test again, watch it fail on the import, then write `report.log`: a heading with the time and the files, then for each table the rows read, one line of counts for each filter in order, and one line for each row a filter fixed, dropped or stopped at.

60.2 Reading a geodatabase

A file geodatabase, a folder ending `.gdb`, is the format many publishers' desktop map programs save in. It may hold curves, heights and coded values, such as 1 standing for 'Café'. The pipeline hands such a source to ogr2ogr, GDAL's own program, which writes each table into a GeoPackage that pyogrio then reads. `extract_arguments` decides the arguments, and a pure test checks them. It came with `etl/` on day 7; reread it, since it proves criterion 6:

```python
def test_extracts_one_layer_flat_and_in_longitude_and_latitude() -> None:
    arguments = seed_script.extract_arguments(
        "/data/macau.gdb", "/work/cafes.gpkg",
        table("food.cafes", layer="Cafes", where="kind = 'cafe'"))

    assert arguments[:5] == ("-f", "GPKG", "/work/cafes.gpkg", "/data/macau.gdb", "Cafes")
    assert "-resolveDomains" in arguments
    assert arguments[-2:] == ("-where", "kind = 'cafe'")
    assert arguments[arguments.index("-dim") + 1] == "XY"
```

`-resolveDomains` writes coded values as their descriptions; `-dim XY` keeps two numbers for each point, dropping the height. Both meet criterion 6 without one line of Python that knows about curves or heights.

60.3 A whole run, from the outside

`etl/tests/test_run.py` runs the tool as a program in a folder holding the fixture geodatabase, and reads what it leaves. The fixture holds seven cafés, six with one fault each and one clean, and two parks, one an area whose edge crosses itself. `tests/fixtures/make_fixtures.py` built it with GDAL; keep it and `conftest.py`, which came with `etl/` on day 7, since building it again needs GDAL's own Python bindings.

```python
assert code == 0
script = (geodatabase / "seed/0002-test-places.sql").read_text(encoding="utf-8")
# Of the seven cafés, the swapped one is mended and kept; four are dropped.
assert len([line for line in script.split("\n")
            if line.startswith('INSERT INTO "food"."cafes"')]) == 3
assert "'Swapped Café'" in script
assert "Far Away" not in script

log = (geodatabase / "seed-logs/0002-test-places.log").read_text(encoding="utf-8")
assert "Row 2: fixed by swapCoordinates: swapped longitude and latitude" in log
assert "Row 5: dropped by dropNearDuplicates: its name matches row 1, 15 m away" in log
assert "Result: wrote seed/0002-test-places.sql: 2 tables, 5 rows." in log
```

Its partner, `test_stops_at_a_bad_row_writes_no_script_and_logs_why`, runs `insideArea` alone with `"onFail": "stop"`. Row 2 has its numbers swapped and nothing swaps them back, so the run ends with exit code 1, writes no script, and still writes the log: 'Row 2: stopped by insideArea: its shape lies outside the area'. Run both files with `uv run pytest tests/test_run.py tests/test_pipeline.py`.

60.4 Run it on the office's places

From your copy's root, run the office's own pipeline file, which lists all nine tables with the eight filters and allows 5 % loss: `npm run etl -- data/macau.pipeline.json`. The shipped places are clean, so every filter keeps every row. In a run for this book the program ended 'Done: wrote db/seed/0002-macau-places-cleaned.sql: 9 tables, 900 rows.', with the log in `db/seed-logs/0002-macau-places-cleaned.log`, and listed three next steps: read and commit both, run `npm start`, run `npm run publish`.

Read the log and the script, then decide before you commit. `Tessera.csproj` copies every file in `db/seed/` into the release, and `Tessera seed` runs each one DbUp has not run, so a script left there replaces the nine tables at the next `npm start`, and DbUp, which knows a script by its name alone, never runs an edited copy again. Keep this one out of `db/seed`, as the worked answer does, unless you mean the office to load it.

> WATCH OUT The story calls the log the record, and ADR 0009 says the maintainer commits the script and its log together. Tessera's `.gitignore` lets the log through: `git check-ignore db/seed-logs/0002-macau-places-cleaned.log` prints nothing, so `git add db/seed-logs/` adds it. Keep your `.gitignore` that way; a broad `*.log` line would hide the record.

> IN INK Commit at each green, with messages such as `US-36: clean a geodatabase end to end, or stop with no script`, and push.

> THE PROOF `uv run pytest`, `uv run pyright`, `uv run ruff check .` and `uv run lint-imports` all pass in `etl/`, and `npm test` runs them beside every other check. Every story now has its code and its tests in your copy, and the burn-up reads 52 of 52. Monday's first step is a clean clone of your repository, not a test.

61

Day 60 · Monday 18 January: the third review, every story in ink

> IN SHORT Today you build your copy from a clean clone, run every test, and check all thirty stories against the worked answer, one row at a time, with no new code. The second plan's third review closes the plan, and the book.

The build is finished when a stranger's clone passes every test and every difference from the worked answer is written down. The review has the delivery plan's usual shape: every test from a clean start, the work checked against its evidence, the week's comparison notes read, and each difference you mean to keep recorded. This time the evidence is the whole of *Phase Oct*: 30 stories and 52 points over sixty working days.

EXHIBIT 61.1 The last review ends when a clean clone passes every test and each story has its proof

Goal	Your copy, cloned fresh, passes `npm test`; every story has a test or a recorded reason; every difference from the worked answer is written down
Story	All thirty, in `docs/user-stories/user-stories.md`
In the worked answer	`docs/requirements/*.md`, whose last column names each requirement's proof; `docs/guides/delivery-plan.md`
Look for	Rows marked 'No test yet' or 'in part', and whether your copy now proves them
Leave for later	Nothing: what you leave today, you leave on purpose, in writing
Done when	`npm test` passes in a fresh clone; `npm run e2e` ends with 'Nginx checks passed'; Parts E and F shown; your record of differences is committed; the tag `review-ef` is pushed

Source: `docs/guides/delivery-plan.md`, *'The weekly and daily rhythm'*

61.1 A clean start

Your working folder holds files no commit records, such as built output, downloaded packages and a `.env`, and a test may pass there only because of them. Clone the repository into a new folder, as the office machine or a reader would, and run everything from there, as Exhibit 61.1 asks:

```bash
cd ~/src
git clone https://github.com/YOUR-NAME/tessera-rebuild.git review-copy
cd review-copy
cp ~/src/tessera-rebuild/.env .env       # the passwords, which never reach GitHub
npm ci
npm test
```

`npm test` runs every check in order: the scripts' types and tests, the pipeline's four checks, the map's checks, the build, the three .NET test projects, architecture first, then the browser tests and the edge checks, ending with day 56's line, 'Nginx checks passed: …'. A failure here but not in your working folder means a file is missing from Git: find it, commit it, and run again.

> WATCH OUT Check the build on GitHub too. Your last push ran the same steps on a machine that has never seen your folder, and a green run there is the record the delivery plan asks for. If the pipeline's property tests fail there once on a time limit and pass when run again, note it: under load, hypothesis can exceed its deadline, as it did once in a run for this book.

61.2 Thirty stories, one table

Go down Exhibit 61.2, whose last three rows and S-07's hold Parts E and F, and open each proof in your copy. A story is done when its criteria have tests (for US-35, a check on the real machine), `npm test` passes, its commits name it, and its comparison is written down.

EXHIBIT 61.2 Every story, the days that built it, and where the worked answer proves it

STORY	DAYS	PROOF IN THE WORKED ANSWER
US-00 Walking skeleton	2	`SettingsTests`, `StartTests`, `ServiceTests`
US-01 One database	3	`DatabaseTests`, on the image `db/Dockerfile` builds
US-03 Publish by grant	4–6	Both `CatalogueTests`
US-24 Places in nine layers	7–8	`SeedingTests`; `etl/tests`
US-07 Serve a tile	9, 11	`TileTests`, `ServiceTests`
S-01 No SQL in a request	12–13	`SqlTests`, `TileTests`, `ServiceTests`, `BoundaryTests`
S-06 Malformed URLs	14	`ServiceTests`, `scripts/lib/edge.ts`
US-35 Port 80 for the office	15	`scripts/lib/office.test.ts`, and a request from another office machine
US-23 The map of the area	16–18	`MapTests`, `ServiceTests`, `App.test.tsx`, `map.test.ts`, `web/e2e/map.spec.ts`
UI-09 Levels 0 to 22	20–21	`controls.test.tsx`, `map.test.ts`
UI-15 A calm page	22–23	`rules.test.ts`, `design.test.ts`
UI-12 Shapes as well as colours	24	`design.test.ts`
UI-19 Scale, compass, position	25–26	`readouts.test.ts`, `controls.test.tsx`
UI-10 Clusters	27, 28, 30	`clusters.test.ts`
UI-11 Names from level 16	31	`style.test.ts`
UI-17 Names that never overlap	32–33	`style.test.ts`; screenshots in `web/e2e/__screenshots__`
S-07 Office machines alone	34	`plan.test.ts`, `scripts/lib/edge.ts`

N-18 A browser keeps a tile	36	`TileTests`, `ServiceTests`
N-19 A kept tile costs no query	37–38	`TileTests`, `ServiceTests`, `checkEdge`
N-20 Packed answers	39	`checkEdge`
N-21 One settings file	40–41	`SettingsTests`, `StartTests`
N-22 503 and the query limit	42, 44	`DatabaseErrorTests`, `CatalogueTests`, `TileTests`
N-23 Grants without a restart	45–46	Both `CatalogueTests`, `StartTests`
N-24 One query for a crowd	47	`checkEdge`
N-25 /health	48	`ServiceTests`, `CatalogueTests`, `StartTests`
N-26 Order of arrival	49	`PoolTests`
N-28 Kept tiles while it hangs	50–51	`checkEdge`, with both copies stopped; `StartTests`
S-30 A cap for each visitor	52–53	`checkEdge`: the flood and the forty
N-29 Two copies	55–56	`plan.test.ts`, `supervise.test.ts`, `checkEdge`
US-36 Clean places	57–59	`test_chain.py`, `test_filters.py`, `test_pipeline.py`, `test_run.py`

Source: `docs/guides/delivery-plan.md`; `docs/requirements/*.md`; *this book, days 2 to 60*

Mark each row 'proved', 'proved in part' or 'not proved', with the reason. The reviews of days 10, 19, 29, 35, 43 and 54 covered everything up to S-30: reread their notes rather than repeat the work, and look only for what has changed since.

61.3 Where your copy proves more

Of the requirements files' 151 rows, the worked answer proves 99 in full and 33 in part, and marks 19 'No test yet', every one of them from a story's criterion. Of the 52 rows not proved in full, count how many your copy now proves. The last twelve days alone add:

- N-26 c2: the request's own cancel signal, in `TileTests`, against the story's wrong sentence;
- N-28 c5 (NFR-33, in part there): `/health` held to one second;
- N-28 c1, c2 and c4: your notes on a stale tile and a paused database, observations rather than tests, unless you wrote day 51's optional check;
- S-30 c2 (SR-21, in part) and c5 (SR-24, none): `no-store` on 429, and this machine's own flood with no 429;
- N-29 c5 (NFR-38, none): a database outage that rests neither copy.

Where a row is still unproved, write why: some need the real office machine, such as US-35's four, and some need a person's eye, such as UI-15's elevation and corners. N-29 c3, a copy rested after three failures (NFR-36), you watched on day 56 but did not test; say so.

> KEY IDEA A worked answer is a guide, not a ceiling. Where it says 'No test yet', the honest copy either writes the test or writes down why not.

61.4 Compare the two folders

Set your copy beside the worked answer one last time, a folder at a time, and read the summary before any single file:

```bash
git diff --no-index --stat ~/src/tessera/src ~/src/review-copy/src
git diff --no-index --stat ~/src/tessera/tests ~/src/review-copy/tests
git diff --no-index --stat ~/src/tessera/scripts ~/src/review-copy/scripts
git diff --no-index --stat ~/src/tessera/nginx ~/src/review-copy/nginx
git diff --no-index --stat ~/src/tessera/etl/src ~/src/review-copy/etl/src
```

Sort each difference into one of three kinds. The first is planned: tests and fixes beyond the worked answer, such as the logged 500 of day 44 and the cancel-signal test of day 49. The second is a choice of your own that meets the same criteria, such as a different name or order of lines; keep it, and say why. The third is a gap: a criterion the worked answer meets and your copy does not. Each gap becomes a task with a test first, or a line in the record saying you accept it.

Write the record as a file in your repository, `docs/differences.md`, with one line for each difference: the file, what differs, which kind, and why. Add the findings of these days that concern the worked answer itself: N-26's Known limit names the wrong signal, and the template's comment on `proxy_read_timeout` does not hold for a database that hangs. It is the last document of the build, and the first a later reader will want.

> IN INK Commit `docs/differences.md` with the message `Review: record every difference from the worked answer`, push, then tag it with `git tag -a review-ef -m 'Third review of the second plan: every story in ink'` and push the tag.
>
> The tag fixes a name to the commit that holds all thirty stories, as `m1` to `m4` marked the milestones and `review-c` and `review-d` the first two reviews.

> THE PROOF A clone that has never seen your folder builds the service and the map, starts its own database and Nginx, and passes every test: the scripts', the pipeline's 253, the map's, the service's 106 and your own beside them, the browser's and Nginx's. The build on GitHub says the same. On day 2 your copy was an empty folder. Now every story is in ink, with a record anyone can read, one commit at a time.

> WHAT TO CARRY AWAY
>
> - **Read, then type.** The worked answer taught you; your copy, typed by hand, proved you learnt it.
> - **Red first, or break it on purpose.** A test you have never seen fail proves nothing.
> - **Write down the differences.** A difference you can explain is a decision; one you cannot is a fault you have not found yet.

A
Planned differences

IN SHORT The worked answer leaves 19 requirements with no test of their own, and your copy differs from it by design in the places listed here, each with a reason and, for most, an end.

Your copy and the worked answer should differ, and you should know exactly where. The worked answer holds all thirty stories from the first day, so for weeks your copy lacks what a later story adds. Where the worked answer proves nothing, the book has you write the test, so by the last review your copy proves more.

A.1 Parts with no test of their own

The worked answer's three requirements files, in `docs/requirements/`, trace 151 requirements to the stories and name the test that proves each: 99 in full, 33 in part, and 19 'No test yet'. Each of the 19 comes from a story's criterion, and the day that builds the story has you meet it, as Exhibit A.1 shows.

EXHIBIT A.1 Nineteen requirements in twelve stories have no test in the worked answer

STORY	CRITERIA WITH NO TEST	REQUIREMENT	THIS BOOK'S ANSWER
US-35	c1 to c4: the office reaches port 80, Nginx sees each machine, the address and the port survive a restart	FR-55 to FR-57, SR-14	Days 15, 19 and 35: checks by hand from another office machine
UI-09	c6: a zoom keeps each pin's size	FR-37	Days 21 and 24: 'keeps each pin's size at every level'
UI-15	c3 to c5: elevation, corners and state layers	NFR-51 to NFR-53	Day 23: a test in `rules.test.ts`
UI-10	c2: pins part from a cluster within 0.3 seconds	NFR-58	Day 30: 'parts a cluster within 300 ms', fragile
UI-17	c6: a name at a tile's edge gives way as one in the middle does	FR-48	Day 33: 'keeps a place from two tiles once'
N-18	c4: new places show once `max-age` passes	NFR-04	Days 36 and 38: checked by hand with a short `Tiles__CacheSeconds`
N-19	c4: 64 MB, least used first; c5: every kept tile expires	NFR-09, NFR-10	Day 38: `scripts/lib/nginx.test.ts`, which reads the template
N-24	c2: the query goes on when one browser leaves	NFR-16	Day 47: the five who stay, in `checkEdge`
N-25	c4: `npm start` waits for `UP`	NFR-48	Day 48: `scripts/lib/database.test.ts`

N-28	c2: an expired tile served while a fresh one is fetched	NFR-30	Day 51: watched by hand; an optional check in `edge.ts`
S-30	c5: no limit on the machine itself	SR-24	Day 53: this machine's flood meets no 429
N-29	c3: three failures rest a copy; c5: a database outage rests neither	NFR-36, NFR-38	Day 56: c5 in `scripts/e2e.ts`; c3 watched by hand

Source: `docs/requirements/functional-requirements.md`, `non-functional-requirements.md` and `security-requirements.md`; this book, days 15 to 56

Some rows resist a test: US-35's four need the real office machine and another beside it, and N-28 c2 and N-29 c3 need a long wait, so the day has you check by hand and write what you saw. The 33 rows proved 'in part' have a test that covers less than the criterion; the 'Checked by' column names what is missing, and the days meet many of them.

A.2 Tests beyond the worked answer

Each test your copy holds and the worked answer lacks is a planned difference that closes a gap the requirements name, or a known fault. Exhibit A.2 lists them in the order you write them.

EXHIBIT A.2 The tests and checks of your own, the day each arrives, and the gap it closes

DAY	TEST OR CHECK	WHERE	WHAT IT CLOSES
2	`Answers_a_request_on_its_port`, until day 11	`StartTests`	US-00 c2, with no database
2	`Names_no_machine_in_the_code`	`SettingsTests`	SR-25, in part
4	`A_failure_skips_every_later_step`, a learning test	`ResultTests`	A library rule
9	Rows `12/3339/1788` and `12/4096/0`	`TileTests`	The example tile; the grid's edge
10	`Runs_the_scripts_in_the_order_of_their_names`, optional	`SeedingTests`	FR-19, in part
11	`Refuses_an_unknown_layer_before_any_query_runs`	`TileTests`	US-07 test 1
13	`Answers_an_attack_as_a_problem_document_and_keeps_all_nine_tables`	`ServiceTests`	SR-08, in part
14	The target `/x/%2e%2e/food.cafes/0/0/0.pbf`, and Nginx's rule against `%2e`; a raw tab and a backslash, optional	`edge.ts`, the template	Known fault of S-06 c3 (SR-11); SR-09
16	`Names_the_same_nine_layers_as_the_map`	`LayerListTests`	US-23 c7
21, 24	'keeps each pin's size at every level (UI-09, criterion 6)'	`style.test.ts`	FR-37

23	‘follows Material’s elevation, corners and state layers (UI-15, c3 to c5)’	`rules.test.ts`	NFR-51 to NFR-53
25	‘draws four sections that change within 0.3 s (UI-19, c2, c3)’	`controls.test.tsx`	FR-50; NFR-60, in part
26	‘points north with a red needle and a red ‘N’ (UI-19, criterion 5)’	`controls.test.tsx`	FR-52, in part
28	‘changes the fingerprint only when the places change (ADR 0005, step 5)’	`style.test.ts`	A rule with no test
30	‘UI-10 c2 (beyond the worked answer): parts a cluster within 300 ms’	`map.spec.ts`	NFR-58 (fragile)
33	`Leaves_a_view_without_an_order`; ‘UI-17 c6 (beyond the worked answer): keeps a place from two tiles once’	`TileTests`; `style.test.ts`	UI-17 test 2; FR-48
38	‘keeps at most 64 MB of tiles’; ‘keeps a tile no longer than the service allows’	`scripts/lib/nginx.test.ts`	NFR-09, NFR-10
39	Plain on request, `Vary` on the plain answer, the page packed, 304 for the packed form	`checkEdge`	NFR-11 to NFR-14, in part
40	`Keeps_five_sections_and_a_unit_in_each_time`	`SettingsTests`	NFR-41, in part
41	`Lets_a_variable_with_two_underscores_override_the_file`	`StartTests`	NFR-43, in part
41	`Names_a_misspelt_key_with_every_other_problem`, with `KnownKeys` in `Settings.Section`	`SettingsTests`	Known fault: a misspelt key
44	`Answers_500_and_logs_the_cause`, with `TileServer`’s optional `ILogger`	`TileTests`	Known fault (NFR-21)
46	The map’s page answers 200 with the database down	`StartTests`	NFR-24, in part
47	The first asker leaves; the five who stay wait for one query	`checkEdge`	NFR-16
48	`no-store` on `DOWN`; ‘says up only once /health says UP’ and ‘never counts DOWN as ready’	`StartTests`; `scripts/lib/database.test.ts`	NFR-47, in part; NFR-48
49	`Hands_the_tile_query_the_requests_own_cancel_signal`	`TileTests`	Known fault: the wrong signal
51	`/health` held to one second	`StartTests`	NFR-33, in part

52	`no-store` on a 429	`checkEdge`	SR-21, in part
53	600 requests from 127.0.0.1 meet no 429	`checkEdge`	SR-24
56	A database outage rests neither copy	`scripts/e2e.ts`	NFR-38

Source: this book, days 2 to 56; `docs/requirements/*.md`

A.3 Where your copy differs by design

A planned difference is one the book chose, so your comparison notes need not puzzle over it. Exhibit A.3 lists them in the order you meet them; 'Kept' marks a difference that lasts to the end.

EXHIBIT A.3 Planned differences, each with a reason and an end

DIFFERENCE	FROM DAY	WHY	UNTIL
Your commit messages name their story, such as `N-25: …`; the worked answer's do not	2	The record then reads story by story	Kept
No settings file: the environment alone sets each value, and `Urls` puts the service on port 8080	2	`tessera.json` belongs to N-21	Days 40 and 41 (port 7800)
Two test projects; `build.ts` builds the solution alone; CI runs the steps your copy can run	2	S-01 brings the architecture tests, US-23 the map	Days 12 and 16; CI grows with the stories
`SettingsTests`' helper registers `DATABASE_URL` alone; `StartTests` keeps its own `FreePort`	2	The worked answer's helpers read every section	Days 11 (`Http.cs`) and 36
`Signs_in_as_the_shipped_reader_role` asks `SELECT current_user`; `Ships_the_seed_made_from_the_places` reads `db/seed/` from the repository	3	The catalogue and `SeedScripts.Folder` come later	Days 6 and 7, if you switch
Your database is `rebuild-db`, image `tessera-rebuild-db`, on port 5434; the test image is `tessera-rebuild-db-test`	3	The worked answer's `tessera-db` holds 5432, so both can run at once	Kept
`Database.Open` takes the connection string alone	6	The pool's limits belong to N-21	Day 41
`DatabaseError` is one record, and the tile route reads 42501 and 42P01 as text	6	The four kinds belong to N-22	Days 42 and 44
The integration `CatalogueTests`' stage has no rows and no `stage.slow`, so it expects five ids, not six	6	The tile route and N-22 need them	Days 11 and 44
`etl/` copied whole from the worked answer	7	The pipeline holds 253 tests, too many for US-24's two days	Days 57–59
The catalogue loads once, before the service listens; `TileServer` takes a `LayerCatalogue`; `ServiceFixture` waits for no list	11	Background loading belongs to N-23	Days 45 and 46
`TileSettings` has no `CacheSeconds`; `Problems.Shape` writes no `Retry-After`	11	N-18 and N-22 add them	Days 36 and 44

`Starts_from_the_run_command_and_answers` without its two lines on the log	11	Until N-21's `Logging`, the program logs each request	Day 40
Your Nginx template and `e2e.ts` are cut down, and Nginx refuses a target that holds `%2e`	14	Later stories add lines; the rule mends a fault	Day 56; rule kept
Your `publish.ts` names `rebuild-db` and neither waits for the database nor reads its place	8	`scripts/lib/database.ts` arrives with `npm start`	Day 34
You start Nginx and the service by hand, with a `docker run` line of the book's own	15	`npm start` comes on the day in hand	Day 34
`AreaSettings` and `MapSettings` run on their defaults, each bound by a line of its own	16	The settings file belongs to N-21	Day 40
Your map's model holds four fields, not seven; `app.test.ts` runs `tileFailed` in place of `toggleUnits`	17	Later map stories add the rest	Day 25 onwards
Your `openMap` waits for `settle` alone; your browser test of the style expects no `circle` layer	18	The drop is UI-15's; the `circle` feed layers UI-10's	Days 23 and 28
Your approved screenshots differ from `web/e2e/__screenshots__`	33	Each machine draws fonts its own way	Kept
Your `npm start`: one copy, `rebuild-db`, port 8080, no restarts, no wait for `/health`; `nginxArguments` takes one port	34	S-07 c4 needs it before N-21, N-25 and N-29	Days 41, 48 and 56
The Windows start-up task, built before N-29	35	The office machine restarts before 18 January	Kept
Your second plan, `docs/second-plan.md`	36	The delivery plan only sketches it	Kept
`TileResult` carries no tag	36	The tag belongs to N-19	Day 37
`Urls` in `tessera.json` reads port 8080	40	N-21's second day moves it	Day 41
`Settings.Section` refuses an unknown key in a check of its own, `KnownKeys`	41	It mends a known fault	Kept
`TileServer` takes an optional `ILogger`, and logs the cause of a 500	44	It mends a known fault	Kept
`LiveCatalogue` loads once, in the background	45	The loop comes next	Day 46
One copy of the service, with four connections, meets the crowd and the limits	47	The second copy belongs to N-29	Day 56
`checkEdge` takes one stop function	50	One copy runs until N-29	Day 56
`docs/differences.md`, the record of every difference	60	The worked answer has nothing to compare with	Kept

Source: this book, days 2 to 60; the worked answer's `README.md` *and* `git log`

> WATCH OUT A difference missing from these tables is unplanned. Write it in your comparison notes the day you find it, with its reason, and decide at the next review whether to keep it or close it.

B
The words this book uses

> IN SHORT This glossary gives every word the book sets in bold italics, and a few more, each with a plain meaning and the day that first explains it.

The words fall into three groups: the way you work, the code, and the database, the network and the store. Each group runs in alphabetical order; after each meaning, the day and the story that explain the word appear in grey.

B.1 Words about the way you work

Approved image
A screenshot a person has checked and committed; a browser test compares each new one with it. Day 33 · UI-17

Approved snapshot
Output a person has read and accepted, such as the tile query's text, for a test to compare with. Day 12 · S-01

Architecture test
A test that reads the built program and checks which part may use which part. Day 12 · S-01

Beyond the worked answer
Your test for a criterion the worked answer leaves untested or proves in part (Appendix A). Day 2 · US-00

Burn-up
A chart of points planned by each Friday and points done, both climbing towards the scope. Chapter 1

Clean run
A run of every test that starts with nothing left on the machine from earlier days. Day 10 · the first review

Commit
A saved snapshot of your files, with a message saying what changed and which story it serves. Chapter 1

Concept
An important idea, set in bold italics where the book explains it. How to read this book

Continuous integration
CI for short: a machine on GitHub that builds the project and runs every test after each push. Chapter 1

Criterion
One line of a story that says when it is done, as given, when, then; 'N-25 c2' is N-25's second. Chapter 1

In ink
Work committed and pushed, and so kept in the record; work left on your machine overnight is pencil. Chapter 1

Iteration
About two weeks of stories, ended by a review; the first plan has four, the second three. Chapter 1

jsdom
A pretend browser with no screen: fast for tests, but blind to drawing. Day 18 · US-23

Learning test
A test of a library you did not write, which fails when an upgrade changes what you rely on. Day 4 · US-03

Milestone
A state of the product someone could see and test, such as M1 'The places are published by grant'. Chapter 1

Property test
A test that states a rule and lets a library, here FsCheck, try it on a hundred made-up inputs. Day 4 · US-03

Release
Not a file you send but a tagged commit that the office machine clones and builds. Day 35 · the release

Review
An iteration's last day: every test from a clean start, the goal checked, the next iteration planned. Chapter 1

Shrinking
After a failure, a property test tries ever smaller inputs that still fail, to find the smallest. Day 4 · US-03

Start-up task
A task that makes Windows run `npm start` whenever the office machine switches on. Day 35 · the release

Story point

A measure of how much work a story holds; one point is about one working day, tests included. Chapter 1

Tag

A name fixed to one commit, such as `m1` or `review-ef`. Day 10 · the first review

Test-first loop

Watch a new test fail (red), write the least code that passes it (green), then tidy the code. Chapter 1

User story

One thing a person needs, as 'As …, I want …, so that …', with criteria that say when it is done. Chapter 1

Walking skeleton

The thinnest program that works from end to end: it builds, tests, starts and answers. Day 2 · US-00

Worked answer

The finished repository, Tessera, which you read and run but never change. Chapter 1

B.2 Words about the code

Accessible name

The words a screen reader says for a control; a test finds each button by them. Day 21 · UI-09

Anchor

A side of a pin where its name may sit; MapLibre tries four, then drops the name. Day 32 · UI-17

Atomic operation

A step no other thread can see half done, such as `Interlocked.Exchange`. Day 45 · N-23

Back-off

Waiting longer after each quick failure before trying again: 1 s, then 2, 4 and so on, up to 30. Day 55 · N-29

Branded type

A number marked for the compiler alone, so a web-map level and a MapLibre zoom never mix. Day 20 · UI-09

Cluster

One round mark that stands for several places close together on the screen. Day 27 · UI-10

Collision box

The patch of screen a name or icon claims; MapLibre drops a later symbol that overlaps it. Day 32 · UI-17

Colour role

A named use of a colour in Material 3, such as `primary` for actions, built from one seed colour. Day 22 · UI-15

Configuration binding

ASP.NET Core copying each key of a settings section into the record's property of the same name. Day 40 · N-21

Content Security Policy

The header that tells a browser which scripts a page may run. Day 16 · US-23

Content type

The label that names an answer's kind of bytes, such as `application/vnd.mapbox-vector-tile`. Day 11 · US-07

Contrast ratio

How far a colour stands out from its background, up to 21 to 1; text needs 4.5 to 1. Day 24 · UI-12

DbUp

A library that runs SQL scripts against a database, each once, in the order of their names. Day 7 · US-24

Elmish loop

A message reaches one pure update, which returns a new model and maybe a command. Day 17 · US-23

Endpoint filter

A step ASP.NET Core runs around each route of a group, such as the page's headers. Day 16 · US-23

Environment variable

A named value the operating system hands a program as it starts, such as `DATABASE_URL`. Day 2 · US-00

Exit code

The whole number a program hands back as it ends: 0 for success, any other for a kind of failure. Day 2 · US-00

Expansion level

The level at which MapLibre says a cluster breaks apart into its places. Day 30 · UI-10

Feed layer

An invisible circle layer whose one job is to make MapLibre load a layer's tiles. Day 28 · UI-10

Filter

One step of the pipeline's cleaning that judges a table's rows and fixes, drops or stops. Day 57 · US-36

Fingerprint

The loaded places' ids as one string, so the map skips clustering when nothing changed. Day 28 · UI-10

Hosted service

A background loop that .NET starts and stops with the program, such as the list's reload. Day 45 · N-23

List pattern

A pattern in a C# `switch` that matches a list by its items: `[]` matches none, `["seed"]` exactly one. Day 2 · US-00

Module

A part of the program with one door; Tessera has one, `Modules/Tessera`, in three layers. Day 4 · US-03

Namespace

The name that groups C# types; in Tessera each folder's path is its namespace. Day 2 · US-00

Port of the update

A function the map's update asks to move the map; a test replaces it with a fake. Day 17 · US-23

Problem document

A short report of a failure in JSON, in RFC 9457's shape, so every client reads errors alike. Day 11 · US-07

Pure function

A function that reads only its inputs and changes nothing outside itself. Day 5 · US-03

Record

A type with named fields that never changes once made, like the `RECORD` of your pseudocode. Day 5 · US-03

Reduced motion

A visitor's request, through `prefers-reduced-motion`, that nothing on the page should move. Day 23 · UI-15

Smart constructor

A private constructor with static functions that keep a rule, so no wrong value can be built. Day 4 · US-03

Solution

The file that lists every project the build compiles; Tessera's is `Tessera.slnx`. Day 2 · US-00

Sort key

A number on each place, so MapLibre sets names in a fixed order and one name wins each clash. Day 33 · UI-17

State machine

A stage, such as `waiting` or `landed`, and a function that moves it on at each event. Day 23 · UI-15

Stop rule

The pipeline's last step, which stops a table that lost more of its rows than the file allows. Day 58 · US-36

Tile address

Three whole numbers, level, column and row, that name one square of the map. Day 9 · US-07

Web-map level

The zoom every web map shares, from 0, the world in one square, to 22; MapLibre's runs one lower. Day 20 · UI-09

B.3 Words about the database, the network and the store

Bound parameter

A value sent beside a query's text, never inside it, like an answer written in a form's box. Day 12 · S-01

Burst

The requests a visitor may send above the steady rate before the limit bites: 300 tiles. Day 52 · S-30

Cache

A store of answers kept to give again, such as Nginx's 64 MB of tiles. Day 38 · N-19

Cache lock

Nginx letting one request fetch a missing tile while the others for it wait for that one answer. Day 47 · N-24

Cache-Control

The header that tells a browser, and every store on the way, what it may keep and for how long. Day 36 · N-18

Catalogue

The list of layers the service may serve, read from the database as the service's own role. Day 5 · US-03

Connection pool

A few database connections kept open for requests to share, in turn. Day 41 · N-21

Container

A program run by Docker in a closed box with its own files. Day 1

ETag

A label for one version of an answer, which a browser sends back to ask if its copy holds. Day 37 · N-19

Firewall

The program that decides which requests may enter a machine; Windows and Hyper-V each keep one. Day 15 · US-35

Geodatabase

A folder ending `.gdb` in which many desktop map programs save their layers. Day 59 · US-36

Grant

The act by which a table's owner lets a role read it; only a grant publishes a layer. Day 4 · US-03

gzip

A way to pack bytes without loss: unpacking gives back every byte. Day 39 · N-20

Hash

A fixed-size fingerprint computed from bytes, which changes whenever a byte does. Day 37 · N-19

Health check

An address a watcher asks to learn whether the service can do its job: Tessera's `/health`. Day 48 · N-25

Image

A packed set of files from which Docker starts containers. Day 3 · US-01

Journal

The table in which DbUp records each script it has run: `tessera.schemaversions`. Day 7 · US-24

Layer

A table the service may serve, named by its schema and table, such as `food.cafes`. Day 4 · US-03

Loopback address

`127.0.0.1`, which every computer uses to mean 'this computer'. Day 1

Malformed URL

An address that looks clean until a server tidies it, such as `..` hidden as `%2e%2e`. Day 14 · S-06

max-age

The part of `Cache-Control` that says for how many seconds a copy stays good. Day 36 · N-18

Mirrored networking

WSL2's mode in which Ubuntu shares the Windows machine's address, so the office reaches Nginx. Day 15 · US-35

Network prefix

The count of leading bits the addresses of a network share: `/24` leaves 256 addresses. Day 34 · S-07

Parameter

A value sent beside a query, named in it as `@z`, so the database never runs it as a command. Day 6 · US-03

Passive health check

Nginx watching real requests and resting a copy of the service that fails. Day 56 · N-29

Port

The number that picks one program among the many that listen on a machine. Day 1

Privilege

A role's right over a table or schema, such as `SELECT` to read or `USAGE` to look inside. Day 8 · US-24

Rate limit

A cap on how many requests one visitor may make in a second: 100 tiles for each office machine. Day 52 · S-30

Reverse proxy

A server in front of another that receives each request first and passes on those it accepts. Day 14 · S-06

Role

A name and a password that can sign in to PostgreSQL, with a list of what it may do. Day 3 · US-01

Round robin

Sending requests to the members of a group in turn: 7800, 7801, 7800 … Day 56 · N-29

Schema

A named folder of tables inside a database, such as `food`. Day 4 · US-03

SHA-256 fingerprint

A 64-digit number computed from every byte of a file: change one byte, and the number changes. Day 3 · US-01

SQL injection

SQL hidden inside a request in the hope that the database runs it. Day 4 · US-03

SQLSTATE

PostgreSQL's five-character code for a refusal; its first two characters give the class. Day 42 · N-22

SRID

The number of a coordinate system: 4326 for longitude and latitude, 3857 for Web Mercator. Day 5 · US-03

Stale tile

A kept tile past its `max-age`, served while the service cannot answer or a fresh copy is fetched. Day 50 · N-28

Transaction

A group of changes the database makes all together or not at all. Day 7 · US-24

Upstream

Nginx's named group of servers; Tessera's, `tessera`, holds the service's copies. Day 14 · S-06

Virtual machine

A computer made of software; WSL2 runs Linux in a light one inside Windows. Day 1

Weak ETag

An ETag marked `W/`: the same content, not the same bytes, so it fits a tile packed or plain. Day 37 · N-19